

Matemáticas y Algoritmos en Ciencia de Datos Aplicada

Libro del curso MATE-2105

César Galindo

2026-07-07

Tabla de contenidos

Prefacio	23
Comenzar la lectura	24
Tesis pedagógica	24
Relación con el portal del curso	25
Fuente canónica	25
Cómo usar este libro	26
Antes de clase	26
Después de clase	26
Antes de una evaluación	26
Qué queda fuera del libro	27
Mapa del curso y del libro	28
Parte I: Fundamentos	30
Parte I: Fundamentos	31
Resultados de aprendizaje de la parte	31
Mapa de la parte	32
Criterio de salida	32
Capítulo 0: Ciencia de datos aplicada	33
Vista previa	34
0.1 Observaciones, variables y datasets	35
Ejemplo 0.1: una tabla pequeña	35
Punto de control 0.1	36
0.2 Preguntas aplicadas y tareas de aprendizaje	37
Tipos de tareas	37
0.3 Modelos, parámetros y predicciones	39
Ejemplo 0.2: una predicción lineal	39
Punto de control 0.2	40
0.4 Pérdida, métrica y baseline	41
Ejemplo 0.3: baseline de regresión	41
Ejemplo 0.4: baseline de clasificación	42
Accuracy	42
Error absoluto medio	42

Error cuadrático medio	43
0.5 Entrenamiento, prueba y generalización	44
Ejemplo 0.5: una mala evaluación	44
0.6 Primer flujo aplicado en Python	45
Cómo leer este código	46
0.7 De número a conclusión técnica	47
Plantilla de conclusión	47
Ejemplo 0.6: conclusión débil y conclusión mejor	47
0.8 Errores comunes al empezar	48
Error 1: empezar por el algoritmo	48
Error 2: confundir pérdida con métrica	48
Error 3: no tener baseline	48
Error 4: evaluar con datos de entrenamiento	48
Error 5: prometer más de lo que los datos muestran	48
Resumen del capítulo	49
Terminos clave	50
Ejercicios	51
Conceptos	51
Calculo manual	51
Interpretacion	52
Codigo	52
Proyecto	52
Respuestas seleccionadas	53
Interludio 0.5: Incertidumbre y datos observados	54
Pregunta aplicada	54
Objetivos del capítulo	54
De tabla observada a variable incierta	55
Observación como realización	55
Distribución como patrón de variabilidad	56
Promedio, valor esperado y varianza	56
Ruido: por qué el modelo no debe acertar exactamente	57
Condicionamiento: mirar Y dado $X = x$	57
Pérdida empírica y riesgo esperado	58
Por qué separar entrenamiento y prueba	59
Qué debes llevar a los próximos capítulos	59
Punto de control	59
Verificaciones seleccionadas	60
Para explorar más	60
Revisión del capítulo	60
Términos clave	60

Ejercicios	61
Parte II: Regresión, optimización y clasificación	62
Parte II: Regresión, optimización y clasificación	63
Resultados de aprendizaje de la parte	63
Mapa de la parte	64
Criterio de salida	64
Capítulo 1: Regresión lineal múltiple	65
Pregunta aplicada	65
Objetivos del capítulo	65
El problema de regresión	66
De una recta a una matriz	67
Dimensiones: la primera defensa contra errores	68
Residuales y pérdida	68
Derivación del gradiente	69
Ecuación normal	70
Interpretación geométrica	71
Ejemplo resuelto 1: datos exactos	71
Ejemplo resuelto 2: datos con ruido	72
Baseline de regresión	72
Interpretación de coeficientes	73
Multicolinealidad y rango	73
Errores comunes	74
Checklist de estudio	74
Para explorar más	74
Revisión del capítulo	74
Términos clave	75
Ejercicios	75
Capítulo 2: Descenso del gradiente	77
Pregunta aplicada	77
Objetivos del capítulo	77
Optimización como búsqueda	77
Idea geométrica del gradiente	78
Algoritmo básico	80
Cómo leer una curva de pérdida	81
Ejemplo resuelto: una variable escalada	81
Escalamiento y geometría	82
Condicionamiento	82
Comparación con ecuación normal	82
Batch, stochastic y mini-batch	83
Tasa de aprendizaje: una rutina práctica	83
Errores comunes	84
Checklist de estudio	84
Para explorar más	84

Revisión del capítulo	84
Términos clave	85
Ejercicios	85
Capítulo 3: Validación y regularización	87
Pregunta aplicada	87
Objetivos del capítulo	87
Feature engineering	87
Regresión polinomial	88
Underfitting y overfitting	89
Separación de datos	89
Leakage	90
Regularización	90
Ridge	91
Efecto de λ	92
Lasso	92
Ridge vs Lasso	92
Estandarización antes de regularizar	93
Selección de modelo	93
Ejemplo resuelto: grado polinomial y Ridge	94
Errores comunes	94
Checklist de estudio	94
Para explorar más	94
Revisión del capítulo	95
Términos clave	95
Ejercicios	95
Capítulo 4: Regresión logística	97
Pregunta aplicada	97
Objetivos del capítulo	97
Clasificación binaria	98
La función sigmoide	99
Modelo logístico	99
Log-odds	100
Por qué no usamos MSE	100
Gradiente de la pérdida logística	101
De probabilidades a clases	101
Matriz de confusión	102
Métricas	102
Ejemplo resuelto: umbral y matriz de confusión	103
Elegir métricas según la decisión	104
Umbrales	104
Baselines de clasificación	105
Calibración	105
Errores comunes	106
Checklist de estudio	106
Para explorar más	106
Revisión del capítulo	106

Términos clave	107
Ejercicios	107
Ejercicios integradores: regresión, optimización y clasificación	109
Cómo usar este banco	110
Resultados de práctica	110
Mapa del banco	110
Semana 2: regresión lineal	111
A. Dimensiones y representación	111
B. Mínimos cuadrados	111
C. Implementación	111
D. Interpretación	112
Semana 3: descenso del gradiente	113
A. Gradiente	113
B. Iteraciones	113
C. Diagnóstico	113
D. Código	113
Semana 4: features y validación	114
A. Complejidad	114
B. Particiones	114
C. Feature engineering	114
D. Código	114
Semana 5: regularización	115
A. Ridge	115
B. Lasso	115
C. Selección	115
D. Código	115
Semana 6: clasificación logística	116
A. Probabilidades	116
B. Log-loss	116
C. Métricas	116
D. Umbrales	116
Problemas integradores	117
Problema 1: regresión de principio a fin	117
Problema 2: regularización y estabilidad	117
Problema 3: clasificación con umbral	118
Problema 4: diseño de parcial	118
Rúbrica de autoevaluación	119
Guía de verificación seleccionada	120

Parte III: Redes neuronales	121
Parte III: Redes neuronales	122
Resultados de aprendizaje de la parte	122
Mapa de la parte	123
Criterio de salida	123
Perceptron, neurona logística y forward propagation	124
Pregunta aplicada	124
Objetivos del capítulo	124
Perceptron como unidad básica	124
Neurona logística	125
De una neurona a una capa	125
Red de una capa oculta	125
ReLU	126
Softmax	126
Entropía cruzada multiclase	126
Ejemplo resuelto: softmax y pérdida	127
Ejemplo resuelto: dimensiones	128
Caso longitudinal: dígitos escritos a mano	128
Punto de control	128
Verificaciones seleccionadas	129
Ejemplo resuelto: auditoría de formas en dos capas	129
Para explorar más	129
Revisión del capítulo	130
Términos clave	130
Ejercicios	130
Backpropagation	132
Pregunta aplicada	132
Objetivos del capítulo	132
El grafo de cómputo	132
Gradiente de softmax con entropía cruzada	133
Gradientes de la segunda capa	133
Propagar hacia la capa oculta	134
Backpropagation como algoritmo	134
Verificación por diferencias finitas	135
Ejemplo resuelto: diferencia finita escalar	135
Ejemplo resuelto: formas de los gradientes	136
Errores frecuentes	137
Punto de control	137
Caso longitudinal: backpropagation en dígitos	137
Verificaciones seleccionadas	137
Para explorar más	138
Revisión del capítulo	138
Términos clave	138
Ejercicios	138

Activaciones, inicialización, regularización y diagnóstico	140
Pregunta aplicada	140
Objetivos del capítulo	140
Activaciones	140
Inicialización	141
Regularización L2 en redes	141
Ejemplo resuelto: L2 en pérdida y gradiente	142
Dropout como idea	142
Curvas de entrenamiento	143
Diagnósticos comunes	143
Ejemplo resuelto: leer una curva	143
Checkpoint técnico del proyecto	144
Caso longitudinal: curva de entrenamiento en dígitos	144
Verificaciones seleccionadas	144
Ejemplo resuelto: regla de parada	144
Lectura de curvas: tres patrones frecuentes	145
Punto de control	145
Para explorar más	145
Revisión del capítulo	146
Términos clave	146
Ejercicios	146
 Optimizadores y PyTorch	 148
Pregunta aplicada	148
Objetivos del capítulo	149
Mini-batch gradient descent	150
Momentum	150
Ejemplo resuelto: dos pasos con momentum	150
Derivación guiada: momentum como memoria exponencial	151
RMSProp y Adam	151
Ejemplo resuelto: Adam en una coordenada	152
PyTorch: qué automatiza	152
Loop mínimo	152
Ejemplo resuelto: lectura de un loop	153
Comparación justa	153
Caso longitudinal: experimento reproducible en dígitos	154
Punto de control	154
Verificaciones seleccionadas	154
Ejemplo resuelto: presupuesto de entrenamiento	154
Convergencia no es validación	155
Para explorar más	155
Revisión del capítulo	155
Términos clave	156
Ejercicios	156
 Ejercicios integradores: redes neuronales	 157
Resultados de práctica	157
Mapa del banco	157

Cómo usar esta sección	157
Bloque A: dimensiones y forward	158
Bloque B: backpropagation	158
Bloque C: diagnóstico	158
Bloque D: PyTorch	158
Respuestas seleccionadas	159
Parte IV: Estrategia de machine learning	160
Parte IV: Estrategia de machine learning	161
Resultados de aprendizaje de la parte	161
Mapa de la parte	162
Criterio de salida	162
Métricas y particiones	163
Pregunta aplicada	163
Objetivos del capítulo	163
Métrica principal	164
Accuracy no siempre basta	165
Ejemplo resuelto: accuracy engañosa	165
Ejemplo resuelto: costo de falsos negativos	166
Derivación guiada: F1 como equilibrio operativo	166
Baseline	167
Particiones	167
Test se toca una vez	167
Estratificación	167
Ejemplo resuelto: conteos en una partición estratificada	168
Leakage	168
Pipeline como defensa	168
Caso longitudinal: clasificación con slice reciente	169
Punto de control	169
Verificaciones seleccionadas	169
Para explorar más	170
Revisión del capítulo	170
Términos clave	170
Ejercicios	170
Validación cruzada y curvas de aprendizaje	172
Pregunta aplicada	172
Objetivos del capítulo	172
Cross-validation	172
Qué reportar	172
Ejemplo resuelto: elegir entre dos modelos	173
Derivación guiada: media y variabilidad de CV	173
Grid search	173
Learning curve	174
Ejemplo resuelto: leer una learning curve	175

Validation curve	175
Caso longitudinal: estabilidad del modelo de clasificación	175
Ejemplo resuelto: decisión con diferencia pequeña	176
Sesgo, varianza y ruido	176
Punto de control	176
Verificaciones seleccionadas	177
Ejemplo resuelto: cuándo no hacer grid más grande	177
Para explorar más	177
Criterio práctico de decisión	177
Revisión del capítulo	177
Términos clave	178
Ejercicios	178
Error analysis y data mismatch	180
Pregunta aplicada	180
Objetivos del capítulo	180
Error analysis	180
Slices	181
Data mismatch	182
Diagnóstico de mismatch	182
Ejemplo resuelto: mismatch por proporción de clase	182
Derivación guiada: promedio global como mezcla de slices	183
Recomendación técnica	183
Ejemplo resuelto: dos modelos con el mismo F1	183
Ejemplo resuelto: brecha contra el promedio	184
Caso longitudinal: slice reciente	184
Punto de control	185
Verificaciones seleccionadas	185
Procedimiento de auditoría por slice	185
Para explorar más	185
Revisión del capítulo	186
Términos clave	186
Ejercicios	186
Ejercicios integradores: estrategia de machine learning	188
Resultados de práctica	188
Mapa del banco	188
Criterio de respuesta profesional	188
Bloque A: métricas	189
Bloque B: particiones y leakage	189
Bloque C: validación y curvas	189
Bloque D: error analysis	189
Respuestas seleccionadas	189

Parte V: Aprendizaje no supervisado	191
Parte V: Aprendizaje no supervisado	192
Resultados de aprendizaje de la parte	192
Mapa de la parte	193
Criterio de salida	193
SVD y PCA	194
Pregunta aplicada	194
Objetivos del capítulo	194
SVD	195
PCA como SVD del dato centrado	195
Proyección	195
Varianza explicada	196
Ejemplo resuelto: error desde valores singulares	196
Derivación guiada: reconstrucción y pérdida de información	197
Ejemplo resuelto: elegir k	197
Reconstrucción	197
Ejemplo resuelto: proyección y reconstrucción	198
Caso longitudinal: PCA sintético	199
PCA no es interpretabilidad automática	199
Punto de control	199
Verificaciones seleccionadas	200
Ejemplo resuelto: PCA dentro de un pipeline	200
Para explorar más	200
Revisión del capítulo	200
Términos clave	201
Ejercicios	201
K-Means y clustering	202
Pregunta aplicada	202
Objetivos del capítulo	202
Función objetivo	203
Algoritmo	203
Inertia	203
Silhouette score	203
Ejemplo resuelto: por qué escalar cambia clusters	204
Ejemplo resuelto: asignación a centroides	204
Ejemplo resuelto: una iteración completa	204
Derivación guiada: por qué el centroide es el promedio	205
Escalamiento	206
Limitaciones	206
Caso longitudinal: clusters sintéticos	206
Estabilidad por inicialización	206
Punto de control	207
Verificaciones seleccionadas	207
Ejemplo resuelto: elegir k con evidencia incompleta	207
Para explorar más	208

Revisión del capítulo	208
Términos clave	208
Ejercicios	208
Recomendación básica y estructura latente	210
Pregunta aplicada	210
Objetivos del capítulo	210
Matriz usuario-ítem	210
Similitud coseno	211
Ejemplo resuelto: similitud coseno	211
Derivación guiada: qué ignora la similitud coseno	211
Recomendación item-item	212
Ejemplo resuelto: recomendación item-item	212
Evaluación offline: Precision@k	212
Baseline de popularidad	213
SVD para factores latentes	213
Caso longitudinal: interacciones de recomendación	214
Riesgos de recomendación	215
Punto de control	215
Verificaciones seleccionadas	215
Ejemplo resuelto: cobertura mínima	215
Para explorar más	216
Revisión del capítulo	216
Términos clave	216
Ejercicios	216
Ejercicios integradores: aprendizaje no supervisado	218
Resultados de práctica	218
Mapa del banco	218
Antes de responder	218
Bloque A: PCA	219
Bloque B: K-Means	219
Bloque C: recomendación	219
Respuestas seleccionadas	219
Parte VI: Reproducibilidad, comunicación y defensa	220
Parte VI: Reproducibilidad, comunicación y defensa	221
Resultados de aprendizaje de la parte	221
Mapa de la parte	222
Criterio de salida	222
Síntesis conceptual: redes, estrategia y no supervisado	223
Pregunta aplicada	223
Alcance	223
Objetivos del capítulo	223
Mapa de contenidos	223

Redes neuronales	224
Softmax y entropía cruzada	224
Diagnóstico de modelos	224
No supervisado	225
Forma del Parcial 2	225
Regla de preparación	225
Cómo leer una pregunta integradora	225
Ejemplo resuelto: softmax estable en síntesis	226
Ejemplo resuelto: evidencia no supervisada dentro de una decisión	226
Ejemplo resuelto: respuesta integradora	227
Caso longitudinal: respuesta integradora	227
Verificaciones seleccionadas	227
Ejemplo resuelto: matriz de decisión integradora	228
Mini-rúbrica para una respuesta integradora	228
Para explorar más	229
Revisión del capítulo	229
Términos clave	229
Ejercicios	229
Reproducibilidad y tracking	231
Pregunta aplicada	231
Qué significa reproducir	231
Objetivos del capítulo	232
El paquete mínimo reproducible	232
Semillas	232
Manifest de experimento	232
Ejemplo resuelto: manifest mínimo	233
Ejemplo resuelto: resultado no reproducible	233
Ejemplo resuelto: auditar un manifest	233
Tracking de experimentos	234
Persistencia de modelos	234
Hash de artefactos	234
Checklist de reproducibilidad	235
Punto de control	235
Caso longitudinal: manifest del clasificador	235
Verificaciones seleccionadas	236
Ejemplo resuelto: manifest mínimo en JSON	236
Qué debe poder rerunearse	236
Para explorar más	237
Revisión del capítulo	237
Términos clave	237
Ejercicios	237
Comunicación técnica	239
Pregunta aplicada	239
Tesis de un reporte	239
Objetivos del capítulo	240
Estructura recomendada	241

Figuras útiles	241
Lenguaje de decisión	242
Ejemplo resuelto: de bitácora a argumento	242
Ejemplo resuelto: mejorar una conclusión	242
Ejemplo resuelto: mejora absoluta y relativa	243
Respuesta corta defendible	243
Caso longitudinal: conclusión del clasificador	243
Verificaciones seleccionadas	244
Ejemplo resuelto: tabla de evidencia contra afirmaciones	244
Plantilla de resultados defendible	244
De resultado a recomendación	244
Para explorar más	245
Revisión del capítulo	245
Términos clave	245
Ejercicios	246
Entrega final y defensa	247
Pregunta aplicada	247
Paquete final	247
Objetivos del capítulo	247
Estructura del repositorio de proyecto	247
Criterio de paquete ejecutable	248
Ejemplo resuelto: contribución individual	248
Rúbrica del proyecto final	249
Ejemplo resuelto: cálculo de nota ponderada	249
Defensa oral individual	249
Rúbrica de defensa 0/1/2	251
Caso longitudinal: defensa del clasificador	251
Verificaciones seleccionadas	251
Ejemplo resuelto: pregunta difícil	252
Checklist individual antes de entrar a defensa	252
Carga docente mínima	252
Evidencia individual y trazabilidad	253
Para explorar más	253
Revisión del capítulo	253
Términos clave	253
Ejercicios	254
Ejercicios integradores: reproducibilidad y defensa	255
Resultados de práctica	255
Mapa del banco	255
Criterio de calidad	255
Ejercicio 1: Softmax estable	256
Ejercicio 2: Diagnóstico	256
Ejercicio 3: PCA y reconstrucción	256
Ejercicio 4: Reproducibilidad	256
Ejercicio 5: Defensa	257
Respuestas seleccionadas	257

Herramientas de estudio	258
Guía de estudio del estudiante	259
Cómo se conectan las páginas del curso	259
Ruta mínima por semana	259
Cómo leer un capítulo técnico	260
Señales de dominio	260
Uso de respuestas seleccionadas	261
Preparación para parciales	261
Preparación para el proyecto	261
Cuando te bloqueas	262
Checklist antes de publicar una entrega	262
Formatos y descargas	263
Lectura principal	263
Guías de inicio	263
Datos de práctica	264
Trazabilidad de publicación	264
Estado de uso	265
Qué no se descarga desde el libro	265
Datos y recursos reproducibles	266
Paquete descargable del libro	266
Datasets incluidos en scikit-learn	267
Datos generados para práctica	267
Regla de reproducibilidad	267
Advertencias de interpretación	268
Checklist para datasets de proyecto	268
Casos longitudinales del libro	269
Caso A: regresión sintética con escala y regularización	269
Caso B: clasificación binaria con slice reciente	270
Caso C: estructura no supervisada	271
Cómo usar los casos al estudiar	271
Atlas visual de modelos y decisiones	272
Manifiesto editorial	272
Regresión lineal	272
Regularización	273
Descenso del gradiente	274
Validación y curvas de aprendizaje	275
Particiones y evaluación	276
Regresión logística y umbrales	277
Métricas para decidir	277
Error analysis	278
Backpropagation	279
Optimizadores	280
PCA y SVD	281

K-Means	281
Recomendación latente	282
Reproducibilidad	282
Comunicación y defensa	283
Resultados de aprendizaje y alineación	285
Resultados de aprendizaje globales	285
Alineación por parte del libro	285
Alineación con evaluación	286
Criterios de evidencia	286
Progresión de autonomía	287
Cómo usar esta página	287
Mapa de objetivos de aprendizaje	288
Cómo leer los códigos	288
Cobertura por parte	288
Mapa por capítulo principal	289
Uso para estudiantes	291
Uso para instructores	292
Criterio de mantenimiento	292
Mapa de cierres de capítulo	293
Rasgos de cierre	293
Mapa por capítulo principal	293
Bancos integradores	295
Relación con respuestas seleccionadas	295
Criterio editorial	295
Apéndices de consulta	297
Glosario y notación	298
Términos clave	298
Notación frecuente	299
Convención de formas	300
Índice temático	301
Cómo usar este índice	301
Modelación y representación	301
Álgebra lineal y optimización	302
Regresión, regularización y clasificación	302
Validación, métricas y diagnóstico	303
Redes neuronales	303
Aprendizaje no supervisado y recomendación	303
Reproducibilidad y comunicación	304
Recursos de consulta rápida	304
Criterio de mantenimiento	304

Índice de algoritmos, datasets y funciones	306
Cómo usar este apéndice	306
Mapa por primer uso	306
Referencia de API por capítulo	307
Algoritmos principales	308
Datasets y generadores	309
Funciones NumPy frecuentes	310
Herramientas scikit-learn frecuentes	311
Métricas y diagnósticos	311
PyTorch mínimo del curso	312
Regla de lectura rápida	312
Taxonomía de ejercicios y banco de preguntas	313
Propósito	313
Tipos de ejercicio	313
Niveles cognitivos	313
Evidencia esperada	314
Banco público por módulo	314
Criterios de calibración	315
Uso para estudiantes	316
Uso para instructores	316
Señales de mala pregunta	316
Mantenimiento y revisión	317
Prerrequisitos matemáticos mínimos	318
Cómo usar este apéndice	318
Álgebra lineal mínima	318
Normas, distancias y pérdida cuadrática	319
Cálculo y gradientes mínimos	319
Probabilidad mínima	320
Notación NumPy y PyTorch	320
Checklist de formas	321
Ejemplo resuelto: gradiente de MSE en una línea	321
Respuestas rápidas de autoverificación	322
Formularios y respuestas seleccionadas	323
Cómo usar esta parte	323
Mapa de apéndices	323
Política de respuestas	323
Apéndice Módulo 1: formulario y respuestas seleccionadas	325
Cómo usar este apéndice	326
Notación mínima	326
Formulario mínimo	326
Patrones de código	329
Checklist antes de entregar	330
Respuestas seleccionadas	330

Guía de estudio para el Parcial 1	332
Apéndice Módulo 2: formulario y respuestas seleccionadas	333
Cómo usar este apéndice	334
Notación mínima	334
Fórmulas clave de forward	334
Fórmulas clave de backward	335
Diagnóstico de entrenamiento	336
Patrones de código	336
Checklist antes de entregar	336
Respuestas seleccionadas	337
Apéndice Módulo 3: formulario y respuestas seleccionadas	339
Cómo usar este apéndice	340
Notación mínima	340
Métricas de clasificación binaria	340
Validación cruzada	341
Reglas operativas	341
Checklist de evaluación	341
Patrones de decisión	341
Patrones de código	342
Respuestas seleccionadas	342
Apéndice Módulo 4: formulario y respuestas seleccionadas	344
Cómo usar este apéndice	345
Notación mínima	345
SVD	345
PCA	346
K-Means	346
Silhouette	346
Cosine similarity	346
Checklist de interpretación	347
Patrones de código	347
Respuestas seleccionadas	347
Apéndice Módulo 5: formulario y respuestas seleccionadas	350
Cómo usar este apéndice	351
Notación mínima	351
Fórmulas clave	351
Checklist de paquete reproducible	352
Estructura mínima de manifest	352
Lenguaje de defensa	353
Respuestas seleccionadas	353

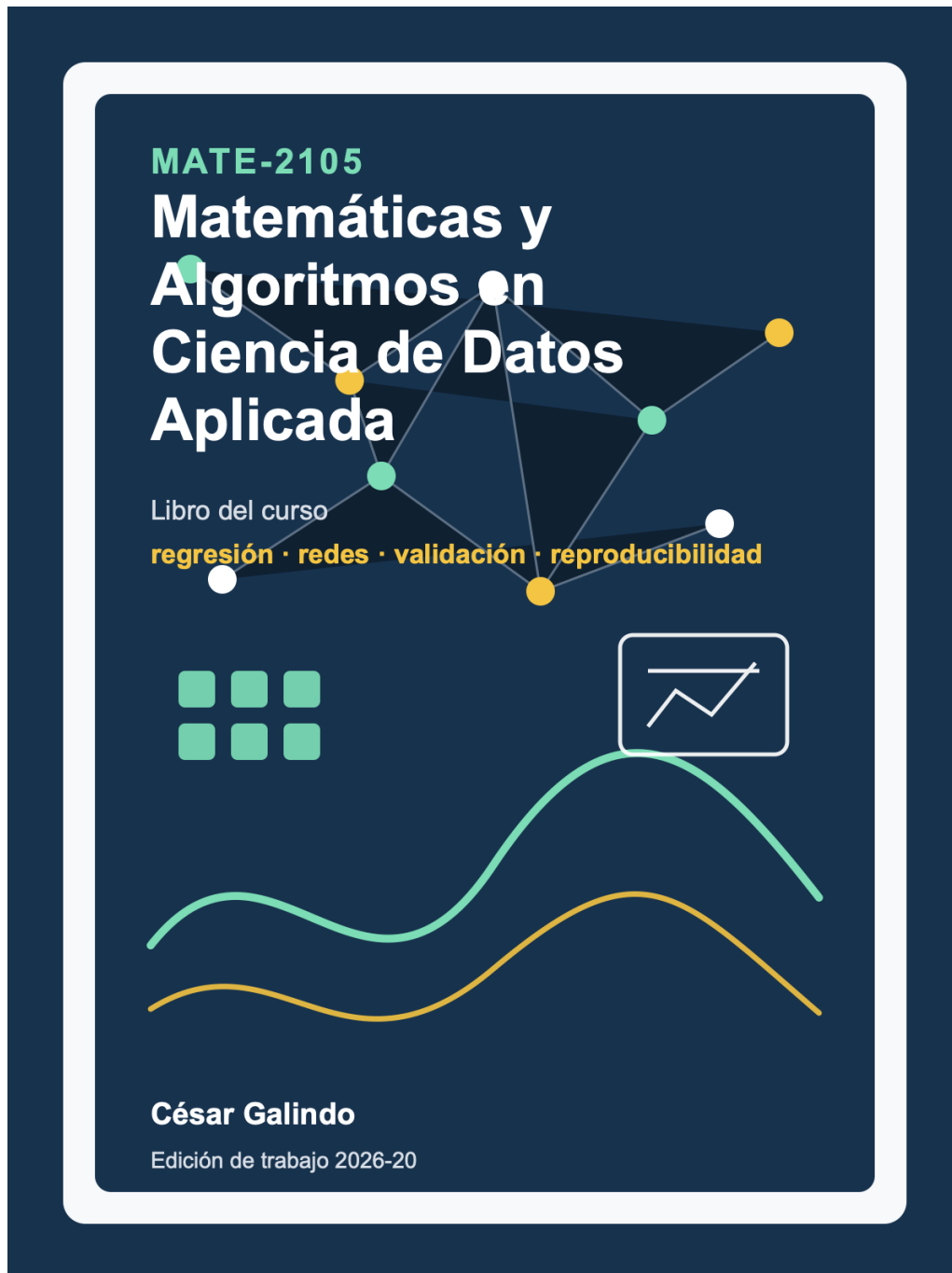
Información editorial y publicación	356
Información editorial	357
Audiencia	357
Prerrequisitos recomendados	357
Alcance	357
Formato recomendado	358
Cómo citar	358
Accesibilidad	358
Errata y mejoras	359
Separación de materiales	359
Accesibilidad	360
Objetivo de accesibilidad	360
Alcance evaluado	360
Evidencia automática	361
Revisión manual requerida	361
Matemática, código y figuras	361
Barreras conocidas	362
Reporte de barreras	362
Plantillas de revisión	363
Criterio de cierre	363
Equipo, revisión y reconocimientos	364
Autoría y responsabilidad editorial	364
Estado de revisión	364
Cómo reportar una corrección	365
Reconocimientos	365
Criterio para versión publicable	365
Protocolo de revisión externa	367
Propósito de la revisión	367
Roles de revisión	367
Evidencia que recibe el revisor	368
Plantillas de revisión	368
Rúbrica matemática	368
Rúbrica pedagógica	369
Rúbrica computacional	369
Dictamen de revisión	369
Registro de revisiones externas	370
Criterio de cierre	370
Matriz de preparación editorial	371
Niveles de preparación	371
Matriz por partes	372
Matriz por capítulo principal	372
Evidencia mínima por nueva edición	374
Señales de riesgo editorial	375

Características pedagógicas	376
Estructura recurrente	376
Figuras guía	377
Tipos de recursos	377
Estándar de ejercicios	378
Relación con evaluación	378
Estándar de revisión	379
Recursos para estudiantes e instructores	380
Recursos públicos para estudiantes	380
Guías públicas de inicio	380
Recursos restringidos para instructores	381
Cómo estudiar con los recursos	382
Cómo usar los recursos como instructor	382
Flujo de publicación	382
Política de respuestas	383
Guía pública de adopción docente	384
Sistema de sincronización del curso	384
Rutina mínima del profesor	384
Usos posibles del libro	385
Diseño semanal recomendado	385
Separación entre recursos públicos y privados	386
Evaluación con baja carga manual	386
Alineación mínima antes de dictar	386
Adaptación a otro contexto	387
Revisión de calidad antes de publicar	387
Señales de una adopción saludable	387
Referencias y atribuciones	388
Referencia editorial	388
Referencias matemáticas y algorítmicas	388
Documentación técnica	389
Evaluación automatizada	389
Datasets y generadores	389
Figuras	390
Cómo citar este libro	390
Integridad académica y uso de IA	391
Principio central	391
Uso permitido	391
Uso que requiere declaración	391
Uso no permitido	392
Evidencia de trabajo propio	392
Defensa oral	392
Regla práctica antes de entregar	393

Respuestas, solucionarios y recursos restringidos	394
Principio editorial	394
Tipos de recurso	394
Qué puede aparecer en respuestas públicas	395
Qué no debe publicarse	395
Política por tipo de pregunta	395
Respuestas de proyectos	396
Proceso para liberar una respuesta	396
Plantillas de control	396
Criterio de auditoría	397
Errata, revisión y control de calidad	398
Qué cuenta como errata	398
Severidad	398
Flujo de corrección	398
Plantillas de errata	399
Evidencia de cierre	399
Registro público	399
Separación de errata pública y docente	400
Registro público de errata	401
Estado actual	401
Estados de un reporte	401
Errata abierta	401
Errata cerrada	402
Cambios editoriales de edición	402
Criterio de publicación de errata	402
Reportes privados	402
Cómo reportar	403
Criterio de cierre	403
Edición, versión y publicación	404
Ficha de edición	404
Criterio de versión estable	404
Canales de publicación	405
Checklist de publicación	406
Registro de edición	406
Registros de candidato	406
Paquete de publicación	408
Artefactos públicos	408
Manifiesto de publicación	409
Comandos de construcción	410
Comandos de validación	410
Criterio de aceptación	411
Guía de impresión y PDF	412
Relación entre web, PDF e impresión	412

Especificación de PDF	412
Revisión visual obligatoria	413
Prueba de impresión	413
Plantillas de revisión	414
Estado de impresión de esta edición	414
Criterio de aceptación para impresión	414
Licencia y uso del material	415
Estado de licencia	415
Uso permitido en el curso	415
Uso no permitido sin autorización	415
Cita sugerida	416
Materiales de terceros	416
Integridad académica	416
Decisión de licencia abierta	417
Por qué esta decisión importa	417
Opciones razonables	417
Separación por tipo de material	418
Criterios de decisión	418
Registro de decisión pendiente	419
Procedimiento de cierre	419

Prefacio



Este libro acompaña el curso **MATE-2105 Matemáticas y Algoritmos en Ciencia de Datos**

Aplicada. Su propósito es organizar el conocimiento del curso como un texto continuo: definiciones, derivaciones, ejemplos resueltos, ejercicios, respuestas seleccionadas y apéndices.

Autor: César Galindo, profesor del Departamento de Matemáticas de la Universidad de los Andes.

La página del curso responde preguntas operativas:

- qué se hace esta semana;
- qué laboratorio se entrega;
- qué notebook se usa en clase;
- qué fecha tiene cada hito;
- cómo configurar el entorno.

Este libro responde una pregunta distinta:

¿qué ideas matemáticas y algorítmicas debe dominar un estudiante para tomar decisiones técnicas defendibles con datos?

Comenzar la lectura

Si estás estudiando el curso, no necesitas leer primero las páginas editoriales de publicación. La ruta natural es:

Momento	Abre
Primera lectura	Cómo usar este libro
Mapa semanal	Mapa del curso y del libro
Inicio conceptual	Capítulo 0: Ciencia de datos aplicada
Puente probabilístico	Interludio 0.5: Incertidumbre y datos observados
Repaso de base	Prerrequisitos matemáticos mínimos

Las páginas de accesibilidad, revisión, licencia, errata y publicación quedan al final del libro como información editorial, no como lectura inicial del curso.

Tesis pedagógica

El curso no trata los algoritmos como recetas. Cada tema sigue una secuencia:

`pregunta aplicada -> modelo matemático -> algoritmo -> implementación -> validación -> decisión`

El estudiante debe poder explicar qué se optimiza, cómo se calcula, cómo se evalúa y qué limitaciones tiene la conclusión.

Relación con el portal del curso

El portal del curso sigue siendo el punto de entrada para clases, entregas, laboratorios y proyecto:

<https://cesarneyit-code.github.io/mate2105/>

Este libro es la lectura de fondo. Cada capítulo corresponde a una o más semanas del curso, pero está escrito para poder estudiarse como texto completo.

Regla de uso: si necesitas saber qué entregar, usa el portal. Si necesitas entender y estudiar, usa este libro.

Fuente canónica

Los capítulos de este libro se ensamblan desde los capítulos públicos de cada módulo. Eso evita mantener dos versiones paralelas del mismo contenido. Cuando un capítulo del módulo mejora, el libro completo hereda esa mejora al renderizarse de nuevo.

Cómo usar este libro

Este libro está pensado para tres usos distintos.

Antes de clase

Lee el capítulo asignado con una meta concreta:

1. identificar la pregunta aplicada;
2. escribir las definiciones centrales;
3. reproducir al menos un ejemplo resuelto;
4. marcar dudas matemáticas o computacionales.

No es necesario dominar todo antes de clase. Sí es necesario llegar con el vocabulario mínimo.

Si el capítulo usa un caso longitudinal, abre también la página [Casos longitudinales del libro](#). Allí puedes ver qué archivo de datos se está reutilizando, qué decisión está en juego y cómo se conecta con capítulos anteriores o posteriores.

Si una derivación usa notación que no recuerdas, consulta primero [Prerrequisitos matemáticos mínimos](#). Ese apéndice no reemplaza un curso de álgebra lineal, cálculo o probabilidad; sirve como puente operativo para seguir el libro.

Después de clase

Vuelve al capítulo y trabaja los puntos de control. Si puedes responderlos sin mirar la solución, estás listo para pasar al laboratorio o al notebook de práctica.

Antes de una evaluación

Usa los ejercicios integradores y los apéndices. La preparación no debe ser memorizar fórmulas aisladas, sino practicar esta estructura:

```
evidencia -> diagnóstico -> acción -> límite
```

Qué queda fuera del libro

El libro no reemplaza:

- los notebooks de clase;
- los laboratorios autocalificados;
- las rúbricas de entrega;
- las guías docentes;
- los autograders privados.

Esos materiales viven en el portal o en el repositorio docente privado, según corresponda.

Mapa del curso y del libro

La siguiente tabla conecta el calendario del curso con el libro. El portal organiza el trabajo semanal; el libro organiza las ideas.

Semana	Curso	Capítulos del libro
1	Arranque, lenguaje común, incertidumbre mínima y flujo aplicado	Capítulo 0 e Interludio 0.5
2	Regresión lineal múltiple y ecuación normal	Capítulo 1
3	Descenso del gradiente y escalamiento	Capítulo 2
4	Validación, complejidad y overfitting	Capítulo 3
5	Regularización L1/L2	Capítulo 3
6	Regresión logística, métricas y Parcial 1	Capítulos 4 y 5
7	Perceptron, capas densas y forward propagation	Capítulo 6
8	Backpropagation	Capítulo 7
9	Diagnóstico, inicialización y regularización en redes	Capítulo 8
10	Optimizadores y PyTorch	Capítulos 9 y 10
11	Estrategia ML, métricas, validación y análisis de errores	Capítulos 11-14
12	SVD y PCA	Capítulo 15
13	K-Means y recomendación básica	Capítulos 16-18
14	Síntesis conceptual, Parcial 2 y reproducibilidad	Capítulos 19-20
15	Comunicación técnica y taller de proyecto final	Capítulo 21
16	Entrega final y defensa	Capítulos 22-23

En el portal, cada semana enlaza directamente a su capítulo. En este libro, puedes leer de forma lineal o saltar por partes.

Para estudiar de forma transversal, usa también [Casos longitudinales del libro](#) y [Prerrequisitos matemáticos mínimos](#). El primer recurso conecta datasets y decisiones entre capítulos; el segundo

resume la notación mínima de álgebra lineal, cálculo, probabilidad y NumPy/PyTorch que se usa durante el curso.

Parte I: Fundamentos

Parte I: Fundamentos

La primera parte fija el lenguaje común del curso: observaciones, variables, datasets, tareas de aprendizaje, pérdida, métrica, baseline, partición de datos e incertidumbre mínima para modelar.

El objetivo es que todos los estudiantes compartan un mismo vocabulario antes de entrar a regresión, optimización y modelos más complejos.

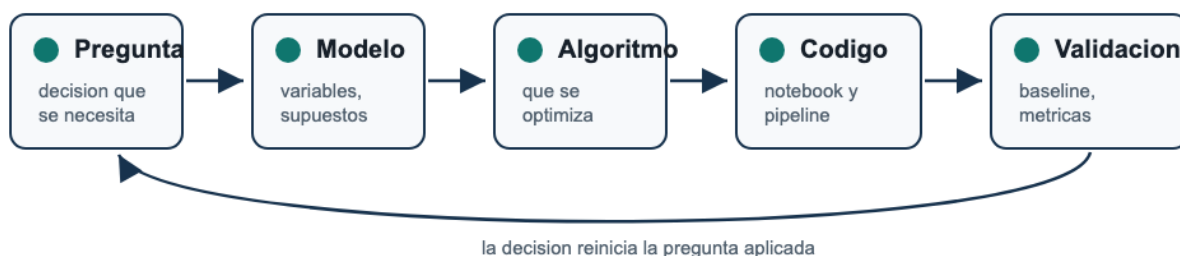


Figura 1: Ciclo de modelación aplicada del curso: una pregunta aplicada se traduce en modelo matemático, algoritmo, código, validación y decisión.

Lectura de la figura. El curso no empieza por algoritmos aislados. Empieza por una pregunta que debe convertirse en objetos matemáticos, implementación, validación y decisión defendible.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. distinguir observación, variable, dataset, modelo, pérdida, métrica y baseline;
2. explicar por qué train, dev y test cumplen funciones distintas;
3. reconocer una tarea de regresión, clasificación o análisis no supervisado;
4. leer una matriz de datos como objeto matemático y computacional;
5. distinguir datos observados de variables aleatorias;
6. interpretar ruido, distribución empírica, probabilidad condicional y pérdida empírica;
7. preparar el vocabulario mínimo para estudiar regresión, optimización, validación y clasificación.

Mapa de la parte

Capítulo	Pregunta guía	Producto de estudio
Lenguaje común	¿Qué objetos aparecen en todos los modelos del curso?	Glosario operativo y checklist inicial.
Incertidumbre y datos observados	¿Qué mínimo de probabilidad necesitamos para modelar sin tomar otro curso?	Puente entre tablas, variables aleatorias, ruido, pérdida empírica y generalización.

Criterio de salida

Antes de pasar a la Parte II, deberías poder tomar un problema aplicado sencillo y escribir: unidad de observación, variables disponibles, salida deseada, variable aleatoria objetivo, fuente de ruido, métrica principal, baseline y regla de partición.

Capítulo 0: Ciencia de datos aplicada

Datos, modelos, métricas y decisiones

Vista previa

Este capítulo introduce el lenguaje básico que usaremos durante el curso. Su función es parecida a la de un capítulo inicial de un libro de cálculo: no pretende resolver todos los problemas del semestre, sino fijar los objetos, la notación, los ejemplos y los hábitos de pensamiento que se repetirán una y otra vez.

En cálculo, antes de derivar funciones complicadas, uno aprende qué es una función, qué significa una gráfica, qué es una tasa de cambio y qué información entrega una aproximación. En ciencia de datos aplicada ocurre algo similar: antes de estudiar regresión, redes neuronales o PCA, necesitamos entender qué es un dataset, qué es un modelo, qué significa entrenar, cómo se mide el desempeño y por qué una métrica no basta para tomar una decisión.

Al terminar este capítulo deberías poder:

1. Distinguir observaciones, variables, entradas y objetivos.
2. Representar un dataset supervisado mediante una matriz $\mathbf{X}$ y un vector $\mathbf{y}$.
3. Explicar qué es un modelo y qué papel juegan sus parámetros.
4. Diferenciar pérdida, métrica y baseline.
5. Explicar por qué se separan datos de entrenamiento y prueba.
6. Escribir una conclusión técnica breve sin exagerar lo que muestran los datos.

Idea central

La ciencia de datos aplicada no empieza con un algoritmo. Empieza con una pregunta, una fuente de datos y una decisión que se quiere informar.

0.1 Observaciones, variables y datasets

Un **dataset** es una colección organizada de datos. En el formato más común para este curso, un dataset se puede imaginar como una tabla.

- Cada fila representa una **observación**.
- Cada columna representa una **variable**.
- Algunas variables se usan como entradas.
- Una variable, cuando existe, se usa como objetivo.

Definición 0.1: Observacion.

Una observación es una unidad individual sobre la que se registran datos. Puede ser un paciente, una vivienda, una imagen, una transacción, un estudiante, una molecula o un texto.

Definición 0.2: Variable.

Una variable es una cantidad, categoría o atributo registrado para cada observación. Puede ser numérica, categorica, ordinal, binaria, temporal o textual.

Ejemplo 0.1: una tabla pequeña

Supongamos que queremos estudiar viviendas. Un dataset pequeño podría tener esta forma:

vivienda	area_m2	habitaciones	antigüedad	precio_millones
1	62	2	12	310
2	85	3	5	455
3	47	1	20	240
4	110	4	8	620

Si la pregunta es predecir `precio_millones`, entonces:

- una observación es una vivienda;
- las variables de entrada podrían ser `area_m2`, `habitaciones` y `antigüedad`;
- la variable objetivo sería `precio_millones`.

En notación matricial escribimos:

$$X = \begin{bmatrix} 62 & 2 & 12 \\ 85 & 3 & 5 \\ 47 & 1 & 20 \\ 110 & 4 & 8 \end{bmatrix}, \quad y = \begin{bmatrix} 310 \\ 455 \\ 240 \\ 620 \end{bmatrix}.$$

La matriz $\mathbf{X}$ contiene las variables de entrada. El vector $\mathbf{y}$ contiene el objetivo.

 Lectura de la notación

Si $\mathbf{X}$ tiene n filas y d columnas, diremos que hay n observaciones y d variables de entrada. En símbolos, $\mathbf{X}$ tiene forma $n \times d$.

Punto de control 0.1

En el ejemplo de viviendas:

1. Cuántas observaciones hay?
2. Cuántas variables de entrada hay?
- 3.Cuál es la forma de $\mathbf{X}$?
- 4.Cuál es la forma de $\mathbf{y}$?

Respuesta esperada: hay 4 observaciones, 3 variables de entrada, $\mathbf{X}$ tiene forma 4×3 y $\mathbf{y}$ tiene longitud 4.

0.2 Preguntas aplicadas y tareas de aprendizaje

Una pregunta aplicada debe ser más concreta que “quiero analizar datos”. Debe decir que se quiere saber o decidir.

Comparemos:

Formulación débil	Formulación más útil
Quiero usar datos de salud	Quiero estimar riesgo de reingreso hospitalario en 30 días
Quiero hacer redes neuronales	Quiero clasificar imágenes de defectos en piezas manufacturadas
Quiero analizar precios	Quiero predecir precio de arriendo a partir de ubicación y características
Quiero hacer clustering	Quiero segmentar usuarios para entender patrones de uso

Definición 0.3: Pregunta aplicada.

Una pregunta aplicada es una pregunta formulada de manera que conecta datos disponibles con una decisión, explicación, predicción o clasificación posible.

Tipos de tareas

En este curso aparecerán cuatro familias de tareas.

Regresión

La variable objetivo es numérica.

Ejemplos:

- precio de una vivienda;
- demanda de energía;
- tiempo de entrega;
- concentración de una sustancia;
- progresión de una enfermedad medida en una escala continua.

Clasificación

La variable objetivo es una clase o categoría.

Ejemplos:

- fraude/no fraude;
- tumor benigno/maligno;
- tipo de vino;
- aprobado/no aprobado;
- categoría de documento.

Aprendizaje no supervisado

No hay una variable objetivo directa. Buscamos estructura en los datos.

Ejemplos:

- segmentar usuarios;
- reducir dimensión;
- encontrar grupos de observaciones parecidas;
- visualizar datos de alta dimensión.

Recomendación

Buscamos sugerir ítems a usuarios o completar información faltante en una matriz de interacciones.

Ejemplos:

- recomendar películas;
- sugerir productos;
- priorizar lecturas;
- ordenar contenidos.

Advertencia

El nombre del algoritmo no define el problema. Primero se formula la pregunta; después se decide si hace falta regresión, clasificación, clustering, PCA, recomendación u otra herramienta.

0.3 Modelos, parámetros y predicciones

Un **modelo** es una familia de funciones que transforma entradas en predicciones.

Si una observación tiene variables de entrada

$$x = (x_1, x_2, \dots, x_d),$$

entonces un modelo produce una predicción:

$$\hat{y} = f_{\theta}(x).$$

Aquí:

- x es la entrada;
- y es el valor real;
- $\hat{y}$ es la predicción;
- f_{θ} es el modelo;
- θ representa parámetros ajustables.

Definición 0.4: Parametros.

Los parámetros son cantidades internas del modelo que se ajustan usando datos. En una regresión lineal, los parámetros son los coeficientes. En una red neuronal, son los pesos y sesgos.

Ejemplo 0.2: una predicción lineal

Supongamos que usamos el modelo:

$$\hat{y} = 40 + 4x_1 + 30x_2 - 2x_3,$$

donde:

- x_1 es el área en metros cuadrados;
- x_2 es el número de habitaciones;
- x_3 es la antigüedad en años;
- $\hat{y}$ es el precio predicho en millones.

Para una vivienda con:

$$x_1 = 80, \quad x_2 = 3, \quad x_3 = 10,$$

la predicción es:

$$\hat{y} = 40 + 4(80) + 30(3) - 2(10) = 40 + 320 + 90 - 20 = 430.$$

El modelo predice 430 millones.

Ejemplo resuelto 0.2.

Si el precio real fuera 455 millones, el error de predicción sería:

$$y - \hat{y} = 455 - 430 = 25.$$

El modelo subestima el precio por 25 millones.

Punto de control 0.2

Usa el mismo modelo:

$$\hat{y} = 40 + 4x_1 + 30x_2 - 2x_3.$$

Calcula la predicción para una vivienda de 60 m2, 2 habitaciones y 15 años de antigüedad.

Respuesta:

$$\hat{y} = 40 + 4(60) + 30(2) - 2(15) = 310.$$

0.4 Pérdida, métrica y baseline

Una vez un modelo produce predicciones, necesitamos evaluar si son buenas. Aquí aparecen tres ideas que suelen confundirse: pérdida, métrica y baseline.

Definición 0.5: Pérdida.

Una pérdida mide qué tan mala es una predicción para una observación o un conjunto de observaciones. Durante el entrenamiento, muchos algoritmos ajustan parámetros minimizando una pérdida.

Definición 0.6: Métrica.

Una métrica resume el desempeño del modelo en una forma útil para comparar, reportar o tomar decisiones.

Definición 0.7: Baseline.

Un baseline es una regla simple que sirve como punto mínimo de comparación. Un modelo complejo debe justificar su complejidad superando un baseline razonable.

Ejemplo 0.3: baseline de regresión

Supongamos que tenemos precios reales:

$$y = \begin{bmatrix} 310 \\ 455 \\ 240 \\ 620 \end{bmatrix}.$$

Un baseline sencillo predice siempre el promedio:

$$\bar{y} = \frac{310 + 455 + 240 + 620}{4} = 406.25.$$

Entonces el baseline predice:

$$\hat{y} = \begin{bmatrix} 406.25 \\ 406.25 \\ 406.25 \\ 406.25 \end{bmatrix}.$$

Este baseline no usa variables de entrada. Precisamente por eso es útil: si un modelo que usa área, habitaciones y antigüedad no supera al promedio, algo está mal o la información de entrada no es suficiente.

Ejemplo 0.4: baseline de clasificación

Supongamos un problema con 100 observaciones:

clase	cantidad
A	70
B	20
C	10

Un baseline que predice siempre la clase más frecuente, A, tendría accuracy:

$$\frac{70}{100} = 0.70.$$

Si un modelo obtiene accuracy 0.72, técnicamente supera el baseline, pero la mejora es pequeña. Habría que mirar:

- matriz de confusion;
- precision y recall por clase;
- estabilidad con otra partición;
- costo de los errores;
- tamaño del dataset.

! Principio

Nunca reportes una métrica sin decir contra qué baseline se compara.

Accuracy

En clasificación, la accuracy mide la proporción de predicciones correctas:

$$\text{accuracy} = \frac{\text{número de predicciones correctas}}{\text{número total de predicciones}}.$$

La accuracy es fácil de entender, pero puede ser engañosa si las clases están desbalanceadas.

Error absoluto medio

En regresión, una métrica común es el error absoluto medio:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

El MAE se interpreta en las mismas unidades de la variable objetivo.

Error cuadrático medio

Otra métrica común es el error cuadrático medio:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

El MSE penaliza más los errores grandes.

0.5 Entrenamiento, prueba y generalización

Un modelo no se evalúa seriamente con los mismos datos que usó para aprender. Si hacemos eso, podemos medir memoria en lugar de generalización.

La separacion más básica es:

```
datos disponibles -> datos de entrenamiento
                  -> datos de prueba
```

El modelo ajusta sus parámetros con los datos de entrenamiento. Luego se evalúa en los datos de prueba.

Definición 0.8: Generalización.

Generalización es la capacidad de un modelo de funcionar razonablemente bien en datos no usados para entrenarlo.

Ejemplo 0.5: una mala evaluación

Un estudiante entrena un modelo con 200 observaciones y reporta accuracy 0.98 en esas mismas 200 observaciones. Esa información no basta para afirmar que el modelo sirve.

Falta saber:

- si se evaluo en datos separados;
- si hay leakage;
- si las clases están balanceadas;
- cuál era el baseline;
- que errores comete;
- si el resultado se mantiene con otra partición.

Error común

Un modelo puede verse excelente en entrenamiento y fallar en datos nuevos. Esto se llama sobreajuste u overfitting.

0.6 Primer flujo aplicado en Python

En la Semana 1 usaremos el dataset `load_wine` de `scikit-learn`. El objetivo es clasificar vinos a partir de mediciones químicas.

El siguiente código muestra un flujo completo. En este capítulo el código se presenta como lectura; el laboratorio lo ejecuta paso a paso.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_wine(as_frame=True)
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=2105,
    stratify=y
)

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)
baseline_acc = accuracy_score(y_test, baseline_pred)

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)
model.fit(X_train, y_train)
model_pred = model.predict(X_test)
model_acc = accuracy_score(y_test, model_pred)
```

```
print("Baseline:", baseline_acc)
print("Modelo:", model_acc)
```

Cómo leer este código

No necesitas dominar todos los detalles todavía. Pero si debes identificar las piezas conceptuales:

Linea conceptual	Codigo
Cargar datos	<code>load_wine(as_frame=True)</code>
Separar entradas y objetivo	<code>X = data.data, y = data.target</code>
Crear train/test	<code>train_test_split(...)</code>
Crear baseline	<code>DummyClassifier(...)</code>
Crear modelo	<code>LogisticRegression(...)</code>
Medir desempeño	<code>accuracy_score(...)</code>

i Por qué aparece **StandardScaler**

Muchas técnicas de machine learning son sensibles a la escala de las variables. En la Semana 2 veremos una razón matemática de fondo: la geometría del problema y el condicionamiento pueden afectar la optimización.

0.7 De número a conclusión técnica

Una conclusión técnica no es simplemente:

El modelo dio 0.91.

Una conclusión técnica debe incluir contexto:

1. Qué se intentó predecir o clasificar.
2. Qué datos se usaron.
3. Contra que baseline se comparo.
4. Qué métrica se reporto.
5. Qué limitacion principal tiene el resultado.

Plantilla de conclusión

Una buena primera plantilla es:

En este experimento, el modelo _____ al baseline usando la métrica _____. La evaluación se hizo sobre _____. Esta conclusión es limitada porque _____.

Ejemplo 0.6: conclusión débil y conclusión mejor

Conclusion débil:

El modelo es muy bueno porque saco 0.96.

Problemas:

- no dice que mide 0.96;
- no menciona baseline;
- no dice con qué datos;
- no reconoce limitaciones.

Conclusion mejor:

En este experimento inicial, la regresión logística supera al baseline de clase mayoritaria usando accuracy en una partición de prueba estratificada. Sin embargo, la conclusión es limitada porque el dataset es pequeño y limpio; antes de usar el modelo en una decisión real habría que revisar errores por clase y validar con datos externos.

0.8 Errores comunes al empezar

Error 1: empezar por el algoritmo

Mala pregunta:

Qué puedo hacer con redes neuronales?

Mejor pregunta:

Qué decisión quiero informar y que datos tengo para hacerlo?

Error 2: confundir pérdida con métrica

Una función puede servir para entrenar, pero no ser la mejor forma de comunicar resultados. Por ejemplo, un modelo puede entrenarse con log-loss y reportarse con F1 o ROC-AUC según el problema.

Error 3: no tener baseline

Sin baseline, una métrica queda flotando. Accuracy 0.80 puede ser buena si el baseline es 0.50, pero mala si el baseline es 0.79 y el modelo es mucho más costoso.

Error 4: evaluar con datos de entrenamiento

Esto infla el desempeño. Separar entrenamiento y prueba no es una formalidad; es una defensa mínima contra autoengano.

Error 5: prometer más de lo que los datos muestran

Un modelo puede mostrar asociación predictiva sin demostrar causalidad. Puede funcionar en un dataset y fallar en otra población. Puede ser útil para priorizar revisión humana, pero no para automatizar una decisión sensible.

Resumen del capítulo

1. Un dataset tabular se organiza en observaciones y variables.
2. En aprendizaje supervisado distinguimos entradas $\mathbf{X}$ y objetivo y .
3. Un modelo transforma entradas en predicciones.
4. Los parámetros se ajustan usando datos.
5. Una pérdida ayuda a entrenar; una métrica ayuda a evaluar y comunicar.
6. Un baseline es una comparación mínima indispensable.
7. Separar entrenamiento y prueba ayuda a estimar generalización.
8. Una conclusión técnica debe mencionar métrica, baseline, datos y limitaciones.

Terminos clave

- accuracy;
- baseline;
- clasificación;
- dataset;
- entrada;
- generalización;
- matriz de datos;
- métrica;
- modelo;
- observación;
- parámetro;
- pérdida;
- regresión;
- train/test;
- variable objetivo.

Ejercicios

Conceptos

1. En tus propias palabras, explica la diferencia entre una pregunta aplicada y un algoritmo.
2. Da un ejemplo de observación y variable en un dataset de transporte.
3. Qué diferencia hay entre y y $\hat{y}$?
4. Por qué un baseline de clase mayoritaria puede tener accuracy alta en un dataset desbalanceado?
5. ¿Por qué no basta evaluar un modelo en los datos de entrenamiento?

Calculo manual

6. Considera el modelo:

$$\hat{y} = 10 + 2x_1 - 3x_2.$$

Calcula la predicción para $x_1 = 8$ y $x_2 = 4$.

7. Los valores reales son:

$$y = (4, 7, 9, 10)$$

y las predicciones son:

$$\hat{y} = (5, 6, 8, 12).$$

Calcula el MAE.

8. En un problema de clasificación hay 50 observaciones de clase A, 30 de clase B y 20 de clase C. ¿Cuál es la accuracy del baseline que predice siempre la clase más frecuente?

Interpretacion

9. Un modelo obtiene accuracy 0.84. El baseline obtiene 0.82. Escribe dos razones por las cuales no deberiamos concluir automáticamente que el modelo es bueno.
10. Un modelo de regresión obtiene MAE de 12 unidades. Qué información adicional necesitas para interpretar si ese valor es pequeño o grande?
11. Escribe una conclusión técnica de tres frases para un modelo que supera el baseline, pero fue evaluado en un dataset pequeño.

Codigo

12. En el laboratorio, identifica en que línea se separan `X_train`, `X_test`, `y_train`, `y_test`.
13. Modifica el tamaño de prueba de 0.25 a 0.30. Qué cambia en las dimensiones?
14. Quita `stratify=y` y compara la distribución de clases en entrenamiento y prueba. Qué observas?

Proyecto

15. Propone una idea de proyecto. Especifica:
 - unidad de observación;
 - variable objetivo, si existe;
 - métrica inicial;
 - baseline razonable;
 - decisión que podría informar.
16. Para tu idea de proyecto, identifica una limitacion ética o técnica que debería aparecer en el reporte final.

Respuestas seleccionadas

6.

$$\hat{y} = 10 + 2(8) - 3(4) = 14.$$

7.

Los errores absolutos son:

$$|4 - 5| = 1, \quad |7 - 6| = 1, \quad |9 - 8| = 1, \quad |10 - 12| = 2.$$

Por tanto:

$$\text{MAE} = \frac{1 + 1 + 1 + 2}{4} = 1.25.$$

8.

La clase más frecuente es A, con 50 de 100 observaciones. El baseline tiene accuracy:

$$50/100 = 0.50.$$

9.

Dos razones posibles: la mejora sobre el baseline es pequeña y podría no ser estable; además, la accuracy puede ocultar errores graves en clases minoritarias.

10.

Hace falta conocer la escala de la variable objetivo, el baseline, el contexto de decisión y el costo práctico de un error de 12 unidades.

Interludio 0.5: Incertidumbre y datos observados

Variable aleatoria, muestra, ruido y generalización sin un curso de probabilidad

Pregunta aplicada

Cuando una tabla ya está en **pandas**, es fácil olvidar que cada número observado es solo una realización de un proceso más variable. Este interludio responde:

¿qué lenguaje mínimo de incertidumbre necesitamos para entender modelos, pérdidas, validación y probabilidades sin convertir el curso en probabilidad?

Objetivos del capítulo

Al terminar este interludio deberías poder:

1. distinguir una columna observada de una variable aleatoria;
2. explicar una observación como realización de un par (X, Y) ;
3. interpretar una distribución como patrón de variabilidad;
4. usar promedio y varianza como resúmenes empíricos de una variable;
5. leer $Y = f(X) + \varepsilon$ como modelo con ruido, no como igualdad exacta;
6. interpretar $P(Y = 1 \mid X = x)$ como probabilidad condicional;
7. distinguir pérdida empírica de desempeño esperado;
8. explicar por qué validamos en datos no usados para ajustar.

i Alcance

Este no es un capítulo de probabilidad formal. No estudia axiomas, conteo, distribuciones especiales ni pruebas largas. Es una capa de lenguaje para que regresión, clasificación y validación tengan sentido.

De tabla observada a variable incierta

En el Capítulo 0 dijimos que una columna de una tabla es una variable. Esa idea es correcta para describir datos observados. Pero al modelar necesitamos una distinción adicional.

Nivel	Pregunta	Ejemplo
Columna observada	¿qué valores tengo en esta tabla?	<code>precio_millones</code> en 400 viviendas medidas
Variable aleatoria	¿qué valor podría aparecer antes de observar una vivienda nueva?	Y , precio de una vivienda nueva tomada del mismo proceso

Definición: variable aleatoria.

Una variable aleatoria es una cantidad cuyo valor no conocemos antes de observar el caso. En este curso la usaremos como modelo conceptual de una columna antes de verla: precio, clase, tiempo, demanda, error o resultado de una medición.

La diferencia importa porque los modelos se usan en casos nuevos. Si ya conocemos y_i , no estamos prediciendo; estamos verificando. La predicción aparece cuando todavía no observamos el valor de Y .

Observación como realización

Un problema supervisado se puede escribir como pares observados:

$$(x_1, y_1), \dots, (x_n, y_n).$$

Cada par observado se interpreta como una realización concreta de un par incierto:

$$(X, Y).$$

Aquí:

- X representa las variables de entrada antes de observar un caso;
- Y representa el objetivo antes de observarlo;
- (x_i, y_i) es lo que quedó registrado en la fila i .

Ejemplo resuelto 0.5.1: vivienda observada y vivienda futura.

En una tabla de viviendas, una fila puede decir:

area_m2	habitaciones	antigüedad	precio_millones
85	3	5	455

La fila observada es (x_i, y_i) . Una vivienda futura con área, habitaciones y antigüedad conocidas tiene entrada x , pero su precio Y todavía es incierto. El modelo intenta producir una predicción o una distribución plausible para ese Y .

Distribución como patrón de variabilidad

Una **distribución** describe cómo se reparten los valores posibles de una variable aleatoria. En este curso la idea operativa es:

distribución = patrón de variabilidad.

No necesitamos empezar con fórmulas abstractas. Podemos mirar una columna:

```
y.describe()
y.hist()
```

Eso no revela la distribución verdadera del mundo, pero muestra la distribución empírica de los datos disponibles.

Definición: distribución empírica.

La distribución empírica es el patrón de valores observado en una muestra. Se resume con tablas, histogramas, proporciones, promedio, cuantiles y varianza.

Promedio, valor esperado y varianza

El promedio muestral de valores observados es:

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

El **valor esperado** es la versión idealizada de esa idea: el promedio que obtendríamos si pudiéramos repetir el proceso muchísimas veces bajo las mismas condiciones.

Definición: valor esperado.

El valor esperado de una variable aleatoria Y , escrito $E[Y]$, es su centro promedio ideal bajo el proceso que genera los datos. En la práctica lo aproximamos con promedios muestrales.

La varianza mide dispersión alrededor del promedio. En datos observados:

$$\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2.$$

La desviación estándar es la raíz de esa cantidad y queda en las mismas unidades que Y .

Ejemplo resuelto 0.5.2: baseline como promedio.

Si los precios observados son

310, 455, 240, 620,

el baseline que predice siempre el promedio usa

$$\bar{y} = \frac{310 + 455 + 240 + 620}{4} = 406.25.$$

Ese número no es “el precio verdadero” de una vivienda nueva. Es una regla simple que resume la información disponible antes de usar variables de entrada.

Ruido: por qué el modelo no debe acertar exactamente

En problemas reales, dos observaciones con entradas parecidas pueden tener salidas distintas. Una vivienda puede costar más por una negociación, una transacción puede parecer normal y ser fraude, una medición puede tener error.

Una forma mínima de escribir esto es:

$$Y = f(X) + \varepsilon.$$

Aquí:

- $f(X)$ es la parte sistemática que las variables disponibles pueden explicar;
- ε es ruido, variabilidad no explicada o error de medición;
- Y no tiene que ser exactamente igual a $f(X)$.

Criterio aplicado. Un residual distinto de cero no prueba que el modelo esté mal. Puede reflejar ruido, variables faltantes, medición imperfecta o una forma funcional insuficiente.

Este punto prepara la regresión lineal. Cuando minimizamos errores cuadrados, no estamos esperando que todos los residuales sean cero. Estamos buscando una regla que reduzca el error promedio en datos representativos.

Condicionamiento: mirar Y dado $X = x$

La ciencia de datos supervisada casi siempre pregunta por Y **dado** que conocemos X .

En regresión:

$$E[Y \mid X = x]$$

se interpreta como el valor promedio esperado de Y entre casos con entradas como x .

En clasificación binaria:

$$P(Y = 1 \mid X = x)$$

se interpreta como la probabilidad de clase positiva para un caso con entradas como x .

Definición: probabilidad condicional.

Una probabilidad condicional describe qué tan probable es un evento cuando ya se conoce cierta información. En clasificación, $P(Y = 1 \mid X = x)$ pregunta por la probabilidad de clase positiva para una observación con características x .

Ejemplo resuelto 0.5.3: de score a probabilidad.

Si un modelo de riesgo produce

$$P(Y = 1 \mid X = x) = 0.82,$$

eso no significa que el caso “sea 82 % positivo”. Significa que, bajo el modelo y sus datos, casos con características como x se tratan como altamente compatibles con la clase positiva. La decisión final todavía requiere umbral, métrica y costo de error.

Pérdida empírica y riesgo esperado

Durante entrenamiento no conocemos el desempeño verdadero del modelo en todos los casos futuros. Solo tenemos una muestra. Por eso usamos un promedio de pérdidas observadas.

Si $\ell(f_\theta(x_i), y_i)$ mide el error en la observación i , la pérdida empírica es:

$$\frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

El desempeño que realmente nos importa es el desempeño esperado en casos nuevos:

$$E[\ell(f_\theta(X), Y)].$$

Definición: pérdida empírica.

La pérdida empírica es el promedio de errores observado en un conjunto de datos. Es lo que podemos calcular.

Definición: riesgo esperado.

El riesgo esperado es el error promedio que el modelo tendría sobre casos nuevos del proceso relevante. Es lo que nos gustaría controlar, pero no podemos observar directamente.

La tensión central del curso es esta:

`minimizamos una pérdida empírica, pero queremos buen riesgo esperado.`

Por qué separar entrenamiento y prueba

Si minimizamos una pérdida sobre entrenamiento, ese número puede ser optimista. El modelo ya tuvo oportunidad de adaptarse a esos datos. Para estimar desempeño en observaciones nuevas, reservamos datos no usados en el ajuste.

Conjunto	Qué aproxima	Uso
Train	pérdida empírica usada para aprender	ajustar parámetros
Dev/validación	desempeño en datos no usados para ajustar parámetros	elegir modelo, complejidad o umbral
Test	estimación final de desempeño en datos no usados para diseño	reporte final

Advertencia

El test no revela el desempeño verdadero del mundo. Solo da una estimación en una muestra reservada. Si se usa repetidamente para tomar decisiones, también se vuelve parte del diseño.

Qué debes llevar a los próximos capítulos

No necesitas saber probabilidad avanzada para continuar. Sí necesitas estas cuatro ideas:

1. Una columna observada es evidencia; una variable aleatoria representa lo que podría pasar en un caso nuevo.
2. Un modelo intenta aprender una relación entre X y Y , pero hay ruido.
3. Entrenar minimiza pérdidas observadas; validar estima desempeño fuera del ajuste.
4. Una probabilidad predicha no es una decisión: necesita umbral, métrica y contexto.

Punto de control

Para cada frase, decide si habla de datos observados o de incertidumbre futura.

1. “El promedio de `precio_millones` en esta tabla es 406.25”.
2. “El precio de una vivienda nueva con estas características es incierto”.
3. “El modelo tuvo MSE 1200 en validación”.
4. “Queremos bajo error esperado en viviendas futuras”.

Respuesta esperada:

1. datos observados;
2. incertidumbre futura;
3. datos observados reservados para evaluación;
4. incertidumbre futura.

Verificaciones seleccionadas

1. Si una tabla tiene 400 filas, el promedio de una columna es una propiedad de esa muestra observada. $E[Y]$ es una idealización del proceso que podría generar casos nuevos; no se observa directamente.
2. En $Y = f(X) + \varepsilon$, el término ε no es un “error de programación”. Representa variabilidad no explicada, medición imperfecta o información que no está en X .
3. Una pérdida baja en entrenamiento no estima por sí sola el riesgo esperado. Para hablar de desempeño futuro se necesita evidencia en datos que no hayan participado en el ajuste.
4. Una probabilidad predicha como $P(Y = 1 \mid X = x) = 0.65$ todavía no decide una acción. Falta escoger umbral, métrica y costo de error.

Para explorar más

- **Extensión matemática:** compara $\bar{y}$ con $E[Y]$. Explica por qué el primero se calcula y el segundo se idealiza.
- **Extensión computacional:** simula $y = 3 + 2x + \varepsilon$ con dos niveles de ruido y compara los residuales.
- **Conexión con proyecto:** escribe cuál sería X , cuál sería Y , qué fuente de ruido anticipas y qué partición usarías para estimar desempeño.

Revisión del capítulo

Este interludio introduce la mínima probabilidad necesaria para el curso. La tabla observada contiene realizaciones; las variables aleatorias representan valores inciertos antes de observar un caso nuevo. Una distribución describe variabilidad, y una distribución empírica resume la muestra disponible.

Regresión y clasificación preguntan por Y dado X . En regresión suele interesar un valor esperado condicional; en clasificación binaria interesa $P(Y = 1 \mid X = x)$. El ruido explica por qué no todo residual se elimina con un modelo más complejo.

Entrenar minimiza pérdida empírica, pero el objetivo aplicado es buen desempeño en casos nuevos. Por eso validación, particiones y cuidado con leakage no son detalles técnicos: son la forma práctica de razonar bajo incertidumbre.

Términos clave

- **Variable aleatoria:** cantidad incierta antes de observar un caso.
- **Realización:** valor concreto observado de una variable aleatoria.
- **Distribución empírica:** patrón de valores observado en una muestra.
- **Valor esperado:** promedio ideal bajo el proceso que genera los datos.
- **Ruido:** variabilidad no explicada por las variables disponibles.
- **Probabilidad condicional:** probabilidad calculada dado que se conoce cierta información.
- **Pérdida empírica:** promedio de errores observado en un dataset.

- **Riesgo esperado:** error promedio ideal en casos nuevos del proceso relevante.

Ejercicios

Preguntas conceptuales

1. Explica con tus palabras la diferencia entre una columna observada y una variable aleatoria.
2. ¿Por qué un residual grande no implica automáticamente que el modelo esté mal?

Problemas cuantitativos

3. Para los valores 2, 4, 4, 10, calcula el promedio y la varianza empírica con denominador n .
4. Si una clase positiva aparece 30 veces en 200 observaciones, estima la proporción empírica de positivos.

Práctica computacional

5. Simula 100 valores con $y = 5 + 0.8 * x + \text{ruido}$ usando dos desviaciones estándar distintas para el ruido. Compara histogramas de residuales.
6. Toma una columna numérica de un dataset y reporta promedio, desviación estándar, mínimo, máximo y un histograma.

Interpretación y pensamiento crítico

7. Un modelo tiene error de entrenamiento muy bajo y error de validación alto. Interpreta el resultado usando pérdida empírica y desempeño esperado.
8. Un modelo reporta $P(Y = 1 \mid X = x) = 0.65$. Explica por qué todavía falta una regla de decisión.

Proyecto de cierre

9. Para una idea de proyecto, escribe qué sería X , qué sería Y , qué ruido esperas y qué variable podría producir leakage.
10. Describe qué muestra tu distribución empírica inicial y qué no permite concluir sobre la población o casos futuros.

Parte II: Regresión, optimización y clasificación

Parte II: Regresión, optimización y clasificación

Esta parte desarrolla el primer bloque matemático fuerte del curso. La regresión lineal introduce modelos paramétricos, mínimos cuadrados y forma matricial. El descenso del gradiente conecta cálculo con algoritmo. La regularización y la validación introducen el problema central de generalización. La regresión logística cierra el bloque con clasificación binaria y métricas.

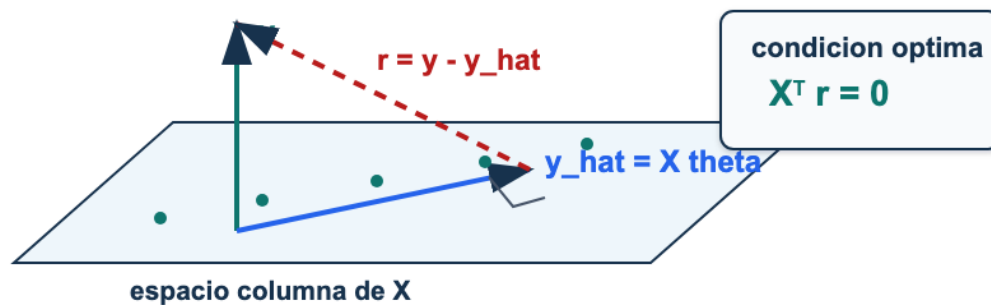


Figura 2: Geometría de mínimos cuadrados: las predicciones forman una proyección de y sobre el espacio columna de X ; el residual queda perpendicular a ese espacio.

Lectura de la figura. Esta parte instala una idea que se repite en todo el curso: ajustar un modelo es escoger una estructura matemática que reduzca una pérdida, pero la decisión final depende de validación y contexto.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. escribir regresión lineal en forma matricial;
2. derivar gradientes y ecuaciones normales para mínimos cuadrados;
3. implementar descenso del gradiente y diagnosticar su convergencia;
4. detectar overfitting mediante validación;
5. usar regularización Ridge/Lasso con escalamiento apropiado;
6. entrenar e interpretar una regresión logística con métricas y umbral.

Mapa de la parte

Capítulo	Pregunta guía	Evidencia esperada
Regresión lineal	¿Qué significa ajustar un modelo lineal con matrices?	Derivación, implementación y baseline.
Descenso del gradiente	¿Cómo se convierte una derivada en algoritmo?	Curva de pérdida y diagnóstico de tasa.
Regularización y validación	¿Cómo elegimos un modelo que generalice?	Comparación train/dev/test sin leakage.
Regresión logística	¿Cómo se convierten puntajes en probabilidades y decisiones?	Matriz de confusión, métrica y umbral.
Ejercicios integradores	¿Puedes conectar cálculo, código e interpretación?	Solución trazable y conclusión breve.

Criterio de salida

La parte queda dominada cuando puedes tomar un dataset tabular, construir un baseline, entrenar un modelo lineal o logístico, justificar preprocesamiento, elegir regularización con validación y escribir una conclusión que no use el test como herramienta de diseño.

Capítulo 1: Regresión lineal múltiple

Matrices, mínimos cuadrados, ecuación normal e interpretación

Pregunta aplicada

¿Cómo podemos predecir una cantidad numérica usando varias variables, escribir el modelo como álgebra matricial y defender si supera una regla simple?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. representar un problema de regresión con una matriz de diseño;
2. distinguir variables, parámetros, predicciones y residuales;
3. escribir MSE como norma cuadrática;
4. derivar el gradiente de MSE;
5. derivar la ecuación normal;
6. implementar mínimos cuadrados con `np.linalg.solve`;
7. comparar un modelo lineal contra un baseline;
8. interpretar coeficientes con cuidado.

La regresión lineal parece elemental, pero contiene la gramática de casi todos los modelos que veremos después: datos como matrices, parámetros como vectores, predicciones como operaciones vectorizadas, pérdidas como funciones a minimizar y desempeño como comparación fuera de entrenamiento.

En este capítulo usaremos la distinción del Interludio 0.5: los valores observados y_i son realizaciones de una variable objetivo Y . Por eso una regresión no intenta memorizar cada fila, sino aprender una regla que reduzca el error promedio y pueda generalizar a observaciones nuevas.

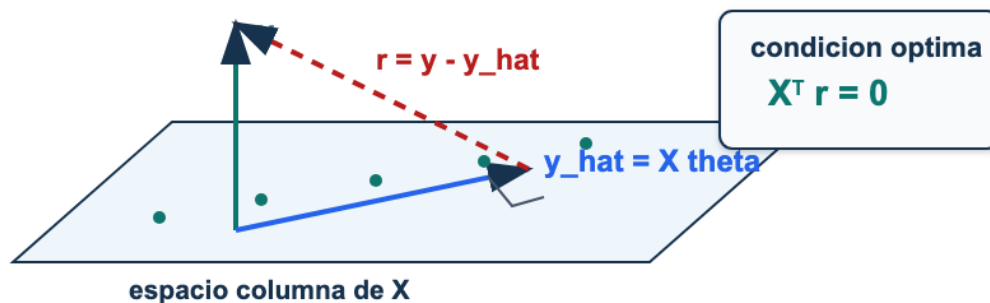


Figura 3: Geometría de mínimos cuadrados: las predicciones forman una proyección de y sobre el espacio columna de X ; el residual queda perpendicular a ese espacio.

Lectura de la figura. Mínimos cuadrados busca una predicción $\hat{y} = X\theta$ dentro del espacio generado por las columnas de X . En la solución óptima, el residual $r = y - \hat{y}$ queda perpendicular a ese espacio.

El problema de regresión

En regresión queremos predecir una cantidad numérica. Algunos ejemplos:

- precio de una vivienda;
- consumo de energía;
- tiempo de entrega;
- concentración de una sustancia;
- puntaje de riesgo continuo.

Cada observación tiene variables de entrada y una respuesta.

Objeto	Ejemplo
Entrada x_i	área, habitaciones, ubicación codificada
Salida y_i	precio
Modelo	regla que transforma x_i en y_{hat_i}
Error	diferencia entre y_i y y_{hat_i}

Definición 1.1: problema supervisado de regresión.

Un conjunto de entrenamiento de regresión es una colección de pares (x_i, y_i) , donde x_i contiene variables explicativas y y_i es una cantidad real. El objetivo es aprender una función f tal que $f(x_i)$ aproxime y_i y generalice a observaciones no usadas para entrenar.

De una recta a una matriz

La recta clásica es

$$\hat{y}_i = \theta_0 + \theta_1 x_i.$$

Con varias variables:

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_d x_{id}.$$

Para escribir esto como producto matricial agregamos una columna de unos. Si hay n observaciones y d variables originales, definimos:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1d} \\ 1 & x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

El vector de parámetros es

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}.$$

Entonces todas las predicciones se escriben como:

$$\hat{y} = X\theta.$$

Definición 1.2: matriz de diseño.

La matriz de diseño X contiene las variables que el modelo usa para producir predicciones. Puede incluir una columna de intercepto, variables originales, variables transformadas, interacciones o codificaciones de variables categóricas.

En Python:

```
import numpy as np

def add_intercept(X):
    X = np.asarray(X, dtype=float)
    ones = np.ones((X.shape[0], 1))
    return np.hstack([ones, X])

X_design = add_intercept(X_raw)
y_hat = X_design @ theta
```

Dimensiones: la primera defensa contra errores

Antes de derivar o programar, revisa dimensiones.

Objeto	Forma
<code>X</code>	(n, p)
<code>theta</code>	$(p,)$ o $(p, 1)$
<code>X @ theta</code>	$(n,)$ o $(n, 1)$
<code>y</code>	$(n,)$
<code>X.T @ X</code>	(p, p)
<code>X.T @ y</code>	$(p,)$

Chequeo rápido. Si `X @ theta` no produce un vector con una predicción por observación, el problema no es estadístico todavía: es de representación.

Residuales y pérdida

El residual de la observación i es

$$r_i = y_i - \hat{y}_i.$$

En vector:

$$r = y - X\theta.$$

La pérdida de mínimos cuadrados mide el promedio de errores cuadrados:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{2n} \|X\theta - y\|_2^2.$$

El factor $1/2$ se usa para que el 2 de la derivada desaparezca.

Definición 1.3: MSE y pérdida cuadrática.

El error cuadrático medio es

$$\text{MSE}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2.$$

La pérdida

$$J(\theta) = \frac{1}{2} \text{MSE}(\theta)$$

tiene el mismo minimizador, pero un gradiente más limpio.

Implementación:

```
def mse_loss(X, y, theta):
    residual = X @ theta - y
    return np.mean(residual ** 2)

def half_mse_loss(X, y, theta):
    residual = X @ theta - y
    return 0.5 * np.mean(residual ** 2)
```

Derivación del gradiente

Partimos de:

$$J(\theta) = \frac{1}{2n} (X\theta - y)^\top (X\theta - y).$$

Expandiendo:

$$J(\theta) = \frac{1}{2n} (\theta^\top X^\top X\theta - 2y^\top X\theta + y^\top y).$$

Usamos dos reglas:

$$\nabla_\theta(\theta^\top A\theta) = (A + A^\top)\theta,$$

y si A es simétrica:

$$\nabla_\theta(\theta^\top A\theta) = 2A\theta.$$

Como $X^\top X$ es simétrica:

$$\nabla J(\theta) = \frac{1}{2n} (2X^\top X\theta - 2X^\top y).$$

Por tanto:

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Proposición 1.1: gradiente de MSE escalado.

Para

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

el gradiente es

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Implementación:

```
def mse_gradient(X, y, theta):
    n = X.shape[0]
    return (X.T @ (X @ theta - y)) / n
```

Ecuación normal

Un mínimo interior de una función diferenciable satisface:

$$\nabla J(\theta) = 0.$$

Entonces:

$$\frac{1}{n} X^\top (X\theta - y) = 0.$$

Multiplicando por n:

$$X^\top X\theta - X^\top y = 0.$$

Reordenando:

$$X^\top X\theta = X^\top y.$$

Esta es la **ecuación normal**.

Si $X.T @ X$ es invertible:

$$\theta^* = (X^\top X)^{-1} X^\top y.$$

Proposición 1.2: solución de mínimos cuadrados.

Si X tiene rango columna completo, entonces $X.T @ X$ es invertible y el minimizador único de la pérdida cuadrática es

$$\theta^* = (X^\top X)^{-1} X^\top y.$$

En código profesional evitamos la inversa explícita:

```
def normal_equation(X, y):
    return np.linalg.solve(X.T @ X, X.T @ y)
```

Criterio numérico. `np.linalg.inv(A) @ b` suele ser menos estable y menos eficiente que `np.linalg.solve(A, b)`. En modelos reales, preferimos resolver el sistema o usar implementaciones robustas como `LinearRegression`.

Interpretación geométrica

El modelo busca un vector $\mathbf{X} \theta$ en el espacio generado por las columnas de $\mathbf{X}$. Ese espacio es un subespacio de $\mathbb{R}^n$. El vector observado y puede no estar exactamente en ese subespacio. Mínimos cuadrados encuentra la proyección de y sobre el espacio columna de $\mathbf{X}$.

En la solución:

$$X^T(y - X\theta^*) = 0.$$

Esto dice que los residuales son ortogonales a cada columna de $\mathbf{X}$. Es decir: después de ajustar el modelo, ninguna variable incluida en $\mathbf{X}$ explica linealmente lo que queda en los residuales.

Ejemplo resuelto 1: datos exactos

Supongamos:

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}.$$

El patrón exacto es:

$$y = 1 + 2x.$$

La solución esperada es:

$$\theta = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

```
X = np.array([[1, 0], [1, 1], [1, 2]], dtype=float)
y = np.array([1, 3, 5], dtype=float)
theta = np.linalg.solve(X.T @ X, X.T @ y)
theta
```

Ejemplo resuelto 2: datos con ruido

Ahora supón:

$$y = \begin{bmatrix} 1.1 \\ 2.9 \\ 5.2 \\ 6.8 \end{bmatrix}, \quad x = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}.$$

No existe una recta que pase exactamente por todos los puntos. Mínimos cuadrados elige la recta que minimiza la suma de residuales cuadrados.

```
x = np.array([0, 1, 2, 3], dtype=float).reshape(-1, 1)
y = np.array([1.1, 2.9, 5.2, 6.8], dtype=float)
X = add_intercept(x)
theta = normal_equation(X, y)
y_hat = X @ theta
residuals = y - y_hat
mse = np.mean(residuals ** 2)
```

Una buena explicación debe incluir:

- los coeficientes estimados;
- el MSE;
- una inspección de residuales;
- comparación contra baseline.

Baseline de regresión

Un baseline es una regla simple que el modelo debe superar. Para regresión, el baseline más común predice siempre el promedio de `y_train`:

$$\hat{y}_i = \bar{y}_{\text{train}}.$$

Su MSE en prueba es:

$$\text{MSE}_{\text{baseline}} = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} (y_i - \bar{y}_{\text{train}})^2.$$

Criterio aplicado. Un modelo lineal que no mejora el baseline en datos de prueba no justifica una decisión, aunque tenga coeficientes matemáticamente calculados.

```
baseline = np.mean(y_train)
y_pred_base = np.full_like(y_test, fill_value=baseline, dtype=float)
mse_base = np.mean((y_test - y_pred_base) ** 2)
```


Interpretación de coeficientes

En un modelo lineal:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d.$$

El coeficiente `theta_j` se interpreta como el cambio esperado en la predicción cuando `x_j` aumenta una unidad y las demás variables se mantienen constantes.

Esta frase tiene tres condiciones:

1. habla de la predicción, no necesariamente de causalidad;
2. depende de la escala de `x_j`;
3. mantiene las demás variables constantes, lo cual puede ser poco realista si las variables están correlacionadas.

No confundas asociación con causalidad. Un coeficiente grande no prueba que una variable cause el resultado. Para causalidad se necesita diseño, supuestos y argumento adicional.

Multicolinealidad y rango

Si una columna de `X` puede escribirse como combinación lineal de otras columnas, entonces `X` no tiene rango columna completo. En ese caso `X.T @ X` no es invertible.

Ejemplo:

```
x1 = np.array([1, 2, 3, 4], dtype=float)
x2 = 2 * x1
X = np.column_stack([np.ones_like(x1), x1, x2])
```

Aquí la tercera columna contiene exactamente la misma información que la segunda.

Síntomas prácticos:

- error de matriz singular al resolver la ecuación normal;
- coeficientes muy inestables;
- cambios grandes en coeficientes cuando se modifica poco el dataset;
- interpretaciones contradictorias.

Errores comunes

1. Olvidar la columna de intercepto.
2. Usar `np.linalg.inv` innecesariamente.
3. Entrenar y evaluar sobre los mismos datos.
4. Reportar solo coeficientes sin métrica.
5. Comparar modelos sin baseline.
6. Interpretar coeficientes sin revisar escala.
7. Ignorar multicolinealidad.
8. Confundir buen ajuste en entrenamiento con generalización.

Checklist de estudio

Antes de pasar al Capítulo 2, deberías poder responder:

- ¿Cuál es la forma de X , θ , y y $X @ \theta$?
- ¿Por qué agregamos una columna de unos?
- ¿Qué mide un residual?
- ¿Por qué usamos errores cuadrados?
- ¿Cómo se deriva $X.T @ X \theta = X.T @ y$?
- ¿Cuándo puede fallar la ecuación normal?
- ¿Cuál es el baseline natural en regresión?

Para explorar más

- **Extensión matemática:** rehace la derivación del gradiente usando diferenciales y verifica que la forma final sea compatible con las dimensiones de X , y y θ .
- **Extensión computacional:** compara `np.linalg.solve`, `np.linalg.lstsq` y `LinearRegression` sobre el mismo dataset; registra cuándo cambian los coeficientes.
- **Conexión con proyecto:** documenta la matriz de diseño de tu baseline: variables, intercepto, transformaciones y forma final de X .

Revisión del capítulo

En este capítulo conectamos un problema de predicción numérica con álgebra matricial. La matriz de diseño organiza las variables, el vector de parámetros define el modelo y la pérdida MSE mide el error promedio de predicción.

El resultado central es que mínimos cuadrados puede leerse de dos formas equivalentes: como optimización de una norma cuadrática y como proyección geométrica de y sobre el espacio columna de X . La ecuación normal $X^T X \theta = X^T y$ aparece cuando el gradiente de la pérdida se anula.

Para usar regresión lineal profesionalmente, no basta resolver la ecuación. Hay que comparar contra un baseline, revisar formas, evitar inversiones numéricas innecesarias e interpretar coeficientes solo bajo las condiciones del diseño y del preprocesamiento.

Términos clave

- **Regresión supervisada:** aprendizaje de una función que predice una cantidad real a partir de variables observadas.
- **Matriz de diseño:** matriz que organiza intercepto, variables y transformaciones usadas por el modelo.
- **Residual:** diferencia entre el valor observado y la predicción del modelo.
- **MSE:** promedio de errores cuadrados usado para medir ajuste en regresión.
- **Ecuación normal:** sistema $X^T X \theta = X^T y$ que caracteriza mínimos cuadrados.
- **Baseline:** regla simple que un modelo debe superar para justificar su complejidad.
- **Multicolinealidad:** dependencia lineal o casi lineal entre columnas de la matriz de diseño.

Ejercicios

Preguntas conceptuales

1. Explica en tus palabras qué significa que X sea una matriz de diseño.
2. ¿Por qué el intercepto puede representarse con una columna de unos?
3. ¿Qué diferencia hay entre residual y error de generalización?
4. ¿Por qué un modelo puede tener MSE bajo en entrenamiento y aun así ser malo?
5. Explica por qué un coeficiente no implica causalidad.

Problemas cuantitativos

6. Sea

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 2 \\ 5 \end{bmatrix}.$$

Calcula $X.T @ X$, $X.T @ y$ y resuelve la ecuación normal.

7. Para

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

reproduce la derivación del gradiente sin mirar el texto.

8. Demuestra que si $X.T @ (y - X \theta) = 0$, los residuales son ortogonales a cada columna de X .

Práctica computacional

9. Implementa `add_intercept(X)` sin usar ciclos.
10. Implementa `mse_loss(X, y, theta)`.
11. Implementa `normal_equation(X, y)` usando `np.linalg.solve`.
12. Simula datos con $y = 3 - 2x + \text{ruido}$ y estima los parámetros.
13. Compara el MSE del modelo contra el baseline del promedio.

Interpretación y pensamiento crítico

14. Un modelo obtiene MSE de entrenamiento 4.1 y MSE de prueba 18.7. ¿Qué hipótesis harías?
15. Un modelo no mejora el baseline. Escribe dos posibles razones técnicas.
16. Si una variable está medida en pesos y otra en millones de pesos, ¿por qué sus coeficientes no son directamente comparables?

Proyecto de cierre

17. Elige un dataset tabular de regresión o genera uno pequeño con semilla fija. Define la matriz de diseño, ajusta mínimos cuadrados, compara contra el baseline del promedio y escribe una recomendación de máximo 150 palabras que incluya métrica, limitación y siguiente verificación.

Capítulo 2: Descenso del gradiente

Optimización, escalamiento y condicionamiento

Pregunta aplicada

¿Cómo ajustamos parámetros cuando no queremos o no podemos depender de una fórmula cerrada, y cómo diagnosticamos si el algoritmo realmente está aprendiendo?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. explicar qué significa minimizar una función de pérdida;
2. implementar descenso del gradiente para MSE;
3. interpretar una curva de pérdida;
4. escoger una tasa de aprendizaje razonable por diagnóstico;
5. explicar por qué el escalamiento afecta la optimización;
6. reconocer problemas de condicionamiento;
7. comparar una solución iterativa con la ecuación normal.

En el Capítulo 1 resolvimos mínimos cuadrados con una fórmula cerrada. En este capítulo estudiamos un método iterativo. Esta transición es importante porque muchos modelos útiles no tienen una solución cerrada conveniente.

Optimización como búsqueda

Tenemos una función de pérdida:

$$J(\theta).$$

Queremos encontrar parámetros con pérdida pequeña:

$$\theta^* = \arg \min_{\theta} J(\theta).$$

En regresión lineal con MSE, J es convexa. Eso significa que no hay mínimos locales malos: cualquier mínimo local es global. Pero que el problema sea convexo no significa que cualquier algoritmo llegue rápido.

Definición 2.1: optimización.

Optimizar un modelo es buscar parámetros que minimicen una función objetivo. En aprendizaje supervisado, esa función suele medir discrepancia entre predicciones y observaciones, posiblemente más una penalización.

Idea geométrica del gradiente

El gradiente $\text{grad } J(\theta)$ apunta en la dirección de mayor crecimiento local de la función. Para bajar la pérdida, nos movemos en la dirección contraria:

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \alpha \nabla J(\theta_{\text{actual}}).$$

Aquí α es la tasa de aprendizaje.

Descenso del gradiente: trayectoria, escala

Cada actualización usa el gradiente local para reducir la pérdida, pero el tamar

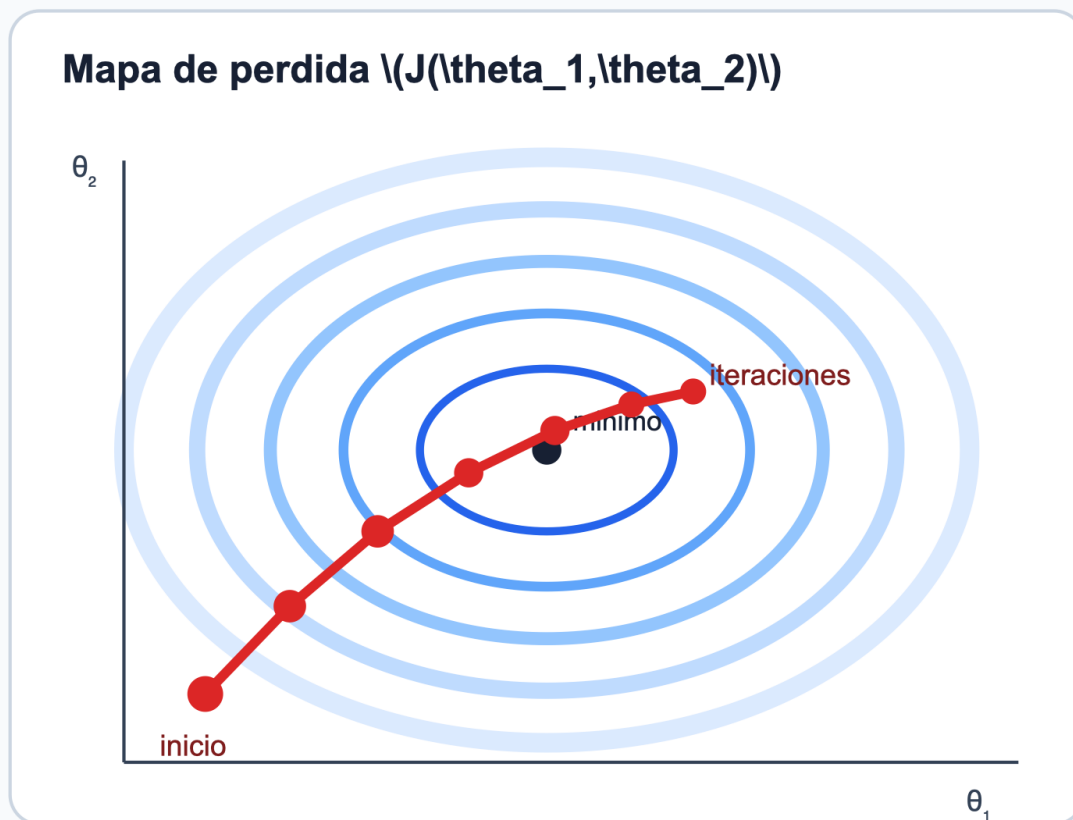


Figura 4: Descenso del gradiente: la trayectoria avanza sobre contornos de pérdida y la tasa de aprendizaje controla si el avance es lento, estable u oscilatorio.

Lectura de la figura. Los puntos rojos no son evaluaciones independientes: son iteraciones del mismo algoritmo. El gradiente define la dirección local de descenso y la tasa de aprendizaje define cuánto se avanza. Un paso pequeño puede ser correcto pero lento; un paso grande puede saltar sobre el valle de la pérdida y producir oscilación.

Definición 2.2: tasa de aprendizaje.

La tasa de aprendizaje **alpha** controla el tamaño del paso en cada iteración del descenso del gradiente. Si es demasiado pequeña, el algoritmo avanza lentamente. Si es demasiado grande, puede oscilar o divergir.

Algoritmo básico

Para MSE:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

el gradiente es:

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

El algoritmo es:

```
inicializar theta
para t = 1, ..., max_iter:
    grad = (X.T @ (X @ theta - y)) / n
    theta = theta - alpha * grad
    guardar pérdida
```

Implementación:

```
def gradient_descent(X, y, alpha=0.1, max_iter=1000, tol=1e-8):
    n, p = X.shape
    theta = np.zeros(p)
    history = []

    for _ in range(max_iter):
        residual = X @ theta - y
        loss = 0.5 * np.mean(residual ** 2)
        grad = (X.T @ residual) / n

        history.append(loss)
        theta_next = theta - alpha * grad

        if np.linalg.norm(theta_next - theta) < tol:
            theta = theta_next
            break

    theta = theta_next

    return theta, np.array(history)
```


Cómo leer una curva de pérdida

La curva de pérdida es una herramienta de diagnóstico. No basta con mirar el resultado final.

Patrón observado	Interpretación probable	Acción
Baja de forma suave	alpha razonable	Continuar
Baja muy lentamente	alpha pequeño o mala escala	Aumentar alpha o escalar
Oscila	alpha grande	Reducir alpha
Explota a infinito	alpha demasiado grande	Reducir mucho alpha
Baja y se estanca alto	modelo limitado o pocas iteraciones	revisar features/modelo
Entrena bien, prueba mal	overfitting o leakage	revisar validación

Criterio práctico. Una curva de pérdida sin eje, sin escala y sin comentario no es evidencia. Debes decir si converge, si oscila, si se estabiliza y qué harías después.

Ejemplo resuelto: una variable escalada

Supón dos variables:

- **x1:** edad en años, rango aproximado 18 a 80;
- **x2:** ingreso anual en pesos, rango aproximado 10,000,000 a 500,000,000.

En la matriz **X**, la segunda columna puede dominar el gradiente por escala, aunque no sea más importante conceptualmente. El algoritmo tendrá que usar una tasa de aprendizaje muy pequeña para no explotar en esa dirección.

La solución no es cambiar la pregunta, sino cambiar la representación:

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

Donde μ_j y s_j se calculan con entrenamiento.

```
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)
sigma[sigma == 0] = 1

X_train_scaled = (X_train - mu) / sigma
X_test_scaled = (X_test - mu) / sigma
```

Regla anti-leakage. El promedio y la desviación estándar se calculan con entrenamiento. Luego se aplican a validación y prueba. No se ajustan con todo el dataset.

Escalamiento y geometría

Si las variables tienen escalas muy distintas, las curvas de nivel de la pérdida pueden ser alargadas. El descenso del gradiente avanza en zigzag porque la dirección de máximo descenso local no apunta directamente al mínimo.

Escalar variables suele transformar el problema para que las curvas de nivel sean menos alargadas. Esto no cambia la información esencial, pero sí cambia la geometría del algoritmo.

Definición 2.3: estandarización.

Estandarizar una variable es restarle su media de entrenamiento y dividirla por su desviación estándar de entrenamiento. La variable resultante tiene media cercana a cero y escala comparable con otras variables.

Condicionamiento

El condicionamiento mide qué tan sensible es un problema numérico a cambios pequeños. Para mínimos cuadrados, una matriz $X.T @ X$ mal condicionada puede producir coeficientes inestables y descenso lento.

El número de condición de una matriz invertible A se define como:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Con norma espectral:

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

Si κ es grande, hay direcciones donde la curvatura de la pérdida cambia mucho.

```
cond = np.linalg.cond(X.T @ X)
```

Lectura práctica. Un número de condición grande no invalida automáticamente un modelo, pero alerta sobre inestabilidad, colinealidad o necesidad de regularización.

Comparación con ecuación normal

Para regresión lineal con MSE, la ecuación normal y descenso del gradiente buscan el mismo mínimo. Por eso podemos usarlas como verificación mutua:

```
theta_ne = np.linalg.solve(X.T @ X, X.T @ y)
theta_gd, history = gradient_descent(X, y, alpha=0.1)

np.linalg.norm(theta_ne - theta_gd)
```

Si la diferencia es grande, puede ocurrir:

- faltaron iteraciones;
- la tasa de aprendizaje es inadecuada;
- las variables no están escaladas;
- hay un error en el gradiente;
- la matriz está mal condicionada.

Batch, stochastic y mini-batch

En este módulo usamos descenso del gradiente batch: cada paso calcula el gradiente con todos los datos.

Variante	Usa en cada paso	Ventaja	Costo
Batch	todo el dataset	gradiente estable	caro en datasets grandes
Stochastic	una observación	pasos baratos	ruidoso
Mini-batch	un subconjunto	balance práctico	requiere más diseño

En redes neuronales usaremos mini-batches. La idea central sigue siendo la misma: mover parámetros en dirección contraria al gradiente.

Tasa de aprendizaje: una rutina práctica

Una estrategia razonable para este módulo:

1. Escala las variables.
2. Prueba **alpha** en una lista: 0.001, 0.01, 0.05, 0.1, 0.5.
3. Grafica la pérdida.
4. Descarta valores que divergen u oscilan.
5. Elige el mayor valor que baje de forma estable.
6. Verifica desempeño en validación o prueba.

```
alphas = [0.001, 0.01, 0.05, 0.1, 0.5]
results = {}
for alpha in alphas:
    theta, history = gradient_descent(X_train_scaled, y_train, alpha=alpha)
    results[alpha] = history
```

Errores comunes

1. No guardar la historia de pérdida.
2. Reportar solo el último `theta` sin diagnóstico de convergencia.
3. Usar `alpha` grande y no detectar divergencia.
4. Escalar entrenamiento y prueba por separado.
5. Incluir la columna de intercepto en el escalamiento.
6. Comparar gradiente contra ecuación normal sin usar la misma matriz `X`.
7. Creer que más iteraciones arreglan un mal modelo.

Checklist de estudio

- ¿Puedes escribir la actualización de descenso del gradiente?
- ¿Puedes explicar el papel de `alpha`?
- ¿Puedes implementar el gradiente de MSE sin ciclos?
- ¿Puedes interpretar una curva que oscila?
- ¿Sabes por qué se escala con parámetros de entrenamiento?
- ¿Puedes comparar `theta_gd` con `theta_ne`?

Para explorar más

- **Extensión matemática:** analiza cómo cambia la cota de estabilidad del paso cuando el valor propio dominante de $X^T X$ aumenta.
- **Extensión computacional:** grafica curvas de pérdida para tres tasas de aprendizaje y marca divergencia, convergencia lenta y convergencia estable.
- **Conexión con proyecto:** registra en tu bitácora qué variables necesitan escalamiento antes de entrenar y cómo lo verificaste.

Revisión del capítulo

El descenso del gradiente convierte cálculo diferencial en un algoritmo iterativo. En lugar de resolver directamente una ecuación cerrada, actualiza los parámetros en la dirección opuesta al gradiente de la pérdida.

La tasa de aprendizaje controla el tamaño de cada paso. Una tasa demasiado pequeña produce avance lento; una tasa demasiado grande puede provocar oscilación o divergencia. El escalamiento de variables mejora el condicionamiento y hace que las direcciones de descenso sean más comparables.

Una implementación confiable debe registrar la pérdida, verificar que las formas de matrices coincidan y comparar el resultado contra una referencia. La curva de pérdida no es decoración: es evidencia de convergencia o de un problema de escala, tasa o código.

Términos clave

- **Optimización:** búsqueda de parámetros que minimizan una función objetivo.
- **Gradiente:** dirección de mayor aumento local de una función diferenciable.
- **Tasa de aprendizaje:** escala del paso en cada actualización iterativa.
- **Convergencia:** estabilización del algoritmo hacia pérdida o gradiente suficientemente pequeños.
- **Estandarización:** transformación que centra y escala variables usando estadísticas de entrenamiento.
- **Condicionamiento:** sensibilidad numérica de un sistema o problema ante pequeñas perturbaciones.
- **Mini-batch:** subconjunto de observaciones usado para aproximar un paso de optimización.

Ejercicios

Preguntas conceptuales

1. Explica por qué el gradiente apunta en la dirección de mayor aumento.
2. ¿Por qué usamos `theta - alpha * grad` y no `theta + alpha * grad`?
3. ¿Qué evidencia usarías para afirmar que el algoritmo convergió?
4. ¿Por qué escalar variables puede acelerar el descenso del gradiente?
5. Explica qué problema produce escalar prueba con su propia media.

Problemas cuantitativos

6. Para

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

calcula el gradiente de `J(theta)`.

7. Ejecuta manualmente dos pasos de descenso del gradiente con `alpha=0.1` para el ejercicio anterior.
8. Si una curva de pérdida pasa de 100 a 80, luego a 130, luego a 70, ¿qué sospechas sobre `alpha`?

Práctica computacional

9. Implementa `gradient_descent(X, y, alpha, max_iter)`.
10. Agrega un criterio de parada por norma del gradiente.
11. Compara tres tasas de aprendizaje en un dataset generado con semilla fija.
12. Estandariza variables sin tocar la columna de intercepto.
13. Calcula `np.linalg.cond(X.T @ X)` antes y después de escalar.

Interpretación y pensamiento crítico

14. Tu modelo con gradiente obtiene MSE mayor que la ecuación normal. Escribe tres hipótesis técnicas.
15. Tu curva converge en entrenamiento, pero el error de prueba es malo. ¿El problema es necesariamente el optimizador?
16. ¿Qué decisión tomarías si $\alpha=0.1$ converge en 200 iteraciones y $\alpha=0.01$ converge en 5000?

Proyecto de cierre

17. Construye un experimento pequeño con dos variables en escalas muy distintas. Ejecuta descenso del gradiente antes y después de estandarizar, grafica la pérdida y redacta un diagnóstico que explique qué cambió y qué no demuestra el experimento.

Capítulo 3: Validación y regularización

Features, complejidad, Ridge, Lasso y selección de modelo

Pregunta aplicada

¿Cómo elegimos la complejidad de un modelo sin engañarnos con el entrenamiento ni contaminar la evaluación final?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. construir features polinomiales e interacciones;
2. explicar underfitting y overfitting;
3. separar entrenamiento, validación y prueba;
4. evitar leakage en preprocesamiento;
5. formular Ridge y Lasso;
6. interpretar el efecto de `lambda`;
7. seleccionar modelos con una métrica de validación;
8. justificar un modelo final sin mirar la prueba repetidamente.

El Capítulo 1 mostró cómo ajustar un modelo lineal. El Capítulo 2 mostró cómo optimizarlo. Este capítulo responde una pregunta más importante: ¿cómo sabemos si ese modelo sirve fuera de los datos con los que fue ajustado?

La respuesta usa el lenguaje del Interludio 0.5. Entrenamiento minimiza una pérdida empírica sobre datos observados, pero la meta aplicada es bajo error en casos futuros. Validación y prueba son aproximaciones prácticas a ese desempeño fuera de la muestra.

Feature engineering

Un modelo lineal es lineal en sus parámetros, no necesariamente en las variables originales. Podemos transformar entradas para darle más flexibilidad.

Si tenemos una variable $\mathbf{x}$, podemos crear:

$$1, \quad x, \quad x^2, \quad x^3.$$

El modelo:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

sigue siendo lineal en **theta**, aunque sea polinomial en **x**.

Definición 3.1: feature engineering.

Feature engineering es el proceso de construir variables de entrada a partir de datos originales usando conocimiento del problema, transformaciones matemáticas o codificaciones computacionales.

Ejemplos:

Transformación	Uso
x^2, x^3	curvatura
$x_1 * x_2$	interacción
$\log(x)$	escala multiplicativa
$1\{x > c\}$	umbral
one-hot encoding	variables categóricas
estandarización	optimización y regularización

Regresión polinomial

Con una variable:

```
def polynomial_features_1d(x, degree):
    x = np.asarray(x, dtype=float).reshape(-1, 1)
    cols = [np.ones((x.shape[0], 1))]
    for k in range(1, degree + 1):
        cols.append(x ** k)
    return np.hstack(cols)
```

Para grados pequeños, la regresión polinomial puede capturar curvatura. Para grados altos, puede memorizar ruido.

Criterio aplicado. Una feature nueva debe tener una razón: mejorar ajuste, capturar conocimiento del dominio, resolver no linealidad o facilitar una decisión. Agregar features sin validación no es progreso.

Underfitting y overfitting

Definición 3.2: underfitting.

Ocurre cuando el modelo es demasiado simple para capturar la estructura relevante de los datos. Suele verse como error alto tanto en entrenamiento como en validación.

Definición 3.3: overfitting.

Ocurre cuando el modelo se ajusta demasiado a patrones accidentales del conjunto de entrenamiento. Suele verse como error bajo en entrenamiento y error alto en validación o prueba.

Situación	Error entrenamiento	Error validación	Diagnóstico
Underfitting	alto	alto	falta complejidad o mejores features
Buen ajuste	bajo/moderado	bajo/moderado	generaliza razonablemente
Overfitting	muy bajo	alto	demasiada complejidad o poca regularización

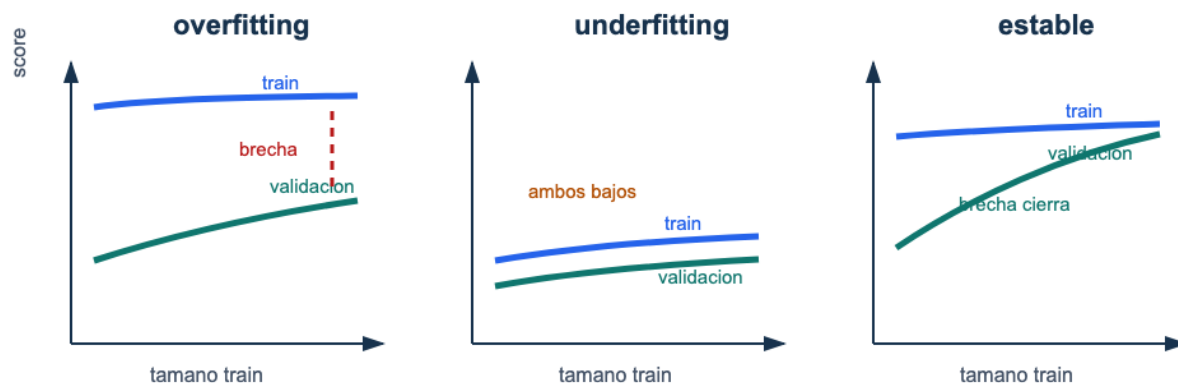


Figura 5: Curvas de aprendizaje: un caso de overfitting mantiene una brecha entre entrenamiento y validación, un caso de underfitting mantiene ambos desempeños bajos, y un caso estable cierra la brecha con más datos.

Lectura de la figura. Una curva de aprendizaje no es un resultado decorativo: debe sugerir una acción técnica. La brecha entre entrenamiento y validación apunta a regularización, más datos, menor complejidad o revisión de leakage.

Separación de datos

Usaremos tres conjuntos cuando sea posible:

Conjunto	Uso
Entrenamiento	ajustar parámetros y preprocesamiento
Validación	elegir grado, <code>lambda</code> , umbral o variante
Prueba	estimar desempeño final una sola vez

Si el dataset es pequeño, puede usarse validación cruzada. Pero el principio no cambia: no se debe usar la prueba como herramienta de diseño.

Regla de prueba. El conjunto de prueba no es un tablero para experimentar. Si lo consultas muchas veces para tomar decisiones, deja de ser una estimación honesta del desempeño final.

Leakage

Leakage ocurre cuando información que no estaría disponible en producción entra al entrenamiento o a la validación.

Ejemplos comunes:

- escalar con media y desviación de todo el dataset;
- imputar valores usando prueba;
- seleccionar features mirando el desempeño de prueba;
- crear una variable que usa información posterior al momento de predicción;
- duplicar observaciones de un mismo individuo en entrenamiento y prueba.

Pregunta anti-leakage. En el momento real de hacer la predicción, ¿esa información ya existiría? Si la respuesta es no, no debe entrar al modelo.

Regularización

La regularización modifica el objetivo para penalizar modelos demasiado complejos. En lugar de minimizar solo error de entrenamiento, minimizamos:

pérdida de datos + penalización.

La intuición es que, entre modelos con desempeño parecido, preferimos uno más estable y menos extremo.

Regularización: controlar complejidad sin cambiar la pregunta

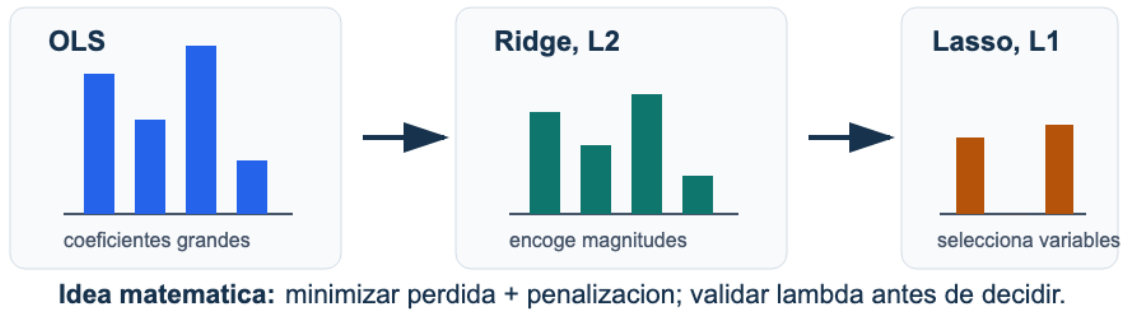


Figura 6: Regularización: OLS puede producir coeficientes grandes, Ridge reduce magnitudes y Lasso puede llevar coeficientes exactamente a cero.

Lectura de la figura. Ridge y Lasso expresan dos preferencias distintas: Ridge encoge coeficientes hacia valores más estables; Lasso puede producir soluciones esparsas. Ninguna de las dos preferencias reemplaza la validación.

Ridge

Ridge agrega una penalización L2:

$$J_{\text{ridge}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \frac{\lambda}{2} \|\theta_{1:p}\|_2^2.$$

Usualmente no penalizamos el intercepto `theta_0`.

El gradiente para los coeficientes no intercepto es:

$$\nabla J_{\text{ridge}}(\theta) = \frac{1}{n} X^\top (X\theta - y) + \lambda \tilde{\theta},$$

donde:

$$\tilde{\theta} = \begin{bmatrix} 0 \\ \theta_1 \\ \vdots \\ \theta_{p-1} \end{bmatrix}.$$

Si penalizamos todos los parámetros, la solución cerrada es:

$$\theta_{\text{ridge}}^* = (X^\top X + n\lambda I)^{-1} X^\top y.$$

Si no penalizamos el intercepto, usamos una matriz diagonal D con $D_{00}=0$ y $D_{jj}=1$ para $j>0$:

$$\theta_{\text{ridge}}^* = (X^\top X + n\lambda D)^{-1} X^\top y.$$

```
def ridge_closed_form(X, y, lam):
    p = X.shape[1]
    D = np.eye(p)
    D[0, 0] = 0
    return np.linalg.solve(X.T @ X + X.shape[0] * lam * D, X.T @ y)
```

Efecto de lambda

lambda	Efecto
0	regresión lineal sin regularización
pequeño	reduce coeficientes extremos suavemente
mediano	mejora estabilidad si hay colinealidad
grande	puede causar underfitting

Lectura aplicada. Ridge no elimina variables; reduce coeficientes. Es útil cuando muchas variables aportan algo y queremos estabilidad.

Lasso

Lasso agrega una penalización L1:

$$J_{\text{lasso}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \lambda \|\theta_{1:p}\|_1.$$

La norma L1 es:

$$\|\theta\|_1 = \sum_j |\theta_j|.$$

Lasso puede llevar algunos coeficientes exactamente a cero. Por eso se usa como herramienta de selección de variables, aunque esa selección debe interpretarse con cuidado.

Lectura aplicada. Lasso es atractivo cuando se espera que pocas variables sean dominantes. Pero con variables muy correlacionadas puede elegir una y descartar otra de forma inestable.

Ridge vs Lasso

Aspecto	Ridge	Lasso
Penalización	L2	L1
Coefficientes	se encogen	algunos quedan en cero
Colinealidad	reparte peso	puede escoger una variable
Solución cerrada simple	sí	no en general
Uso típico	estabilidad	selección de variables

Estandarización antes de regularizar

Ridge y Lasso penalizan coeficientes. Si las variables están en escalas distintas, la penalización no es comparable. Por eso se estandarizan variables antes de regularizar.

Pipeline correcto:

1. separar entrenamiento, validación y prueba;
2. ajustar `StandardScaler` con entrenamiento;
3. transformar entrenamiento, validación y prueba con ese scaler;
4. ajustar modelos regularizados;
5. elegir `lambda` con validación;
6. reportar prueba una vez.

Selección de modelo

Una rutina básica:

```

lambdas = [0.0, 0.001, 0.01, 0.1, 1.0, 10.0]
records = []

for lam in lambdas:
    theta = ridge_closed_form(X_train_scaled, y_train, lam)
    pred_train = X_train_scaled @ theta
    pred_val = X_val_scaled @ theta
    records.append({
        "lambda": lam,
        "mse_train": np.mean((y_train - pred_train) ** 2),
        "mse_val": np.mean((y_val - pred_val) ** 2),
        "coef_norm": np.linalg.norm(theta[1:])
    })

```

La selección no se hace por el menor error de entrenamiento, sino por validación.

Ejemplo resuelto: grado polinomial y Ridge

Supón que comparas grados 1, 3, 8 con `lambda` en [0, 0.01, 0.1, 1].

Un resultado típico puede ser:

grado	lambda	MSE train	MSE val	diagnóstico
1	0	alto	alto	underfitting
8	0	muy bajo	alto	overfitting
8	0.1	bajo	moderado	mejor control
3	0.01	moderado	bajo	candidato final

La conclusión debe explicar el compromiso, no solo señalar el mínimo.

Errores comunes

1. Crear polinomios después de mezclar entrenamiento y prueba.
2. Elegir grado por error de entrenamiento.
3. Usar prueba para escoger `lambda`.
4. Regularizar sin escalar.
5. Penalizar el intercepto sin intención.
6. Concluir que Lasso descubrió causalidad.
7. Comparar modelos con particiones distintas.
8. No reportar baseline.

Checklist de estudio

- ¿Puedes explicar overfitting con errores train/val?
- ¿Puedes construir features polinomiales?
- ¿Sabes qué es leakage?
- ¿Puedes escribir el objetivo de Ridge?
- ¿Puedes explicar por qué Lasso puede hacer selección de variables?
- ¿Sabes por qué se escala antes de regularizar?
- ¿Puedes elegir un modelo usando validación y no prueba?

Para explorar más

- **Extensión matemática:** compara el efecto de Ridge y Lasso sobre dos variables altamente correlacionadas y explica por qué sus coeficientes pueden comportarse distinto.
- **Extensión computacional:** construye una curva de validación para λ y reporta no solo el mejor valor, sino la región de desempeño estable.
- **Conexión con proyecto:** define qué transformación, partición o hiperparámetro queda prohibido ajustar con el conjunto de test.

Revisión del capítulo

Este capítulo introduce el problema central de generalización. Un modelo puede reducir mucho el error de entrenamiento y, aun así, fallar en datos nuevos. La validación separa el ajuste de parámetros de la selección de modelo.

Feature engineering y modelos polinomiales aumentan capacidad, pero también aumentan el riesgo de overfitting. Ridge controla coeficientes grandes mediante penalización L2; Lasso usa penalización L1 y puede llevar algunos coeficientes a cero. En ambos casos, el escalamiento es parte del modelo, no un detalle secundario.

La regla profesional es simple: entrenamiento ajusta parámetros, validación elige configuraciones y test estima desempeño final una sola vez. Cualquier transformación aprendida antes de partir los datos puede introducir leakage.

Términos clave

- **Feature engineering:** construcción de variables a partir de datos originales y conocimiento del problema.
- **Underfitting:** error alto por modelo demasiado simple o mal especificado.
- **Overfitting:** ajuste excesivo al entrenamiento con pobre generalización.
- **Train/validation/test:** separación de datos para ajustar, seleccionar y estimar desempeño final.
- **Leakage:** uso accidental de información que no estaría disponible en predicción real.
- **Ridge:** regularización L2 que penaliza la magnitud cuadrática de coeficientes.
- **Lasso:** regularización L1 que puede llevar coeficientes exactamente a cero.
- **Hiperparámetro:** decisión de configuración elegida por validación, no aprendida directamente como parámetro del modelo.

Ejercicios

Preguntas conceptuales

1. Explica por qué un polinomio de grado alto puede sobreajustar.
2. ¿Por qué no se debe escoger `lambda` con el conjunto de prueba?
3. ¿Qué diferencia conceptual hay entre Ridge y Lasso?
4. ¿Por qué la estandarización es especialmente importante antes de Lasso?
5. Da dos ejemplos de leakage en un proyecto real.

Problemas cuantitativos

6. Escribe el objetivo Ridge para un modelo con intercepto y dos variables.
7. Deriva el gradiente Ridge cuando no se penaliza el intercepto.

8. Para

$$\theta = (3, 4, -1),$$

calcula la penalización L2 y L1 sin incluir el intercepto.

9. Explica qué ocurre con la solución Ridge cuando `lambda -> infinity`.

Práctica computacional

10. Implementa `polynomial_features_1d(x, degree)`.
11. Implementa `ridge_closed_form(X, y, lam)` sin penalizar intercepto.
12. Divide un dataset en train/validation/test con semilla fija.
13. Ajusta scaler en train y aplícalo a validation/test.
14. Compara al menos cinco valores de `lambda`.
15. Grafica `mse_train` y `mse_val` contra `lambda`.

Interpretación y pensamiento crítico

16. Un modelo con `lambda=0` tiene MSE train 1.2 y MSE val 30.0; con `lambda=0.1` tiene MSE train 4.5 y MSE val 8.0. ¿Cuál prefieres y por qué?
17. Lasso deja una variable en cero. ¿Qué puedes afirmar y qué no puedes afirmar?
18. Si Ridge mejora validación pero empeora entrenamiento, ¿eso es necesariamente malo?

Proyecto de cierre

19. Diseña una comparación reproducible entre regresión polinomial sin regularización y Ridge. Usa una partición train/validation/test, reporta una tabla de resultados y escribe una conclusión que mencione explícitamente cómo evitaste leakage.

Capítulo 4: Regresión logística

Clasificación binaria, log-loss, métricas y umbrales

Pregunta aplicada

¿Cómo convertimos datos en una probabilidad de clase positiva y luego en una decisión defendible bajo costos de error?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. formular clasificación binaria con probabilidades;
2. explicar la función sigmoide;
3. interpretar log-odds;
4. escribir la log-loss binaria;
5. derivar el gradiente de la pérdida logística;
6. calcular matriz de confusión, accuracy, precision, recall y F1;
7. elegir un umbral según costos de decisión;
8. comparar un clasificador contra un baseline.

La regresión logística se llama “regresión”, pero se usa para clasificación. La idea central es modelar una probabilidad:

$$P(y = 1 \mid x).$$

Con la notación del Interludio 0.5, escribiremos con más cuidado $P(Y = 1 \mid X = x)$: la probabilidad de que la variable aleatoria objetivo tome la clase positiva para una observación con características x . En código y tablas seguiremos usando y , pero la interpretación probabilística corresponde a la variable Y .

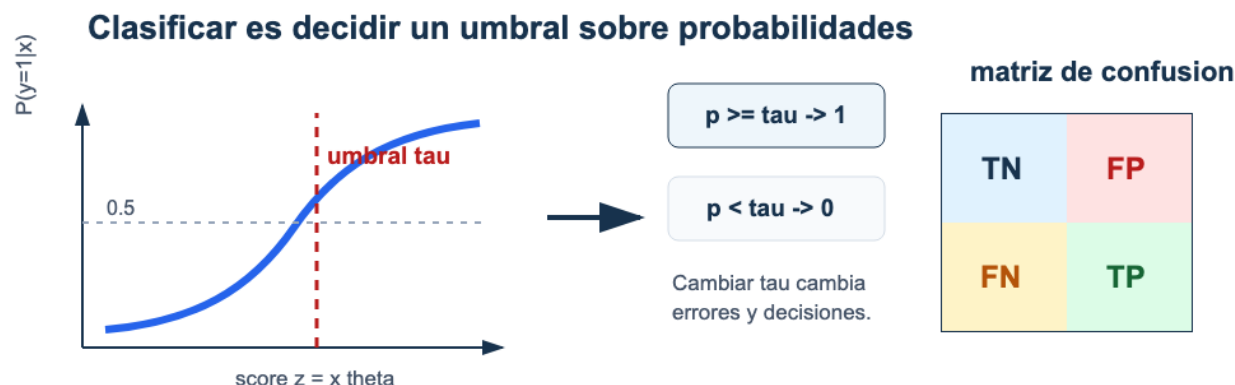


Figura 7: Regresión logística: la curva sigmoide produce probabilidades; un umbral convierte probabilidades en clases y modifica la matriz de confusión.

Lectura de la figura. El modelo y la decisión son dos pasos distintos. La sigmoide produce probabilidades; el umbral τ convierte esas probabilidades en clases y cambia los errores observados.

Clasificación binaria

En clasificación binaria, la variable objetivo toma dos valores:

$$y_i \in \{0, 1\}.$$

Ejemplos:

Contexto	Clase positiva 1
diagnóstico	paciente con condición
fraude	transacción fraudulenta
retención	cliente abandona
calidad	producto defectuoso
admisión	solicitud aprobada

La elección de cuál clase es positiva no es neutral. Las métricas y los costos se interpretan respecto a esa clase.

Criterio aplicado. Antes de calcular métricas, declara qué significa la clase positiva y por qué importa.

La función sigmoide

La regresión lineal produce valores en toda la recta real. Para modelar probabilidades necesitamos valores entre 0 y 1. Usamos:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Propiedades:

- si z es grande positivo, $\sigma(z)$ se acerca a 1;
- si z es grande negativo, $\sigma(z)$ se acerca a 0;
- si $z=0$, $\sigma(z)=0.5$.

```
def sigmoid(z):  
    z = np.asarray(z, dtype=float)  
    out = np.empty_like(z)  
    pos = z >= 0  
    out[pos] = 1 / (1 + np.exp(-z[pos]))  
    exp_z = np.exp(z[~pos])  
    out[~pos] = exp_z / (1 + exp_z)  
    return out
```

La implementación anterior evita overflow numérico cuando z es muy negativo o muy positivo.

Modelo logístico

Definimos:

$$z_i = x_i^\top \theta.$$

La probabilidad estimada es:

$$\hat{p}_i = P(y_i = 1 \mid x_i) = \sigma(x_i^\top \theta).$$

En forma vectorizada:

$$\hat{p} = \sigma(X\theta).$$

```
z = X @ theta  
p_hat = sigmoid(z)
```

Log-odds

La regresión logística es lineal en los log-odds:

$$\log\left(\frac{p}{1-p}\right) = x^\top \theta.$$

Esto significa que un aumento de una unidad en $\mathbf{x}_j$, manteniendo lo demás constante, aumenta los log-odds en θ_j .

Si exponenciamos:

$$e^{\theta_j}$$

es el factor multiplicativo sobre los odds asociado con una unidad adicional de $\mathbf{x}_j$, manteniendo las demás variables constantes.

Cuidado interpretativo. Los coeficientes logísticos no son cambios directos en probabilidad. Son cambios en log-odds. La variación en probabilidad depende del punto donde se evalúe.

Por qué no usamos MSE

Podríamos intentar ajustar probabilidades con MSE, pero la pérdida natural para clasificación probabilística es la log-loss. Viene de máxima verosimilitud para una variable Bernoulli.

Si $y_i=1$, queremos que p_i sea grande. Si $y_i=0$, queremos que $1-p_i$ sea grande. La verosimilitud de una observación es:

$$p_i^{y_i} (1 - p_i)^{1-y_i}.$$

La log-verosimilitud promedio es:

$$\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

Como optimizamos minimizando, usamos la negativa:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Definición 4.1: log-loss binaria.

La log-loss penaliza probabilidades confiadas en la dirección equivocada. Una predicción $p=0.99$ cuando $y=0$ recibe una penalización mucho mayor que una predicción $p=0.55$ cuando $y=0$.

Implementación estable:

```
def log_loss_binary(y, p, eps=1e-15):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

Gradiente de la pérdida logística

Para regresión logística:

$$\hat{p} = \sigma(X\theta).$$

La pérdida es:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

El gradiente tiene una forma sorprendentemente parecida al de regresión lineal:

$$\nabla J(\theta) = \frac{1}{n} X^\top (\hat{p} - y).$$

Proposición 4.1: gradiente logístico.

Para regresión logística binaria con log-loss, el gradiente respecto a **theta** es

$$\nabla J(\theta) = \frac{1}{n} X^\top (\sigma(X\theta) - y).$$

Implementación:

```
def logistic_gradient(X, y, theta):
    p = sigmoid(X @ theta)
    return (X.T @ (p - y)) / X.shape[0]
```

De probabilidades a clases

El modelo produce probabilidades. Para tomar una decisión binaria usamos un umbral:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases}$$

El valor común es **tau=0.5**, pero no siempre es el mejor.

```
def predict_class(p, threshold=0.5):
    return (p >= threshold).astype(int)
```

Matriz de confusión

	Predicho 1	Predicho 0
Real 1	TP	FN
Real 0	FP	TN

Donde:

- TP: verdaderos positivos;
- TN: verdaderos negativos;
- FP: falsos positivos;
- FN: falsos negativos.

```
def confusion_counts(y_true, y_pred):  
    y_true = np.asarray(y_true)  
    y_pred = np.asarray(y_pred)  
    tp = np.sum((y_true == 1) & (y_pred == 1))  
    tn = np.sum((y_true == 0) & (y_pred == 0))  
    fp = np.sum((y_true == 0) & (y_pred == 1))  
    fn = np.sum((y_true == 1) & (y_pred == 0))  
    return tp, fp, tn, fn
```

Métricas

Accuracy:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$\frac{TP}{TP + FP}$$

Recall:

$$\frac{TP}{TP + FN}$$

F1:

$$2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

```
def classification_metrics(y_true, y_pred):
    tp, fp, tn, fn = confusion_counts(y_true, y_pred)
    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = (
        2 * precision * recall / (precision + recall)
        if (precision + recall) else 0.0
    )
    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

Ejemplo resuelto: umbral y matriz de confusión

Supón seis observaciones con etiquetas reales:

$$y = (1, 0, 1, 0, 1, 0)$$

y probabilidades predichas:

$$\hat{p} = (0.92, 0.61, 0.55, 0.40, 0.20, 0.10).$$

Con umbral $\tau = 0.5$:

$$\hat{y} = (1, 1, 1, 0, 0, 0).$$

Los conteos son:

- $TP = 2$: observaciones 1 y 3;
- $FP = 1$: observación 2;
- $TN = 2$: observaciones 4 y 6;
- $FN = 1$: observación 5.

Por tanto:

$$\text{accuracy} = \frac{2+2}{6} = 0.667, \quad \text{precision} = \frac{2}{2+1} = 0.667, \quad \text{recall} = \frac{2}{2+1} = 0.667.$$

Como precision y recall son iguales, $F1 = 0.667$.

Si subimos el umbral a $\tau = 0.7$, entonces:

$$\hat{y} = (1, 0, 0, 0, 0, 0).$$

Ahora $TP = 1$, $FP = 0$, $TN = 3$, $FN = 2$. La precision sube a 1.0, pero recall baja a 1/3. El modelo no cambió; cambió la regla de decisión.

Elegir métricas según la decisión

No hay una métrica universal.

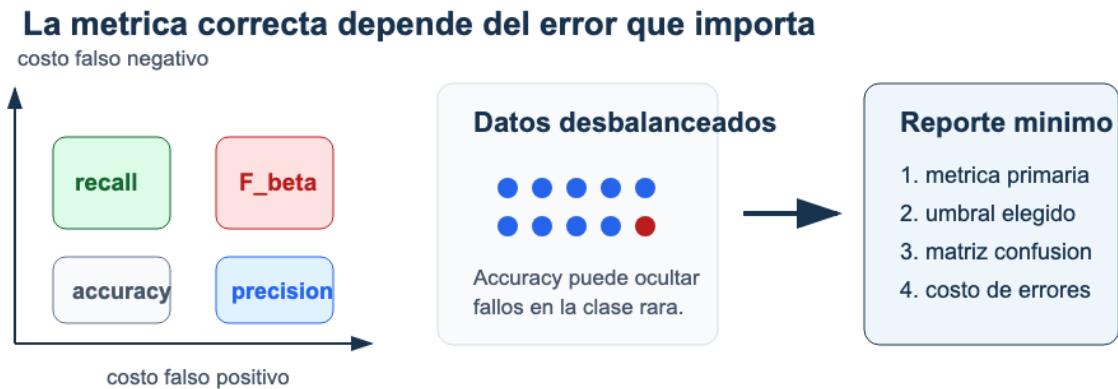


Figura 8: Selección de métricas: el costo relativo de falsos positivos y falsos negativos orienta si conviene mirar accuracy, precisión, recall o una métrica F beta.

Lectura de la figura. La métrica traduce el costo de equivocarse. Si el falso negativo y el falso positivo no tienen el mismo impacto, accuracy rara vez basta como criterio principal.

Situación	Riesgo principal	Métrica prioritaria
enfermedad grave	falso negativo	recall
fraude con revisión humana costosa	falso positivo	precision
clases balanceadas y costos similares	error global	accuracy
balance precision/recall	ambos errores importan	F1
ranking de candidatos	calidad del orden	ROC-AUC o PR-AUC

Criterio aplicado. La métrica se elige antes de mirar cuál modelo gana. Si eliges la métrica después, puedes estar optimizando la historia que te conviene.

Umbrales

Cambiar el umbral cambia la matriz de confusión.

- Umbral bajo: más positivos predichos, mayor recall, menor precision.
- Umbral alto: menos positivos predichos, mayor precision, menor recall.

Ejemplo de función para elegir el umbral más alto que mantiene un recall mínimo:

```
def choose_threshold_for_recall(y_true, p, min_recall=0.90):
    candidates = np.sort(np.unique(p))
    best = None
    for threshold in candidates:
        y_pred = (p >= threshold).astype(int)
        metrics = classification_metrics(y_true, y_pred)
        if metrics["recall"] >= min_recall:
            best = threshold
    return best
```

La lógica depende del problema. En algunos contextos preferimos el umbral que maximiza F1; en otros imponemos un recall mínimo o un máximo de falsos positivos.

Baselines de clasificación

Baselines comunes:

1. predecir siempre la clase mayoritaria;
2. predecir con frecuencia igual a la prevalencia;
3. regla simple basada en una variable fuerte;
4. modelo logístico sin features complejas.

Si el dataset tiene 95 % de clase 0, un clasificador que siempre predice 0 tiene 95 % de accuracy. Por eso accuracy puede ser engañosa en clases desbalanceadas.

Calibración

Un modelo está calibrado si, entre observaciones con probabilidad predicha cercana a 0.8, aproximadamente 80 % pertenecen a la clase positiva.

La calibración es distinta de la clasificación. Un modelo puede ordenar bien pero producir probabilidades mal calibradas.

En este módulo basta con recordar:

- la salida logística se interpreta como probabilidad bajo supuestos del modelo;
- esa probabilidad debe evaluarse empíricamente;
- el umbral transforma probabilidad en decisión.

Errores comunes

1. Tratar probabilidades como clases sin declarar umbral.
2. Usar accuracy en clases muy desbalanceadas sin mirar precision/recall.
3. Elegir umbral con el conjunto de prueba.
4. No definir la clase positiva.
5. Confundir coeficientes logísticos con cambios lineales en probabilidad.
6. Reportar F1 sin explicar el costo de FP y FN.
7. Comparar modelos usando umbrales distintos sin justificar.

Checklist de estudio

- ¿Puedes explicar por qué usamos sigmoide?
- ¿Puedes escribir la log-loss?
- ¿Puedes derivar o reconocer el gradiente logístico?
- ¿Puedes calcular TP, FP, TN, FN?
- ¿Puedes distinguir precision y recall?
- ¿Puedes justificar un umbral?
- ¿Puedes identificar cuándo accuracy engaña?

Para explorar más

- **Extensión matemática:** deriva el gradiente de la log-loss para una observación y luego generaliza a forma matricial.
- **Extensión computacional:** dibuja precisión, recall y F1 como funciones del umbral; identifica si existe un rango estable de decisión.
- **Conexión con proyecto:** escribe cuál error es más costoso en tu problema y justifica el umbral desde esa asimetría.

Revisión del capítulo

La regresión logística modela probabilidades para clasificación binaria. La función sigmoide transforma puntajes lineales en valores entre 0 y 1, y la log-loss penaliza predicciones probabilísticas incompatibles con la etiqueta observada.

El gradiente logístico tiene una forma compacta, $X^T(\hat{p} - y)/n$, que conecta de nuevo matrices, pérdida y optimización. La clasificación final no aparece hasta elegir un umbral, y ese umbral cambia la matriz de confusión.

Por eso la evaluación debe ir más allá de accuracy. Precision, recall, F1 y la selección de umbral dependen del costo relativo de falsos positivos y falsos negativos. Una recomendación técnica debe declarar ese costo o, al menos, el riesgo que está priorizando.

Términos clave

- **Clase positiva:** evento o condición que se codifica como 1 y define la interpretación de métricas.
- **Sigmoide:** función que transforma un score real en un valor entre 0 y 1.
- **Log-odds:** logaritmo de la razón $p/(1-p)$.
- **Log-loss:** pérdida que penaliza probabilidades equivocadas, especialmente si son confiadas.
- **Matriz de confusión:** conteo de verdaderos/falsos positivos y negativos.
- **Precision:** proporción de predicciones positivas que fueron correctas.
- **Recall:** proporción de positivos reales detectados.
- **Umbral:** regla que convierte una probabilidad o score en clase.
- **Calibración:** correspondencia entre probabilidades predichas y frecuencias observadas.

Ejercicios

Preguntas conceptuales

1. ¿Por qué regresión logística no produce directamente una clase?
2. Explica qué significa $p=0.8$ para una observación.
3. ¿Por qué una predicción muy confiada y equivocada recibe alta log-loss?
4. ¿Qué diferencia hay entre precision y recall?
5. Da un ejemplo donde prefieras recall sobre precision.

Problemas cuantitativos

6. Calcula $\text{sigma}(0)$, $\text{sigma}(2)$ y $\text{sigma}(-2)$ aproximadamente.
7. Si $TP=40$, $FP=10$, $TN=80$, $FN=20$, calcula accuracy, precision, recall y F1.
8. Para $y=1$ y $p=0.9$, calcula la contribución a log-loss. Repite con $p=0.1$.
9. Si $\text{theta}_j=0.7$, ¿cuánto se multiplican los odds por una unidad adicional de x_j ?

Práctica computacional

10. Implementa `sigmoid(z)` de forma estable.
11. Implementa `log_loss_binary(y, p)`.
12. Implementa `logistic_gradient(X, y, theta)`.
13. Implementa `classification_metrics(y_true, y_pred)`.
14. Evalúa métricas para umbrales 0.2, 0.5 y 0.8.
15. Elige un umbral que mantenga recall al menos 0.95.

Interpretación y pensamiento crítico

16. Un modelo tiene accuracy 0.94, pero recall de la clase positiva 0.20. ¿Qué puede estar pasando?
17. En fraude, subir el umbral aumenta precision y baja recall. ¿Cómo explicarías el tradeoff a una persona no técnica?
18. Si dos modelos tienen F1 similar, pero uno tiene mejor calibración, ¿en qué situación preferirías el calibrado?

Proyecto de cierre

19. Entrena o simula un clasificador binario con probabilidades. Evalúa tres umbrales, reporta matriz de confusión, precision, recall y F1, y recomienda un umbral para un contexto donde el falso negativo sea más costoso que el falso positivo.

Ejercicios integradores: regresión, optimización y clasificación

Práctica para laboratorios, parcial y proyecto

Cómo usar este banco

Este capítulo contiene ejercicios para estudiar el Módulo 1 de forma activa. No están pensados para resolverse todos de una vez. Usa esta ruta:

1. resuelve primero los ejercicios conceptuales de cada bloque;
2. pasa a cálculo manual;
3. implementa funciones pequeñas;
4. cierra con interpretación aplicada;
5. compara tus respuestas con el apéndice solo después de intentar.

Regla de práctica. Si puedes ejecutar código pero no puedes explicar el objeto matemático que estás calculando, todavía falta estudio. Si puedes derivar una fórmula pero no puedes implementarla, todavía falta práctica computacional.

Resultados de práctica

Al terminar este banco deberías poder:

1. representar regresión lineal y logística con matrices;
2. calcular pérdidas, gradientes y métricas en casos pequeños;
3. implementar mínimos cuadrados, descenso del gradiente y Ridge;
4. diagnosticar underfitting, overfitting y leakage;
5. justificar regularización, métrica y umbral con evidencia.

Mapa del banco

Bloque	Propósito
Semanas 2-3	álgebra matricial, mínimos cuadrados y optimización
Semanas 4-5	validación, features, Ridge, Lasso y selección
Semana 6	clasificación logística, métricas y umbrales
Problemas integradores	síntesis para laboratorio, parcial y proyecto

Semana 2: regresión lineal

A. Dimensiones y representación

1. Tienes un dataset con $n=250$ observaciones y $d=6$ variables originales. Si agregas intercepto, ¿cuál es la forma de X y de θ ?
2. Explica qué significa cada fila y cada columna de la matriz de diseño.
3. Escribe la predicción de la observación i usando sumatoria.
4. Escribe la misma predicción usando producto punto.
5. ¿Qué error esperas si θ tiene forma $(d,)$ pero X incluye intercepto?

B. Mínimos cuadrados

6. Para

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix},$$

calcula $X.T @ X$ y $X.T @ y$.

7. Resuelve la ecuación normal del ejercicio anterior.
8. Calcula las predicciones y los residuales.
9. Calcula el MSE.
10. Compara contra el baseline que predice el promedio de y .

C. Implementación

11. Escribe una función `add_intercept(X)`.
12. Escribe una función `predict_linear(X, theta)`.
13. Escribe una función `mse_loss(X, y, theta)`.
14. Escribe una función `normal_equation(X, y)`.
15. Simula $y = 5 + 0.8x + \text{ruido}$ y verifica si recuperas parámetros cercanos.

D. Interpretación

16. Tu modelo mejora el baseline en entrenamiento, pero no en prueba. Da tres posibles explicaciones.
17. Un coeficiente estimado es negativo. ¿Qué significa y qué no significa?
18. ¿Por qué debes revisar unidades antes de comparar coeficientes?

Semana 3: descenso del gradiente

A. Gradiente

1. Escribe la pérdida $J(\mathbf{\theta})$ para mínimos cuadrados.
2. Deriva el gradiente usando expansión matricial.
3. Deriva el gradiente usando residuales.
4. Explica por qué el gradiente tiene la misma dimensión que $\mathbf{\theta}$.
5. ¿Qué significa que el gradiente sea cero?

B. Iteraciones

6. Con

$$\theta^{(0)} = (0, 0), \quad \alpha = 0.1,$$

ejecuta dos pasos manuales para el dataset:

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

7. ¿Qué pasa si duplicas todas las columnas de X excepto el intercepto?
8. ¿Por qué la tasa de aprendizaje estable puede cambiar?

C. Diagnóstico

9. Describe qué harías si la pérdida aumenta en las primeras iteraciones.
10. Describe qué harías si la pérdida baja, pero muy lentamente.
11. Describe qué harías si la pérdida de entrenamiento baja y la de validación sube.
12. ¿Cómo usarías la ecuación normal para probar tu implementación de gradiente?

D. Código

13. Implementa `mse_gradient(X, y, theta)`.
14. Implementa `gradient_descent`.
15. Guarda `history` y grafica pérdida contra iteración.
16. Compara `theta_gd` contra `theta_ne`.
17. Repite con datos sin escalar y escalados.

Semana 4: features y validación

A. Complejidad

1. Explica por qué un modelo polinomial de grado 1 puede subajustar.
2. Explica por qué un modelo polinomial de grado 20 puede sobreajustar.
3. ¿Qué patrón esperas en errores train/validation para cada caso?
4. ¿Qué tipo de problema no se arregla solo aumentando el grado?

B. Particiones

5. Define train, validation y test en una frase cada uno.
6. ¿Por qué no se debe usar test para escoger grado?
7. ¿Qué harías si solo tienes 80 observaciones?
8. Da un ejemplo de leakage temporal.

C. Feature engineering

9. Construye manualmente features para x : 1, x , x^2 , x^3 .
10. Para dos variables x_1 , x_2 , propone tres interacciones posibles.
11. ¿Cuándo usarías $\log(x)$?
12. ¿Qué problema aparece si x puede ser cero o negativo?

D. Código

13. Implementa `polynomial_features_1d`.
14. Entrena grados 1 a 10.
15. Grafica MSE train y MSE validation contra grado.
16. Escoge un grado y justifica la elección.

Semana 5: regularización

A. Ridge

1. Escribe el objetivo Ridge.
2. Explica por qué no se suele penalizar el intercepto.
3. Deriva la solución cerrada cuando se penalizan todos los coeficientes.
4. Modifica la solución para no penalizar intercepto.
5. Explica qué ocurre cuando `lambda=0`.

B. Lasso

6. Escribe el objetivo Lasso.
7. ¿Por qué Lasso puede producir coeficientes exactamente cero?
8. ¿Qué precaución tomarías con variables correlacionadas?
9. ¿Por qué Lasso no prueba causalidad?

C. Selección

10. Diseña una tabla para comparar `lambda`, MSE train, MSE validation y norma de coeficientes.
11. ¿Qué `lambda` escogerías si el menor MSE validation está empatado entre dos valores?
12. ¿Qué evidencia reportarías para defender el modelo final?

D. Código

13. Implementa `ridge_closed_form`.
14. Compara `lambda` en escala logarítmica.
15. Grafica norma de coeficientes contra `lambda`.
16. Compara con `sklearn.linear_model.Ridge`.
17. Si usas Lasso de `sklearn`, revisa cuántos coeficientes quedan en cero.

Semana 6: clasificación logística

A. Probabilidades

1. Explica por qué una regresión lineal no es ideal para probabilidades.
2. Grafica mentalmente la sigmoide.
3. ¿Qué significa $\sigma(z)=0.5$?
4. ¿Qué significa log-odds?

B. Log-loss

5. Escribe la log-loss binaria.
6. Calcula la pérdida para $y=1$, $p=0.9$.
7. Calcula la pérdida para $y=1$, $p=0.1$.
8. Explica por qué la segunda es mayor.
9. Deriva el gradiente $X.T @ (p - y) / n$.

C. Métricas

10. Con TP=25, FP=5, TN=60, FN=10, calcula accuracy, precision, recall y F1.
11. Explica cuándo accuracy puede engañar.
12. Da un caso donde un falso negativo sea más costoso que un falso positivo.
13. Da un caso donde un falso positivo sea más costoso que un falso negativo.

D. Umbrales

14. Si bajas el umbral, ¿qué suele pasar con recall?
15. Si subes el umbral, ¿qué suele pasar con precision?
16. Diseña una regla de umbral para un sistema de alerta médica.
17. Diseña una regla de umbral para revisión manual de fraude con capacidad limitada.

Problemas integradores

Problema 1: regresión de principio a fin

Toma un dataset de regresión, por ejemplo `load_diabetes`.

1. Separa train/test.
2. Define un baseline de promedio.
3. Ajusta regresión lineal.
4. Reporta MSE train y test.
5. Estandariza variables y repite.
6. Ajusta Ridge con varios `lambda`.
7. Escoge un modelo con validación.
8. Escribe una conclusión técnica de máximo 150 palabras.

La conclusión debe responder:

- ¿mejora el baseline?
- ¿cuál métrica usaste?
- ¿qué limitación principal observas?
- ¿qué harías después?

Problema 2: regularización y estabilidad

Construye un dataset generado con variables correlacionadas:

```
rng = np.random.default_rng(42)
n = 200
x1 = rng.normal(size=n)
x2 = x1 + 0.05 * rng.normal(size=n)
x3 = rng.normal(size=n)
y = 2 + 3 * x1 + 0.5 * x3 + rng.normal(scale=0.5, size=n)
```

1. Ajusta regresión lineal.
2. Ajusta Ridge.
3. Ajusta Lasso.
4. Compara coeficientes.
5. Explica qué ocurre con `x1` y `x2`.

Problema 3: clasificación con umbral

Usa `load_breast_cancer` o un dataset binario similar.

1. Define clase positiva.
2. Separa train/test con estratificación.
3. Estandariza variables.
4. Ajusta regresión logística.
5. Reporta matriz de confusión para umbral 0.5.
6. Escoge un umbral para recall mínimo 0.95.
7. Explica el tradeoff en lenguaje no técnico.

Problema 4: diseño de parcial

Escribe una pregunta de parcial para cada tema:

1. forma matricial;
2. gradiente de MSE;
3. escalamiento;
4. Ridge;
5. log-loss;
6. precision/recall.

Para cada pregunta incluye:

- respuesta correcta;
- error común esperado;
- por qué la pregunta evalúa comprensión y no memoria.

Rúbrica de autoevaluación

Nivel	Evidencia
Inicial	puedo ejecutar notebooks siguiendo instrucciones
Básico	puedo completar funciones pequeñas con ayuda
Competente	puedo derivar fórmulas centrales e implementarlas
Sólido	puedo diagnosticar fallas y justificar decisiones
Excelente	puedo conectar matemática, código, métrica y contexto aplicado

Antes del Parcial 1, deberías estar al menos en nivel competente en todos los temas y sólido en los temas que uses en tu proyecto.

Guía de verificación seleccionada

Semana 2, ejercicio 1. Si hay $d = 6$ variables originales y se agrega intercepto, entonces $\mathbf{X}$ tiene forma $(250, 7)$ y $\mathbf{\theta}$ debe tener 7 entradas.

Semana 5, ejercicio 8. Para $\theta = (3, 4, -1)$, sin incluir intercepto, la penalización L2 usa $4^2 + (-1)^2 = 17$ y la penalización L1 usa $|4| + |-1| = 5$.

Semana 6, ejercicio 10. Con TP=25, FP=5, TN=60, FN=10, accuracy es $85/100 = 0.85$, precision es $25/30$, recall es $25/35$ y F1 se calcula como media armónica de precision y recall.

Parte III: Redes neuronales

Parte III: Redes neuronales

Esta parte construye redes neuronales desde primeros principios. Primero se estudia forward propagation, luego backpropagation como aplicación organizada de la regla de la cadena. Después se analizan activaciones, inicialización, regularización, optimizadores y PyTorch.

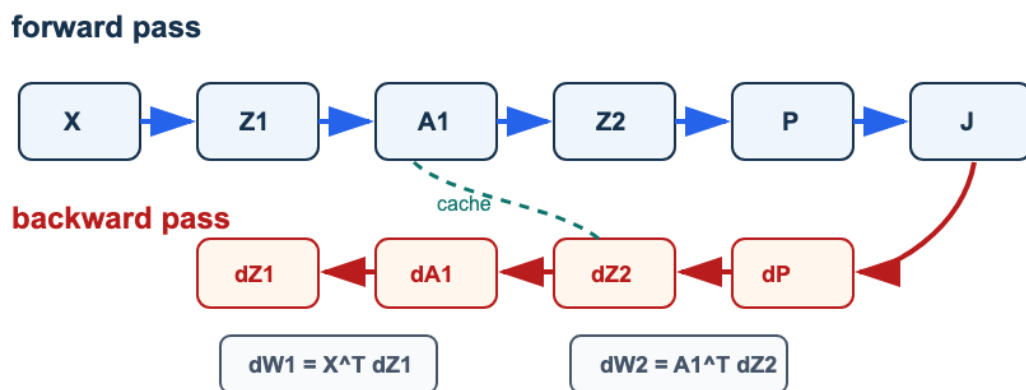


Figura 9: Backpropagation como grafo: el forward calcula activaciones y pérdida; el backward propaga gradientes usando la cache.

Lectura de la figura. Una red neuronal no se estudia como caja negra. Se estudia como grafo de cómputo: cada tensor tiene forma, cada operación produce un objeto y cada gradiente debe poder auditarse.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. implementar forward propagation para una red densa pequeña;
2. seguir las formas de pesos, sesgos, activaciones y logits;
3. derivar backpropagation para dos capas usando regla de la cadena;
4. diagnosticar saturación, overfitting y problemas de learning rate;
5. comparar una implementación NumPy con un loop PyTorch equivalente;
6. escribir un checkpoint técnico con evidencia, riesgo y siguiente experimento.

Mapa de la parte

Capítulo	Pregunta guía	Evidencia esperada
Perceptron y forward	¿Qué computa una red antes de aprender?	Tabla de formas y forward estable.
Backpropagation	¿Cómo viaja el gradiente por el grafo?	Derivación y verificación numérica.
Diagnóstico y regularización	¿Qué muestran las curvas de entrenamiento?	Diagnóstico con acción proporcional.
Optimizadores y PyTorch	¿Qué automatiza una librería y qué debe revisar el usuario?	Loop legible y comparación justa.
Ejercicios integradores	¿Puedes auditar una red de extremo a extremo?	Formas, gradientes, curva y conclusión.

Criterio de salida

La parte queda dominada cuando puedes entrenar una red pequeña, explicar cada objeto intermedio, verificar gradientes básicos, leer curvas de entrenamiento y evitar afirmar que una mejora numérica es suficiente sin validación.

Perceptron, neurona logística y forward propagation

Pregunta aplicada

Suponga que tenemos imágenes pequeñas de dígitos escritos a mano. Una regresión logística puede separar dos clases simples, pero ¿qué ocurre cuando la frontera entre clases requiere interacciones no lineales entre píxeles?

Una red neuronal densa permite aprender representaciones intermedias. La idea central es sencilla:

Una **red neuronal feedforward** es una composición de transformaciones lineales y funciones no lineales.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. reconocer una neurona logística como una capa lineal seguida de una activación;
2. escribir las dimensiones correctas de una capa densa;
3. explicar por qué una activación no lineal cambia la capacidad del modelo;
4. implementar softmax de forma estable;
5. interpretar entropía cruzada como pérdida para clasificación multiclase.

Perceptron como unidad básica

Dada una observación $x \in \mathbb{R}^p$, un perceptron calcula:

$$z = w^\top x + b.$$

Si se aplica una regla de decisión:

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0, \end{cases}$$

obtenemos un clasificador lineal.

Advertencia

El perceptron clásico produce una decisión dura. Para entrenar con gradientes resulta más conveniente usar funciones diferenciables y pérdidas suaves.

Neurona logística

La regresión logística ya puede verse como una neurona:

$$\hat{p} = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

La salida $\hat{p}$ se interpreta como probabilidad de la clase positiva. Esta interpretación conecta el Módulo 1 con redes neuronales:

`regresión logística = una capa lineal + sigmoide`

De una neurona a una capa

Una capa densa con h neuronas calcula:

$$Z^{[1]} = XW^{[1]} + b^{[1]},$$

donde:

- $X \in \mathbb{R}^{n \times p}$;
- $W^{[1]} \in \mathbb{R}^{p \times h}$;
- $b^{[1]} \in \mathbb{R}^h$;
- $Z^{[1]} \in \mathbb{R}^{n \times h}$.

Luego aplicamos una activación elemento a elemento:

$$A^{[1]} = g(Z^{[1]}).$$

Red de una capa oculta

Para clasificación multiclase con k clases:

$$Z^{[1]} = XW^{[1]} + b^{[1]},$$

$$A^{[1]} = \text{ReLU}(Z^{[1]}),$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]},$$

$$\hat{P} = \text{softmax}(Z^{[2]}).$$

ReLU

$$\text{ReLU}(z) = \max(0, z).$$

ReLU introduce no linealidad y evita parte de la saturación que aparece con sigmoides en redes profundas.

Su derivada por partes es:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Softmax

Para logits $z_1, \dots, z_k$:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_\ell}}.$$

En implementación usamos una versión estable:

```
shifted = Z - np.max(Z, axis=1, keepdims=True)
exp_scores = np.exp(shifted)
probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)
```

Restar el máximo no cambia las probabilidades, pero evita overflow.

Entropía cruzada multiclase

Si Y es one-hot y $\hat{P}$ son probabilidades:

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k Y_{ij} \log(\hat{P}_{ij}).$$

La pérdida premia asignar alta probabilidad a la clase correcta.

Ejemplo resuelto: softmax y pérdida

Antes del cálculo numérico, conviene entender por qué una activación no lineal importa. Supón dos capas lineales sin activación:

$$H = XW^{[1]} + b^{[1]}, \quad Z = HW^{[2]} + b^{[2]}.$$

Sustituyendo H :

$$Z = (XW^{[1]} + b^{[1]})W^{[2]} + b^{[2]} = X(W^{[1]}W^{[2]}) + b^{[1]}W^{[2]} + b^{[2]}.$$

Si definimos $W^* = W^{[1]}W^{[2]}$ y $b^* = b^{[1]}W^{[2]} + b^{[2]}$, entonces:

$$Z = XW^* + b^*.$$

Dos capas lineales sin activación equivalen a una sola capa lineal. La activación no lineal es lo que impide colapsar la red a un modelo lineal.

Supón que una observación produce tres logits:

$$z = (2, 1, 0).$$

Para calcular softmax de forma estable restamos el máximo, que es 2:

$$z - \max(z) = (0, -1, -2).$$

Entonces:

$$e^{z - \max(z)} = (1, e^{-1}, e^{-2}) \approx (1, 0.368, 0.135).$$

La suma es aproximadamente 1.503, así que:

$$\text{softmax}(z) \approx (0.665, 0.245, 0.090).$$

Si la clase correcta es la primera, la entropía cruzada para esta observación es:

$$-\log(0.665) \approx 0.408.$$

Si la clase correcta fuera la tercera, la pérdida sería:

$$-\log(0.090) \approx 2.408.$$

La misma predicción puede ser buena o mala según la etiqueta verdadera. Por eso no basta mirar logits grandes: hay que mirar la probabilidad asignada a la clase correcta.

Ejemplo resuelto: dimensiones

Suponga:

- $n = 100$ observaciones;
- $p = 64$ pixeles por imagen;
- $h = 32$ neuronas ocultas;
- $k = 10$ clases.

Entonces:

Objeto	Forma
X	100×64
$W^{[1]}$	64×32
$b^{[1]}$	32
$A^{[1]}$	100×32
$W^{[2]}$	32×10
$\hat{P}$	100×10

Caso longitudinal: dígitos escritos a mano

En los capítulos de redes usaremos como referencia conceptual el dataset `load_digits`: imágenes pequeñas de 8×8 pixeles, convertidas en vectores de 64 entradas. Este dataset no representa visión por computador moderna; su valor pedagógico es que permite ver todas las formas de una red pequeña sin depender de internet ni de archivos pesados.

Objeto	Forma típica	Interpretación
X	$n \times 64$	intensidad de pixeles aplanados
$W^{[1]}$	$64 \times h$	pesos de entrada a capa oculta
$A^{[1]}$	$n \times h$	representación intermedia
$W^{[2]}$	$h \times 10$	pesos hacia diez clases
$\hat{P}$	$n \times 10$	probabilidades por dígito

La primera comparación profesional no debe ser contra una red grande. Debe ser contra un baseline simple: clase mayoritaria, regresión logística o una red pequeña sin tuning agresivo. Si la red mejora el baseline, todavía falta revisar validación, errores por clase y reproducibilidad del entrenamiento.

Punto de control

Antes de pasar a backpropagation, verifica que puedes responder sin mirar la tabla:

1. Si una capa recibe 64 variables y produce 32 activaciones, ¿qué forma tiene su matriz de pesos?
2. Si una red produce 10 logits por observación, ¿por qué softmax se aplica por fila?

3. Si se quita ReLU, ¿qué tipo de frontera queda después de componer capas lineales?

Respuesta corta. La matriz tiene forma 64×32 . Softmax se aplica por fila porque cada fila contiene las probabilidades de una observación sobre las 10 clases. Sin activaciones no lineales, la composición de capas sigue siendo una transformación lineal.

Verificaciones seleccionadas

1. Si X tiene forma 50×64 y $W^{[1]}$ tiene forma 64×16 , entonces $Z^{[1]}$ tiene forma 50×16 .
2. Si una red multiclase produce logits 50×10 , la suma de softmax debe ser 1 en cada fila, no en cada columna.
3. Si se eliminan todas las activaciones no lineales, la red completa equivale a una transformación lineal; por eso no aumenta capacidad geométrica real.

Ejemplo resuelto: auditoría de formas en dos capas

Supón un lote $X \in \mathbb{R}^{32 \times 8}$, una capa oculta con 5 neuronas y una salida de 3 clases. Una implementación consistente debe declarar

$$W^{[1]} \in \mathbb{R}^{8 \times 5}, \quad b^{[1]} \in \mathbb{R}^5, \quad W^{[2]} \in \mathbb{R}^{5 \times 3}, \quad b^{[2]} \in \mathbb{R}^3.$$

Entonces $Z^{[1]} = XW^{[1]} + b^{[1]}$ tiene forma 32×5 , la activación $A^{[1]}$ conserva esa forma y los logits finales tienen forma 32×3 . Softmax se aplica sobre las 3 columnas de cada fila, porque cada fila representa una observación. Si softmax se aplicara por columna, se estarían normalizando observaciones distintas entre sí, lo cual no tiene interpretación como distribución de clases para un caso individual.

Esta auditoría debe hacerse antes de entrenar. Un notebook profesional imprime o verifica formas en puntos intermedios, porque un error de broadcasting puede producir un arreglo numérico sin representar el modelo matemático esperado.

Para explorar más

- **Extensión matemática:** calcula manualmente una pasada forward con dos observaciones, dos entradas y dos neuronas ocultas.
- **Extensión computacional:** implementa la misma red con `numpy` y luego con `PyTorch`; compara formas y valores intermedios.
- **Conexión con proyecto:** identifica si tu problema necesita una arquitectura no lineal o si un modelo lineal regularizado sigue siendo un baseline suficiente.

Revisión del capítulo

Este capítulo construye una red neuronal desde sus operaciones de forward pass. Una neurona combina una transformación lineal con una activación; una red densa apila esas operaciones para producir logits y probabilidades.

La no linealidad es esencial. Sin activaciones no lineales, varias capas lineales se pueden reescribir como una sola transformación lineal. ReLU introduce capacidad adicional y softmax convierte logits multiclase en una distribución de probabilidad por observación.

El criterio de dominio de este capítulo es la trazabilidad de formas. Antes de entrenar, debes poder decir la forma de W , b , Z , A , logits, probabilidades y etiquetas. Sin esa auditoría, los errores de broadcasting pueden parecer resultados válidos.

Términos clave

- **Perceptron:** unidad que combina entradas con pesos y sesgo para producir un score.
- **Capa densa:** transformación $XW + b$ aplicada a un lote de observaciones.
- **Activación:** función no lineal aplicada a los scores de una capa.
- **ReLU:** activación $\max(0, z)$ usada para introducir no linealidad.
- **Logit:** score real antes de convertirlo en probabilidad.
- **Softmax:** transformación que convierte logits multiclase en probabilidades por fila.
- **One-hot:** codificación de clases como vectores indicadores.
- **Entropía cruzada:** pérdida que penaliza baja probabilidad asignada a la clase correcta.

Ejercicios

Preguntas conceptuales

1. Explique por qué una red sin activaciones no lineales se reduce a una transformación lineal.
2. ¿Qué diferencia hay entre logits y probabilidades?

Problemas cuantitativos

3. Si X tiene forma $(250, 20)$ y una capa oculta tiene 12 neuronas, ¿qué forma deben tener $W1$, $b1$ y $Z1$?
4. Para una salida con 5 clases y batch de 32 observaciones, indique la forma de los logits, las probabilidades softmax y la matriz one-hot Y .

Práctica computacional

5. Implemente una función `softmax_stable(Z)` y verifique que cada fila suma 1.
6. Implemente un `forward_pass(X, params)` para una red con una capa oculta ReLU y salida softmax.

Interpretación y pensamiento crítico

7. Una red obtiene probabilidades casi uniformes para todas las clases. Proponga dos hipótesis técnicas.

Proyecto de cierre

8. Use un dataset multiclase generado con semilla fija o una muestra pequeña de imágenes. Defina dimensiones, implemente forward propagation y entregue una tabla con formas de cada objeto intermedio.

Backpropagation

Pregunta aplicada

Forward propagation nos dice cómo una red produce predicciones. Pero entrenar exige responder:

¿Cómo cambia la pérdida si modificamos cada peso de la red?

Backpropagation es una forma eficiente de aplicar la regla de la cadena a una composición de funciones.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. leer una red como grafo de cómputo;
2. derivar las formas de $dW^{[2]}$, $db^{[2]}$, $dW^{[1]}$ y $db^{[1]}$;
3. explicar por qué softmax con entropía cruzada produce $dZ^{[2]} = (\hat{P} - Y)/n$;
4. usar una cache de forward para calcular backward;
5. diseñar una prueba básica de gradientes.

El grafo de cómputo

Para una red de una capa oculta:

- entrada X ;
- preactivación Z_1 ;
- activación A_1 ;
- logits Z_2 ;
- probabilidades P ;
- pérdida J .

Cada flecha representa una operación diferenciable o casi diferenciable. Backpropagation recorre el grafo en sentido inverso:

- primero J ;
- luego P , Z_2 , A_1 y Z_1 ;
- finalmente los parámetros.

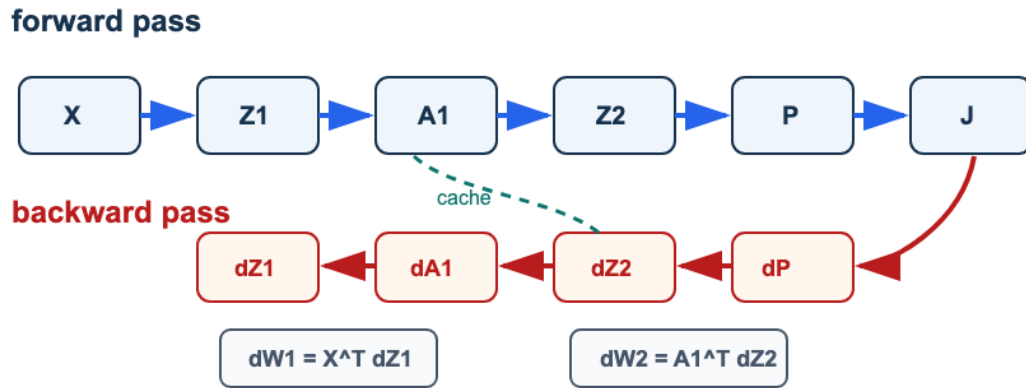


Figura 10: Backpropagation como grafo: el forward calcula X , $Z1$, $A1$, $Z2$, P y J ; el backward propaga dJ hacia $dZ2$, $dW2$, $dA1$, $dZ1$ y $dW1$ usando la cache.

Lectura de la figura. Forward produce valores intermedios; backward reutiliza esa cache para construir gradientes. Si una forma no coincide con el parámetro correspondiente, el error está en el recorrido del grafo o en una transposición.

Gradiente de softmax con entropía cruzada

Para softmax + entropía cruzada, el gradiente se simplifica:

$$\frac{\partial J}{\partial Z^{[2]}} = \frac{1}{n}(\hat{P} - Y).$$

Esta fórmula es una de las razones por las que esta combinación es estándar en clasificación multiclase.

Gradientes de la segunda capa

Con:

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]},$$

definimos:

$$dZ^{[2]} = \frac{1}{n}(\hat{P} - Y).$$

Entonces:

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]},$$

$$db^{[2]} = \sum_i dZ_i^{[2]}.$$

Propagar hacia la capa oculta

Primero pasamos el gradiente hacia las activaciones:

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^\top.$$

Luego atravesamos ReLU:

$$dZ^{[1]} = dA^{[1]} \odot \mathbf{1}\{Z^{[1]} > 0\}.$$

Finalmente:

$$dW^{[1]} = X^\top dZ^{[1]},$$

$$db^{[1]} = \sum_i dZ_i^{[1]}.$$

Backpropagation como algoritmo

La identidad de salida merece una derivación corta. Para una observación, la entropía cruzada multiclase es:

$$L = -\sum_{j=1}^k y_j \log p_j, \quad p_j = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_\ell}}.$$

Cuando y es one-hot, solo una clase c tiene $y_c = 1$, de modo que $L = -\log p_c$. Al derivar con respecto a un logit z_j , el resultado compacto es:

$$\frac{\partial L}{\partial z_j} = p_j - y_j.$$

Para un lote de n observaciones y pérdida promedio:

$$dZ^{[2]} = \frac{\hat{P} - Y}{n}.$$

Esta identidad explica por qué el código de salida es tan breve y por qué la forma de $dZ^{[2]}$ coincide con la forma de los logits.

```
def backward_pass(X, Y, cache, params):
    P = cache["P"]
    A1 = cache["A1"]
    Z1 = cache["Z1"]
    W2 = params["W2"]

    dZ2 = (P - Y) / len(X)
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0)

    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

Verificación por diferencias finitas

Una forma de detectar errores en backpropagation es comparar contra una aproximación numérica:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta_j + \varepsilon) - J(\theta_j - \varepsilon)}{2\varepsilon}.$$

Esta técnica no se usa para entrenar porque es costosa, pero es excelente para depurar.

Ejemplo resuelto: diferencia finita escalar

Considera la función simple:

$$J(w) = (w - 3)^2.$$

Su derivada exacta es:

$$J'(w) = 2(w - 3).$$

En $w = 2$, la derivada exacta es:

$$J'(2) = 2(2 - 3) = -2.$$

Ahora usa diferencia finita central con $\varepsilon = 0.01$:

$$\frac{J(2 + \varepsilon) - J(2 - \varepsilon)}{2\varepsilon} = \frac{J(2.01) - J(1.99)}{0.02}.$$

Calculamos:

$$J(2.01) = (-0.99)^2 = 0.9801, \quad J(1.99) = (-1.01)^2 = 1.0201.$$

Por tanto:

$$\frac{0.9801 - 1.0201}{0.02} = -2.$$

La aproximación coincide con la derivada exacta. En una red real no esperamos igualdad perfecta, pero sí una diferencia pequeña. Si la diferencia es grande, hay que revisar signos, escalas, transpuestas o divisiones por n .

Ejemplo resuelto: formas de los gradientes

Supón que:

- $X \in \mathbb{R}^{100 \times 64}$;
- $W^{[1]} \in \mathbb{R}^{64 \times 32}$;
- $A^{[1]} \in \mathbb{R}^{100 \times 32}$;
- $W^{[2]} \in \mathbb{R}^{32 \times 10}$;
- $\hat{P}, Y \in \mathbb{R}^{100 \times 10}$.

Entonces:

Gradiente	Cálculo	Forma
$dZ^{[2]}$	$(\hat{P} - Y)/100$	100×10
$dW^{[2]}$	$(A^{[1]})^\top dZ^{[2]}$	32×10
$db^{[2]}$	suma por filas de $dZ^{[2]}$	10
$dA^{[1]}$	$dZ^{[2]}(W^{[2]})^\top$	100×32
$dW^{[1]}$	$X^\top dZ^{[1]}$	64×32

El chequeo profesional es simple: cada gradiente debe tener la misma forma que el parámetro que actualiza.

Errores frecuentes

1. Olvidar dividir por n .
2. Confundir $Y - P$ con $P - Y$.
3. Sumar `db` sin `axis=0`.
4. Usar las formas transpuestas incorrectas.
5. Recalcular forward dentro de backward sin guardar cache.
6. No estabilizar softmax.

Punto de control

Si el autograder dice que `dW2` tiene forma incorrecta, no empieces cambiando signos. Primero revisa:

1. ¿`A1.T @ dZ2` produce la misma forma que `W2`?
2. ¿`db2` suma sobre observaciones y no sobre clases?
3. ¿dividiste por n exactamente una vez?

Caso longitudinal: backpropagation en dígitos

En el caso `load_digits`, el forward produce probabilidades para diez clases. El backward debe producir gradientes con las mismas formas que los parámetros:

Parámetro	Forma si $h = 32$	Gradiente esperado
$W^{[1]}$	64×32	$dW^{[1]}$, misma forma
$b^{[1]}$	32	$db^{[1]}$, una entrada por neurona
$W^{[2]}$	32×10	$dW^{[2]}$, misma forma
$b^{[2]}$	10	$db^{[2]}$, una entrada por clase

La prueba de gradiente no se hace sobre todo el dataset. Se hace sobre un caso pequeño, con pocas observaciones y parámetros controlados, porque ahí se puede comparar contra diferencias finitas. Una red que “entrena” sin pasar una prueba de gradiente puede estar reduciendo la pérdida por accidente o por un error de implementación no detectado.

Verificaciones seleccionadas

1. Si $A^{[1]}$ tiene forma $n \times h$ y $dZ^{[2]}$ tiene forma $n \times k$, entonces $dW^{[2]} = (A^{[1]})^\top dZ^{[2]}$ tiene forma $h \times k$.
2. Una diferencia finita central usa $(J(\theta + \epsilon) - J(\theta - \epsilon))/(2\epsilon)$; suele ser más estable que usar solo $J(\theta + \epsilon) - J(\theta)$.
3. Un error de forma que NumPy resuelve con broadcasting puede producir un valor numérico sin representar el gradiente correcto.

Para explorar más

- **Extensión matemática:** verifica por diferencias finitas un solo peso de la primera capa y un solo peso de la segunda capa.
- **Extensión computacional:** agrega chequeos automáticos de forma para cada gradiente antes de actualizar parámetros.
- **Conexión con proyecto:** decide qué prueba mínima de gradientes o sanity check usarías antes de confiar en un entrenamiento propio.

Revisión del capítulo

Backpropagation aplica la regla de la cadena de forma organizada sobre un grafo de cómputo. El forward produce activaciones, logits, probabilidades y pérdida; el backward usa esos valores guardados para calcular gradientes.

La simplificación $dZ^{[2]} = (\hat{P} - Y)/n$ para softmax con entropía cruzada permite propagar gradientes hacia la segunda capa y luego hacia la capa oculta. Cada gradiente debe tener exactamente la misma forma que el parámetro que actualiza.

La verificación por diferencias finitas no reemplaza la derivación, pero sí detecta errores de signo, escala o forma. Una red que no mejora aunque sus formas sean correctas exige revisar inicialización, activaciones, tasa de aprendizaje y datos.

Términos clave

- **Grafo de cómputo:** representación de operaciones intermedias usadas para calcular una salida.
- **Cache:** valores guardados durante forward propagation para reutilizarlos en backward propagation.
- **Backpropagation:** aplicación eficiente de la regla de la cadena desde la pérdida hacia los parámetros.
- **Gradiente:** derivada de la pérdida respecto a un parámetro o activación.
- **Diferencias finitas:** aproximación numérica de derivadas para verificar gradientes.
- **Forma del gradiente:** dimensión que debe coincidir con el parámetro correspondiente.

Ejercicios

Preguntas conceptuales

1. Explique por qué backpropagation necesita valores guardados del forward pass.
2. Explique por qué $dZ1 = dA1 * (Z1 > 0)$ para ReLU.

Problemas cuantitativos

3. Derive la forma de dW_2 a partir de $Z_2 = A_1 @ W_2 + b_2$.
4. Si $A^{[1]} \in \mathbb{R}^{80 \times 16}$ y $dZ^{[2]} \in \mathbb{R}^{80 \times 4}$, calcule la forma de $dW^{[2]}$ y $db^{[2]}$.

Práctica computacional

5. Diseñe una prueba pequeña para verificar que `backward_pass` devuelve gradientes con la misma forma que los parámetros.
6. Implemente una verificación por diferencias finitas para un parámetro escalar.

Interpretación y pensamiento crítico

7. Un gradiente tiene la forma correcta pero el entrenamiento no mejora. Proponga tres causas posibles.

Proyecto de cierre

8. Tome una red pequeña de dos capas y documente una auditoría de gradientes: formas esperadas, formas obtenidas, norma de cada gradiente y resultado de una prueba numérica para un parámetro.

Activaciones, inicialización, regularización y diagnóstico

Pregunta aplicada

Una red puede fallar aunque el código no tenga errores. El entrenamiento puede divergir, saturarse, memorizar datos o quedarse sin aprender.

El trabajo profesional no termina al implementar backpropagation:

hay que diagnosticar el entrenamiento.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. identificar fallas de saturación, overfitting y learning rate;
2. explicar por qué la inicialización afecta el entrenamiento;
3. incorporar regularización L2 en la pérdida y en los gradientes;
4. leer curvas de entrenamiento y validación;
5. convertir una curva en una decisión técnica.

Activaciones

Activación	Fórmula	Riesgo principal
Sigmoide	$\sigma(z)$	saturación y gradientes pequeños
Tanh	$\tanh(z)$	saturación en valores extremos
ReLU	$\max(0, z)$	neuronas muertas si muchos $z \leq 0$

ReLU suele ser una buena opción inicial en redes densas pequeñas por simplicidad y estabilidad.

Inicialización

La inicialización controla la escala inicial de las señales. Si una capa calcula:

$$z = \sum_{j=1}^d w_j x_j,$$

la varianza de z crece con d si los pesos se inicializan sin controlar escala. Bajo independencia aproximada y media cero:

$$\text{Var}(z) \approx d \text{Var}(w) \text{Var}(x).$$

Para que la señal no crezca sin control al pasar por capas, se escoge $\text{Var}(w)$ proporcional a $1/d$. En redes con ReLU se usa a menudo la inicialización He porque compensa que ReLU anula una parte de las activaciones.

Si los pesos empiezan demasiado grandes, las activaciones pueden saturarse. Si empiezan demasiado pequeños, las señales pueden desaparecer.

Una regla práctica para ReLU es inicialización He:

$$W \sim \mathcal{N}\left(0, \frac{2}{\text{fan in}}\right).$$

En NumPy:

```
W = rng.normal(0, np.sqrt(2 / fan_in), size=(fan_in, fan_out))
```

Regularización L2 en redes

La pérdida puede incluir una penalización:

$$J_\lambda = J + \frac{\lambda}{2} (\|W^{[1]}\|_F^2 + \|W^{[2]}\|_F^2).$$

En los gradientes:

$$dW^{[\ell]} \leftarrow dW^{[\ell]} + \lambda W^{[\ell]}.$$

No penalizamos sesgos por defecto.

Ejemplo resuelto: L2 en pérdida y gradiente

Supón una capa con:

$$W = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}, \quad \lambda = 0.1.$$

La norma de Frobenius al cuadrado es:

$$\|W\|_F^2 = 2^2 + (-1)^2 + 0^2 + 3^2 = 14.$$

Si usamos la penalización $\frac{\lambda}{2}\|W\|_F^2$, el aporte a la pérdida es:

$$\frac{0.1}{2} \cdot 14 = 0.7.$$

Si el gradiente de datos fuera:

$$dW_{\text{datos}} = \begin{bmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{bmatrix},$$

el gradiente regularizado sería:

$$dW = dW_{\text{datos}} + \lambda W = \begin{bmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{bmatrix} + 0.1 \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.3 \\ 0.1 & 0.6 \end{bmatrix}.$$

La regularización no “apaga” pesos automáticamente; agrega una fuerza que empuja los pesos hacia valores más pequeños durante la actualización.

Dropout como idea

Dropout apaga unidades al azar durante entrenamiento. En este curso se introduce conceptualmente, no como requisito central del autograder.

Su intuición:

- evita dependencia excesiva de una neurona;
- funciona como ensamble implícito;
- requiere cuidado al pasar de entrenamiento a evaluación.

Curvas de entrenamiento

Una curva útil muestra:

- pérdida de entrenamiento;
- pérdida de validación;
- métrica principal;
- número de épocas;
- configuración del experimento.

Diagnósticos comunes

Señal	Diagnóstico probable	Acción
Train y validación altos	underfitting	más capacidad, más épocas, revisar features
Train bajo y validación alto	overfitting	regularización, menos capacidad, más datos
Pérdida explota	learning rate alto o gradiente incorrecto	reducir paso, revisar backward
Pérdida plana	paso bajo, mala inicialización o activaciones saturadas	ajustar inicialización y paso

Ejemplo resuelto: leer una curva

Supón que una red obtiene:

Época	Train loss	Val loss	Train acc	Val acc
5	0.82	0.88	0.74	0.71
20	0.24	0.43	0.94	0.86
50	0.05	0.78	0.99	0.78

Lectura:

1. El entrenamiento sí aprende: **train loss** baja.
2. La validación mejora hasta cierto punto y luego empeora.
3. El diagnóstico probable es overfitting después de la época 20.
4. Acciones razonables: early stopping, más regularización, menor capacidad o más datos.

La conclusión no es “la red es mala”; la conclusión es “esta configuración debe controlarse”.

Checkpoint técnico del proyecto

En semana 9, el proyecto debe mostrar avance real desde el baseline:

1. Métrica de baseline.
2. Modelo mejorado o experimento técnico nuevo.
3. Análisis de errores.
4. Decisión de siguiente experimento.
5. Riesgo técnico o de datos.

Caso longitudinal: curva de entrenamiento en dígitos

Supón que en `load_digits` una red con $h = 64$ obtiene este patrón:

Configuración	Train acc	Dev acc	Lectura
sin regularización	0.99	0.86	posible overfitting
L2 moderado	0.96	0.90	mejor generalización
red muy pequeña	0.82	0.80	posible underfitting

La conclusión no es que “L2 siempre gana”. La conclusión es que, para esta partición y esta arquitectura, L2 moderado reduce la brecha train/dev. Un reporte profesional debe conservar semilla, partición, tamaño de red, learning rate y número de épocas para que la comparación sea reproducible.

Verificaciones seleccionadas

1. Si train accuracy sube y dev accuracy cae después de cierta época, el diagnóstico principal es overfitting, no falta de capacidad.
2. Si train y dev son bajos, aumentar regularización rara vez es la primera acción; primero se revisan capacidad, features, épocas o learning rate.
3. Si cambias arquitectura y regularización al mismo tiempo, no puedes atribuir la mejora a una sola causa sin un experimento adicional.

Ejemplo resuelto: regla de parada

Supón que el dev loss por época es:

Época	Dev loss
10	0.58
20	0.44
30	0.41
40	0.43

Época	Dev loss
50	0.49

Una regla de early stopping con paciencia de 2 revisiones guardaría el modelo de la época 30 y se detendría después de observar que las épocas 40 y 50 no mejoran. La acción correcta no es usar el modelo de la última época por costumbre; es usar el mejor modelo según validación y reportar la regla usada.

Lectura de curvas: tres patrones frecuentes

Una curva profesional se lee comparando entrenamiento y validación, no mirando una sola línea. Hay tres patrones básicos.

Primero, si train loss baja de forma sostenida y dev loss también baja, el entrenamiento todavía está produciendo evidencia útil. No conviene detenerlo solo porque ya pasaron muchas épocas. Segundo, si train loss baja pero dev loss sube, la red está memorizando estructura específica del conjunto de entrenamiento. La acción natural es guardar el mejor modelo de validación, revisar regularización, reducir capacidad o aumentar datos. Tercero, si train y dev loss se mantienen altos, el problema no se resuelve aumentando regularización; probablemente faltan capacidad, variables, épocas, una tasa de aprendizaje adecuada o una revisión de etiquetas.

El reporte debe nombrar el patrón observado. Es distinto decir “el modelo no mejora” que decir “hay overfitting después de la época 30” o “hay underfitting persistente en train y dev”. Esa diferencia reduce ambigüedad para el lector y para quien califica.

Punto de control

Antes de reportar una red como mejora, responde:

- ¿contra qué baseline compite?
- ¿la mejora aparece en validación o solo en entrenamiento?
- ¿qué error específico sigue siendo importante?
- ¿qué hiperparámetro cambiarías primero y por qué?

Para explorar más

- **Extensión matemática:** relaciona la penalización L_2 con el tamaño de los pesos y explica por qué puede reducir varianza.
- **Extensión computacional:** compara curvas train/dev con y sin dropout, manteniendo fija la semilla y el tamaño de red.
- **Conexión con proyecto:** define una regla de parada basada en dev loss y explica qué evidencia evitaría sobreentrenamiento.

Revisión del capítulo

Entrenar una red no termina cuando baja la pérdida de entrenamiento. La lectura profesional compara entrenamiento y validación para distinguir aprendizaje, overfitting, underfitting, saturación y problemas de tasa.

La inicialización afecta la escala de activaciones y gradientes. La regularización L2, dropout y early stopping son herramientas para controlar capacidad efectiva, pero ninguna reemplaza una partición de validación ni un análisis de errores.

El producto final del capítulo es un diagnóstico accionable. Una curva debe convertirse en evidencia, la evidencia en causa probable y la causa en una acción proporcional: cambiar tasa, capacidad, regularización, datos o siguiente experimento.

Términos clave

- **Activación saturada:** zona donde una activación cambia poco y dificulta el flujo de gradientes.
- **Inicialización:** regla para asignar valores iniciales a pesos antes de entrenar.
- **Regularización L2:** penalización sobre pesos que busca reducir sobreajuste.
- **Dropout:** técnica que desactiva aleatoriamente unidades durante entrenamiento.
- **Curva de entrenamiento:** evolución de pérdida o métrica por época.
- **Early stopping:** detención del entrenamiento cuando validación deja de mejorar.
- **Análisis de error:** revisión de fallas concretas para decidir el siguiente experimento.

Ejercicios

Preguntas conceptuales

1. Explique por qué una red con train accuracy 99 % y validation accuracy 70 % no debe reportarse como exitosa.
2. ¿Por qué la regularización no reemplaza una partición de validación?

Problemas cuantitativos

3. Dada una tabla de pérdidas por época, identifique la primera señal de overfitting y proponga una época candidata para early stopping.
4. Si la penalización L2 es $\lambda \sum_j w_j^2$, calcule la penalización para $w = (2, -1, 0.5)$ y $\lambda = 0.1$.

Práctica computacional

5. Diseñe un experimento para comparar dos valores de `lambda` sin usar el conjunto de test.
6. Grafique pérdida de entrenamiento y validación para al menos dos configuraciones.

Interpretación y pensamiento crítico

7. ¿Qué señal en la curva de pérdida sugeriría revisar el learning rate antes de cambiar la arquitectura?
8. Un modelo mejora la métrica global pero empeora un subconjunto importante. ¿Qué decisión técnica tomaría?

Proyecto de cierre

9. Redacte un memo técnico de una página sobre una red entrenada: baseline, curva, diagnóstico, cambio propuesto, riesgo y siguiente experimento.

Optimizadores y PyTorch

Pregunta aplicada

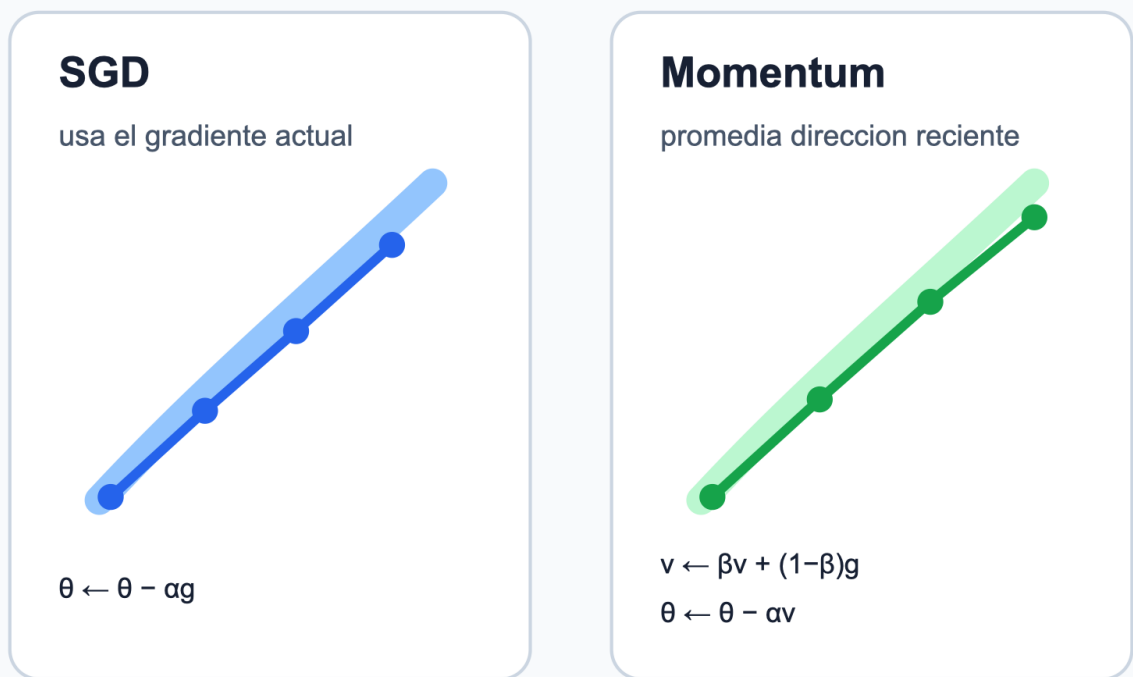
Hasta ahora actualizamos parámetros con descenso del gradiente simple:

$$\theta \leftarrow \theta - \alpha \nabla J(\theta).$$

Pero redes neuronales reales suelen entrenarse con variantes que usan memoria del gradiente, escalamiento adaptativo o mini-batches.

Optimizadores: gradiente, memoria y escala

La regla de actualización cambia, pero la validación sigue decidiendo si el entrenamiento



La eleccion del optimizador es un hiperparametro experimental: s

Figura 11: Optimizadores: SGD usa el gradiente actual, momentum suaviza direcciones recientes y Adam combina memoria con escala adaptativa por coordenada.

Lectura de la figura. SGD, momentum y Adam no cambian el objetivo del entrenamiento; cambian la regla para convertir gradientes en actualizaciones. Por eso el optimizador debe reportarse como parte del experimento, junto con tasa de aprendizaje, batch size, épocas y métrica de validación.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. distinguir batch, mini-batch y época;
2. explicar la intuición de momentum, RMSProp y Adam;
3. ubicar forward, loss, backward y actualización en un loop de PyTorch;
4. comparar una implementación desde cero con PyTorch sin cambiar la pregunta experimental;
5. reconocer que un optimizador no reemplaza validación.

Mini-batch gradient descent

En lugar de usar todo el dataset en cada paso, usamos lotes pequeños:

```
dataset -> mini-batches -> actualización por batch -> época completa
```

Esto reduce costo por actualización y agrega ruido útil al entrenamiento.

Momentum

Momentum acumula una dirección suavizada:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta_t),$$

$$\theta_{t+1} = \theta_t - \alpha v_t.$$

Intuición: evita que cada gradiente aislado cambie la dirección con demasiada fuerza.

Ejemplo resuelto: dos pasos con momentum

Supón un parámetro escalar $\theta_0 = 5$, tasa $\alpha = 0.1$, momentum $\beta = 0.9$ y velocidad inicial $v_0 = 0$. Los dos primeros gradientes son:

$$g_1 = 4, \quad g_2 = 2.$$

Primer paso:

$$v_1 = 0.9(0) + 0.1(4) = 0.4, \quad \theta_1 = 5 - 0.1(0.4) = 4.96.$$

Segundo paso:

$$v_2 = 0.9(0.4) + 0.1(2) = 0.56, \quad \theta_2 = 4.96 - 0.1(0.56) = 4.904.$$

Aunque el segundo gradiente es menor que el primero, la velocidad conserva parte de la dirección previa. Esa memoria suaviza actualizaciones ruidosas, pero también puede sostener una mala dirección si el learning rate o β no son adecuados.

Derivación guiada: momentum como memoria exponencial

La fórmula

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

parece local, pero al expandirla se ve su memoria. Como

$$v_{t-1} = \beta v_{t-2} + (1 - \beta)g_{t-1},$$

entonces

$$v_t = (1 - \beta)g_t + \beta(1 - \beta)g_{t-1} + \beta^2 v_{t-2}.$$

Si repetimos la sustitución:

$$v_t = (1 - \beta)g_t + \beta(1 - \beta)g_{t-1} + \beta^2(1 - \beta)g_{t-2} + \dots.$$

Los gradientes recientes pesan más que los antiguos, pero los antiguos no desaparecen de inmediato. Por eso momentum puede avanzar de forma más estable en valles alargados de la pérdida. La contraparte es que una memoria demasiado alta puede retrasar la reacción cuando la dirección útil cambia.

RMSProp y Adam

RMSProp escala cada coordenada según magnitudes recientes del gradiente.

Adam combina:

- promedio móvil del gradiente;
- promedio móvil del gradiente al cuadrado;
- corrección de sesgo al inicio.

En práctica, Adam suele ser un buen punto de partida, pero no elimina la necesidad de validar learning rate, regularización y arquitectura.

Ejemplo resuelto: Adam en una coordenada

Supón una coordenada con gradiente inicial $g_1 = 4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $m_0 = 0$ y $s_0 = 0$. El primer momento queda:

$$m_1 = 0.9(0) + 0.1(4) = 0.4.$$

El segundo momento queda:

$$s_1 = 0.999(0) + 0.001(4^2) = 0.016.$$

Como ambos promedios empiezan en cero, Adam usa corrección de sesgo:

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1} = 4, \quad \hat{s}_1 = \frac{s_1}{1 - \beta_2} = 16.$$

La dirección adaptativa es aproximadamente:

$$\frac{\hat{m}_1}{\sqrt{\hat{s}_1} + \epsilon} \approx 1.$$

La lectura no es que Adam normaliza todo perfectamente. La lectura es que usa información sobre magnitud reciente del gradiente para ajustar el tamaño efectivo del paso por coordenada.

PyTorch: qué automatiza

PyTorch ofrece:

- tensores;
- `autograd`;
- módulos `nn.Module`;
- pérdidas;
- optimizadores;
- entrenamiento en CPU/GPU.

El ciclo conceptual no cambia:

```
forward -> loss -> backward -> optimizer.step()
```

Loop mínimo


```

model.train()
for X_batch, y_batch in loader:
    optimizer.zero_grad()
    logits = model(X_batch)
    loss = criterion(logits, y_batch)
    loss.backward()
    optimizer.step()

```

La diferencia es que PyTorch construye el grafo de cómputo y calcula los gradientes.

Ejemplo resuelto: lectura de un loop

En el loop mínimo:

```

optimizer.zero_grad()
logits = model(X_batch)
loss = criterion(logits, y_batch)
loss.backward()
optimizer.step()

```

la interpretación es:

Línea	Significado
<code>zero_grad()</code>	borra gradientes acumulados
<code>model(X_batch)</code>	forward propagation
<code>criterion(...)</code>	calcula la pérdida
<code>backward()</code>	backpropagation automático
<code>step()</code>	actualiza parámetros

Si olvidas `zero_grad()`, PyTorch acumula gradientes entre batches. Eso rara vez es lo que quieres en este curso.

Comparación justa

Al comparar NumPy y PyTorch:

- use la misma partición de datos;
- use la misma métrica;
- reporte semilla;
- compare curvas o resultados finales;
- no confunda diferencia de implementación con diferencia de modelo.

Caso longitudinal: experimento reproducible en dígitos

Para `load_digits`, una comparación defendible entre NumPy y PyTorch debe fijar:

Elemento	Valor que debe registrarse
partición	semilla y proporciones train/dev/test
arquitectura	64 entradas, h ocultas, 10 salidas
pérdida	entropía cruzada multiclase
optimizador	SGD, momentum o Adam
presupuesto	épocas, batch size y learning rate
evidencia	curva train/dev y métrica final

Si PyTorch obtiene mejor resultado, hay que preguntar si cambió el modelo, el optimizador, el número de épocas o la inicialización. La biblioteca no es la evidencia; la evidencia es una comparación con condiciones controladas.

Punto de control

Una comparación NumPy vs PyTorch solo es defendible si:

1. usa la misma partición;
2. usa la misma métrica;
3. entrena una arquitectura comparable;
4. reporta semilla y número de épocas;
5. interpreta diferencias con cautela.

Verificaciones seleccionadas

1. Con 1024 observaciones y batch size 64 hay $1024/64 = 16$ batches por época.
2. Si se entrenan 20 épocas con 16 batches por época, `optimizer.step()` se ejecuta 320 veces.
3. `zero_grad()` debe ir antes de `backward()` en cada batch para evitar acumular gradientes de batches anteriores.

Ejemplo resuelto: presupuesto de entrenamiento

Comparar dos optimizadores exige presupuesto comparable. Si Adam se entrena 100 épocas y SGD se entrena 20, la diferencia de métrica no aísla el efecto del optimizador. Una comparación mínima fija:

Elemento	Mismo valor
partición	sí
arquitectura	sí

Elemento	Mismo valor
batch size	sí
épocas máximas	sí
criterio de parada	sí
métrica final	sí

Solo después de fijar esos elementos se puede discutir si momentum o Adam mejoraron estabilidad, velocidad o desempeño.

Convergencia no es validación

Un optimizador puede hacer que la pérdida de entrenamiento baje con más rapidez sin producir el mejor modelo para datos nuevos. Por eso el criterio de éxito no es “Adam bajó la pérdida antes”, sino “bajo la misma partición, presupuesto y métrica, el modelo validado sostiene una mejora defendible”. En particular, una curva suave de train loss puede esconder overfitting si no se acompaña de dev loss o de una métrica externa.

En una entrega, la comparación debe separar tres afirmaciones: velocidad de optimización, estabilidad de la curva y desempeño de validación. La primera se mide por épocas o pasos; la segunda por oscilaciones o sensibilidad a la tasa; la tercera por una métrica en datos no usados para ajustar parámetros.

Para explorar más

- **Extensión matemática:** compara una actualización de SGD con una de Adam en una dimensión y explica qué cambia por los momentos adaptativos.
- **Extensión computacional:** guarda `seed`, arquitectura, optimizador, tasa de aprendizaje y época del mejor modelo en un registro reproducible.
- **Conexión con proyecto:** define qué configuración mínima necesitas reportar para que otra persona pueda rerunear tu entrenamiento.

Revisión del capítulo

Los optimizadores modernos modifican la forma de usar gradientes, pero no cambian la lógica experimental. Mini-batch gradient descent estima el gradiente con subconjuntos; momentum acumula dirección; RMSProp y Adam adaptan escalas por parámetro.

PyTorch automatiza autograd y actualización de parámetros, pero el usuario sigue siendo responsable de particiones, métricas, semillas, arquitectura y lectura de curvas. `zero_grad()`, `backward()` y `step()` son operaciones con significado, no rituales de código.

Una comparación justa entre NumPy y PyTorch exige misma pregunta experimental: datos, partición, arquitectura comparable, métrica y presupuesto de entrenamiento. Velocidad de entrenamiento no equivale automáticamente a mejor modelo.

Términos clave

- **Mini-batch:** subconjunto usado para estimar un paso de gradiente.
- **Momentum:** acumulación suavizada de direcciones de actualización pasadas.
- **RMSProp:** adaptación de pasos según magnitudes recientes de gradientes.
- **Adam:** optimizador adaptativo que combina ideas de momentum y RMSProp.
- **Autograd:** mecanismo de diferenciación automática de PyTorch.
- **zero_grad():** instrucción que limpia gradientes acumulados.
- **backward():** cálculo automático de gradientes desde la pérdida.
- **step():** actualización de parámetros del optimizador.

Ejercicios

Preguntas conceptuales

1. Explique por qué Adam no reemplaza la validación.
2. ¿Por qué PyTorch acumula gradientes y por qué normalmente llamamos `zero_grad()`?

Problemas cuantitativos

3. Si un dataset tiene 1024 observaciones y batch size 64, ¿cuántos batches tiene una época?
4. Si se entrenan 20 épocas con 16 batches por época, ¿cuántas veces se ejecuta `optimizer.step()`?

Práctica computacional

5. Identifique en el loop de PyTorch dónde ocurren forward, pérdida, backward y actualización.
6. Implemente un loop mínimo que registre pérdida promedio por época.

Interpretación y pensamiento crítico

7. Proponga una comparación justa entre una red NumPy pequeña y una red PyTorch equivalente.
8. Si PyTorch entrena más rápido que NumPy, ¿qué evidencia necesita antes de afirmar que el modelo es mejor?

Proyecto de cierre

9. Reproduzca el mismo experimento con NumPy y PyTorch usando la misma partición, semilla y métrica. Entregue una tabla comparativa y una conclusión cautelosa.

Ejercicios integradores: redes neuronales

Estos ejercicios están diseñados para estudiar el módulo como un sistema completo: forward, backward, diagnóstico y entrenamiento con herramientas modernas. La meta no es memorizar fórmulas aisladas, sino verificar que cada objeto matemático tiene forma, propósito e interpretación.

Resultados de práctica

Al terminar este banco deberías poder:

1. seguir las formas de tensores en una red densa;
2. explicar forward propagation como composición de funciones;
3. derivar gradientes de segunda y primera capa;
4. diagnosticar entrenamiento con curvas y métricas;
5. leer un loop PyTorch sin tratarlo como caja negra.

Mapa del banco

Bloque	Evidencia de aprendizaje
A	compatibilidad de dimensiones y forward pass
B	backpropagation y verificación de gradientes
C	diagnóstico de entrenamiento y generalización
D	lectura profesional de PyTorch

Cómo usar esta sección

Trabaja primero sin mirar las respuestas. Después revisa:

1. si las formas de matrices son compatibles;
2. si cada gradiente actualiza un parámetro con la misma forma;
3. si tu diagnóstico distingue evidencia, causa probable y acción;
4. si puedes explicar cada línea crítica de un loop de entrenamiento.

Bloque A: dimensiones y forward

A1. Una red recibe `X.shape == (400, 64)`, tiene 20 unidades ocultas y 10 clases. Escriba las formas de `W1`, `b1`, `Z1`, `A1`, `W2`, `b2`, `Z2` y `P`.

A2. Explique por qué softmax se aplica por fila cuando `Z2` tiene forma `(n, k)`.

A3. Muestre con una frase por qué restar `max(Z, axis=1)` antes de `exp` no cambia softmax.

Bloque B: backpropagation

B1. Partiendo de `dZ2 = (P - Y) / n`, derive las formas de `dW2` y `db2`.

B2. Describa qué debe guardarse en `cache` durante forward para poder ejecutar backward sin recalcular todo.

B3. Diseñe una prueba de gradientes por diferencias finitas para un solo peso de `W1`.

Bloque C: diagnóstico

C1. Una red tiene pérdida de entrenamiento decreciente y pérdida de validación creciente después de la época 20. ¿Qué diagnóstico haría y qué dos acciones probaría?

C2. Si al aumentar el learning rate la pérdida explota, ¿qué evidencia reportaría y qué ajuste haría?

C3. Explique por qué un checkpoint técnico de proyecto debe incluir análisis de errores, no solo una métrica agregada.

Bloque D: PyTorch

D1. En un loop de entrenamiento de PyTorch, explique el propósito de `optimizer.zero_grad()`.

D2. ¿Qué significa que `loss.backward()` use diferenciación automática?

D3. Proponga dos razones por las que una implementación NumPy y una PyTorch podrían producir métricas distintas aun usando el mismo dataset.

Respuestas seleccionadas

A1. Una convención compatible es: `W1.shape == (64, 20)`, `b1.shape == (20,)`, `Z1.shape == (400, 20)`, `A1.shape == (400, 20)`, `W2.shape == (20, 10)`, `b2.shape == (10,)`, `Z2.shape == (400, 10)` y `P.shape == (400, 10)`.

B1. Si `dZ2.shape == (400, 10)` y `A1.shape == (400, 20)`, entonces `dW2 = A1.T @ dZ2` tiene forma `(20, 10)`, igual que `W2`. Además, `db2 = dZ2.sum(axis=0)` tiene forma `(10,)`, igual que `b2`.

C1. El patrón sugiere overfitting después de la época 20. Dos acciones defendibles son early stopping y aumentar regularización L2; también podría reducirse capacidad o ampliar datos si el proyecto lo permite.

D1. `optimizer.zero_grad()` borra gradientes acumulados antes del siguiente batch. Si no se llama, PyTorch suma gradientes de batches anteriores y la actualización ya no corresponde al gradiente del batch actual.

Parte IV: Estrategia de machine learning

Parte IV: Estrategia de machine learning

Esta parte cambia el foco: ya no basta con entrenar un modelo. Hay que elegir métricas, diseñar particiones, evitar leakage, validar con variabilidad, analizar errores y comunicar decisiones técnicas proporcionales a la evidencia.



Figura 12: Selección de métricas: el costo relativo de falsos positivos y falsos negativos orienta si conviene mirar accuracy, precisión, recall o una métrica F beta.

Lectura de la figura. En esta parte, una métrica se entiende como una decisión de diseño. Elegir accuracy, precision, recall o F-score solo tiene sentido si se declara qué error cuesta más y cómo se usará el modelo.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. elegir una métrica principal según costo de error;
2. diseñar particiones train/dev/test sin contaminar la evaluación;
3. usar pipelines para evitar leakage de preprocesamiento;
4. interpretar cross-validation con media y variabilidad;
5. leer learning curves y validation curves;
6. convertir análisis de errores en recomendación técnica.

Mapa de la parte

Capítulo	Pregunta guía	Evidencia esperada
Métricas y particiones	¿Qué estamos optimizando y con qué datos lo medimos?	Ficha de evaluación sin leakage.
Validación y curvas	¿El modelo necesita más datos, menor complejidad o mejor señal?	Curva y diagnóstico sesgo/varianza.
Error analysis	¿Dónde falla el modelo y qué acción sigue?	Tabla por slice y recomendación.
Ejercicios integradores	¿Puedes defender una estrategia de evaluación completa?	Evidencia, diagnóstico, acción y límite.

Criterio de salida

La parte queda dominada cuando puedes rechazar una mejora aparente si usa la métrica equivocada, toca test indebidamente, oculta un slice crítico o no declara el límite de la recomendación.

Métricas y particiones

Pregunta aplicada

Un modelo puede tener buena accuracy y aun así ser inaceptable si falla en los casos importantes. Antes de entrenar más modelos, debemos decidir:

¿qué error cuesta más y qué evidencia será suficiente para elegir?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. elegir una métrica principal alineada con el costo de error;
2. explicar por qué accuracy falla en clases desbalanceadas;
3. construir un baseline útil;
4. separar train, dev y test sin leakage;
5. usar pipelines como defensa contra preprocesamiento incorrecto.

Particiones: ajustar, seleccionar y estimar

La particion no es un detalle administrativo: define que evidencia puede usarse

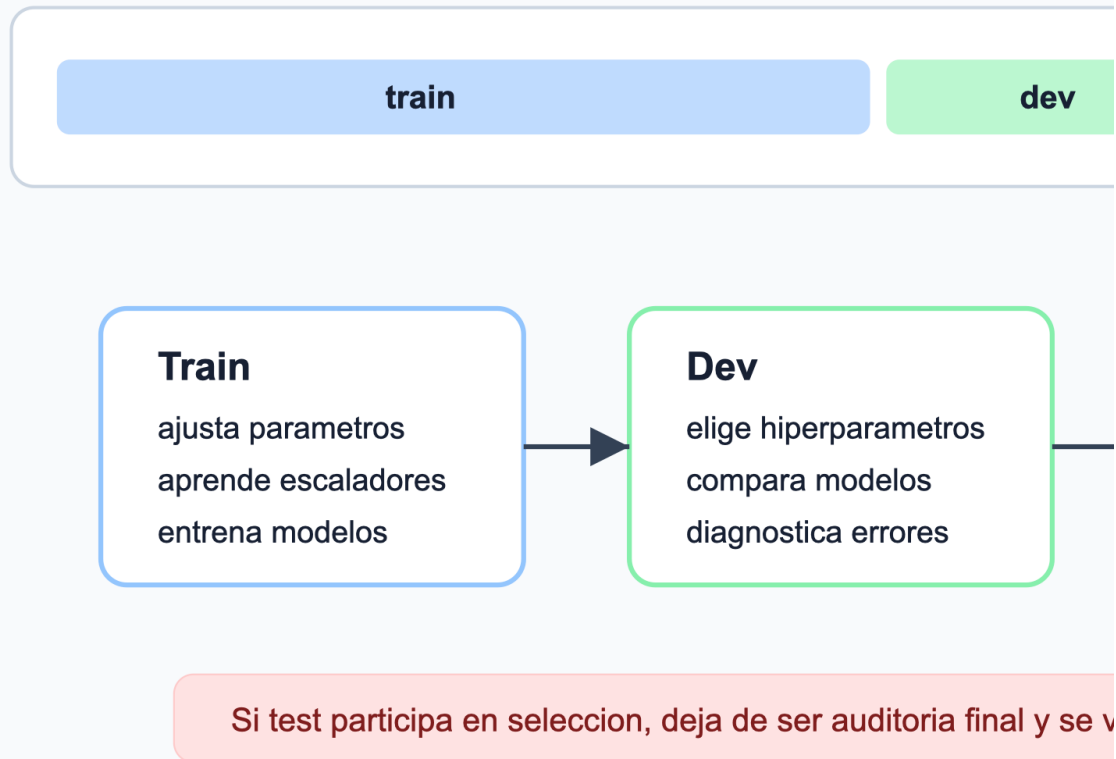


Figura 13: Particiones: train ajusta parámetros, dev selecciona modelos y test estima desempeño final; mezclar esos roles produce evaluaciones optimistas.

Lectura de la figura. La partición es un contrato de evidencia. Train puede aprender parámetros y transformaciones; dev puede guiar decisiones de modelo; test se reserva para estimar el desempeño final. Si test participa en selección, la comparación queda contaminada.

Métrica principal

La **métrica principal** es la regla cuantitativa que se usará para seleccionar modelos durante el desarrollo.

Ejemplos:

Problema	Métrica razonable	Motivo
Diagnóstico médico	recall o F1	falsos negativos costosos
Filtro de alertas	precision	falsos positivos saturan al usuario
Ranking general	ROC-AUC o PR-AUC	importa ordenar riesgos
Regresión de demanda	MAE o RMSE	depende del costo de error

Accuracy no siempre basta

En clases desbalanceadas, un modelo que predice siempre la clase mayoritaria puede tener alta accuracy.

Por eso, en clasificación binaria se debe revisar:

- matriz de confusión;
- precision;
- recall;
- F1;
- ROC-AUC o PR-AUC;
- costo de falsos positivos y falsos negativos.

Ejemplo resuelto: accuracy engañosa

Supón un problema con 1000 observaciones:

- 950 son negativas;
- 50 son positivas;
- un modelo predice todo como negativo.

La accuracy es:

$$\frac{950}{1000} = 0.95.$$

Pero el recall de la clase positiva es:

$$\frac{TP}{TP + FN} = \frac{0}{0 + 50} = 0.$$

Si detectar positivos era el objetivo, este modelo no sirve aunque su accuracy parezca alta.

Ejemplo resuelto: costo de falsos negativos

Supón un problema con 50 positivos y 950 negativos. Comparamos dos umbrales:

Umbral	TP	FP	FN	TN
A	35	40	15	910
B	25	10	25	940

El umbral A tiene:

$$\text{precision}_A = \frac{35}{35 + 40} \approx 0.467, \quad \text{recall}_A = \frac{35}{35 + 15} = 0.70.$$

El umbral B tiene:

$$\text{precision}_B = \frac{25}{25 + 10} \approx 0.714, \quad \text{recall}_B = \frac{25}{25 + 25} = 0.50.$$

Si un falso positivo cuesta 1 unidad y un falso negativo cuesta 10 unidades, el costo esperado de cada umbral es:

$$C_A = 40(1) + 15(10) = 190, \quad C_B = 10(1) + 25(10) = 260.$$

Aunque B tiene mayor precision, A es preferible bajo esta función de costo porque reduce falsos negativos. La métrica correcta depende de la decisión, no de cuál número se ve más alto.

Derivación guiada: F1 como equilibrio operativo

Precision y recall miden errores distintos:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}.$$

F1 es la media armónica de ambas:

$$F1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

La media armónica penaliza valores muy bajos. Por eso un modelo con precision alta pero recall casi cero no obtiene buen F1. Esto es útil cuando ambos tipos de desempeño importan, pero no reemplaza una función de costo si los errores tienen costos muy asimétricos.

Baseline

Ningún modelo se reporta sin baseline.

Un baseline puede ser:

- clase mayoritaria;
- promedio de entrenamiento;
- modelo lineal simple;
- regla de negocio;
- modelo anterior del proyecto.

Particiones

La estructura estándar es:

```
train -> ajustar parámetros  
dev/validation -> elegir modelo e hiperparámetros  
test -> estimar desempeño final
```

Test se toca una vez

Si se usa test para tomar decisiones, deja de ser una estimación honesta del desempeño final.

Advertencia

El conjunto de test no es una herramienta de diseño. Es una auditoría final.

Estratificación

En clasificación, la partición debe preservar proporciones de clase cuando sea razonable:

```
train_test_split(X, y, stratify=y, random_state=42)
```

Si no se estratifica un dataset pequeño o desbalanceado, una partición puede cambiar artificialmente la dificultad.

Ejemplo resuelto: conteos en una partición estratificada

Supón un dataset con 2000 observaciones y clase positiva de 8 %. Entonces hay:

$$2000(0.08) = 160$$

observaciones positivas. En una división 60/20/20 estratificada, los tamaños esperados son:

Conjunto	Observaciones	Positivos esperados	Negativos esperados
train	$2000(0.60) = 1200$	$160(0.60) = 96$	1104
dev	$2000(0.20) = 400$	$160(0.20) = 32$	368
test	$2000(0.20) = 400$	$160(0.20) = 32$	368

La tabla no garantiza que cada partición sea perfecta en todos los rasgos del problema. Solo preserva, aproximadamente, la proporción de la clase usada para estratificar. Si hay tiempo, grupos, hospitales, sedes o usuarios repetidos, esas restricciones pueden ser más importantes que una estratificación simple.

Leakage

Leakage ocurre cuando información no disponible en producción entra al entrenamiento o a la evaluación.

Ejemplos:

- escalar usando todo el dataset antes de partir;
- seleccionar variables usando test;
- usar variables creadas después del evento a predecir;
- duplicados entre train y test;
- elegir hiperparámetros mirando test.

Pipeline como defensa

Los pipelines ayudan a aplicar `fit` solo donde corresponde:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

En cross-validation, cada fold ajusta sus transformaciones solo con el train fold.

Caso longitudinal: clasificación con slice reciente

El archivo `clasificacion_binaria_sintetica.csv` acompaña varios capítulos. Tiene variables `x1`, `x2`, una etiqueta binaria `target` y una columna `slice` con valores `base` y `reciente`. El objetivo pedagógico es discutir qué pasa cuando un modelo parece razonable en promedio, pero cambia su comportamiento en una subpoblación o periodo.

Para este caso, una primera partición defendible debe declarar:

Decisión	Criterio
baseline	clase mayoritaria o logística simple
métrica primaria	F1 o recall si la clase positiva importa
partición	train/dev/test con semilla fija
estratificación	preservar proporción de <code>target</code> cuando sea razonable
auditoría	revisar desempeño por <code>slice</code> antes de recomendar

La columna `slice` no debe usarse automáticamente como variable predictora. A veces se usa para diagnóstico, no para entrenamiento. Esa distinción evita que el modelo aprenda un marcador de origen que no estará disponible o que no se quiere usar en producción.

Punto de control

Antes de comparar modelos, escribe explícitamente:

1. métrica principal;
2. baseline;
3. partición;
4. transformación dentro de pipeline;
5. regla para no tocar test.

Verificaciones seleccionadas

1. Si la clase positiva representa 8% de 2000 observaciones, hay 160 positivos.
2. En una división estratificada 60/20/20, se esperan aproximadamente 96 positivos en train, 32 en dev y 32 en test.
3. Si se ajusta `StandardScaler` antes de partir datos, el promedio de dev/test ya contaminó el preprocesamiento.

Para explorar más

- **Extensión matemática:** calcula precisión, recall, F1 y balanced accuracy para dos matrices de confusión con el mismo accuracy.
- **Extensión computacional:** repite una partición estratificada con varias semillas y mide la variación de la métrica principal.
- **Conexión con proyecto:** escribe qué métrica será primaria, cuál será secundaria y qué baseline debe superar tu modelo.

Revisión del capítulo

Este capítulo separa entrenamiento de evaluación. La métrica principal debe elegirse antes de comparar modelos y debe estar alineada con el costo de error. Accuracy puede ser engañosa cuando las clases están desbalanceadas o cuando un tipo de error pesa más que otro.

Las particiones train, dev y test cumplen funciones distintas. Train ajusta parámetros, dev guía decisiones de modelo y test estima desempeño final. La estratificación, las particiones temporales o por grupo deben elegirse según el problema, no por costumbre.

El leakage suele entrar por preprocesamiento aprendido con demasiados datos. Un **Pipeline** no es solo comodidad de código: es una barrera operativa para que escalamiento, imputación y selección de variables se ajusten donde corresponde.

Términos clave

- **Métrica principal:** cantidad que guía la selección del modelo.
- **Baseline:** regla simple que fija un punto mínimo de comparación.
- **Train/dev/test:** separación para entrenar, seleccionar y estimar desempeño final.
- **Estratificación:** partición que preserva proporciones de clase.
- **Leakage:** entrada de información no disponible en producción al entrenamiento o evaluación.
- **Pipeline:** objeto que encadena preprocesamiento y modelo para evitar ajustes fuera de lugar.

Ejercicios

Preguntas conceptuales

1. Para un problema de fraude, explique cuándo preferiría recall y cuándo precision.
2. ¿Por qué accuracy puede ser una mala métrica en clases desbalanceadas?

Problemas cuantitativos

3. Diseñe una partición `train/dev/test` para un dataset de 2000 observaciones con clases desbalanceadas.
4. Si una clase positiva representa 8 % de los datos, estime cuántos positivos deberían aparecer en cada partición de una división 60/20/20 estratificada.

Práctica computacional

5. Implemente una partición estratificada con semilla fija.
6. Construya un `Pipeline` que escale variables numéricas y ajuste un modelo.

Interpretación y pensamiento crítico

7. Identifique el leakage: se imputa la media de cada variable usando todo el dataset antes de `train_test_split`.
8. Escriba una regla concreta para decidir cuándo se permite tocar el conjunto de test.

Proyecto de cierre

9. Para su proyecto, escriba una ficha de evaluación con métrica principal, baseline, partición, riesgo de leakage y regla de uso del test.

Validación cruzada y curvas de aprendizaje

Pregunta aplicada

Un solo split puede ser afortunado o desafortunado. Necesitamos estimar variabilidad:

¿el modelo funciona de forma estable o solo tuvo suerte en una partición?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. explicar qué estima cross-validation;
2. reportar media y variabilidad de una métrica;
3. usar grid search sin contaminar test;
4. leer learning curves y validation curves;
5. distinguir problemas de sesgo, varianza y ruido.

Cross-validation

En k -fold cross-validation:

1. Se divide el conjunto de entrenamiento en k folds.
2. Se entrena k veces.
3. Cada fold actúa una vez como validación.
4. Se reporta media y variabilidad.

```
scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring="f1")
```

Qué reportar

Un reporte profesional no dice solo:

```
F1 = 0.84
```

Dice:

F1 CV = 0.84 ± 0.03

La desviación informa estabilidad.

Ejemplo resuelto: elegir entre dos modelos

Supón dos modelos evaluados con 5-fold CV:

Modelo	F1 medio	Desviación
A	0.84	0.02
B	0.86	0.10

El modelo B tiene mayor promedio, pero también mucha mayor variabilidad. Si el contexto exige estabilidad, A puede ser preferible. Si se elige B, debe justificarse y revisarse en slices o con más datos.

La decisión profesional no mira solo el máximo promedio.

Derivación guiada: media y variabilidad de CV

Si los puntajes de cinco folds son:

0.80, 0.84, 0.83, 0.86, 0.87,

la media es:

$$\bar{s} = \frac{0.80 + 0.84 + 0.83 + 0.86 + 0.87}{5} = 0.84.$$

La desviación resume qué tanto cambian los resultados entre folds. No convierte la validación cruzada en garantía absoluta, pero sí evita tratar una partición afortunada como evidencia universal. En reportes del curso, escribe media y variabilidad juntas:

F1 CV = 0.84 ± 0.03

Si dos modelos tienen medias parecidas, la variabilidad puede decidir cuál es más prudente.

Grid search

Para seleccionar hiperparámetros:

```
GridSearchCV(
    estimator=pipeline,
    param_grid=grid,
    scoring="f1",
    cv=5
)
```

El grid search debe ocurrir dentro de entrenamiento/desarrollo, no sobre test.

Learning curve

Una curva de aprendizaje compara desempeño al aumentar el tamaño del conjunto de entrenamiento.

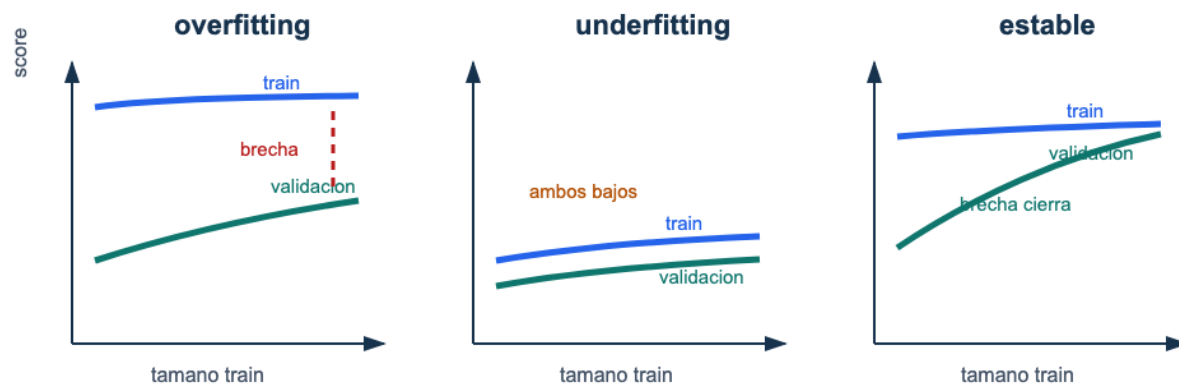


Figura 14: Curvas de aprendizaje: un caso de overfitting mantiene una brecha entre entrenamiento y validación, un caso de underfitting mantiene ambos desempeños bajos, y un caso estable cierra la brecha con más datos.

Lectura de la figura. Una learning curve no se interpreta por el valor final aislado, sino por la relación entre entrenamiento y validación. La brecha sugiere varianza; valores bajos en ambas curvas sugieren sesgo; curvas que se acercan pueden indicar que más datos ayudan.

Señales típicas:

Señal	Diagnóstico	Acción
Train alto y val bajo	overfitting	regularizar, simplificar, más datos
Train bajo y val bajo	underfitting	modelo más expresivo, mejores features
Train y val se acercan Val mejora con más datos	estabilidad conseguir más datos puede ayudar	evaluar límite por ruido

Ejemplo resuelto: leer una learning curve

Supón que una curva de aprendizaje produce:

Tamaño train	F1 train	F1 validación
200	0.98	0.63
800	0.95	0.74
2000	0.92	0.82
5000	0.90	0.86

Lectura:

1. El desempeño de entrenamiento baja al aumentar datos, lo cual es normal.
2. El desempeño de validación sube de forma sostenida.
3. La brecha train-validación se reduce de 0.35 a 0.04.
4. El diagnóstico es alta varianza inicial que mejora con más datos.

La acción razonable no es aumentar complejidad. Primero conviene conseguir más datos, usar regularización moderada o mantener el modelo si el costo de más datos es alto.

Validation curve

Una validation curve varía un hiperparámetro:

```
regularización baja -> modelo flexible  
regularización alta -> modelo simple
```

Permite detectar si el hiperparámetro está empujando al modelo hacia overfitting o underfitting.

Caso longitudinal: estabilidad del modelo de clasificación

En `clasificacion_binaria_sintetica.csv`, una validación de cinco folds puede producir algo como:

Modelo	F1 CV medio	Desviación	Lectura
logística simple	0.71	0.03	baseline estable
logística con features polinomiales	0.76	0.09	mejora media, alta varianza
modelo regularizado	0.74	0.04	balance entre mejora y estabilidad

La selección no es automática. Si el contexto tolera variabilidad y luego se hará error analysis por `slice`, podría explorarse el modelo polinomial. Si se quiere una recomendación conservadora, el modelo regularizado puede ser más defendible. El punto es que media y desviación se leen juntas.

Ejemplo resuelto: decisión con diferencia pequeña

Supón que dos modelos tienen:

Modelo	F1 CV medio	Desviación
A	0.812	0.018
B	0.819	0.061

La diferencia de medias es 0.007, menor que la variabilidad reportada de B. Una respuesta profesional no diría que B “gana” de forma clara. Diría que B tiene una mejora media pequeña, pero su inestabilidad exige revisar folds, slices o una partición temporal antes de cambiar la recomendación. En validación, ganar por una milésima no equivale a ganar una decisión.

Sesgo, varianza y ruido

La generalización puede entenderse como una combinación de:

- **sesgo:** error sistemático por modelo demasiado restrictivo;
- **varianza:** sensibilidad a cambios en entrenamiento;
- **ruido:** parte no explicable por las variables disponibles.

No todo error se arregla con un modelo más complejo.

Punto de control

Si una learning curve muestra train alto y validación bajo, no concluyas “necesito una red más grande”. Primero pregunta:

1. ¿hay leakage?
2. ¿la métrica está alineada con la decisión?
3. ¿el modelo ya tiene demasiada varianza?
4. ¿más datos podría cerrar la brecha?

Verificaciones seleccionadas

1. Evaluar 4 valores de `C`, 3 valores de `penalty` y 5 folds entrena $4 \cdot 3 \cdot 5 = 60$ ajustes.
2. Si la media CV mejora poco pero la desviación se triplica, la mejora puede no justificar el cambio de modelo.
3. `GridSearchCV` debe recibir el pipeline completo cuando hay escalamiento, imputación o selección de variables.

Ejemplo resuelto: cuándo no hacer grid más grande

Si una grilla inicial muestra que todos los valores de regularización fuerte tienen train y dev bajos, aumentar la grilla alrededor de regularización más fuerte probablemente no ayudará. La acción más razonable es revisar features, capacidad del modelo o métrica. Grid search no reemplaza diagnóstico: solo prueba combinaciones dentro del espacio que tú definiste.

Para explorar más

- **Extensión matemática:** interpreta una curva con alto sesgo y otra con alta varianza usando brechas train/dev.
- **Extensión computacional:** genera curvas de aprendizaje para dos tamaños de entrenamiento y reporta si más datos parecen útiles.
- **Conexión con proyecto:** decide qué experimento harías primero: más datos, más regularización, más features o modelo más flexible.

Criterio práctico de decisión

Cuando dos modelos tienen métricas cercanas, la elección no debe depender solo del tercer decimal. Un criterio razonable compara desempeño medio, variabilidad, complejidad, costo de explicación y estabilidad por slice. Si el modelo más complejo mejora 0.003 en F1, pero tiene mayor varianza entre folds y errores más difíciles de explicar, la recomendación profesional puede ser conservar el modelo simple como baseline operativo.

La validación no busca declarar un campeón universal. Busca decidir qué modelo es proporcional a la evidencia disponible. Esa frase importa para el curso: elegir bien también significa saber cuándo no hay evidencia suficiente para preferir una alternativa más complicada.

Revisión del capítulo

La validación cruzada estima desempeño con variabilidad. Reportar solo una media oculta si el modelo es estable o si depende demasiado de una partición particular. La desviación entre folds puede cambiar una decisión técnica.

Grid search debe evaluar pipelines completos para no contaminar cada fold. Las learning curves y validation curves ayudan a distinguir sesgo, varianza, falta de datos, hiperparámetros mal calibrados y ruido irreducible.

La pregunta profesional no es únicamente qué modelo gana. Es si la evidencia justifica cambiar complejidad, recolectar datos, ajustar regularización o aceptar un límite. La curva debe terminar en una acción, no solo en una imagen.

Términos clave

- **Cross-validation:** estimación de desempeño usando varias particiones train/validation.
- **Grid search:** búsqueda sistemática sobre combinaciones de hiperparámetros.
- **Learning curve:** curva que compara desempeño al variar tamaño de entrenamiento.
- **Validation curve:** curva que compara desempeño al variar un hiperparámetro.
- **Sesgo:** error sistemático por modelo demasiado restrictivo.
- **Varianza:** sensibilidad del modelo a cambios en datos de entrenamiento.
- **Ruido:** variación no explicable con las variables disponibles.

Ejercicios

Preguntas conceptuales

1. Una learning curve muestra train F1 0.99 y validation F1 0.70. Diagnostique y proponga dos acciones.
2. ¿Por qué reportar desviación estándar de cross-validation puede cambiar una decisión de modelo?

Problemas cuantitativos

3. Compare dos modelos con medias CV similares pero desviaciones estándar diferentes. ¿Cuál elegiría si la estabilidad es prioritaria?
4. Si se evalúan 4 valores de `C`, 3 valores de `penalty` y 5 folds, ¿cuántos ajustes se entrenan?

Práctica computacional

5. Explique por qué `GridSearchCV` debe recibir un `Pipeline` completo si hay escalamiento.
6. Genere una tabla con media y desviación estándar de CV para al menos tres configuraciones.

Interpretación y pensamiento crítico

7. Una validation curve mejora hasta cierto punto y luego cae. Explique qué sugiere sobre el hiperparámetro.
8. ¿Cuándo buscar más datos es más razonable que cambiar de algoritmo?

Proyecto de cierre

9. Construya una learning curve o validation curve para un modelo de su proyecto y escriba un diagnóstico de sesgo/varianza/ruido con una acción siguiente.

Error analysis y data mismatch

Pregunta aplicada

Dos modelos pueden tener la misma métrica agregada y fallar en casos completamente distintos.

El análisis de errores pregunta:

¿dónde falla el modelo, para quién falla y qué decisión técnica se desprende?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. separar errores por tipo: falsos positivos, falsos negativos y casos ambiguos;
2. definir slices relevantes para un proyecto;
3. detectar señales de data mismatch;
4. escribir una recomendación técnica con evidencia, diagnóstico y acción;
5. reconocer cuándo una métrica agregada oculta daño o riesgo.

Error analysis

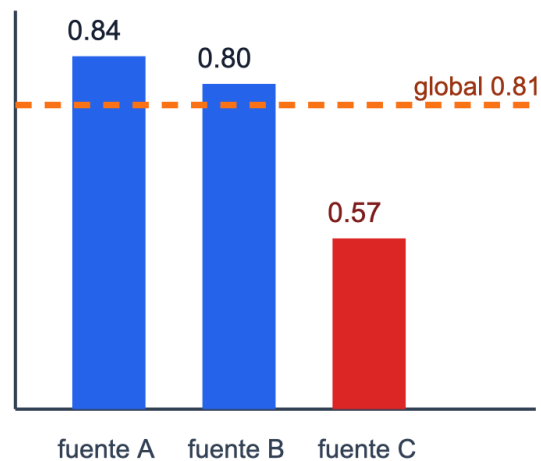
Un análisis mínimo incluye:

1. Casos correctamente clasificados.
2. Falsos positivos.
3. Falsos negativos.
4. Subgrupos con peor desempeño.
5. Variables o condiciones asociadas al error.

Error analysis: el promedio no cuenta toda

Un modelo con métrica global aceptable puede fallar en un slice que importa p

F1 por slice



1. Eviden

2. Diagn

3. Accior

4. Limite

Figura 15: Error analysis por slices: el promedio global puede ocultar un slice crítico con desempeño bajo y exige una recomendación con evidencia, diagnóstico, acción y límite.

Lectura de la figura. El promedio global puede ser suficiente para comparar dos prototipos, pero no para recomendar una decisión si un slice importante falla. La figura fuerza la secuencia mínima: evidencia, diagnóstico, acción y límite.

Slices

Un **slice** es un subconjunto interpretable de datos:

- pacientes mayores de cierta edad;
- observaciones con medición faltante;
- imágenes de baja calidad;
- transacciones de monto alto;
- ejemplos de una fuente específica.

El objetivo es detectar fallas ocultas por el promedio.

Data mismatch

Data mismatch ocurre cuando train, dev o test no vienen de la misma distribución relevante.

Ejemplos:

- entrenar con datos de una ciudad y probar en otra;
- entrenar con datos históricos y usar en una temporada distinta;
- usar datos limpios de laboratorio y desplegar en datos ruidosos;
- combinar fuentes sin controlar origen.

Diagnóstico de mismatch

Compare:

- distribución de variables clave;
- proporción de clases;
- desempeño por fuente;
- error por periodo;
- desempeño en dev vs test.

Si validación y test difieren mucho, no basta con cambiar hiperparámetros. Puede existir mismatch.

Ejemplo resuelto: mismatch por proporción de clase

Supón la siguiente auditoría:

Conjunto	Observaciones	Positivos	Porcentaje positivo	Recall
train	5000	1000	20 %	0.84
dev	1000	210	21 %	0.82
test	1000	80	8 %	0.55

Train y dev se parecen en proporción de clase y desempeño. Test, en cambio, tiene una clase positiva mucho menos frecuente y recall mucho menor.

La primera hipótesis no debería ser “el modelo necesita más hiperparámetros”. Una explicación más plausible es que test proviene de otra población, fuente o periodo. La acción técnica sería:

1. revisar cómo se construyó test;
2. comparar distribuciones de variables clave;
3. reportar desempeño por fuente o periodo;
4. decidir si se necesita un dev set más parecido a producción.

Si test representa producción real, el resultado indica un riesgo de despliegue, no solo una oportunidad de tuning.

Derivación guiada: promedio global como mezcla de slices

Una métrica global suele mezclar subpoblaciones. Si un conjunto tiene dos slices con tamaños n_A, n_B y métricas m_A, m_B , un promedio ponderado simple se escribe como:

$$m_{\text{global}} = \frac{n_A m_A + n_B m_B}{n_A + n_B}.$$

Esto explica por qué un slice grande puede esconder un slice pequeño con mal desempeño. El promedio global puede subir aunque el grupo crítico empeore, si ese grupo tiene poco peso. Por eso el análisis por slice no es opcional cuando la decisión afecta subpoblaciones distintas.

Recomendación técnica

Una recomendación debe seguir la forma:

Evidencia -> diagnóstico -> acción

Ejemplo:

El recall cae de 0.84 a 0.62 en el grupo de mayor riesgo.

Diagnóstico: el modelo no generaliza bien a ese slice.

Acción: revisar representación de variables, ampliar datos del slice y ajustar métrica de selección.

Ejemplo resuelto: dos modelos con el mismo F1

Supón dos modelos con F1 global 0.82.

Slice	Modelo A	Modelo B
Datos recientes	0.80	0.83
Datos antiguos	0.84	0.81
Grupo de alto riesgo	0.55	0.76

Aunque el F1 global sea igual, el Modelo B es más defendible si el grupo de alto riesgo es central para la decisión. El análisis de errores cambia la recomendación.

Ejemplo resuelto: brecha contra el promedio

Supón que un modelo tiene F1 global de 0.81, pero el análisis por slice muestra:

Slice	Observaciones	F1
fuelle A	1400	0.84
fuelle B	900	0.80
fuelle C	220	0.57

La brecha del slice C contra el promedio global es:

$$0.81 - 0.57 = 0.24.$$

Esa diferencia es grande y afecta una fuente específica. La recomendación no debería ser simplemente “el modelo tiene F1 0.81”. Una recomendación defendible sería:

Evidencia: el F1 global es 0.81, pero en fuente C cae a 0.57 con 220 casos.
Diagnóstico: el modelo generaliza mal a la fuente C o esa fuente tiene un proceso de medición distinto.
Acción: auditar variables, faltantes y distribución de fuente C; entrenar/dev con datos representativos de esa fuente antes de recomendar despliegue.
Límite: el desempeño global no debe extrapolarse a fuente C hasta cerrar la brecha.

La magnitud de la brecha no prueba por sí sola la causa. Sí prueba que el promedio global es insuficiente para tomar la decisión.

Caso longitudinal: slice reciente

En `clasificacion_binaria_sintetica.csv`, la columna `slice` permite construir una tabla mínima:

Slice	Observaciones	F1	Recall	Lectura
base	180	0.82	0.79	desempeño esperado
reciente	60	0.61	0.48	posible mismatch

Aunque el número exacto dependa de la partición y del modelo, la pregunta profesional permanece: si el uso real se parece al slice **reciente**, el promedio global no justifica despliegue. La acción razonable es construir un dev set más parecido a producción, revisar variables que cambian en el tiempo y reportar la limitación.

Punto de control

Una recomendación técnica es incompleta si no responde:

1. ¿qué evidencia observaste?;
2. ¿qué diagnóstico explica esa evidencia?;
3. ¿qué acción concreta sigue?;
4. ¿qué límite o riesgo debe comunicarse?

Verificaciones seleccionadas

1. Si el F1 global es 0.81 y el F1 de un slice es 0.57, la brecha es 0.24.
2. Una brecha por slice no prueba la causa; prueba que el promedio global es insuficiente para la decisión.
3. Si dev y test tienen proporciones de clase muy distintas, primero se audita la construcción de particiones antes de hacer más tuning.

Procedimiento de auditoría por slice

Una auditoría mínima por slice sigue cuatro pasos. Primero, define slices antes de mirar demasiados resultados: periodo, fuente, región, rango de edad, instrumento de medición o categoría operativa relevante. Segundo, calcula para cada slice tamaño, tasa de clase positiva, métrica principal y tipo de error dominante. Tercero, compara contra el desempeño global y contra el baseline; un slice puede tener bajo desempeño absoluto pero seguir mejorando el baseline, o puede empeorar aunque el promedio global suba. Cuarto, escribe una acción: recoger más datos, cambiar partición, ajustar umbral, crear feature, separar modelo o limitar el uso.

La auditoría también debe proteger contra conclusiones exageradas. Un slice con 12 observaciones no sostiene una afirmación fuerte, pero sí puede activar una revisión. Un slice grande con caída persistente sí cambia la recomendación. En el reporte, el tamaño del slice y la métrica deben aparecer juntos.

Para explorar más

- **Extensión matemática:** calcula tasas de error por slice y estima la brecha respecto al promedio global.
- **Extensión computacional:** crea una tabla de errores con columnas para predicción, etiqueta, slice, score y tipo de fallo.
- **Conexión con proyecto:** define dos slices que podrían revelar data mismatch y explica qué acción tomarías si fallan.

Revisión del capítulo

El análisis de errores convierte una métrica agregada en diagnóstico. Separar falsos positivos, falsos negativos, casos ambiguos y slices críticos revela fallas que un promedio puede ocultar.

Data mismatch aparece cuando train, dev o test no representan la misma población, periodo o proceso de medición. Antes de ajustar hiperparámetros, hay que revisar si la caída de desempeño se explica por cambio de distribución.

Una recomendación técnica debe tener estructura: evidencia observada, diagnóstico probable, acción concreta y límite. Sin esa estructura, el reporte queda como una opinión difícil de auditar.

Términos clave

- **Error analysis:** revisión sistemática de casos donde el modelo falla.
- **Slice:** subconjunto de datos definido por una característica relevante.
- **Data mismatch:** diferencia entre distribuciones o condiciones de dev, test o producción.
- **Diagnóstico:** explicación técnica probable de un patrón de error.
- **Recomendación técnica:** acción proporcional a evidencia, diagnóstico y límites.
- **Límite:** condición bajo la cual la conclusión no debe extrapolarse.

Ejercicios

Preguntas conceptuales

1. Proponga tres slices relevantes para un proyecto de predicción de abandono de clientes.
2. ¿Por qué un F1 global puede ocultar una falla grave?

Problemas cuantitativos

3. Dada una tabla de métricas por slice, identifique el slice crítico y calcule la brecha contra el desempeño global.
4. Si dev tiene 30 % de clase positiva y test 12 %, ¿qué sospecha antes de ajustar hiperparámetros?

Práctica computacional

5. Escriba una función que calcule una métrica por grupo.
6. Compare distribución de una variable clave entre dev y test con una tabla o gráfico.

Interpretación y pensamiento crítico

7. ¿Qué evidencia sugeriría data mismatch entre dev y test?
8. Escriba una recomendación técnica con la estructura evidencia-diagnóstico-acción.

Proyecto de cierre

9. Para su proyecto, entregue una tabla de error analysis con al menos tres slices, diagnóstico por slice y una acción concreta.

Ejercicios integradores: estrategia de machine learning

Esta sección entrena la habilidad central del módulo: convertir métricas y particiones en decisiones defendibles. Una buena respuesta debe nombrar la evidencia, justificar por qué esa evidencia es relevante y reconocer qué todavía no se sabe.

Resultados de práctica

Al terminar este banco deberías poder:

1. elegir una métrica según costo de error;
2. diseñar particiones que respeten el problema;
3. detectar leakage antes de confiar en una métrica;
4. interpretar curvas de aprendizaje;
5. convertir error analysis en una recomendación técnica.

Mapa del banco

Bloque	Evidencia de aprendizaje
A	métrica principal, métrica secundaria y baseline
B	particiones, validación y leakage
C	curvas, variabilidad y validación cruzada
D	análisis de errores por slices

Criterio de respuesta profesional

En problemas aplicados, una respuesta completa suele tener esta forma:

`métrica elegida -> baseline -> partición -> riesgo de leakage -> decisión o siguiente experimento`

Si falta una de esas piezas, la comparación queda débil.

Bloque A: métricas

- A1.** En un problema con falsos negativos costosos, explique por qué accuracy puede ser una mala métrica principal.
- A2.** Dada una matriz de confusión, calcule precision, recall y F1.
- A3.** Proponga una métrica principal y una métrica secundaria para un proyecto de detección temprana de riesgo.

Bloque B: particiones y leakage

- B1.** Explique por qué escalar antes de partir datos puede contaminar validación.
- B2.** Diseñe una partición para datos temporales. ¿Por qué `shuffle=True` puede ser inadecuado?
- B3.** Identifique dos formas de leakage en proyectos estudiantiles.

Bloque C: validación y curvas

- C1.** Interprete una curva con train alto y validation bajo.
- C2.** Explique cuándo más datos probablemente ayudarían.
- C3.** ¿Qué información agrega cross-validation frente a un único split?

Bloque D: error analysis

- D1.** Proponga slices para analizar errores en clasificación de dígitos escritos a mano.
- D2.** Escriba una recomendación técnica basada en un slice donde el F1 cae de 0.80 a 0.55.
- D3.** Explique por qué una métrica agregada no reemplaza inspección de errores.

Respuestas seleccionadas

- A1.** Accuracy puede ser alta aunque el modelo ignore la clase importante. Si los falsos negativos son costosos, recall, F1 o una métrica ponderada por costo pueden reflejar mejor la decisión.
- B1.** Escalar antes de partir datos usa media y desviación calculadas con información de validación. Eso contamina la evaluación porque el preprocesamiento ya vio datos que debían permanecer independientes.
- C3.** Cross-validation agrega una estimación de variabilidad. Un único split puede favorecer o perjudicar accidentalmente a un modelo; varios folds muestran si el desempeño es estable.

D2. Una recomendación mínima sería: “El F1 global cae de 0.80 a 0.55 en el slice crítico. El diagnóstico probable es que el modelo no representa bien ese subgrupo. Antes de recomendar despliegue, se debe analizar variables de ese slice, ajustar métrica de selección y recolectar o reponderar datos relevantes.”

Parte V: Aprendizaje no supervisado

Parte V: Aprendizaje no supervisado

Esta parte estudia estructura sin etiquetas: SVD, PCA, K-Means y recomendación básica. La advertencia permanente es que una estructura matemática no equivale automáticamente a una conclusión de dominio.

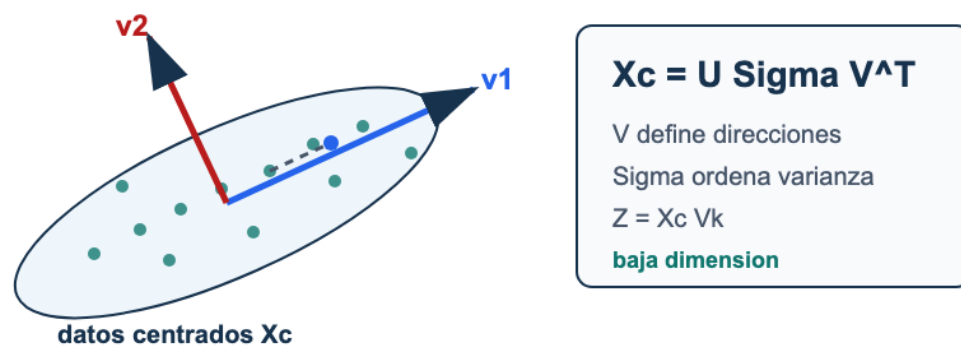


Figura 16: PCA como proyección: datos centrados se factorizan con SVD; los primeros vectores de V definen ejes principales y la matriz Z contiene coordenadas de baja dimensión.

Lectura de la figura. Los métodos no supervisados producen coordenadas, grupos o similitudes. La interpretación no viene gratis: debe justificarse con estructura matemática, estabilidad, variables originales y límites de uso.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. escribir una SVD y usarla para explicar PCA;
2. calcular proyecciones, varianza explicada y reconstrucción;
3. ajustar K-Means y leer inertia/silhouette con cautela;
4. construir una recomendación item-item mínima;
5. distinguir estructura matemática de interpretación aplicada;
6. declarar riesgos como cold start, popularidad y sobreinterpretación.

Mapa de la parte

Capítulo	Pregunta guía	Evidencia esperada
SVD y PCA	¿Qué información conserva una proyección?	Forma, varianza explicada y advertencia.
K-Means	¿Qué significa agrupar por distancia?	Escalamiento, curva de k e interpretación.
Recomendación latente	¿Cómo se usan similitudes para sugerir ítems?	Matriz usuario-ítem y límite de cold start.
Ejercicios integradores	¿Puedes separar cálculo, visualización y conclusión?	Resultado verificable y cautela aplicada.

Criterio de salida

La parte queda dominada cuando puedes usar PCA, K-Means o recomendación para explorar estructura sin afirmar causalidad, identidad de grupos o valor aplicado sin evidencia adicional.

SVD y PCA

Pregunta aplicada

Un dataset puede tener decenas o miles de variables. ¿Podemos representarlo en menos dimensiones sin perder la estructura principal?

PCA responde:

buscar direcciones ortogonales que capturan la mayor varianza posible.

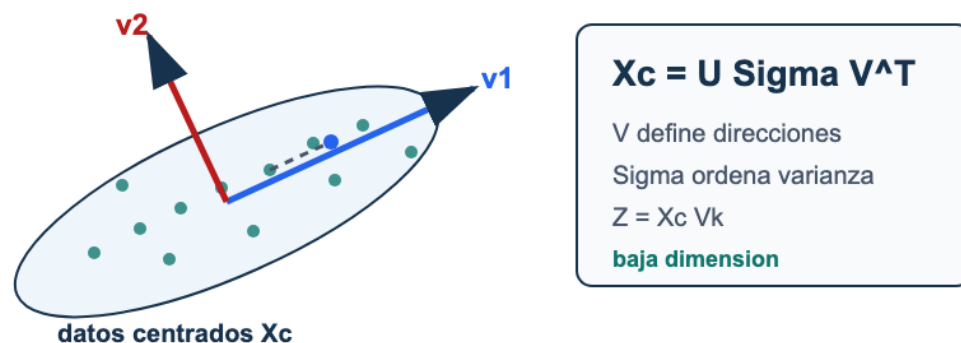


Figura 17: PCA como proyección: datos centrados se factorizan con SVD; los primeros vectores de V definen ejes principales y la matriz Z contiene coordenadas de baja dimensión.

Lectura de la figura. PCA empieza con datos centrados, usa la SVD para encontrar direcciones de alta varianza y proyecta sobre V_k . La proyección puede servir para visualizar, comprimir o alimentar otro modelo, pero no prueba causalidad.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir la SVD de una matriz;
2. explicar por qué PCA requiere centrar datos;
3. calcular la forma de una proyección $Z = X_c V_k$;
4. interpretar varianza explicada y error de reconstrucción;
5. distinguir reducción de dimensión de interpretación causal.

SVD

Para una matriz $X \in \mathbb{R}^{n \times p}$:

$$X = U\Sigma V^\top.$$

Donde:

- U contiene direcciones en el espacio de observaciones;
- Σ contiene valores singulares no negativos;
- V contiene direcciones en el espacio de variables.

PCA como SVD del dato centrado

Primero centramos:

$$X_c = X - \bar{X}.$$

Luego:

$$X_c = U\Sigma V^\top.$$

Los componentes principales son las primeras filas de $V^\top$, o columnas de V , asociadas a los valores singulares más grandes.

Proyección

Si tomamos k componentes:

$$Z = X_c V_k.$$

Aquí $Z \in \mathbb{R}^{n \times k}$ es la representación reducida.

Varianza explicada

La varianza explicada por el componente j es proporcional a:

$$\sigma_j^2.$$

La razón de varianza explicada:

$$\frac{\sigma_j^2}{\sum_{\ell} \sigma_{\ell}^2}.$$

Ejemplo resuelto: error desde valores singulares

Supón que una matriz centrada tiene valores singulares:

$$\sigma_1 = 9, \quad \sigma_2 = 4, \quad \sigma_3 = 1.$$

La energía total bajo la norma de Frobenius es:

$$9^2 + 4^2 + 1^2 = 98.$$

Si usamos solo $k = 1$, descartamos σ_2 y σ_3 . El error de reconstrucción cuadrático es:

$$4^2 + 1^2 = 17.$$

La varianza explicada acumulada por el primer componente es:

$$\frac{9^2}{98} \approx 0.827.$$

La lectura profesional es doble: un componente conserva cerca de 82.7% de la variabilidad medida por PCA, pero todavía descarta una parte que puede ser importante para ciertos grupos, variables o decisiones posteriores.

Derivación guiada: reconstrucción y pérdida de información

Si usamos k componentes, retenemos:

$$X_c \approx U_k \Sigma_k V_k^\top.$$

La parte descartada corresponde a componentes $k + 1, \dots, r$. Bajo la norma de Frobenius, el error de reconstrucción queda ligado a los valores singulares descartados:

$$\|X_c - \hat{X}_c\|_F^2 = \sum_{j=k+1}^r \sigma_j^2.$$

Esta fórmula explica por qué la varianza explicada acumulada y el error de reconstrucción son dos caras del mismo diagnóstico. Retener pocos componentes puede ser útil para visualizar, pero siempre descarta variación.

Ejemplo resuelto: elegir k

Supón que los primeros componentes explican:

Componente	Varianza explicada	Acumulada
1	0.42	0.42
2	0.23	0.65
3	0.12	0.77
4	0.06	0.83

Si la meta es visualizar, $k = 2$ puede ser suficiente. Si la meta es reconstruir con poca pérdida, $k = 4$ puede ser más razonable. La elección depende de la decisión, no de un número mágico.

Reconstrucción

Con k componentes:

$$\hat{X}_c = Z V_k^\top.$$

La reconstrucción en escala original:

$$\hat{X} = \hat{X}_c + \bar{X}.$$

El error de reconstrucción ayuda a medir pérdida de información.

Ejemplo resuelto: proyección y reconstrucción

Considera dos observaciones con dos variables:

$$X = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Primero centramos por columnas. La media es $\bar{X} = (1, 1)$, entonces:

$$X_c = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Una dirección principal para estos datos es

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

La proyección con $k = 1$ es:

$$Z = X_c v_1 = \begin{bmatrix} \sqrt{2} \\ -\sqrt{2} \end{bmatrix}.$$

La reconstrucción centrada queda:

$$\hat{X}_c = Z v_1^\top = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Al sumar la media:

$$\hat{X} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

En este ejemplo la reconstrucción con un componente es exacta porque los datos centrados tienen rango 1. En un dataset real, $k = 1$ normalmente conserva solo una parte de la variación.

Evidencia	Pregunta
-----------	----------

Caso longitudinal: PCA sintético

El archivo `pca_sintetico.csv` contiene cuatro variables generadas desde una estructura latente de baja dimensión. El ejercicio profesional no es “hacer que PCA se vea bonito”, sino justificar qué se conserva y qué se pierde.

Evidencia	Pregunta
media de entrenamiento	¿con qué parámetros se centró?
varianza explicada acumulada	¿cuántos componentes se retienen?
error de reconstrucción	¿qué se pierde al reducir dimensión?
cargas principales	¿qué variables dominan cada componente?
advertencia	¿qué interpretación causal no se puede afirmar?

Si los dos primeros componentes explican mucha varianza, puedes usarlos para visualizar o comprimir. Eso no significa que los componentes sean causas, grupos naturales o factores reales del dominio. La interpretación exige evidencia adicional.

PCA no es interpretabilidad automática

Advertencia

Un componente principal es una combinación lineal de variables. Puede ser útil, pero no siempre tiene una interpretación humana directa.

Interpretar PCA requiere:

- revisar cargas de variables;
- medir varianza explicada;
- visualizar proyecciones;
- conectar componentes con el dominio.

Punto de control

Antes de usar PCA en un proyecto, responde:

1. ¿centraste con parámetros aprendidos en entrenamiento?
2. ¿cuánta varianza explican los componentes usados?
3. ¿la proyección se usa para visualizar, modelar o comprimir?
4. ¿qué interpretación no puedes afirmar?

Verificaciones seleccionadas

1. Si X tiene forma 500×64 y se usan $k = 2$ componentes, la proyección $Z = X_c V_k$ tiene forma 500×2 .
2. Si los primeros 10 componentes explican 85 % de la varianza, todavía se pierde 15 % de la variabilidad bajo esa medida.
3. Un scatterplot PCA 2D puede sugerir separación visual, pero no prueba causalidad ni garantiza buen desempeño predictivo.

Ejemplo resuelto: PCA dentro de un pipeline

Si PCA se usa antes de un clasificador, el centrado y los componentes deben aprenderse solo con entrenamiento. En scikit-learn, la forma segura es:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=10)),
    ("model", LogisticRegression())
])
```

En validación cruzada, cada fold aprende su propio **scaler** y su propio **PCA** con el train fold. Hacer PCA con todo el dataset antes de partir datos introduce leakage porque la proyección ya vio validación o test.

Para explorar más

- **Extensión matemática:** reconstruye una matriz pequeña con rango 1 y compara el error de reconstrucción con el de rango 2.
- **Extensión computacional:** grafica varianza explicada acumulada y marca el punto donde agregar componentes deja de cambiar la interpretación.
- **Conexión con proyecto:** decide si PCA se usará para visualización, compresión, diagnóstico o como paso previo de modelado.

Revisión del capítulo

SVD factoriza una matriz en direcciones y magnitudes de variación. PCA usa esa estructura sobre datos centrados para construir proyecciones de baja dimensión que conservan tanta varianza como sea posible bajo el número de componentes elegido.

La forma de los objetos importa: V_k define direcciones, $Z = X_c V_k$ contiene scores y la reconstrucción aproxima los datos originales. La varianza explicada ayuda a elegir k , pero no convierte automáticamente los componentes en causas o conceptos de dominio.

PCA es una herramienta de compresión, visualización o preprocesamiento. Su uso profesional exige declarar qué conserva, qué pierde y qué interpretación no está justificada por la proyección.

Términos clave

- **SVD:** factorización $X = U\Sigma V^T$ que expone direcciones principales.
- **PCA:** proyección lineal que conserva la mayor varianza posible en pocas dimensiones.
- **Dato centrado:** matriz después de restar la media de cada variable.
- **Componente principal:** dirección lineal de alta varianza.
- **Varianza explicada:** proporción de variabilidad capturada por componentes.
- **Reconstrucción:** aproximación del dato original desde una proyección de baja dimensión.

Ejercicios

Preguntas conceptuales

1. Explique por qué PCA debe centrar los datos antes de calcular componentes.
2. ¿Por qué un componente principal no es automáticamente interpretable?

Problemas cuantitativos

3. Si X tiene forma (500, 64) y usamos $k=2$, ¿qué forma tiene la proyección PCA?
4. ¿Qué significa que los primeros 10 componentes expliquen 85 % de la varianza?

Práctica computacional

5. Implemente una proyección PCA usando SVD sobre datos centrados.
6. Calcule y grafique varianza explicada acumulada.

Interpretación y pensamiento crítico

7. Si dos componentes explican poca varianza pero separan visualmente grupos, ¿qué conclusión puede y no puede afirmar?

Proyecto de cierre

8. Aplique PCA a un dataset de su proyecto o a uno pequeño generado para práctica, justifique k , muestre una proyección y escriba una advertencia de interpretación.

K-Means y clustering

Pregunta aplicada

Si no tenemos etiquetas, aún podemos preguntar:

¿hay grupos de observaciones con patrones similares?

K-Means busca particionar datos en k grupos representados por centroides.

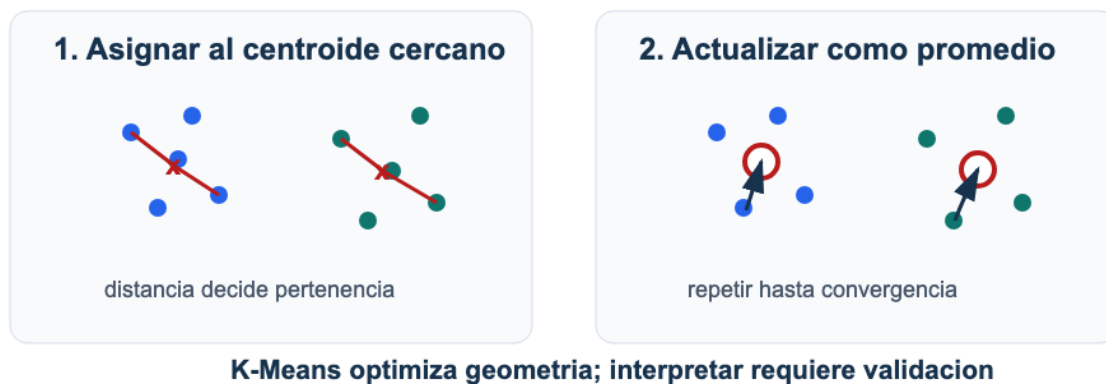


Figura 18: K-Means alterna dos pasos: asignar cada punto al centroide más cercano y actualizar centroides como promedio de los puntos asignados.

Lectura de la figura. K-Means alterna entre asignar observaciones al centroide más cercano y mover cada centroide al promedio de su grupo. La interpretación aplicada viene después de validar que esa geometría tenga sentido.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir la función objetivo de K-Means;
2. explicar los pasos de asignación y actualización;
3. interpretar inercia y silhouette sin absolutizarlos;
4. justificar el escalamiento antes de usar distancias euclídeas;
5. reconocer cuándo un cluster no debe traducirse directamente en una decisión.

Función objetivo

K-Means minimiza la suma de distancias cuadradas a centroides:

$$\sum_{i=1}^n \|x_i - \mu_{c_i}\|_2^2.$$

Donde:

- c_i es el cluster asignado a x_i ;
- μ_{c_i} es el centroide correspondiente.

Algoritmo

1. Inicializar centroides.
2. Asignar cada punto al centroide más cercano.
3. Recalcular cada centroide como promedio del grupo.
4. Repetir hasta convergencia.

Inertia

inertia es la suma de distancias cuadradas dentro de clusters.

Menor *inertia* no siempre significa mejor segmentación:

- baja al aumentar k ;
- no está normalizada;
- puede favorecer clusters poco interpretables.

Silhouette score

Para una observación:

$$s = \frac{b - a}{\max(a, b)}.$$

Donde:

- a : distancia media al propio cluster;
- b : distancia media al cluster vecino más cercano.

Valores cercanos a 1 sugieren separación; valores cercanos a 0 sugieren solapamiento.

Ejemplo resuelto: por qué escalar cambia clusters

Supón dos variables: ingreso mensual en pesos y número de visitas a una página. Dos observaciones son:

$$x_1 = (5,000,000, 10), \quad x_2 = (5,100,000, 80).$$

La diferencia en ingreso es 100,000, mientras la diferencia en visitas es 70. En distancia euclídea sin escalar, el ingreso domina casi todo:

$$\sqrt{100000^2 + 70^2} \approx 100000.$$

K-Means tenderá a agrupar por ingreso aunque visitas sea más relevante para la decisión. Después de estandarizar, cada variable se mide en desviaciones estándar y la distancia refleja patrones relativos, no unidades arbitrarias. Esto no garantiza clusters correctos, pero evita que una unidad de medida decida el resultado sin justificación.

Ejemplo resuelto: asignación a centroides

Supón dos centroides en una dimensión:

$$\mu_1 = 2, \quad \mu_2 = 8.$$

Para una observación $x = 5$:

$$|5 - 2| = 3, \quad |5 - 8| = 3.$$

La observación está empatada. En implementación debe haber una regla de desempate. Este ejemplo muestra que K-Means es geométrico: asigna por distancia, no por significado de negocio.

Ejemplo resuelto: una iteración completa

Supón cuatro puntos en una dimensión:

$$x = (1, 2, 8, 10)$$

y dos centroides iniciales:

$$\mu_1 = 1, \quad \mu_2 = 10.$$

Paso 1: asignar puntos.

Punto	Distancia a μ_1	Distancia a μ_2	Cluster
1	0	9	1
2	1	8	1
8	7	2	2
10	9	0	2

La inercia inicial con estas asignaciones es:

$$(1 - 1)^2 + (2 - 1)^2 + (8 - 10)^2 + (10 - 10)^2 = 5.$$

Paso 2: actualizar centroides.

$$\mu_1^{nuevo} = \frac{1 + 2}{2} = 1.5, \quad \mu_2^{nuevo} = \frac{8 + 10}{2} = 9.$$

Con los nuevos centroides, la inercia para las mismas asignaciones es:

$$(1 - 1.5)^2 + (2 - 1.5)^2 + (8 - 9)^2 + (10 - 9)^2 = 2.5.$$

La inercia bajó, pero eso no prueba que $k = 2$ sea la mejor decisión aplicada. Solo muestra que la actualización mejoró el objetivo geométrico de K-Means.

Derivación guiada: por qué el centroide es el promedio

Para un cluster con puntos $x_1, \dots, x_m$, K-Means escoge un centroide μ que minimiza:

$$\sum_{i=1}^m \|x_i - \mu\|_2^2.$$

En una dimensión, derivar respecto a μ da:

$$\frac{d}{d\mu} \sum_{i=1}^m (x_i - \mu)^2 = -2 \sum_{i=1}^m (x_i - \mu).$$

Igualando a cero:

$$\sum_{i=1}^m x_i - m\mu = 0 \quad \Rightarrow \quad \mu = \frac{1}{m} \sum_{i=1}^m x_i.$$

En varias dimensiones ocurre componente a componente. Por eso la actualización del centroide es el promedio de los puntos asignados.

Escalamiento

K-Means usa distancia euclídea. Por tanto, las escalas importan.

```
Pipeline([
    ("scaler", StandardScaler()),
    ("kmeans", KMeans(n_clusters=3))
])
```

Limitaciones

K-Means funciona mejor cuando los clusters son:

- aproximadamente esféricos;
- de tamaños comparables;
- separables por distancia euclídea.

Puede fallar con formas no convexas, densidades distintas o variables mal escaladas.

Caso longitudinal: clusters sintéticos

El archivo `clusters_sinteticos.csv` fue generado con tres centros compactos. Eso lo hace útil para aprender K-Means, pero también peligroso si se generaliza demasiado: favorece justo la geometría que K-Means prefiere.

Evidencia	Uso
scatterplot escalado	inspección geométrica inicial
inertia por k	reducción de compactación
silhouette	separación relativa
tamaños de clusters	detectar grupos diminutos
estabilidad por semilla	revisar sensibilidad

Si $k = 3$ gana en este dataset, no significa que “siempre hay tres segmentos”. Significa que, bajo esta geometría sintética y esta escala, tres centroides resumen bien la nube de puntos.

Estabilidad por inicialización

K-Means puede converger a soluciones distintas si cambian los centroides iniciales. Por eso un reporte debe fijar `random_state` y usar varias inicializaciones (`n_init`). Si dos corridas con el mismo k producen centroides muy distintos o clusters con tamaños incompatibles, la segmentación no es estable.

La estabilidad no convierte los clusters en categorías reales. Solo indica que el algoritmo encuentra una partición parecida bajo pequeñas variaciones de inicialización. La interpretación de dominio sigue siendo una segunda etapa.

Punto de control

Antes de reportar clusters, verifica:

1. ¿las variables están en escalas comparables?
2. ¿probaste más de un k ?
3. ¿los clusters son interpretables con variables originales?
4. ¿hay un cluster demasiado grande o demasiado pequeño?
5. ¿qué decisión concreta usaría esta segmentación?

Verificaciones seleccionadas

1. La inercia siempre tiende a bajar cuando aumenta k ; por eso no basta para elegir el número de clusters.
2. Si dos variables tienen escalas muy distintas, K-Means puede agrupar por la variable de mayor escala aunque no sea la más relevante.
3. Un cluster no es una categoría real hasta que se interpreta con variables originales y contexto de decisión.

Ejemplo resuelto: elegir k con evidencia incompleta

Supón:

k	Inertia	Silhouette
2	420	0.48
3	260	0.55
4	210	0.51

La inercia baja de 3 a 4, pero silhouette cae. Una decisión defendible podría elegir $k = 3$, siempre que los clusters tengan tamaño razonable y puedan describirse con variables originales. La tabla no prueba que existan tres grupos naturales; solo apoya que $k = 3$ es una representación útil bajo esta geometría.

Para explorar más

- **Extensión matemática:** ejecuta una iteración de K-Means a mano con cuatro puntos y dos centroides iniciales distintos.
- **Extensión computacional:** compara inercia y silueta para varios valores de k , usando la misma escala de variables.
- **Conexión con proyecto:** describe qué significaría cada cluster para una decisión real y qué variables dominan esa interpretación.

Revisión del capítulo

K-Means busca centroides que reduzcan la suma de distancias cuadráticas dentro de clusters. El algoritmo alterna asignación de puntos al centroide más cercano y actualización de centroides como promedios.

Inercia siempre tiende a bajar cuando aumenta k , por lo que no puede usarse sola como prueba de calidad. Silhouette aporta otra señal, pero también depende de geometría, escala y estructura de los datos.

Una segmentación profesional requiere escalamiento, varias inicializaciones, descripción con variables originales y cautela aplicada. Un cluster matemático no es automáticamente un grupo accionable ni una categoría real de dominio.

Términos clave

- **Clustering:** agrupación no supervisada de observaciones.
- **Centroide:** punto representativo de un cluster.
- **Inercia:** suma de distancias cuadradas a centroides.
- **Silhouette:** métrica que compara cercanía al cluster propio frente a otros.
- **Escalamiento:** transformación necesaria cuando las variables tienen unidades distintas.
- **Segmentación:** interpretación aplicada de grupos, que requiere validación de negocio o dominio.

Ejercicios

Preguntas conceptuales

1. Explique por qué inercia siempre tiende a bajar cuando aumenta k .
2. ¿Por qué K-Means debe usarse con escalamiento en variables de unidades distintas?

Problemas cuantitativos

3. Asigne tres puntos en una dimensión a dos centroides dados y calcule inercia.
4. Si un cluster contiene 90 % de las observaciones, ¿qué revisaría antes de reportarlo?

Práctica computacional

5. Ajuste K-Means para varios valores de k y reporte inertia y silhouette.
6. Construya un pipeline con escalamiento y K-Means.

Interpretación y pensamiento crítico

7. Proponga una situación donde clusters matemáticamente separados no deberían convertirse directamente en una decisión aplicada.
8. ¿Qué evidencia usaría para nombrar un cluster sin caer en sobreinterpretación?

Proyecto de cierre

9. Genere una segmentación con K-Means, describa cada cluster usando variables originales y escriba una recomendación cautelosa de uso.

Recomendación básica y estructura latente

Pregunta aplicada

Si sabemos qué ítems han gustado o sido usados por ciertos usuarios, podemos preguntar:

¿qué ítems son similares y qué recomendaríamos a un usuario?

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. construir una matriz usuario-ítem simple;
2. calcular e interpretar similitud coseno;
3. describir una recomendación item-item;
4. explicar la idea de factores latentes por SVD;
5. reconocer problemas de popularidad, cold start y sesgo.

Matriz usuario-ítem

Una matriz R puede representar:

- filas: usuarios;
- columnas: ítems;
- entradas: interacción, rating, compra, vista o preferencia.

Ejemplo:

$$R_{ui} = 1$$

si el usuario u interactuó con el ítem i .

Similitud coseno

Para dos ítems representados por vectores x y y :

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}.$$

La similitud coseno compara dirección más que magnitud.

Ejemplo resuelto: similitud coseno

Supón dos ítems con vectores de interacción:

$$x = (1, 1, 0, 0), \quad y = (1, 0, 1, 0).$$

Entonces:

$$x^\top y = 1, \quad \|x\| = \sqrt{2}, \quad \|y\| = \sqrt{2}.$$

Por tanto:

$$\cos(x, y) = \frac{1}{2} = 0.5.$$

La interpretación es moderada similitud: comparten un usuario, pero cada uno tiene otro usuario distinto.

Derivación guiada: qué ignora la similitud coseno

La similitud coseno normaliza por las normas:

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}.$$

Eso significa que dos ítems pueden tener similitud alta si apuntan en dirección parecida, aunque uno tenga muchas más interacciones que el otro. Esta propiedad puede ser útil para reducir el sesgo por popularidad, pero no lo elimina. Si un ítem aparece en casi todos los perfiles, puede seguir dominando rankings por cobertura y disponibilidad de datos.

La lectura profesional de una similitud debe incluir tres preguntas: qué usuarios comparten los ítems, cuántas interacciones sustentan el cálculo y qué ítems quedan invisibles por falta de datos.

Recomendación item-item

Flujo básico:

1. Construir matriz usuario-ítem.
2. Calcular similitud entre ítems.
3. Para un ítem semilla, buscar ítems similares.
4. Excluir ítems ya vistos si se recomienda a un usuario.

Ejemplo resuelto: recomendación item-item

Supón la siguiente matriz binaria de interacción:

Usuario	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>u1</i>	1	1	0	0
<i>u2</i>	1	1	1	0
<i>u3</i>	0	0	1	1
<i>u4</i>	0	1	0	1

Los vectores de ítems son:

$$A = (1, 1, 0, 0), \quad B = (1, 1, 0, 1), \quad C = (0, 1, 1, 0), \quad D = (0, 0, 1, 1).$$

Para recomendar a partir del ítem *A*, calculamos similitudes:

$$\cos(A, B) = \frac{2}{\sqrt{2}\sqrt{3}} \approx 0.816,$$

$$\cos(A, C) = \frac{1}{\sqrt{2}\sqrt{2}} = 0.5, \quad \cos(A, D) = 0.$$

Si el usuario semilla interactuó con *A*, el ítem más similar es *B*. Si ese usuario ya vio *B*, el siguiente candidato por similitud sería *C*. La recomendación debe excluir ítems ya vistos y declarar que esta regla no resuelve cold start ni sesgo de popularidad.

Evaluación offline: Precision@k

Una evaluación mínima de recomendación separa algunas interacciones conocidas como si fueran futuras. Luego pregunta si el recomendador las recupera entre sus primeras sugerencias.

Si para un usuario ocultamos los ítems relevantes $\{C, D\}$ y el sistema recomienda los tres primeros ítems:

$$[B, C, E],$$

entonces solo uno de los tres recomendados es relevante. La precisión en 3 es:

$$\text{Precision@3} = \frac{1}{3}.$$

Si además queremos saber cuántos relevantes recuperó:

$$\text{Recall@3} = \frac{1}{2}.$$

Estas métricas no prueban satisfacción real. Solo verifican si el ranking recupera interacciones ocultas bajo una simulación offline. Un reporte debe decir qué se ocultó, cómo se partió el dato y qué usuarios o ítems quedaron sin recomendación.

Baseline de popularidad

Antes de celebrar un recomendador personalizado, compáralo contra una regla simple: recomendar los ítems más populares que el usuario no ha visto. Este baseline suele ser fuerte porque muchos datasets tienen concentración de interacciones en pocos ítems.

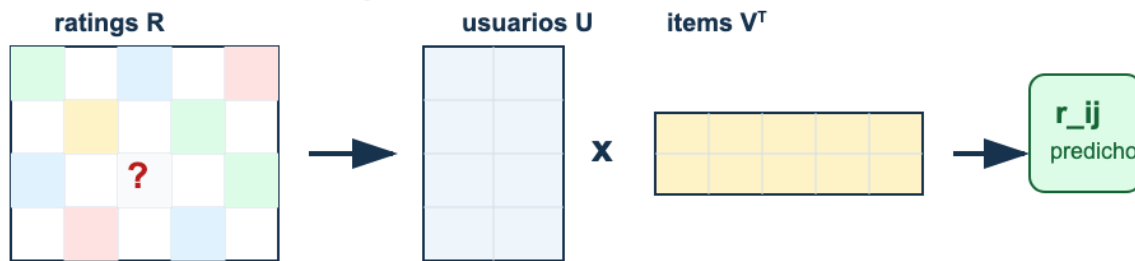
Si el modelo latente no supera popularidad en Precision@k o cobertura, todavía no hay evidencia de personalización útil. Si lo supera, el reporte debe mostrar cuánto mejora y para qué grupos de usuarios. La personalización no se presume: se compara.

SVD para factores latentes

SVD puede aproximar:

$$R \approx U_k \Sigma_k V_k^\top.$$

Recomendacion: completar una matriz con estructura latente



Lectura: factores latentes explican patrones; la validacion decide si recomiendan bien.

Figura 19: Sistemas de recomendación: una matriz usuario-item incompleta se aproxima mediante factores latentes de usuarios e ítems; una calificación faltante se predice con un producto punto.

Lectura de la figura. La factorización representa usuarios e ítems en una baja dimensión compartida. Una recomendación es una predicción incompleta de la matriz, pero también una intervención que puede reforzar sesgos o popularidad.

Esto produce representaciones latentes de usuarios e ítems.

En este curso usamos una versión introductoria, sin entrar en todos los detalles de datos faltantes, sesgos y evaluación offline de recomendadores.

Caso longitudinal: interacciones de recomendación

El archivo `recomendacion_interacciones.csv` contiene usuarios, ítems y ratings ficticios. Su función es mostrar la lógica de una matriz usuario-item pequeña:

Evidencia	Pregunta
matriz usuario-item	¿qué entradas se observaron y cuáles faltan?
promedio por ítem	¿hay sesgo de popularidad?
similitud item-item	¿qué ítems parecen cercanos?
cobertura	¿a cuántos usuarios/ítems puede recomendar?
evaluación offline	¿qué ratings ocultos se predicen?

Un recomendador puede producir rankings plausibles y aun así amplificar popularidad, ignorar ítems nuevos o recomendar sin diversidad. Por eso la pregunta aplicada no es solo “qué ítem tiene mayor score”, sino “qué se omite y a quién afecta”.

Riesgos de recomendación

Los sistemas de recomendación pueden:

- reforzar popularidad;
- encerrar usuarios en burbujas;
- amplificar sesgos;
- optimizar interacción sin optimizar bienestar;
- fallar con usuarios o ítems nuevos.

Punto de control

Antes de recomendar, responde:

1. ¿la similitud refleja preferencia o solo popularidad?
2. ¿se excluyen ítems ya vistos?
3. ¿qué ocurre con usuarios nuevos?
4. ¿qué sesgo puede amplificarse?

Verificaciones seleccionadas

1. Una entrada faltante en la matriz usuario-item no significa rating cero; significa no observado.
2. La similitud coseno alta entre dos ítems no prueba que sean equivalentes; solo indica patrones de interacción parecidos bajo los datos disponibles.
3. Un ranking sin cobertura puede verse preciso para usuarios frecuentes y fallar para usuarios o ítems con pocas interacciones.

Ejemplo resuelto: cobertura mínima

Supón un recomendador que produce al menos una recomendación para 70 de 100 usuarios y cubre 12 de 50 ítems disponibles. Entonces:

$$\text{cobertura de usuarios} = 70/100 = 70\%, \quad \text{cobertura de ítems} = 12/50 = 24\%.$$

Aunque las recomendaciones visibles parezcan razonables, el sistema deja sin recomendación a 30 % de usuarios y concentra exposición en pocos ítems. Esa evidencia debe aparecer antes de afirmar que el recomendador funciona bien.

Para explorar más

- **Extensión matemática:** calcula una similitud coseno item-item y compara la recomendación con una basada en producto punto.
- **Extensión computacional:** oculta algunas interacciones conocidas y mide si el sistema las recupera entre las mejores recomendaciones.
- **Conexión con proyecto:** explica qué sesgo de popularidad podría aparecer y cómo lo reportarías sin exagerar la calidad del sistema.

Revisión del capítulo

Un sistema de recomendación básico parte de una matriz usuario-ítem y mide similitudes o factores latentes para sugerir elementos no observados. La similitud coseno compara dirección de interacción y puede mitigar parcialmente diferencias de escala.

Las recomendaciones item-item son útiles porque se pueden explicar a partir de ítems similares, pero siguen dependiendo del historial observado. La idea de factores latentes resume usuarios e ítems en representaciones de baja dimensión.

El límite central es que el comportamiento pasado no define por completo la preferencia futura. Cold start, popularidad y sesgo deben declararse antes de presentar una recomendación como útil o justa.

Términos clave

- **Matriz usuario-ítem:** tabla que representa interacciones entre usuarios e ítems.
- **Similitud coseno:** medida angular entre vectores de interacción.
- **Recomendación item-item:** recomendación basada en ítems similares a uno observado.
- **Factor latente:** representación de baja dimensión aprendida de patrones de interacción.
- **Cold start:** dificultad para recomendar a usuarios o ítems sin historial.
- **Bucle de retroalimentación:** refuerzo de patrones pasados por recomendaciones futuras.

Ejercicios

Preguntas conceptuales

1. Explique por qué similitud coseno puede ser útil cuando algunos ítems son mucho más populares que otros.
2. ¿Qué problema aparece para recomendar a un usuario nuevo sin historial?

Problemas cuantitativos

3. Calcule similitud coseno entre dos vectores binarios de interacción.
4. Dada una matriz usuario-ítem pequeña, identifique los dos ítems más similares.

Práctica computacional

5. Implemente una función de similitud coseno para matrices.
6. Construya una recomendación item-item simple excluyendo ítems ya vistos.

Interpretación y pensamiento crítico

7. Proponga una limitación ética de un sistema de recomendación basado solo en comportamiento pasado.
8. ¿Cómo detectaría si el recomendador solo amplifica popularidad?

Proyecto de cierre

9. Diseñe un recomendador item-item mínimo, reporte tres recomendaciones y escriba una advertencia sobre cold start, popularidad o sesgo.

Ejercicios integradores: aprendizaje no supervisado

Los ejercicios de este módulo deben resolverse con una advertencia constante: en aprendizaje no supervisado no hay una etiqueta que valide automáticamente el significado. Por eso una solución profesional combina cálculo, visualización, estabilidad e interpretación prudente.

Resultados de práctica

Al terminar este banco deberías poder:

1. calcular formas de SVD y proyecciones PCA;
2. explicar qué sí y qué no justifica una reducción de dimensión;
3. usar inercia y silhouette como evidencia limitada;
4. construir una recomendación item-item mínima;
5. separar estructura matemática de interpretación aplicada.

Mapa del banco

Bloque	Evidencia de aprendizaje
A	PCA, SVD, proyección e interpretación cautelosa
B	K-Means, selección de k y diagnóstico geométrico
C	similitud, recomendación y sesgos de interacción

Antes de responder

Para cada ejercicio, identifica:

1. qué estructura matemática estás usando;
2. qué cantidad se puede calcular;
3. qué conclusión sí está permitida;
4. qué conclusión sería una sobreinterpretación.

Bloque A: PCA

A1. Derive las formas de U , Σ y $V^\top$ si X_c tiene forma $(100, 20)$.

A2. Implemente mentalmente los pasos de PCA desde cero: centrar, SVD, seleccionar componentes, proyectar.

A3. Explique por qué un componente con alta varianza no necesariamente es causal ni justo para tomar decisiones.

Bloque B: K-Means

B1. Escriba la función objetivo de K-Means y explique cada término.

B2. Compare inercia y silhouette como evidencia para elegir k .

B3. Proponga un diagnóstico si K-Means produce un cluster con 95 % de los datos.

Bloque C: recomendación

C1. Calcule similitud coseno entre dos vectores binarios simples.

C2. Explique cómo excluir ítems ya vistos por el usuario.

C3. ¿Qué diferencia hay entre recomendar por similitud de contenido y por comportamiento colaborativo?

Respuestas seleccionadas

A1. En SVD reducida, una forma usual es $U \in \mathbb{R}^{100 \times 20}$, $\Sigma \in \mathbb{R}^{20 \times 20}$ y $V^\top \in \mathbb{R}^{20 \times 20}$. En SVD completa, U podría tener forma 100×100 .

A3. Alta varianza significa que una dirección explica dispersión de los datos, no que represente una causa, una variable ética ni una regla justa de decisión.

B3. Un cluster con 95 % de los datos puede indicar k inadecuado, variables mal escaladas, ausencia de estructura separable o outliers que deforman centroides. Antes de interpretar, conviene revisar escalamiento, probar otros k y visualizar variables originales.

C3. La similitud de contenido compara atributos de los ítems. La similitud colaborativa compara patrones de interacción de usuarios. La segunda puede capturar comportamiento colectivo, pero también amplificar popularidad y sesgos históricos.

Parte VI: Reproducibilidad, comunicación y defensa

Parte VI: Reproducibilidad, comunicación y defensa

La última parte integra el curso. Un resultado técnico debe poder reproducirse, comunicarse y defenderse. El objetivo no es solo obtener una métrica, sino dejar evidencia suficiente para que otra persona pueda auditar la decisión.

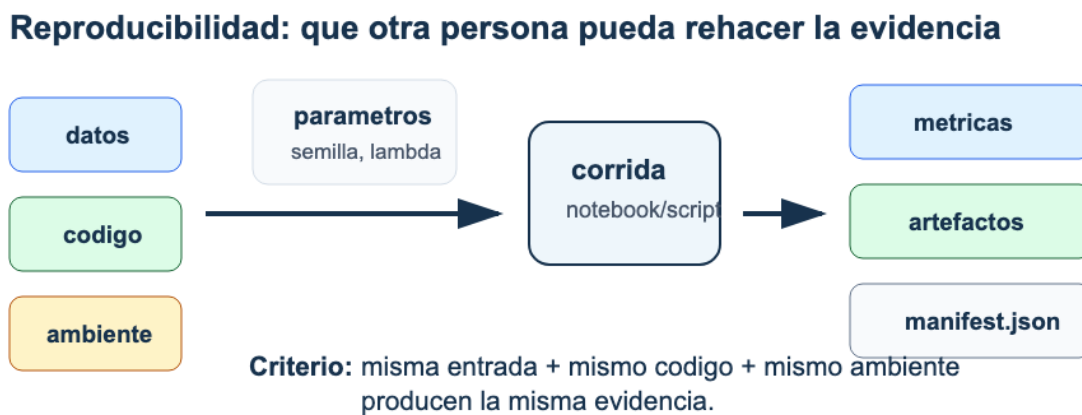


Figura 20: Reproducibilidad: datos, código, ambiente y parámetros alimentan una corrida que produce métricas, artefactos y un manifiesto verificable.

Lectura de la figura. La evidencia final no es solo una tabla de métricas. Es un paquete trazable: datos, código, entorno, parámetros, artefactos, manifiesto y decisión proporcional.

Resultados de aprendizaje de la parte

Al terminar esta parte deberías poder:

1. responder preguntas integradoras del Parcial 2 con evidencia y límites;
2. construir un paquete mínimo reproducible;
3. registrar parámetros, métricas, artefactos y hashes relevantes;
4. escribir una conclusión técnica con evidencia, diagnóstico y acción;
5. preparar una entrega final auditable;
6. defender oralmente tu contribución individual y las limitaciones del proyecto.

Mapa de la parte

Capítulo	Pregunta guía	Evidencia esperada
Síntesis conceptual	¿Puedes conectar modelos, métricas y decisiones?	Respuesta integradora breve.
Reproducibilidad	¿Otra persona puede reconstruir el resultado?	Manifest, entorno y script de evaluación.
Comunicación técnica	¿La conclusión es proporcional a la evidencia?	Resumen ejecutivo y figura funcional.
Entrega y defensa	¿Qué prueba cada archivo y cada afirmación?	Repositorio final y defensa individual.
Ejercicios integradores	¿Puedes cerrar el curso con evidencia auditable?	Paquete, límite y recomendación.

Criterio de salida

El curso queda cerrado cuando puedes entregar un proyecto que otra persona pueda leer, ejecutar o auditar, y cuando puedes defender qué hiciste, qué evidencia obtuviste, qué decisión recomiendas y qué no puedes afirmar.

Síntesis conceptual: redes, estrategia y no supervisado

Pregunta aplicada

¿Cómo integramos redes neuronales, estrategia de machine learning y aprendizaje no supervisado para tomar una decisión técnica sin depender de memoria aislada?

Alcance

Este capítulo prepara el Parcial 2 e integra tres bloques:

- redes neuronales desde primeros principios;
- estrategia ML y diagnóstico;
- aprendizaje no supervisado.

No se evalúa memoria aislada. Se evalúa si puedes tomar una decisión técnica con fundamento matemático y evidencia computacional.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. conectar forward/backward de redes con diagnóstico de entrenamiento;
2. explicar cómo métrica, baseline y partición sostienen una decisión;
3. interpretar PCA, K-Means o recomendación como evidencia estructural;
4. responder preguntas integradoras sin depender de memoria aislada;
5. escribir una recomendación con evidencia, diagnóstico, acción y límite.

Mapa de contenidos

Bloque	Pregunta central	Evidencia esperada
Redes	¿Cómo se calculan predicciones y gradientes?	dimensiones, forward, pérdida y actualización
Estrategia ML	¿Cómo sé si un modelo es defendible?	baseline, partición, métrica, validación y error analysis

Bloque	Pregunta central	Evidencia esperada
No supervisado	¿Qué estructura encontré sin etiqueta?	varianza, reconstrucción, separación, estabilidad e interpretación

Redes neuronales

Forward propagation. Para una capa densa:

$$Z^{[\ell]} = A^{[\ell-1]}W^{[\ell]} + b^{[\ell]}, \quad A^{[\ell]} = g(Z^{[\ell]}).$$

Las dimensiones son parte del razonamiento. Si $A^{[\ell-1]} \in \mathbb{R}^{m \times d}$ y $W^{[\ell]} \in \mathbb{R}^{d \times h}$, entonces $Z^{[\ell]} \in \mathbb{R}^{m \times h}$.

Softmax y entropía cruzada

Para clasificación multiclase, softmax convierte logits en probabilidades:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}.$$

La implementación estable resta el máximo por fila antes de exponenciar. Esto no cambia las probabilidades, pero evita overflow.

Diagnóstico de modelos

Un modelo defendible requiere:

1. baseline explícito;
2. métrica alineada con el costo de error;
3. particiones sin leakage;
4. validación o estimación de variabilidad;
5. análisis de errores;
6. recomendación proporcional a la evidencia.

Un score alto no basta si no sabemos contra qué baseline compite, qué variabilidad tiene y dónde falla.

No supervisado

En PCA se decide cuántos componentes retener usando varias señales:

- varianza explicada acumulada;
- error de reconstrucción;
- interpretabilidad;
- utilidad para la tarea posterior.

En K-Means se evita elegir k solo por inercia porque inercia disminuye al aumentar k . Silhouette ayuda, pero también tiene sesgos geométricos: favorece grupos compactos y separados.

Forma del Parcial 2

Parte	Tipo	Propósito
Notebook autograded	implementación y resultados numéricos	verificar competencias básicas
Preguntas estructuradas	fórmulas, selección, respuestas numéricas	reducir carga manual
Explicación breve	0/1/2 por criterio	verificar interpretación

Regla de preparación

Si puedes explicar una decisión con esta estructura, estás listo:

```
evidencia -> diagnóstico -> acción -> límite
```

Cómo leer una pregunta integradora

Una pregunta integradora suele mezclar cálculo, implementación y decisión. Antes de responder, separa cuatro capas:

Capa	Pregunta que debes responder
Objeto matemático	¿qué función, matriz, métrica o pérdida aparece?
Procedimiento	¿qué algoritmo o transformación se aplica?
Evidencia	¿qué número, curva, tabla o slice sostiene la afirmación?
Decisión	¿qué acción se justifica y qué límite queda?

Por ejemplo, si una pregunta combina PCA y clasificación, no basta con decir que PCA “mejora” el modelo. Debes decir si PCA se ajustó dentro del pipeline, cuánta varianza conserva, cómo cambia la

métrica de validación y qué interpretación no puede hacerse de los componentes. Esta lectura evita respuestas largas pero desordenadas.

Ejemplo resuelto: softmax estable en síntesis

Supón que una red produce logits:

$$z = (1000, 1001, 1002).$$

Calcular e^{1002} directamente puede producir overflow. Restamos el máximo:

$$z - \max(z) = (-2, -1, 0).$$

Entonces:

$$\text{softmax}(z) = \frac{(e^{-2}, e^{-1}, 1)}{e^{-2} + e^{-1} + 1}.$$

Como $e^{-2} \approx 0.135$ y $e^{-1} \approx 0.368$, la suma es aproximadamente 1.503. Por tanto:

$$\text{softmax}(z) \approx (0.090, 0.245, 0.665).$$

La respuesta integradora debe decir dos cosas: la resta del máximo no cambia las probabilidades y evita un problema numérico. Si la clase correcta es la tercera, la pérdida de esa observación sería $-\log(0.665) \approx 0.408$.

Ejemplo resuelto: evidencia no supervisada dentro de una decisión

Supón que PCA muestra separación visual entre dos grupos y K-Means produce dos clusters con silhouette razonable. Esa evidencia puede sugerir estructura, pero no prueba que los grupos correspondan a clases útiles para una decisión supervisada.

Una respuesta integradora adecuada sería:

Evidencia: PCA y K-Means sugieren dos grupos geométricos.

Diagnóstico: puede haber estructura latente, pero no hay etiqueta ni costo de error.

Acción: usar los clusters como hipótesis exploratoria y compararlos con variables de dominio.

Límite: no afirmar que los clusters son segmentos reales ni que predicen una decisión.

La síntesis del curso exige separar evidencia exploratoria de evidencia predictiva. Un método no supervisado puede abrir una pregunta; no siempre puede cerrarla.

Ejemplo resuelto: respuesta integradora

Caso: una red mejora accuracy global, pero el análisis por slice muestra caída fuerte en datos recientes.

Respuesta defendible:

Evidencia: la accuracy global mejora, pero el slice reciente cae.
Diagnóstico: posible data mismatch temporal.
Acción: validar con partición temporal, revisar features y recolectar datos recientes.
Límite: no recomendar despliegue si el uso real se parece al slice reciente.

La clave es no quedarse en “mejoró la métrica”.

Caso longitudinal: respuesta integradora

Retoma `clasificacion_binaria_sintetica.csv`. Una respuesta integradora no debe decir solo “el modelo regularizado ganó”. Debe conectar varias capas de evidencia:

Pieza	Evidencia esperada
modelo	logística regularizada o baseline comparable
validación	media y variabilidad de F1/recall
error analysis	desempeño por slice
reproducibilidad	semilla, partición, pipeline y manifest
comunicación	recomendación y límite

Una respuesta de parcial aceptable sería:

El modelo regularizado mejora el baseline en F1 medio, pero el recall cae en el slice reciente. El diagnóstico probable es mismatch temporal o cambio de distribución. Recomendaría usarlo solo como baseline interno y construir una validación temporal antes de despliegue.

Esta respuesta no memoriza capítulos: integra métrica, validación, diagnóstico y decisión.

Verificaciones seleccionadas

1. Una mejora global no basta si el slice relevante para producción empeora.
2. PCA o K-Means pueden apoyar exploración, pero no sustituyen una métrica supervisada cuando la decisión depende de etiquetas.
3. La forma evidencia -> diagnóstico -> acción -> límite es una herramienta de razonamiento, no una frase decorativa.

Ejemplo resuelto: matriz de decisión integradora

Supón dos modelos:

Evidencia	Modelo A	Modelo B
F1 global	0.83	0.85
F1 slice reciente	0.62	0.51
desviación CV	0.03	0.09
manifest completo	sí	no

Aunque B tiene mayor F1 global, A es más defendible si el slice reciente importa y si la reproducibilidad es requisito de entrega. La respuesta integradora debe explicar la tensión: B gana una métrica agregada, pero A tiene menor riesgo de decisión bajo la evidencia disponible.

Mini-rúbrica para una respuesta integradora

Una respuesta de parcial puede calificarse de forma consistente con cuatro criterios 0/1/2.

Criterio	0	1	2
evidencia	no cita resultados	cita un resultado aislado	conecta métrica global, slice y variabilidad
diagnóstico	no explica causa probable	nombra una causa sin sostenerla	relaciona patrón observado con causa plausible
acción	propone algo genérico	propone una acción parcial	propone acción proporcional y verificable
límite	no declara límite	límite vago	límite específico sobre uso, datos o despliegue

Esta rúbrica también guía el estudio. Antes del parcial, el estudiante debe practicar respuestas que no sean listas de conceptos, sino argumentos breves con evidencia. Para el profesor, la ventaja es que la parte manual se concentra en juicio técnico observable, mientras los cálculos, formas y métricas pueden quedar autocalificados.

Un error común es responder solo con el nombre del método: “usaría regularización” o “miraría PCA”. Eso rara vez basta. La respuesta debe decir qué evidencia activa esa decisión y qué resultado esperaría observar después. Si la acción propuesta no produce una verificación posterior, todavía no es una decisión técnica cerrada. El objetivo es mostrar criterio técnico, no solo vocabulario aislado.

Para explorar más

- **Extensión matemática:** construye un mapa de dependencias entre pérdida, métrica, regularización, validación y decisión.
- **Extensión computacional:** rehace un experimento anterior desde cero y verifica si produce las mismas métricas dentro de tolerancias razonables.
- **Conexión con proyecto:** escribe qué evidencia mínima necesitarías para defender que tu resultado no es accidental.

Revisión del capítulo

Este capítulo reúne los conceptos necesarios para responder preguntas integradoras. Una buena respuesta no enumera fórmulas: conecta modelo, métrica, validación, evidencia y decisión.

El Parcial 2 exige reconocer cuándo una mejora global es insuficiente, cuándo un slice crítico cambia la recomendación y cuándo una técnica no supervisada solo permite una conclusión exploratoria. También exige cálculos básicos estables, como softmax con resta del máximo.

La estructura de respuesta esperada es evidencia -> diagnóstico -> acción -> límite. Esa estructura permite calificar con claridad y, más importante, permite defender una decisión técnica sin exagerar.

Términos clave

- **Respuesta integradora:** explicación que conecta modelo, métrica, evidencia y límite.
- **Data mismatch temporal:** diferencia entre datos antiguos y recientes que afecta generalización.
- **Síntesis conceptual:** capacidad de relacionar contenidos de varios módulos en una decisión.
- **Evidencia proporcional:** resultado suficiente para justificar una acción, sin exagerar alcance.

Ejercicios

Preguntas conceptuales

1. Explique por qué una mejora global no basta si un slice crítico empeora.
2. ¿Qué diferencia hay entre diagnosticar overfitting y diagnosticar data mismatch?

Problemas cuantitativos

3. Compare dos modelos con métricas globales similares y desempeño distinto por slice; elija uno y justifique.
4. Calcule softmax estable para logits (1000, 1001, 1002) usando resta del máximo.

Práctica computacional

5. Escriba una función que evalúe una métrica global y la misma métrica por slice.
6. Cree una tabla resumen para preparar una respuesta de parcial: baseline, métrica, error principal y acción.

Interpretación y pensamiento crítico

7. Escriba una respuesta de 120 palabras con estructura evidencia -> diagnóstico -> acción -> límite.

Proyecto de cierre

8. Tome un resultado real o simulado de su proyecto y conviértalo en una respuesta oral de dos minutos para Parcial 2.

Reproducibilidad y tracking

Pregunta aplicada

Si otra persona recibe nuestros resultados, ¿puede reconstruir qué datos, código, parámetros y decisiones produjeron la conclusión?

Qué significa reproducir

Reproducibilidad: capacidad de volver a ejecutar el análisis con los mismos datos, código, entorno y parámetros para obtener resultados equivalentes.

En ciencia de datos aplicada, reproducibilidad no es un lujo académico. Es lo que permite auditar una decisión, comparar experimentos y depurar errores.

Reproducibilidad: que otra persona pueda rehacer la evidencia

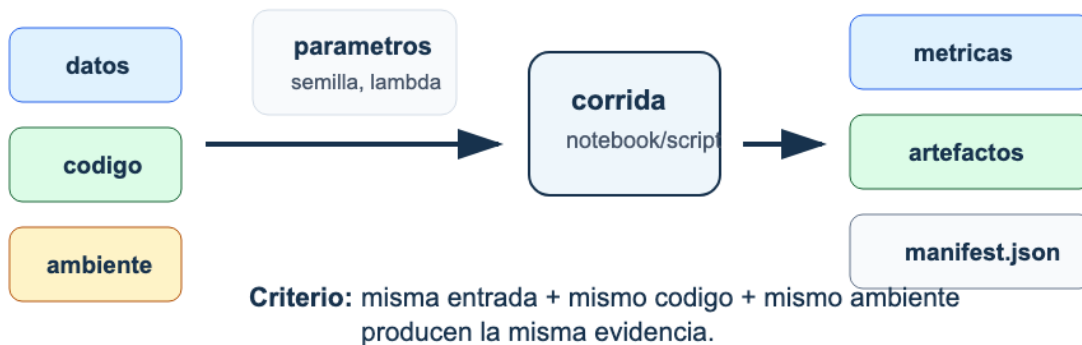


Figura 21: Reproducibilidad: datos, código, ambiente y parámetros alimentan una corrida que produce métricas, artefactos y un manifiesto verificable.

Lectura de la figura. Una corrida reproducible no se identifica solo por el resultado final. Necesita registrar datos, código, ambiente, parámetros, métricas y artefactos para que la evidencia pueda reconstruirse.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. listar los componentes mínimos de un proyecto reproducible;
2. explicar qué controla una semilla y qué no controla;
3. construir un `manifest.json` útil;
4. registrar parámetros, métricas y artefactos;
5. usar hashes para detectar cambios en archivos.

El paquete mínimo reproducible

Elemento	Pregunta que responde
README.md	¿Qué hace el proyecto y cómo se corre?
requirements.txt o environment.yml	¿Qué entorno necesita?
src/ o funciones claras	¿Dónde vive la lógica reutilizable?
notebooks/	¿Dónde está la exploración y evidencia narrativa?
data/README.md	¿Qué datos se usaron y qué restricciones tienen?
reports/	¿Cuál es la conclusión final?
artifacts/	¿Qué modelos, métricas o figuras se generaron?
manifest.json	¿Qué parámetros, métricas y hashes prueban la corrida?

Semillas

La semilla controla generadores pseudoaleatorios:

```
RANDOM_STATE = 42
```

Pero una semilla no garantiza reproducibilidad total si cambian dependencias, hardware, versiones o datos. Por eso se registra junto con el entorno y los artefactos.

Manifest de experimento

Un manifest es un resumen estructurado:

```
{
  "params": {"model": "LogisticRegression", "C": 1.0, "random_state": 42},
  "metrics": {"f1": 0.91, "roc_auc": 0.96},
  "artifacts": {"model.joblib": "sha256:..."},
  "decision": "usar como baseline final"
}
```


El objetivo no es burocracia. Es poder responder: “¿qué corrida produjo este resultado?”.

Ejemplo resuelto: manifest mínimo

Un manifest suficientemente útil para un baseline podría ser:

```
{
  "run_id": "baseline_logreg_2026_09_10",
  "data_split": "train/dev/test random_state=42",
  "params": {"model": "LogisticRegression", "C": 1.0},
  "metrics": {"dev_f1": 0.81, "test_f1": 0.78},
  "artifacts": {"report.html": "sha256:..."},
  "decision": "mantener como baseline de comparación"
}
```

No es necesario que sea largo. Debe ser suficiente para reconstruir la decisión.

Ejemplo resuelto: resultado no reproducible

Supón que un equipo reporta `test_f1 = 0.84`, pero al ejecutar el notebook desde cero aparece `test_f1 = 0.77`. La primera hipótesis no debe ser mala fe; debe ser falta de trazabilidad. Revisa en orden:

1. si la partición usa la misma semilla;
2. si el pipeline aprende transformaciones solo en entrenamiento;
3. si el dataset tiene el mismo hash;
4. si las versiones de librerías cambiaron;
5. si el reporte copió una métrica de una corrida anterior.

Un resultado no reproducible no se defiende con memoria. Se defiende corrigiendo la cadena de evidencia hasta que la métrica pueda reconstruirse.

Ejemplo resuelto: auditar un manifest

Supón que el manifest registra:

```
{
  "run_id": "ridge_v3",
  "seed": 2105,
  "metrics": {"dev_rmse": 12.4},
  "artifacts": {"predictions.csv": "sha256:abc123"},
  "decision": "usar ridge_v3 como modelo candidato"
}
```

Dos semanas después, el archivo `predictions.csv` existe, pero su hash calculado es `sha256:def987`.

La conclusión correcta no es “el modelo está mal”. La conclusión correcta es:

1. el artefacto actual no coincide con el artefacto registrado;
2. la métrica `dev_rmse=12.4` ya no queda respaldada por ese archivo;
3. hay que rerunear la evaluación o recuperar el artefacto original;
4. cualquier reporte que use esa corrida debe marcarse como no verificado hasta resolver la discrepancia.

Un hash no evalúa calidad estadística. Evalúa identidad del archivo.

Tracking de experimentos

Herramientas como MLflow organizan experimentos en corridas con parámetros, métricas y artefactos. En este curso no se exige un servidor MLflow; el patrón conceptual sí se exige:

Concepto	En el curso
experimento	carpeta o manifest
run	una corrida con semilla y parámetros
params	hiperparámetros y decisiones
metrics	resultados numéricos
artifacts	modelo, figuras, reporte, predicciones

Persistencia de modelos

Para proyectos con `scikit-learn`, una práctica común es guardar pipelines completos:

```
from joblib import dump

dump(pipeline, "artifacts/model.joblib")
```

Nunca cargues archivos `pickle` o `joblib` de fuentes no confiables. La persistencia basada en `pickle` puede ejecutar código durante la carga. Para producción o intercambio abierto, considera formatos más seguros o flujos de confianza explícita.

Hash de artefactos

Un hash SHA-256 permite detectar si un archivo cambió:

```
import hashlib

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()
```

Checklist de reproducibilidad

1. ¿Está la semilla definida en un lugar visible?
2. ¿La partición de datos es reproducible?
3. ¿El preprocesamiento está dentro de un pipeline?
4. ¿El entorno está documentado?
5. ¿Las métricas finales se pueden recalcular?
6. ¿Los artefactos tienen nombre, versión o hash?
7. ¿El reporte distingue resultados de decisiones?

Punto de control

Si alguien abre tu proyecto dos semanas después, debería poder responder:

1. ¿qué datos se usaron?;
2. ¿con qué entorno se ejecutó?;
3. ¿qué modelo produjo la métrica reportada?;
4. ¿qué archivos son evidencia y cuáles son auxiliares?;
5. ¿qué decisión se tomó a partir de la corrida?

Caso longitudinal: manifest del clasificador

Para `clasificacion_binaria_sintetica.csv`, un manifest mínimo debería registrar:

Campo	Ejemplo
dataset	<code>clasificacion_binaria_sintetica.csv</code>
hash o versión	SHA-256 del archivo publicado
target	<code>target</code>
slices auditados	<code>base, reciente</code>
partición	semilla y proporciones
pipeline	escalamiento dentro del pipeline y modelo
métrica primaria	F1 o recall
artefactos	tabla de métricas, matriz de confusión, error por slice

El manifest no garantiza que el modelo sea bueno. Garantiza que la evidencia se puede reconstruir. Esa diferencia es central para una defensa técnica.

Verificaciones seleccionadas

1. Un notebook ejecutable sin semilla puede ser reproducible en estructura, pero no necesariamente en resultados numéricos.
2. Si el dataset cambia y no cambia el manifest, ya no sabes qué evidencia sostiene la conclusión.
3. Una tabla de métricas sin partición asociada no es evidencia auditable.

Ejemplo resuelto: manifest mínimo en JSON

Un manifest pequeño puede verse así:

```
{
  "dataset": "clasificacion_binaria_sintetica.csv",
  "target": "target",
  "random_state": 2105,
  "split": "60/20/20 stratified",
  "metric_primary": "f1",
  "artifacts": ["metrics.csv", "confusion_matrix.png", "slice_report.csv"]
}
```

Este archivo no sustituye el reporte, pero permite auditar qué corrida produjo qué evidencia. Si cambian datos, métrica o partición, el manifest debe cambiar.

Qué debe poder rerunearse

Un paquete reproducible no exige que todo el curso se ejecute en segundos, pero sí que exista una ruta verificable. Como mínimo, otra persona debe poder ejecutar un script o notebook que cargue datos permitidos, reconstruya particiones, entrene o cargue el modelo declarado, calcule métricas principales y regenere la tabla final. Si el entrenamiento completo es costoso, se debe incluir una versión reducida o un modo de evaluación que cargue artefactos ya generados y verifique sus métricas.

La trazabilidad se rompe cuando una figura se pega manualmente, una métrica se copia sin archivo de origen o una tabla final no se puede regenerar. En ese caso, la discusión técnica puede ser buena, pero la evidencia no es auditable. El manifest debe apuntar a los archivos que sostienen cada afirmación importante.

Para explorar más

- **Extensión matemática:** identifica qué cantidades numéricas deben quedar fijas para comparar dos corridas de entrenamiento.
- **Extensión computacional:** crea un manifest mínimo con datos, código, semilla, ambiente, métricas y hash de salida.
- **Conexión con proyecto:** define qué archivos entregados permitirían auditar tu resultado sin abrir tu notebook manualmente.

Revisión del capítulo

Reproducibilidad significa que otra persona puede reconstruir el resultado desde datos, código, entorno, parámetros y artefactos. Una semilla ayuda, pero no basta si cambian datos, librerías, hardware o pasos manuales.

El manifest registra qué corrida produjo qué métricas y qué artefactos. Los hashes detectan cambios en archivos, pero no prueban que el contenido sea correcto. El tracking de experimentos ordena comparaciones y evita depender de memoria o nombres informales.

Un paquete reproducible debe permitir auditar la decisión final. Eso implica separar archivos de entrada, scripts, outputs, métricas, modelos y reporte, y declarar qué evidencia sostiene cada afirmación.

Términos clave

- **Reproducibilidad:** capacidad de reconstruir resultados desde datos, código, entorno y decisiones.
- **Manifest:** archivo que registra configuración, rutas, semillas, métricas y artefactos.
- **Run:** corrida individual de un experimento.
- **Artefacto:** salida guardada, como modelo, figura, reporte, predicciones o tabla.
- **Hash:** huella de un archivo usada para detectar cambios.
- **Persistencia:** guardado de un modelo o pipeline para reutilizarlo.

Ejercicios

Preguntas conceptuales

1. ¿Por qué una semilla fija no garantiza reproducibilidad completa?
2. ¿Qué diferencia hay entre código que corre y evidencia reproducible?

Problemas cuantitativos

3. Diseñe una tabla mínima con tres runs, dos hiperparámetros y dos métricas.
4. Explique qué detecta y qué no detecta un hash SHA-256.

Práctica computacional

5. Cree un `manifest.json` mínimo para un experimento.
6. Calcule el hash SHA-256 de un archivo de resultados.

Interpretación y pensamiento crítico

7. ¿Qué riesgos tiene cargar modelos `pickle` o `joblib` de fuentes no confiables?
8. Si no puede reproducir una métrica final, ¿qué partes del proyecto revisaría primero?

Proyecto de cierre

9. Prepare un paquete mínimo reproducible para su proyecto: manifest, entorno, script de evaluación, tabla de métricas y lista de artefactos.

Comunicación técnica

Pregunta aplicada

¿Cómo convertimos experimentos, métricas y errores en una recomendación técnica que una audiencia pueda evaluar y cuestionar?

Tesis de un reporte

Un reporte final no debe ser una bitácora de todo lo que se intentó. Debe defender una conclusión.

Comunicación técnica: de resultados a reco

Un reporte profesional no narra todo lo que se hizo; defiende una conclusión co

Tesis técnica

qué se recomienda, con qué evidencia y ba

Evidencia

baseline
métrica
validación

Diagnóstico

error principal
slice crítico
causa probable

Decisión

acción proporci
siguiente experir
criterio de éxito

Prueba de lectura: una persona externa debe poder cuestion

Figura 22: Comunicación técnica: un reporte organiza tesis, evidencia, diagnóstico, decisión y límites para que una audiencia pueda auditar la recomendación.

Lectura de la figura. La tesis técnica aparece primero, pero no vive sola: debe estar sostenida por evidencia, diagnóstico, decisión y límites. Si una afirmación no puede conectarse con una tabla, figura, cálculo o archivo, debe suavizarse o eliminarse.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. escribir una tesis técnica breve;
2. seleccionar figuras que respondan preguntas específicas;
3. distinguir evidencia, interpretación y recomendación;
4. evitar lenguaje desproporcionado;
5. preparar una respuesta oral defendible.

Tesis técnica: frase que resume la decisión recomendada, la evidencia principal y la limitación más importante.

Ejemplo:

El modelo regularizado mejora el baseline en F1 y mantiene estabilidad en validación cruzada, pero requiere revisar falsos negativos en el subgrupo de mayor riesgo antes de una recomendación operativa.

Estructura recomendada

Sección	Pregunta
Resumen ejecutivo	¿Qué se recomienda y con qué cautela?
Problema y datos	¿Qué pregunta se modeló y qué datos la soportan?
Método	¿Qué pipeline se usó y por qué?
Validación	¿Qué evidencia numérica respalda la decisión?
Error analysis	¿Dónde falla y qué patrón importa?
Reproducibilidad	¿Cómo se rerunea o audita?
Limitaciones	¿Qué no puede concluirse?
Siguiente paso	¿Qué decisión o experimento sigue?

Figuras útiles

Una figura profesional tiene función. No se incluye porque “se ve bien”.

Figura	Uso
curva de aprendizaje	diagnosticar underfitting/overfitting
matriz de confusión	identificar tipos de error
tabla de métricas con baseline	comparar decisiones
slices de error	encontrar fallas por subgrupo
PCA 2D	explorar estructura, no probar causalidad
perfil de clusters	interpretar segmentos

Lenguaje de decisión

Evita:

- “el modelo es bueno”;
- “la gráfica demuestra”;
- “el accuracy es alto”;
- “se recomienda implementar”.

Prefiere:

- “mejora el baseline por...”;
- “la evidencia sugiere...”;
- “el resultado es estable/inestable porque...”;
- “se recomienda el siguiente experimento antes de...”.

Ejemplo resuelto: de bitácora a argumento

Una bitácora dice:

Probamos regresión logística, random forest y varias variables. La mejor métrica fue la del modelo regularizado. También hicimos gráficas y revisamos errores.

Un argumento técnico dice:

La regresión logística regularizada se elige porque mejora el baseline en F1 medio y mantiene menor variabilidad que el modelo más complejo. El análisis por slice muestra caída en observaciones recientes; por eso la recomendación se limita a usarlo como baseline interno y priorizar validación temporal.

La diferencia no es estilo literario. La bitácora enumera acciones; el argumento conecta criterio, evidencia, diagnóstico y límite. Un lector puede auditar el segundo texto porque sabe qué tabla buscar y qué afirmación verificar.

Ejemplo resuelto: mejorar una conclusión

Conclusión débil:

El modelo es bueno porque tiene accuracy alta.

Conclusión profesional:

El modelo mejora el baseline de F1 en validación cruzada y mantiene desempeño similar en test. Sin embargo, el recall cae en el slice de observaciones recientes; por eso se recomienda usarlo solo como baseline interno mientras se revisa data mismatch temporal.

La segunda versión es mejor porque incluye comparación, evidencia, límite y acción proporcional.

Ejemplo resuelto: mejora absoluta y relativa

Supón que el baseline tiene F1 de 0.60 y el modelo final tiene F1 de 0.72.

La mejora absoluta es:

$$0.72 - 0.60 = 0.12.$$

La mejora relativa respecto al baseline es:

$$\frac{0.72 - 0.60}{0.60} = 0.20 = 20\%.$$

Una frase defendible sería:

El modelo mejora el baseline en 0.12 puntos de F1, equivalente a una mejora relativa de 20 %. Sin embargo, esta comparación debe leerse junto con validación, error analysis y limitaciones del dataset.

La mejora relativa puede sonar más grande que la absoluta. Un reporte profesional muestra ambas o explica por qué una es más relevante para la decisión.

Respuesta corta defendible

Una respuesta oral de defensa debe caber en 45 segundos:

```
Elegimos [método] porque [criterio].  
La evidencia principal fue [métrica + validación].  
El mayor error aparece en [slice o caso].  
La limitación más importante es [límite].  
Por eso recomendamos [acción proporcional].
```

Caso longitudinal: conclusión del clasificador

Con `clasificacion_binaria_sintetica.csv`, una conclusión débil sería:

El modelo final tiene buen F1 y se recomienda usarlo.

Una conclusión profesional sería:

El modelo regularizado mejora el baseline en F1 medio de validación, pero el recall del slice **reciente** es menor que el del slice **base**. Por tanto, el modelo puede usarse como baseline interno para priorizar revisión, pero no como sistema de decisión final hasta construir una validación temporal y auditar variables que cambian entre slices.

La segunda versión tiene tesis, evidencia, diagnóstico, recomendación y límite. También deja claro qué experimento falta.

Verificaciones seleccionadas

1. Una mejora relativa de 20 % puede sonar grande; siempre debe leerse junto con la mejora absoluta y el baseline.
2. Una figura funcional debe responder una pregunta: comparación, error, curva, slice o trazabilidad.
3. Si una conclusión no declara límite, probablemente exagera el alcance de la evidencia.

Ejemplo resuelto: tabla de evidencia contra afirmaciones

Antes de entregar un reporte, cruza cada afirmación fuerte contra evidencia:

Afirmación	Evidencia necesaria	Estado
supera el baseline	tabla baseline vs modelo	verificable
generaliza bien	validación y test final	incompleto si no hay test
funciona para casos recientes	métrica por slice	falso si el slice cae
es reproducible	manifest y entorno	verificable

Una afirmación sin evidencia debe cambiar de tono. Por ejemplo, “generaliza bien” puede convertirse en “muestra desempeño estable en validación, pendiente de test final independiente”.

Plantilla de resultados defendible

Una sección de resultados puede tener esta forma:

```
Baseline. [regla simple] obtiene [métrica] en [partición].
Modelo. [modelo elegido] obtiene [métrica media ± desviación] bajo [validación].
Error principal. El peor patrón aparece en [slice/tipo de error].
Decisión. Con esta evidencia se recomienda [acción proporcional].
Límite. No se puede afirmar [conclusión excesiva] porque [razón].
```

Esta plantilla evita dos extremos: una bitácora larga sin tesis y una conclusión fuerte sin evidencia. Si una línea no puede llenarse con una tabla, figura, cálculo o archivo, el reporte todavía no está listo.

De resultado a recomendación

La recomendación final debe tener un verbo de acción. “El modelo alcanza 0.84 de F1” es un resultado; “usar el modelo como filtro preliminar con revisión humana en casos de baja confianza” es una recomendación. Entre ambos debe aparecer el razonamiento: baseline superado, error principal, costo de fallo y límite de uso. En ciencia de datos aplicada, una métrica sin acción suele quedarse corta.

También importa el tono. Si la evidencia viene de validación interna, la recomendación debe decirlo. Si el test final es pequeño, debe aparecer el tamaño. Si un slice falla, no se debe esconder detrás del promedio global. La escritura profesional no consiste en sonar segura; consiste en ser precisa sobre lo que la evidencia permite sostener.

Una buena prueba de lectura es subrayar cada afirmación del reporte y preguntar: ¿qué tabla, figura, cálculo o archivo la sostiene? Si no hay respuesta, la frase debe debilitarse, eliminarse o convertirse en trabajo futuro. Esta disciplina hace que el texto sea más corto, pero más fuerte y auditable. Además, ayuda a distinguir entre una conclusión lista para entrega y una idea que aún necesita validación adicional.

Para explorar más

- **Extensión matemática:** convierte una mejora de métrica en mejora absoluta, relativa y límite de interpretación.
- **Extensión computacional:** genera una tabla final reproducible que combine baseline, modelo elegido, métrica principal, variabilidad y nota de validación.
- **Conexión con proyecto:** prepara una conclusión de 45 segundos que incluya decisión, evidencia, error principal, limitación y siguiente paso.

Revisión del capítulo

La comunicación técnica transforma resultados en una conclusión defendible. Una tesis de reporte debe decir qué problema se abordó, qué evidencia se obtuvo, qué decisión se recomienda y bajo qué límites.

Las figuras funcionales responden preguntas concretas: desempeño contra baseline, errores por slice, curva de validación o estructura de datos. Una figura decorativa puede distraer; una figura funcional reduce ambigüedad.

El lenguaje debe ser proporcional. En lugar de afirmar que un modelo “es bueno”, el reporte debe explicar contra qué baseline mejora, con qué métrica, en qué datos, dónde falla y qué acción se recomienda.

Términos clave

- **Tesis del reporte:** afirmación central que el documento defiende con evidencia.
- **Resumen ejecutivo:** síntesis de decisión, evidencia y límite para lector no técnico.
- **Figura funcional:** visualización incluida porque responde una pregunta técnica.
- **Limitación:** frontera explícita de lo que no se puede concluir.
- **Acción proporcional:** recomendación ajustada al peso de la evidencia.

Ejercicios

Preguntas conceptuales

1. ¿Por qué “el modelo es bueno” no es una conclusión técnica suficiente?
2. ¿Qué diferencia hay entre una figura decorativa y una figura funcional?

Problemas cuantitativos

3. Reduzca una tabla con cinco métricas a las dos más relevantes para una decisión de alto costo.
4. Compare un modelo contra baseline y escriba la mejora absoluta y relativa de la métrica principal.

Práctica computacional

5. Cree una tabla final de resultados con baseline, modelo elegido, métrica, intervalo o desviación y nota de validación.
6. Genere una figura de error analysis con título, eje claro y caption interpretativo.

Interpretación y pensamiento crítico

7. Reescriba una conclusión exagerada para que incluya evidencia, límite y siguiente paso.

Proyecto de cierre

8. Escriba el resumen ejecutivo de su proyecto en 180 palabras: problema, método, evidencia, error principal, limitación y recomendación.

Entrega final y defensa

Pregunta aplicada

¿Qué debe entregar un equipo para que su proyecto pueda leerse, ejecutarse, auditarse y defenderse sin depender de una explicación informal?

Paquete final

El paquete final debe permitir tres acciones:

1. leer la conclusión;
2. ejecutar o auditar el flujo principal;
3. evaluar la contribución individual.

Objetivos del capítulo

Al terminar este capítulo deberías poder:

1. organizar un repositorio de proyecto con evidencia auditable;
2. preparar una entrega que se pueda leer y ejecutar;
3. explicar tu contribución individual;
4. responder preguntas sobre métrica, validación, leakage y limitaciones;
5. distinguir presentación grupal de defensa individual.

Estructura del repositorio de proyecto

```
proyecto-final/  
  README.md  
  requirements.txt  
  data/  
    README.md  
  notebooks/  
    01_exploracion.ipynb  
    02_modelado.ipynb  
    03_reporte_final.ipynb
```

```
src/  
  features.py  
  evaluation.py  
reports/  
  reporte_final.pdf  
artifacts/  
  metrics.json  
  manifest.json
```

No todos los proyectos tendrán exactamente la misma estructura, pero sí deben separar datos, código, evidencia y reporte.

Criterio de paquete ejecutable

Un proyecto no está listo porque “corre en mi computador”. Está listo cuando otra persona puede reconstruir el resultado principal con instrucciones mínimas. El README debe responder:

Pregunta	Evidencia esperada
¿qué problema resuelve?	pregunta aplicada y decisión
¿qué datos usa?	fuentes, licencia, columnas y restricciones
¿cómo se ejecuta?	comandos o notebooks en orden
¿qué resultado debe aparecer?	métrica, tabla o figura esperada
¿qué no se publica?	datos privados, credenciales o archivos sensibles

Si el notebook final depende de rutas locales, celdas ejecutadas fuera de orden o archivos no incluidos, la defensa se vuelve una explicación informal. La entrega profesional reduce esa fragilidad antes de que el profesor tenga que descubrirla.

Ejemplo resuelto: contribución individual

Respuesta vaga:

Ayudé con el modelo.

Respuesta defendible:

Implementé la partición train/dev/test, construí el baseline logístico y comparé F1/recall contra el modelo regularizado. También preparé la tabla de errores por slice de edad y documenté el riesgo de falsos negativos.

La segunda respuesta permite verificar autoría y comprensión.

Rúbrica del proyecto final

Criterio	Peso sugerido
Pregunta, datos y alcance	15 %
Fundamento matemático y algoritmo	20 %
Validación, métricas y error analysis	25 %
Reproducibilidad y organización	15 %
Comunicación visual y escrita	15 %
Limitaciones, ética y decisión	10 %

Ejemplo resuelto: cálculo de nota ponderada

Supón que un proyecto recibe:

Criterio	Puntaje	Peso
Pregunta, datos y alcance	12/15	15 %
Fundamento matemático y algoritmo	16/20	20 %
Validación, métricas y error analysis	20/25	25 %
Reproducibilidad y organización	13/15	15 %
Comunicación visual y escrita	12/15	15 %
Limitaciones, ética y decisión	8/10	10 %

Como los pesos suman 100 puntos y los denominadores coinciden con esos pesos, la nota total es:

$$12 + 16 + 20 + 13 + 12 + 8 = 81.$$

El porcentaje final es $81/100 = 81\%$. La tabla permite calificar rápido y, al mismo tiempo, muestra dónde perdió puntos el proyecto.

Defensa oral individual

La defensa no repite la presentación. Verifica autoría, comprensión y criterio.

Defensa oral: responder con evidencia, no

La defensa verifica autoría, criterio técnico y límites de la recomendación.



Criterio 0/1/2

0: no responde o no hay evidencia
1: idea parcial, evidencia incompleta
2: respuesta correcta, evidencia verificable y límite específico

Figura 23: Defensa oral: la respuesta individual conecta pregunta, respuesta breve, evidencia verificable y límite específico con una rúbrica 0/1/2.

Lectura de la figura. Una defensa fuerte no empieza con “yo hice varias cosas”; empieza con una respuesta breve, la conecta con evidencia verificable y declara un límite. Esa estructura permite evaluar rápido sin convertir la defensa en una conversación improvisada.

Preguntas típicas:

1. ¿Cuál fue tu contribución técnica concreta?
2. ¿Por qué esa métrica es la correcta?
3. ¿Dónde falla el modelo?
4. ¿Qué parte del pipeline podría producir leakage?

5. ¿Qué cambiaría si tuvieras dos semanas más?
6. ¿Qué conclusión no puedes afirmar con estos datos?

Rúbrica de defensa 0/1/2

Criterio	0	1	2
Autoría	no identifica contribución	contribución vaga	contribución concreta y verificable
Matemática	errores conceptuales	idea parcial	explicación correcta y conectada
Validación	no justifica métrica	justifica parcialmente	conecta métrica, baseline y error
Limitaciones	no reconoce límites	límite genérico	límite específico y relevante
Decisión	recomendación desproporcionada	recomendación débil	acción prudente y defendible

Caso longitudinal: defensa del clasificador

Para defender el caso `clasificacion_binaria_sintetica.csv`, prepara una tabla de evidencia antes de hablar:

Pregunta posible	Evidencia que debes abrir
¿por qué esa métrica?	costo de falsos positivos/falsos negativos y baseline
¿dónde falla?	matriz de confusión y tabla por <code>slice</code>
¿hay leakage?	pipeline y partición
¿qué hiciste tú?	commit, notebook, función o tabla verificable
¿qué no puedes afirmar?	límite sobre el <code>slice reciente</code>

Una defensa de 45 segundos:

Mi contribución fue construir el pipeline de evaluación y la tabla por `slice`. El modelo regularizado mejora el baseline en F1, pero el recall cae en el `slice reciente`. Por eso no recomiendo despliegue; recomiendo usarlo como baseline y crear una validación temporal antes de tomar decisiones.

Verificaciones seleccionadas

1. “Ayudé con el modelo” no prueba autoría; “implementé la partición y la tabla por `slice` en el archivo X” sí es verificable.

2. Una defensa fuerte puede reconocer que el resultado no basta para despliegue.
3. La rúbrica 0/1/2 funciona cuando la respuesta tiene evidencia observable y no depende de impresiones.

Ejemplo resuelto: pregunta difícil

Pregunta:

Si tu modelo tiene mejor F1, ¿por qué no recomiendas desplegarlo?

Respuesta defendible:

Porque la mejora global no se mantiene en el slice reciente. La evidencia por slice muestra una caída de recall, y ese slice se parece más al uso esperado. Recomiendo mantener el modelo como baseline interno y construir una validación temporal antes de despliegue.

La respuesta obtiene puntaje alto porque no niega la mejora; la ubica dentro de su límite y propone una acción técnica.

Checklist individual antes de entrar a defensa

Cada estudiante debe poder abrir evidencia concreta para estas preguntas:

Pregunta	Evidencia mínima
¿qué hiciste tú?	archivo, función, notebook, tabla o commit
¿qué fórmula o métrica usaste?	definición y razón aplicada
¿qué baseline superaste?	tabla comparativa
¿dónde falla el modelo?	matriz de confusión, residuos o slice
¿cómo evitas leakage?	partición y pipeline
¿qué no puedes concluir?	limitación específica

Si una respuesta depende de “eso lo hizo mi compañero”, la defensa individual no está lista. La colaboración es válida, pero cada persona debe defender una parte técnica concreta del proyecto.

Carga docente mínima

Para grupos grandes:

- revisar el reporte con rúbrica estructurada;
- usar checklist de reproducibilidad antes de leer detalles;
- evaluar defensa individual con 5 criterios 0/1/2;
- pedir evidencia de contribución en una tabla;
- muestrear notebooks solo cuando haya inconsistencia.

Evidencia individual y trazabilidad

La defensa individual debe apoyarse en evidencia concreta. Una respuesta fuerte no dice solamente “yo hice el modelo”; señala el archivo, la función, el commit, la tabla o la figura que prueba la contribución. Esta trazabilidad evita dos problemas frecuentes: estudiantes que entienden el proyecto pero no pueden mostrar su aporte, y estudiantes que presentan artefactos sin poder defender las decisiones técnicas.

Una forma simple de prepararse es construir una tabla con tres columnas: pregunta posible, evidencia que abriría y respuesta de 30 segundos. Por ejemplo, si la pregunta es “¿cómo evitaron leakage?”, la evidencia puede ser el pipeline de preprocesamiento y la partición; la respuesta debe explicar qué pasos se ajustaron solo con entrenamiento y cuáles se aplicaron después a validación o test. Esta preparación hace la defensa más justa y reduce tiempo de evaluación.

Para explorar más

- **Extensión matemática:** anticipa una pregunta sobre pérdida, métrica o validación y prepara una respuesta con una fórmula o cálculo simple.
- **Extensión computacional:** ejecuta el proyecto en un ambiente limpio y registra cualquier dependencia faltante antes de la defensa.
- **Conexión con proyecto:** arma una tabla de preguntas difíciles con evidencia, archivo fuente y respuesta breve.

Revisión del capítulo

La entrega final debe ser auditable. El repositorio, el reporte, los artefactos y la defensa deben permitir reconstruir qué se hizo, quién lo hizo, qué evidencia se obtuvo y qué decisión se puede sostener.

La defensa individual no repite la presentación grupal. Su función es verificar criterio técnico: contribución concreta, elección de métrica, prevención de leakage, lectura de errores, limitaciones y siguiente experimento razonable.

La carga docente se reduce cuando la evidencia está estructurada. Una rúbrica 0/1/2 funciona si cada criterio se puede observar rápidamente en archivos, tablas, respuestas orales o artefactos reproducibles.

Términos clave

- **Paquete final:** conjunto de reporte, código, datos permitidos, artefactos y evidencia.
- **Defensa oral:** instancia para verificar autoría, comprensión y criterio.
- **Contribución individual:** trabajo técnico concreto que puede ser explicado y verificado.
- **Rúbrica:** criterio explícito para evaluar con consistencia.
- **Carga docente mínima:** diseño de evaluación que concentra revisión manual donde aporta juicio.

Ejercicios

Preguntas conceptuales

1. ¿Por qué una defensa oral no debe repetir simplemente la presentación?
2. ¿Qué evidencia distingue una contribución concreta de una afirmación vaga?

Problemas cuantitativos

3. Con la rúbrica propuesta, calcule la nota de un proyecto que obtiene 12/15, 16/20, 20/25, 13/15, 12/15 y 8/10.
4. Si una defensa usa cinco criterios 0/1/2, diseñe una conversión transparente a porcentaje.

Práctica computacional

5. Cree una estructura de carpetas final para su proyecto y liste qué archivo prueba cada resultado importante.
6. Prepare una tabla de contribuciones individuales con tarea, evidencia y responsable.

Interpretación y pensamiento crítico

7. Escriba una respuesta defendible a: “¿Qué cambiaría si tuviera dos semanas más?”

Proyecto de cierre

8. Arme un checklist final de entrega con 12 ítems verificables antes de subir el proyecto.

Ejercicios integradores: reproducibilidad y defensa

Esta sección sirve como ensayo final del curso. Cada ejercicio conecta matemática, implementación, evidencia y comunicación. La respuesta correcta no es solo la que calcula bien: también debe dejar claro qué decisión se puede defender con ese cálculo.

Resultados de práctica

Al terminar este banco deberías poder:

1. estabilizar cálculos numéricos básicos;
2. transformar métricas en diagnóstico y acción;
3. usar PCA y reconstrucción como herramientas verificables;
4. diseñar un manifest reproducible;
5. defender límites antes de recomendar uso real.

Mapa del banco

Ejercicio	Evidencia de aprendizaje
1	estabilidad numérica en softmax
2	diagnóstico con baseline, CV, test y slice crítico
3	PCA, scores y reconstrucción
4	manifest y trazabilidad de experimento
5	defensa técnica con límites explícitos

Criterio de calidad

Una respuesta final de curso debe:

1. usar una fórmula o procedimiento correcto;
2. señalar la evidencia observada;
3. traducir la evidencia en diagnóstico;
4. proponer una acción proporcional;
5. declarar un límite.

Ejercicio 1: Softmax estable

Sea

$$Z = \begin{bmatrix} 1000 & 1001 & 1002 \end{bmatrix}.$$

1. Explica por qué una implementación directa puede fallar.
2. Calcula softmax después de restar el máximo.
3. Verifica que las probabilidades suman 1.

Ejercicio 2: Diagnóstico

Un modelo tiene:

- baseline F1 = 0.62;
- modelo F1 CV = 0.78 ± 0.09 ;
- test F1 = 0.70;
- slice crítico F1 = 0.48.

Escribe una recomendación en cuatro partes:

```
evidencia -> diagnóstico -> acción -> límite
```

Ejercicio 3: PCA y reconstrucción

Tienes una matriz centrada X_c y su SVD $X_c = U\Sigma V^\top$.

1. Escribe la fórmula de los scores con k componentes.
2. Escribe la reconstrucción aproximada.
3. Explica por qué el error de reconstrucción no aumenta al aumentar k .

Ejercicio 4: Reproducibilidad

Diseña un `manifest.json` mínimo para un experimento de clasificación. Debe incluir:

- semilla;
- modelo;
- hiperparámetros;
- métrica principal;
- hash de artefactos;
- decisión.

Ejercicio 5: Defensa

Responde en 120 palabras:

¿Qué evidencia necesitarías antes de recomendar que un modelo se use en una decisión real?

Respuestas seleccionadas

Ejercicio 1. La implementación directa puede fallar porque e^{1002} excede el rango numérico usual. Al restar el máximo, se calcula softmax sobre $(-2, -1, 0)$. Las probabilidades son proporcionales a $(e^{-2}, e^{-1}, 1)$ y se normalizan dividiendo por $e^{-2} + e^{-1} + 1$. La suma final es 1 por construcción.

Ejercicio 2. Una respuesta defendible sería: “Evidencia: el modelo supera el baseline, pero la variabilidad CV es alta, test cae y el slice crítico tiene F1 0.48. Diagnóstico: desempeño inestable y posible falla en un subgrupo importante. Acción: analizar errores del slice, ajustar métrica de selección y recolectar datos relevantes. Límite: no recomendar uso real si el slice crítico representa casos de alto costo.”

Ejercicio 3. Los scores con k componentes son $Z = X_c V_k$. La reconstrucción aproximada es $\hat{X}_k = Z V_k^\top$. Al aumentar k , el subespacio disponible contiene al anterior, así que la mejor reconstrucción no puede empeorar.

Ejercicio 5. La evidencia mínima incluye baseline, partición independiente, métrica alineada con la decisión, análisis por slices, matriz de errores, revisión de leakage, estabilidad ante variación de datos, limitaciones y costo de falsos positivos/falsos negativos. Sin esa evidencia, una métrica global no justifica una recomendación real.

Herramientas de estudio

Guía de estudio del estudiante

Esta guía explica cómo usar el libro, los notebooks y los laboratorios como un sistema de aprendizaje. No reemplaza el calendario del curso; sirve para tomar buenas decisiones de estudio semana a semana.

Cómo se conectan las páginas del curso

Usa las páginas del curso con funciones distintas:

Página	Cuándo abrirla	Para qué sirve
Portal del curso	al comenzar la semana	ubicar programa, calendario, ruta, materiales, proyecto y configuración
Libro	antes y después de clase	estudiar conceptos, derivaciones, ejemplos, ejercicios y apéndices
Ruta semanal	antes de cada clase	saber qué leer, qué notebook usar y qué entregar
Materiales	al descargar archivos	encontrar slides, notebooks, laboratorios y paquetes por módulo

La regla práctica es: si necesitas saber **qué hacer**, usa el portal y la ruta; si necesitas saber **por qué funciona**, usa el libro; si necesitas **entregar**, usa materiales y conserva el notebook ejecutado con salidas visibles.

Ruta mínima por semana

Cada semana debe producir una evidencia pequeña antes de llegar a la evaluación.

Momento	Acción	Evidencia concreta
Antes de clase	leer la pregunta aplicada, objetivos, mapa de objetivos y definiciones	lista breve de símbolos, dimensiones y dudas

Momento	Acción	Evidencia concreta
Durante clase	ejecutar el notebook y anotar errores	celda reproducida, output revisado y una pregunta técnica
Después de clase	resolver puntos de control del capítulo	cálculo pequeño o explicación de una métrica
Antes del laboratorio	implementar funciones mínimas	pruebas locales, shapes y comparación con referencia
Después del feedback	corregir interpretación	párrafo con evidencia, límite y decisión

Regla práctica. Si solo puedes ejecutar el código pero no puedes explicar qué objeto matemático calcula, todavía no estás listo. Si solo puedes repetir la fórmula pero no puedes probarla con datos, tampoco estás listo.

Cómo leer un capítulo técnico

Lee cada capítulo en tres pasadas.

Pasada	Meta	Qué debes marcar
Primera lectura	ubicar la pregunta aplicada	qué problema se intenta resolver y qué decisión se quiere apoyar
Lectura matemática	seguir objetos y derivaciones	variables, dimensiones, función objetivo, gradiente o métrica
Lectura computacional	conectar con implementación	entradas, salidas, tests, semilla, partición y comparación

No intentes memorizar todo en la primera lectura. El libro está diseñado para que vuelvas a los ejemplos cuando implementes o estudies para evaluación.

El [Mapa de objetivos de aprendizaje](#) te ayuda a convertir la lectura en acciones observables: formular, derivar, implementar, diagnosticar, seleccionar, comunicar o defender. Úsalo antes de estudiar para saber qué evidencia deberías poder producir al cerrar el capítulo.

Señales de dominio

Antes de entregar un laboratorio o presentar una defensa, verifica si puedes hacer lo siguiente sin mirar una solución completa:

- definir el objeto matemático principal del tema;
- calcularlo en un caso pequeño;
- implementar una versión mínima en Python;

- comparar el resultado con una librería profesional sin cambiar el experimento;
- elegir una métrica adecuada;
- explicar una limitación del resultado;
- detectar una fuente posible de leakage, sesgo o sobreajuste;
- escribir una recomendación técnica con evidencia y cautela.

Uso de respuestas seleccionadas

Las respuestas seleccionadas están para calibrar tu estudio, no para sustituir tu solución.

Buen uso	Mal uso
comparar después de intentar el ejercicio	leer antes de pensar el problema
detectar errores de dimensión o signo	copiar el razonamiento sin reproducirlo
verificar una métrica o interpretación	usar la respuesta como solución de laboratorio
preparar preguntas para clase	ocultar que no se entiende la derivación

Si una respuesta seleccionada no basta para destrabar una duda, usa el foro, la clase o la atención docente con una evidencia mínima: fórmula, celda, error, shape, métrica o párrafo que no entiendes.

Preparación para parciales

Un parcial de este curso no debe prepararse como una lista de fórmulas aisladas. Usa esta secuencia:

1. escoger un tema;
2. escribir la función objetivo, métrica o factorización central;
3. resolver un ejemplo pequeño a mano;
4. implementar el cálculo en un notebook limpio;
5. interpretar el output en una frase;
6. cambiar un supuesto y predecir qué debería pasar;
7. revisar respuestas seleccionadas o feedback automático;
8. registrar el error más importante encontrado.

La pregunta típica no es “¿recuerdas la fórmula?”, sino “¿qué se optimiza, cómo se calcula, cómo se valida y qué decisión permite o no permite tomar?”.

Preparación para el proyecto

Desde el inicio del proyecto, conserva un registro de:

- fuente del dataset, licencia o permiso de uso y fecha de descarga;
- qué representa cada fila y cada variable importante;
- variable objetivo o decisión aplicada;
- partición de datos y justificación;

- baseline inicial;
- métrica primaria y métricas secundarias;
- riesgos de privacidad, sesgo, leakage o falta de representatividad;
- cambios de código que alteran resultados;
- ayuda externa o uso sustancial de IA.

El proyecto se evalúa por reproducibilidad, evidencia y defensa técnica. Una métrica alta sin trazabilidad no es un resultado fuerte.

Cuando te bloqueas

Antes de pedir ayuda, prepara una de estas formas de evidencia:

Bloqueo	Evidencia útil
fórmula	símbolos, dimensiones y paso exacto donde aparece la duda
código	celda mínima, error, shape esperado y shape obtenido
métrica	matriz de confusión, tabla de residuos o curva relevante
interpretación	frase actual, evidencia usada y límite que no sabes declarar
proyecto	fuentes de datos, decisión aplicada y riesgo detectado

Pedir ayuda con evidencia acelera la retroalimentación y protege tu autoría.

Checklist antes de publicar una entrega

- ¿El notebook corre desde cero?
- ¿Las semillas y particiones están declaradas?
- ¿Las transformaciones se ajustan solo con entrenamiento?
- ¿Las métricas corresponden al tipo de problema?
- ¿Los outputs importantes tienen interpretación?
- ¿La recomendación declara límites?
- ¿El uso de ayuda externa o IA está declarado cuando corresponde?
- ¿El archivo entregado no contiene datos privados innecesarios?

Formatos y descargas

Esta página reúne los formatos públicos del libro y los recursos descargables de la edición vigente. Su propósito es que estudiantes, instructores y revisores puedan encontrar rápidamente la versión web, el PDF, los datos de práctica y los manifiestos de publicación sin depender del portal operativo del curso.

La versión HTML es el formato principal de lectura y accesibilidad. El PDF se publica como respaldo offline y archivo de versión; en esta edición todavía no es un PDF etiquetado.

Lectura principal

Recurso	Enlace	Uso recomendado
Libro web	Inicio del libro	lectura principal, búsqueda, navegación y enlaces internos
PDF del libro	PDF del libro	lectura offline y archivo de versión
Guía de impresión	Guía	criterios para usar el PDF como copia archivable o imprimible
Accesibilidad	Declaración	estado de accesibilidad, evidencia automática y barreras conocidas

Guías de inicio

Recurso	Enlace	Uso recomendado
Guía del estudiante	Estudiar con el libro	rutina semanal, preparación de parciales y proyecto
Guía docente pública	Adopción docente	adopción modular, separación público/privado y evaluación automatizada
Mapa de objetivos	Objetivos por capítulo	resultados observables, evidencia y alineación por capítulo
Mapa de cierres	Cierres de capítulo	términos clave, repaso, práctica y proyecto por capítulo

Recurso	Enlace	Uso recomendado
Casos longitudinales	Casos longitudinales	hilos de regresión, clasificación y estructura no supervisada
Respuestas y solucionarios	Política	respuestas públicas, manual de estudiante y recursos restringidos
Prerrequisitos mínimos	Prerrequisitos matemáticos	álgebra lineal, gradientes, probabilidad mínima y notación NumPy/PyTorch

Datos de práctica

Recurso	Enlace	Uso recomendado
Manifest de datos	Manifest JSON	filas, columnas, hashes, semillas y uso pedagógico
README de datos	README	descripción del paquete de CSV
Regresión generada	CSV regresión	regresión lineal, gradiente y regularización
Clasificación generada	CSV clasificación	logística, umbrales y métricas
Clustering generado	CSV clustering	K-Means y diagnóstico de escala
PCA generado	CSV PCA	PCA, SVD y reconstrucción
Interacciones de recomendación	CSV recomendación	recomendación item-item pequeña

Trazabilidad de publicación

Recurso	Enlace	Qué prueba
Manifiesto de publicación	Manifest JSON	inventario, tamaños y hashes SHA-256
Manifest de objetivos	Manifest JSON	objetivos por capítulo, evidencia y alineación global
Manifest de figuras	Manifest JSON	SVG, PNG, texto alternativo, atribución y primer uso significativo
Sitemap	sitemap.xml	páginas públicas previstas para descubrimiento
Registro de candidato	libro_curso/registros/publicacion/candidato-2026-07-07.md	construcción y bloqueadores editoriales

Recurso	Enlace	Qué prueba
Registro público de errata	registro_errata_publica.html	reportes abiertos, cerrados y cambios editoriales
Referencias y atribuciones	referencias_atribuciones.html	fuentes técnicas, benchmark editorial y cómo citar
Plantillas de respuestas	README	control editorial de respuestas públicas y recursos restringidos

El registro de candidato vive en el repositorio fuente, no en el libro publicado. El manifiesto público es el artefacto correcto para verificar una carpeta renderizada.

Estado de uso

El repositorio incluye un **LICENSE** formal de uso docente restringido. La lectura y el uso dentro del curso están permitidos; redistribución, adaptación pública o uso comercial requieren autorización explícita. Una licencia abierta futura debe declararse explícitamente antes de tratar el libro como OER.

Para citar esta edición:

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: Libro del curso MATE-2105*. Universidad de los Andes. <https://cesarneyit-code.github.io/mate2105/libro/>

Qué no se descarga desde el libro

El libro público no distribuye:

- soluciones oficiales completas;
- pruebas ocultas;
- autograders;
- paquetes Gradescope;
- guías docentes privadas;
- datos de estudiantes o entregas reales.

Estos materiales permanecen fuera de la publicación para proteger evaluación, privacidad y trazabilidad docente.

Datos y recursos reproducibles

Este libro usa datasets pequeños, públicos o generados para práctica, de modo que los ejemplos puedan ejecutarse sin depender de internet durante clase, laboratorio o autograding. La prioridad es que el estudiante pueda concentrarse en el modelo, la validación y la interpretación.

Los datasets de ejemplo no reemplazan un problema aplicado real. Sirven para practicar conceptos bajo condiciones controladas. En el proyecto final, cada equipo debe documentar fuente, licencia, fecha de descarga, variables, restricciones de uso y riesgos de interpretación.

Paquete descargable del libro

El libro publica un paquete de CSV de práctica para que estudiantes e instructores puedan reproducir ejemplos básicos sin conexión a internet y sin depender de material privado del curso.

Estos archivos también funcionan como casos longitudinales. No se introducen solo para practicar una función de código; reaparecen en capítulos distintos para conectar regresión, regularización, validación, análisis por slice, comunicación técnica y defensa. La guía completa está en [Casos longitudinales del libro](#).

Archivo	Filas	Uso pedagógico
regresion_lineal_sintetica.csv	160	regresión lineal, descenso del gradiente y regularización
clasificacion_binaria_sintetica.csv	240	regresión logística, umbrales, métricas y análisis por slice
clusters_sinteticos.csv	240	K-Means, centroides, inercia y diagnóstico de escala
pca_sintetico.csv	180	PCA, SVD, varianza explicada y reconstrucción
recomendacion_interacciones.csv	32	recomendación item-item y evaluación offline pequeña

El archivo [manifest_datos.json](#) registra columnas, número de filas, tamaño, hash SHA-256, semilla y uso pedagógico de cada CSV. El README del paquete está en [datos/README.md](#).

Estos CSV fueron generados artificialmente. Están diseñados para práctica matemática y computacional, no para inferencias sobre poblaciones reales.

Datasets incluidos en scikit-learn

load_wine. Se usa en el arranque del curso para clasificación multiclase con mediciones químicas de vinos. Tiene 178 observaciones, 13 variables y 3 clases.

Fuente técnica: [scikit-learn: load_wine](#).

load_diabetes. Se usa para regresión, Ridge/Lasso y validación. Tiene 442 observaciones, 10 variables y un objetivo numérico.

Fuente técnica: [scikit-learn: load_diabetes](#).

load_breast_cancer. Se usa para clasificación binaria, métricas, umbrales y diagnóstico. Tiene 569 observaciones, 30 variables y 2 clases.

Fuente técnica: [scikit-learn: load_breast_cancer](#).

load_digits. Se usa para redes neuronales, PCA y clasificación multiclase. Tiene 1797 imágenes 8x8, 64 variables y 10 clases.

Fuente técnica: [scikit-learn: load_digits](#).

Datos generados para práctica

make_regression. Se usa para regresión lineal, descenso del gradiente, ruido y condicionamiento. Controla número de observaciones, variables, ruido y semilla.

Fuente técnica: [scikit-learn: make_regression](#).

make_blobs. Se usa para K-Means, clustering y separación geométrica. Controla centros, desviación de clusters y semilla.

Fuente técnica: [scikit-learn: make_blobs](#).

make_moons. Se usa para fronteras no lineales y limitaciones de modelos lineales. Controla ruido, número de puntos y semilla.

Fuente técnica: [scikit-learn: make_moons](#).

Regla de reproducibilidad

Todo ejemplo computacional debe registrar:

1. función, fuente o archivo del dataset;
2. versión aproximada de la librería cuando sea relevante;
3. `random_state` o semilla;
4. partición usada: train, dev, validation o test;
5. transformaciones aprendidas solo con entrenamiento;
6. métrica principal;
7. advertencia de interpretación.

Advertencias de interpretación

Los datasets clásicos son útiles porque están limpios, documentados y disponibles localmente. Esa misma limpieza limita su valor como evidencia aplicada.

Un buen resultado en `load_breast_cancer`, `load_diabetes` o `load_digits` no justifica por sí solo una recomendación clínica, económica u operativa. Para una decisión real se requiere validación externa, análisis de sesgos, trazabilidad de datos y revisión de dominio.

Checklist para datasets de proyecto

Antes de aprobar un dataset de proyecto, debe poder responderse:

- ¿cuál es la fuente original?
- ¿qué licencia o permiso de uso tiene?
- ¿cuándo se obtuvo?
- ¿qué representa cada fila?
- ¿cuál es la variable objetivo o decisión?
- ¿qué observaciones no están representadas?
- ¿qué filtrados o transformaciones se hicieron?
- ¿qué riesgos de privacidad, sesgo o leakage existen?
- ¿puede reproducirse la carga de datos desde instrucciones claras?

Casos longitudinales del libro

Este libro usa varios datasets pequeños y reproducibles. Algunos aparecen una sola vez; otros funcionan como **casos longitudinales**: regresan en distintos capítulos para que el estudiante vea cómo una misma decisión técnica cambia cuando se estudia desde regresión, clasificación, validación, diagnóstico, reproducibilidad y comunicación.

Un caso longitudinal no es un proyecto final. Es una línea de continuidad para leer el libro: el mismo dataset permite formular la pregunta, entrenar un baseline, validar, diagnosticar errores y escribir una recomendación.

Caso A: regresión sintética con escala y regularización

Archivo público:

```
datos/regresion_lineal_sintetica.csv
```

Uso principal:

Capítulo	Rol del caso
Capítulo 1	construir matriz de diseño, intercepto, MSE y ecuación normal
Capítulo 2	comparar ecuación normal contra descenso del gradiente
Capítulo 3	revisar escalamiento, regularización y selección por validación
Capítulo 20	registrar semilla, datos, métrica y artefactos
Capítulo 21	convertir una tabla de errores en conclusión técnica

Pregunta guía:

¿Cómo cambia la recomendación cuando pasamos de ajustar una regresión a validar si sus errores son estables y reproducibles?

Evidencia mínima:

- forma de X y de y ;
- baseline de promedio de entrenamiento;
- MSE o MAE en partición independiente;

- comparación OLS/Ridge/Lasso;
- diagnóstico de escala o condicionamiento;
- manifest de corrida.

Caso B: clasificación binaria con slice reciente

Archivo público:

`datos/clasificacion_binaria_sintetica.csv`

Este dataset incluye una columna **slice** con dos valores: **base** y **reciente**. El objetivo no es simular una población real, sino crear una situación controlada para discutir generalización y data mismatch.

Uso principal:

Capítulo	Rol del caso
Capítulo 4	interpretar probabilidades, umbral y matriz de confusión
Capítulo 11	escoger métrica, baseline, train/dev/test y pipeline
Capítulo 12	estimar variabilidad con validación cruzada y curvas
Capítulo 13	comparar desempeño global contra desempeño por slice
Capítulo 19	responder preguntas integradoras de parcial
Capítulo 21	escribir una recomendación proporcional
Capítulo 22	defender una decisión con evidencia y límites

Pregunta guía:

Si el modelo mejora el promedio global pero falla en observaciones recientes, ¿qué recomendación técnica es defendible?

Evidencia mínima:

- proporción de clase por partición;
- baseline de clase mayoritaria o regla simple;
- matriz de confusión;
- precision, recall y F1;
- métrica por **slice**;
- diagnóstico de mismatch;
- decisión y límite.

Caso C: estructura no supervisada

Archivos públicos:

```
datos/pca_sintetico.csv
datos/clusters_sinteticos.csv
datos/recomendacion_interacciones.csv
```

Uso principal:

Capítulo	Rol del caso
Capítulo 15	centrar datos, aplicar SVD/PCA y medir varianza explicada
Capítulo 16	ajustar K-Means y revisar estabilidad geométrica
Capítulo 17	construir matriz usuario-item y discutir cobertura
Capítulo 21	comunicar hallazgos exploratorios sin exagerar

Pregunta guía:

¿Qué tipo de estructura puede sugerirse sin etiquetas y qué conclusión no se puede afirmar sin validación adicional?

Evidencia mínima:

- criterio de centrado o escalamiento;
- varianza explicada o error de reconstrucción;
- inercia, silhouette o estabilidad de clusters;
- cobertura y sesgo de popularidad en recomendación;
- advertencia explícita sobre interpretación causal.

Cómo usar los casos al estudiar

Cuando un capítulo mencione un caso longitudinal, haz tres preguntas:

1. ¿Qué evidencia nueva agrega este capítulo al caso?
2. ¿Qué decisión técnica cambia respecto al capítulo anterior?
3. ¿Qué límite debe mantenerse aunque el resultado numérico mejore?

La meta no es memorizar resultados de estos datasets. La meta es aprender a reconstruir una cadena de evidencia: pregunta, datos, modelo, validación, diagnóstico, comunicación y defensa.

Atlas visual de modelos y decisiones

Este atlas reúne figuras originales del curso. Su función es dar mapas visuales para objetos que luego se formalizan con álgebra, cálculo o código. Cada figura debe leerse como guía conceptual, no como sustituto de la derivación.

Manifiesto editorial

El inventario completo de figuras, archivos fuente, texto alternativo, atribución, derechos de uso y primer uso significativo está publicado en [figuras/manifest_figuras.json](#). Este archivo permite auditar que cada SVG/PNG tenga metadatos mínimos antes de una nueva edición del libro.

Regresión lineal

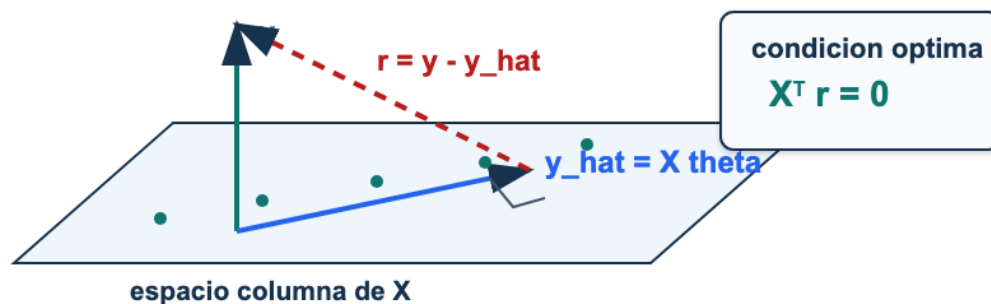


Figura 24: Geometría de mínimos cuadrados: las predicciones forman una proyección de y sobre el espacio columna de X ; el residual queda perpendicular a ese espacio.

Lectura. Mínimos cuadrados elige $\hat{y} = X\theta$ dentro del espacio de predicciones posibles. En la solución óptima, el residual $r = y - \hat{y}$ es ortogonal a las columnas de X . Esa idea geométrica produce la ecuación normal $X^T r = 0$.

Regularización

Regularización: controlar complejidad sin cambiar la pregunta

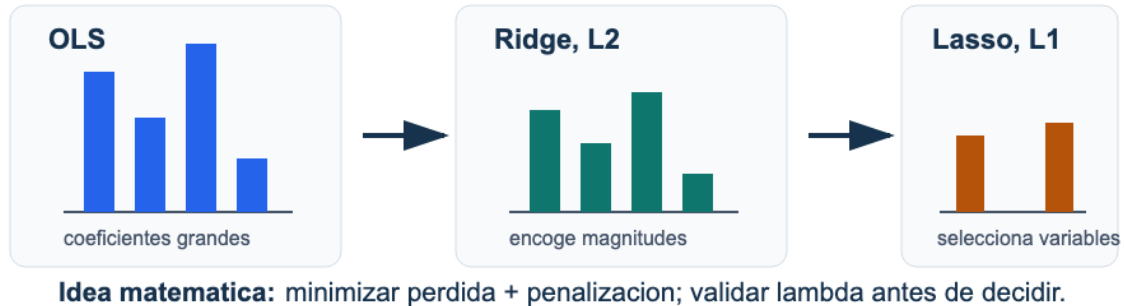


Figura 25: Regularización: OLS puede producir coeficientes grandes, Ridge reduce magnitudes y Lasso puede llevar coeficientes exactamente a cero.

Lectura. Regularizar no es castigar el modelo por estética. Es introducir una preferencia cuantitativa por soluciones más estables. La fuerza de la penalización se valida fuera del entrenamiento.

Descenso del gradiente

Descenso del gradiente: trayectoria, escala

Cada actualización usa el gradiente local para reducir la pérdida, pero el tamar

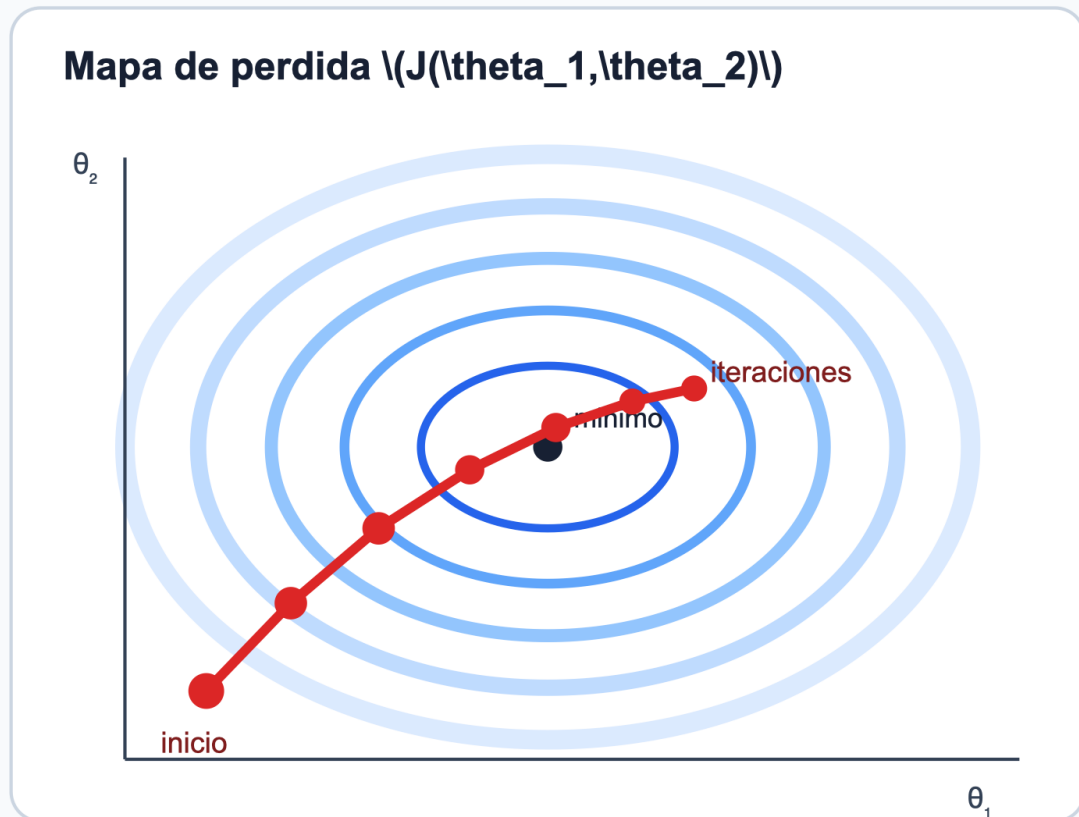


Figura 26: Descenso del gradiente: la trayectoria avanza sobre contornos de pérdida y la tasa de aprendizaje controla si el avance es lento, estable u oscilatorio.

Lectura. El gradiente define una dirección local; la tasa de aprendizaje define cuánto se avanza. La convergencia se diagnostica con pérdida, escala y estabilidad, no solo con el número de iteraciones.

Validación y curvas de aprendizaje

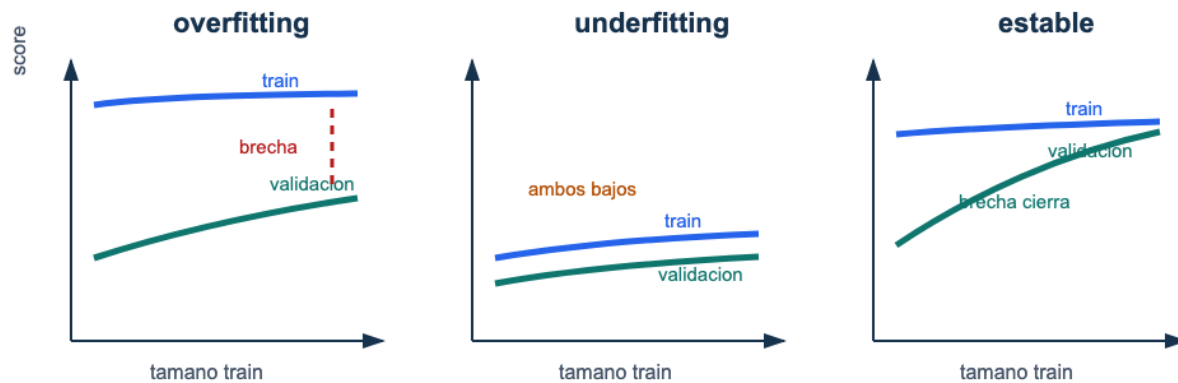


Figura 27: Curvas de aprendizaje: un caso de overfitting mantiene una brecha entre entrenamiento y validación, un caso de underfitting mantiene ambos desempeños bajos, y un caso estable cierra la brecha con más datos.

Lectura. Una curva no es decoración. Debe producir una acción: regularizar, simplificar, aumentar capacidad, recolectar datos, revisar leakage o aceptar un límite por ruido.

Particiones y evaluación

Particiones: ajustar, seleccionar y estimar

La particion no es un detalle administrativo: define que evidencia puede usarse

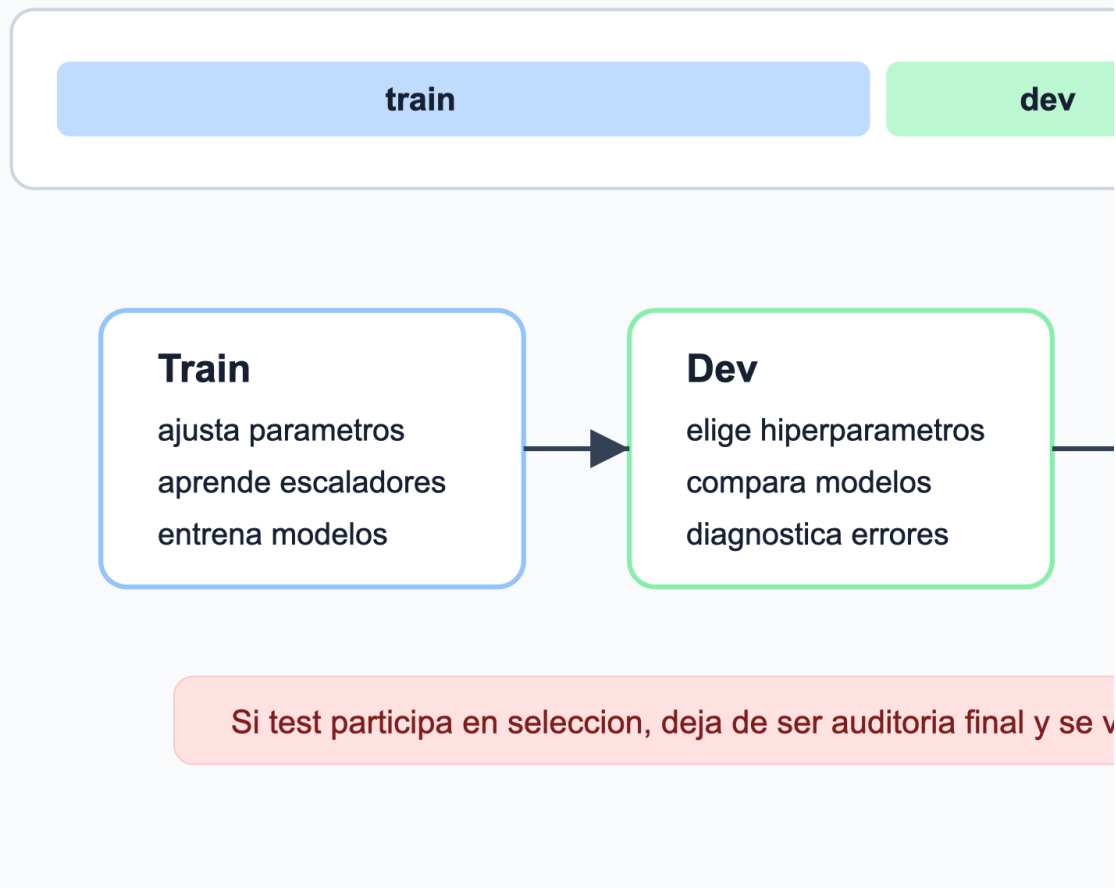


Figura 28: Particiones: train ajusta parámetros, dev selecciona modelos y test estima desempeño final; mezclar esos roles produce evaluaciones optimistas.

Lectura. La partición determina qué evidencia se puede usar. Train aprende, dev decide y test audita. Si test participa en decisiones, ya no estima desempeño final de forma honesta.

Regresión logística y umbrales

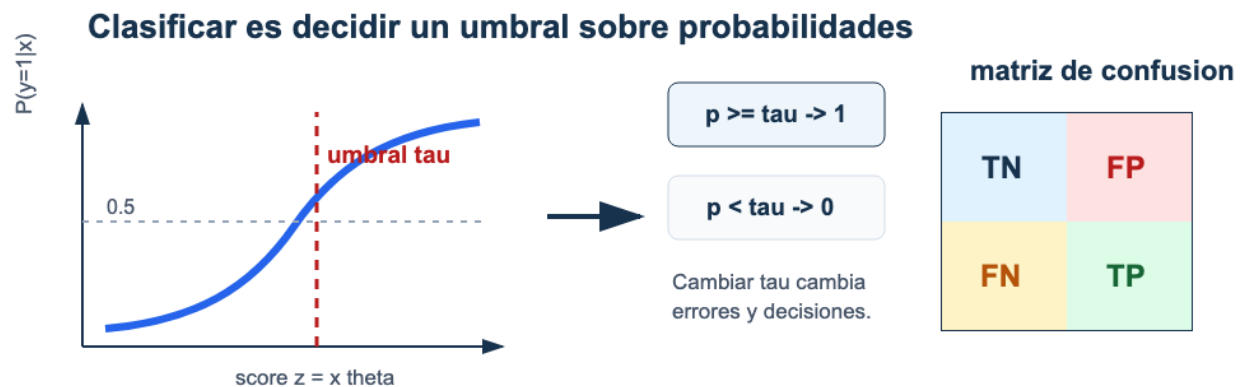


Figura 29: Regresión logística: la curva sigmoide produce probabilidades; un umbral convierte probabilidades en clases y modifica la matriz de confusión.

Lectura. El modelo produce un puntaje probabilístico; la regla de decisión aparece al escoger τ . Dos modelos con probabilidades similares pueden tener consecuencias distintas si el umbral no corresponde al costo del problema.

Métricas para decidir



Figura 30: Selección de métricas: el costo relativo de falsos positivos y falsos negativos orienta si conviene mirar accuracy, precisión, recall o una métrica F beta.

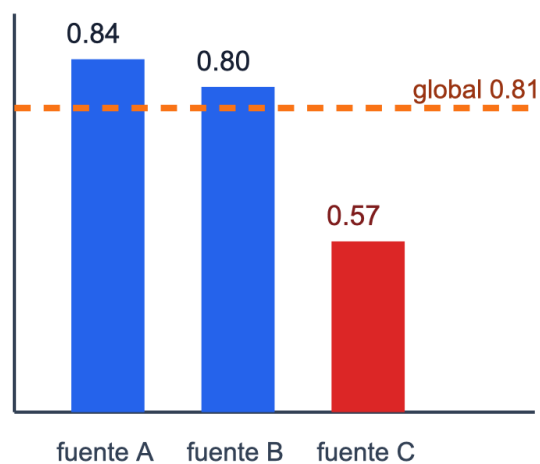
Lectura. Una métrica es una traducción operativa de una preferencia. Reportar solo accuracy en datos desbalanceados puede esconder el error que más importa.

Error analysis

Error analysis: el promedio no cuenta toda

Un modelo con metrica global aceptable puede fallar en un slice que importa p

F1 por slice



1. Eviden

2. Diagn

3. Accior

4. Limite

Figura 31: Error analysis por slices: el promedio global puede ocultar un slice crítico con desempeño bajo y exige una recomendación con evidencia, diagnóstico, acción y límite.

Lectura. El promedio global ayuda a comparar, pero no basta para decidir. Un slice crítico debe convertirse en evidencia específica, diagnóstico probable, acción técnica y límite de interpretación.

Backpropagation

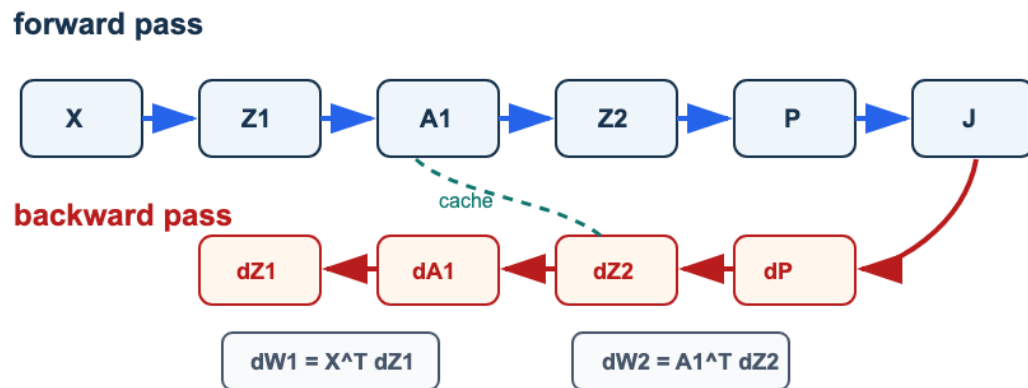


Figura 32: Backpropagation como grafo: el forward calcula X , $Z1$, $A1$, $Z2$, P y J ; el backward propaga dJ hacia $dZ2$, $dW2$, $dA1$, $dZ1$ y $dW1$ usando la cache.

Lectura. Backpropagation no es una fórmula aislada. Es una forma organizada de recorrer el grafo de cómputo en sentido inverso y producir gradientes con las mismas formas que los parámetros.

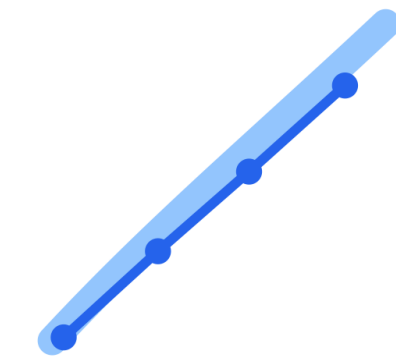
Optimizadores

Optimizadores: gradiente, memoria y escala

La regla de actualización cambia, pero la validación sigue decidiendo si el entrenamiento

SGD

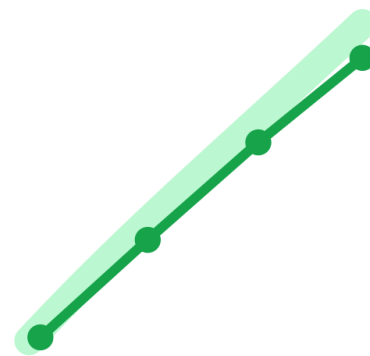
usa el gradiente actual



$$\theta \leftarrow \theta - \alpha g$$

Momentum

promedia dirección reciente



$$v \leftarrow \beta v + (1 - \beta)g$$
$$\theta \leftarrow \theta - \alpha v$$

La elección del optimizador es un hiperparámetro experimental: s

Figura 33: Optimizadores: SGD usa el gradiente actual, momentum suaviza direcciones recientes y Adam combina memoria con escala adaptativa por coordenada.

Lectura. Cambiar de optimizador cambia la actualización, no la lógica científica. Deben reportarse semilla, tasa, épocas, arquitectura y métrica de validación.

PCA y SVD

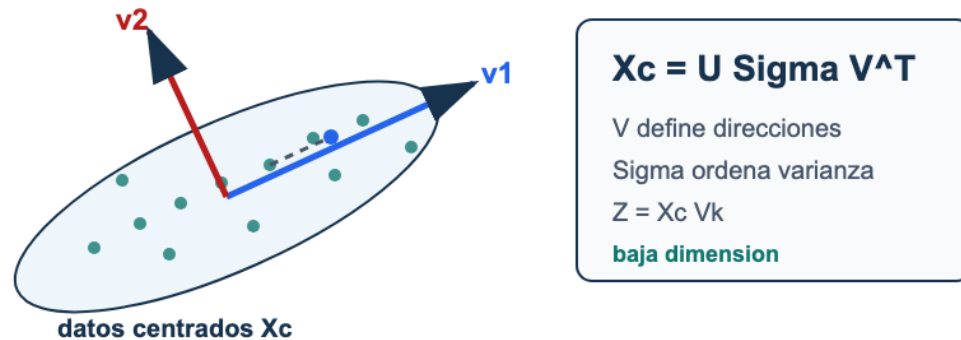


Figura 34: PCA como proyección: datos centrados se factorizan con SVD; los primeros vectores de V definen ejes principales y la matriz Z contiene coordenadas de baja dimensión.

Lectura. PCA requiere centrar datos, elegir cuántas direcciones retener y separar visualización de interpretación. Una proyección útil no prueba causalidad.

K-Means

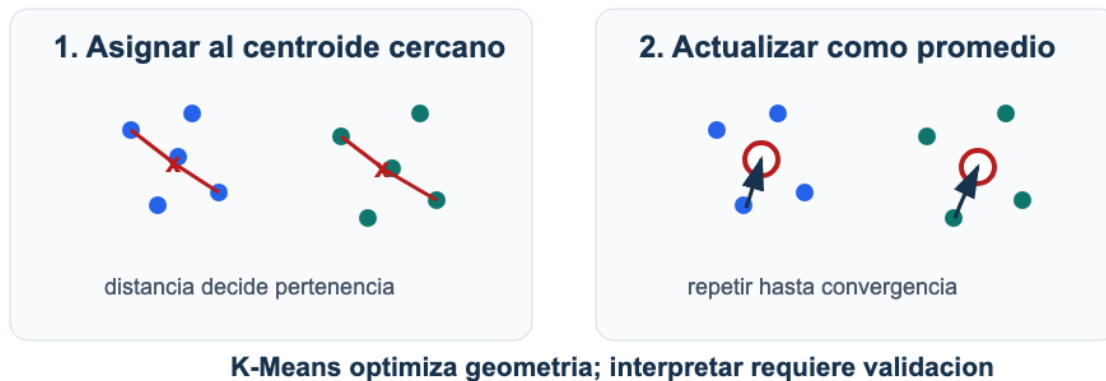


Figura 35: K-Means alterna dos pasos: asignar cada punto al centroide más cercano y actualizar centroides como promedio de los puntos asignados.

Lectura. K-Means optimiza una geometría. Antes de convertir clusters en segmentos reales, hay que revisar escalamiento, estabilidad, tamaño de grupos e interpretación con variables originales.

Recomendación latente



Lectura: factores latentes explican patrones; la validación decide si recomiendan bien.

Figura 36: Sistemas de recomendación: una matriz usuario-item incompleta se aproxima mediante factores latentes de usuarios e ítems; una calificación faltante se predice con un producto punto.

Lectura. La factorización latente convierte preferencias observadas en coordenadas de baja dimensión. La recomendación final debe evaluarse como decisión: a quién se recomienda, qué se omite y qué sesgo puede amplificarse.

Reproducibilidad

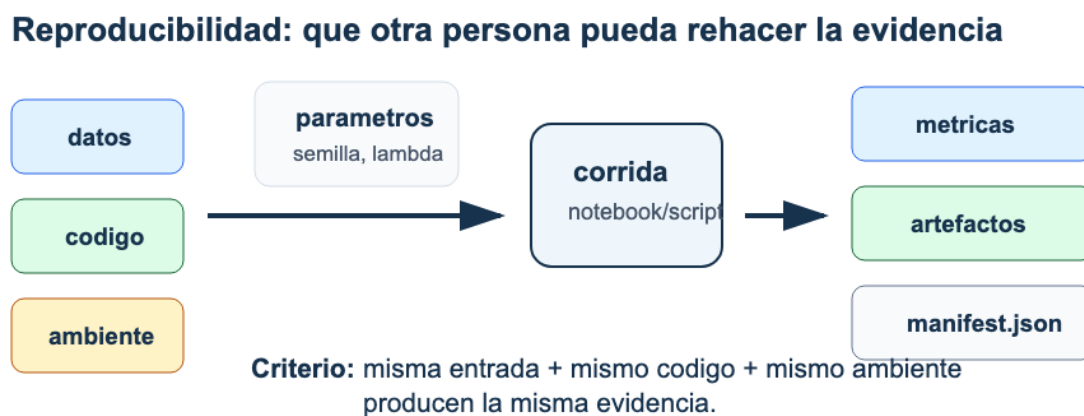


Figura 37: Reproducibilidad: datos, código, ambiente y parámetros alimentan una corrida que produce métricas, artefactos y un manifiesto verificable.

Lectura. Un resultado no es reproducible solo porque el notebook existe. La evidencia reproducible registra entradas, código, ambiente, parámetros y salidas con suficiente precisión para repetir la corrida.

Comunicación técnica: de resultados a recomendación

Un reporte profesional no narra todo lo que se hizo; defiende una conclusión con evidencia.

Tesis técnica

qué se recomienda, con qué evidencia y bajo qué condiciones

Evidencia

baseline
métrica
validación

Diagnóstico

error principal
slice crítico
causa probable

Decisión

acción proporcional
siguiente experiencia
criterio de éxito

Prueba de lectura: una persona externa debe poder cuestionar la tesis.

Figura 38: Comunicación técnica: un reporte organiza tesis, evidencia, diagnóstico, decisión y límites para que una audiencia pueda auditar la recomendación.

Lectura. La comunicación técnica convierte outputs en una tesis defendible. Cada afirmación fuerte necesita una tabla, figura, cálculo o límite que la sostenga.

Defensa oral: responder con evidencia, no

La defensa verifica autoría, criterio técnico y límites de la recomendación.



Criterio 0/1/2

0: no responde o no hay evidencia
1: idea parcial, evidencia incompleta
2: respuesta correcta, evidencia verificable y límite específico

Figura 39: Defensa oral: la respuesta individual conecta pregunta, respuesta breve, evidencia verificable y límite específico con una rúbrica 0/1/2.

Lectura. La defensa no premia recitar el reporte. Premia responder con autoría, evidencia concreta, criterio técnico y límites claros.

Resultados de aprendizaje y alineación

Esta página resume qué debe poder hacer un estudiante al terminar el curso y qué evidencia muestra ese aprendizaje. Su función es que el libro no sea solo una secuencia de temas, sino una ruta evaluable de capacidades.

El detalle operativo por capítulo está en el [Mapa de objetivos de aprendizaje](#). Para auditoría editorial, el inventario machine-readable está en [objetivos/manifest_objetivos_aprendizaje.json](#).

Resultados de aprendizaje globales

Al finalizar MATE-2105, el estudiante debe poder:

1. derivar algoritmos fundamentales de ciencia de datos desde funciones objetivo, gradientes, restricciones o factorizaciones;
2. implementar versiones básicas de esos algoritmos con programación vectorizada y verificaciones numéricas pequeñas;
3. comparar implementaciones propias con librerías profesionales sin cambiar la pregunta experimental;
4. diagnosticar modelos usando métricas, curvas, residuos, particiones y análisis de errores;
5. diseñar particiones `train/dev/test` y justificar validación, selección de métricas y criterios de comparación;
6. reconocer problemas aplicados frecuentes: escalamiento, leakage, overfitting, data mismatch, clases desbalanceadas, sesgo y límites de interpretación;
7. construir un proyecto reproducible con datos reales, baseline, modelos, evidencia y recomendaciones técnicas;
8. comunicar y defender decisiones técnicas con evidencia, límites y autoría clara.

El resultado esperado no es memorizar algoritmos. Es poder explicar qué se optimiza, cómo se calcula, cómo se valida y qué decisión se justifica con la evidencia disponible.

Alineación por parte del libro

Parte	Capacidades dominantes	Evidencia esperada
Fundamentos	lenguaje común, incertidumbre mínima, flujo aplicado, lectura de datos y primer baseline	diagnóstico inicial, pregunta modelable y lectura de ruido/variable objetivo

Parte	Capacidades dominantes	Evidencia esperada
Regresión, optimización y clasificación	derivación, implementación, regularización, validación y métricas	laboratorios y Parcial 1
Redes neuronales	forward propagation, backpropagation, diagnóstico y PyTorch	implementación desde cero y comparación profesional
Estrategia de ML	métricas, particiones, validación cruzada, curvas y error analysis	selección defendible de modelo
No supervisado	SVD/PCA, clustering y recomendación	reducción de dimensión, segmentación o recomendación justificadas
Reproducibilidad y defensa	tracking, comunicación, reporte y defensa oral	proyecto final reproducible y defensa individual

Alineación con evaluación

Evidencia	Qué verifica	Qué no debe verificar sola
Parciales	razonamiento matemático, derivación, diagnóstico y lectura de resultados	memorización de fórmulas aisladas
Laboratorios	implementación, pruebas, comparación con referencia e interpretación corta	copiar una API sin entender el algoritmo
Notebooks de clase	práctica guiada, predicción, ejecución y discusión de errores	evaluación final de desempeño
Proyecto final	flujo completo con datos reales, reproducibilidad y comunicación	una métrica única sin contexto
Defensa oral	comprensión individual, autoría y límites de la recomendación	presentación memorizada sin evidencia

Criterios de evidencia

Una evidencia de aprendizaje es fuerte cuando contiene:

- **objeto matemático:** variables, dimensiones, función objetivo o métrica;
- **procedimiento:** algoritmo, gradiente, factorización o regla de decisión;
- **verificación:** prueba pequeña, comparación, baseline o resultado esperado;
- **validación:** partición, métrica, curva, error o análisis de estabilidad;
- **interpretación:** recomendación, límite y riesgo de extrapolación;
- **trazabilidad:** fuente de datos, semilla, ambiente, ayuda externa y autoría.

Los casos longitudinales del libro ayudan a practicar esa estructura en más de un capítulo. Un mismo dataset puede producir evidencia de ajuste, validación, diagnóstico, comunicación y defensa; por eso el aprendizaje esperado no es solo resolver una tarea local, sino sostener una decisión técnica a través de varias etapas.

Progresión de autonomía

Momento	Apoyo esperado	Autonomía esperada
Semanas 1-3	plantillas, ejemplos pequeños y tests guiados	entender notación, ejecutar notebooks y explicar outputs
Semanas 4-6	funciones incompletas, rúbricas y comparación con <code>sklearn</code>	implementar, validar y justificar regularización o umbral
Semanas 7-10	derivaciones guiadas y referencias profesionales	leer loops de entrenamiento y diagnosticar entrenamiento
Semanas 11-13	casos aplicados, curvas y análisis de errores	seleccionar métrica, partición y diagnóstico defendible
Semanas 14-16	checklist final y defensa	sostener una recomendación técnica reproducible

Cómo usar esta página

Para estudiar, usa los resultados como lista de control: si no puedes explicar un resultado con un ejemplo pequeño y una interpretación, todavía falta trabajo.

Para enseñar, usa la alineación como restricción de diseño: una actividad semanal debe preparar al menos una evidencia mayor del curso. Si una clase introduce un tema que no se conecta con evidencia, laboratorio, parcial o proyecto, debe revisarse su lugar en la secuencia.

Para revisar el libro, usa esta página como auditoría pedagógica: cada capítulo debe aportar a una capacidad observable, no solo cubrir un tema.

Mapa de objetivos de aprendizaje

Esta página convierte los resultados de aprendizaje en una matriz operativa por capítulo. Su propósito es que cada capítulo tenga una promesa verificable: qué debe poder hacer el estudiante, con qué evidencia se observa y cómo se conecta con la evaluación del curso.

El inventario completo está publicado en [objetivos/manifest_objetivos_aprendizaje.json](#). Ese archivo funciona como contrato editorial: cada capítulo principal debe tener objetivos observables, evidencia primaria y alineación con los resultados globales del curso.

Cómo leer los códigos

Los resultados globales usan códigos RA1 a RA8:

Código	Capacidad observable
RA1	derivar algoritmos desde objetivos, gradientes, restricciones o factorizaciones
RA2	implementar algoritmos con programación vectorizada y verificaciones pequeñas
RA3	comparar implementaciones propias con librerías profesionales
RA4	diagnosticar modelos con métricas, curvas, residuos, particiones y errores
RA5	diseñar validación, particiones y criterios de comparación
RA6	reconocer leakage, overfitting, data mismatch, sesgo y límites
RA7	construir un proyecto reproducible con evidencia y recomendación
RA8	comunicar y defender decisiones técnicas con límites y autoría clara

Los objetivos de capítulo usan códigos como C04-02: capítulo 4, objetivo 2.

Cobertura por parte

Parte	Capítulos principales	Énfasis de aprendizaje
Parte I: Fundamentos	2	lenguaje común, incertidumbre mínima, datos observados, ruido y generalización
Parte II: Regresión, optimización y clasificación	4	derivación, implementación, regularización, validación y clasificación
Parte III: Redes neuronales	4	forward pass, backpropagation, diagnóstico y PyTorch mínimo
Parte IV: Estrategia de machine learning	3	métricas, particiones, curvas, error analysis y decisiones de modelo
Parte V: Aprendizaje no supervisado	3	PCA/SVD, clustering, recomendación y cautela interpretativa
Parte VI: Reproducibilidad, comunicación y defensa	4	síntesis, tracking, reporte, entrega y defensa

Mapa por capítulo principal

Capítulo	Objetivos observables	Evidencia primaria	Alineación
Incertidumbre y datos observados	distinguir columna observada y variable aleatoria; interpretar ruido y probabilidad condicional; explicar pérdida empírica frente a riesgo esperado	Lab diagnóstico, ideas iniciales de proyecto, lectura de Semana 1	RA4, RA5, RA6, RA8
Regresión lineal múltiple	formular modelo matricial; derivar ecuación normal; comparar implementación propia con referencia	Lab 1, Parcial 1, baseline de proyecto	RA1, RA2, RA3
Descenso del gradiente	calcular gradiente de MSE; implementar descenso; diagnosticar escalamiento y condicionamiento	Lab 1, Parcial 1	RA1, RA2, RA4, RA6

Capítulo	Objetivos observables	Evidencia primaria	Alineación
Regularización y validación	distinguir train/dev/test; explicar Ridge/Lasso; seleccionar complejidad con validación	Lab 2, Parcial 1, checkpoint de proyecto	RA1, RA4, RA5, RA6, RA8
Regresión logística	derivar pérdida logística; convertir probabilidades en decisiones; justificar métricas	Lab 2, Parcial 1, baseline de clasificación	RA1, RA2, RA4, RA5, RA6, RA8
Perceptrón y forward propagation	representar capas; implementar forward pass; explicar logits y activaciones	notebook de clase, laboratorio de redes	RA1, RA2, RA8
Backpropagation	aplicar regla de la cadena; implementar backward pass; verificar gradientes	notebook de clase, laboratorio de redes	RA1, RA2, RA4
Diagnóstico en redes	leer curvas; aplicar regularización o early stopping; reportar límites	curvas de entrenamiento, laboratorio de redes	RA4, RA5, RA6, RA8
Optimizadores y PyTorch	construir loop mínimo; comparar optimizadores; separar API de razonamiento	notebook PyTorch, laboratorio de redes	RA2, RA3, RA4, RA8
Métricas, particiones y baseline	diseñar particiones; definir baseline; seleccionar métricas por costo de error	checkpoint de proyecto, reporte de baseline	RA4, RA5, RA6, RA7, RA8
Validación y curvas	usar validación sin contaminar test; interpretar curvas; seleccionar modelo por evidencia	checkpoint de proyecto, reporte de selección	RA4, RA5, RA6, RA8
Error analysis y data mismatch	construir slices de error; diagnosticar mismatch; priorizar acciones de mejora	análisis de errores del proyecto, reporte técnico	RA4, RA5, RA6, RA7, RA8
SVD y PCA	explicar PCA como proyección; calcular scores y reconstrucción; interpretar sin causalidad	notebook de PCA, ejercicio integrador	RA1, RA2, RA6, RA8

Capítulo	Objetivos observables	Evidencia primaria	Alineación
K-Means y clustering	ejecutar una iteración; diagnosticar escala e inicialización; comunicar clusters con cautela	notebook de clustering, ejercicio integrador	RA1, RA2, RA4, RA6, RA8
Recomendación latente	representar matriz usuario-item; evaluar sin leakage; discutir sesgos de recomendación	notebook de recomendación, ejercicio integrador	RA1, RA2, RA4, RA5, RA6, RA8
Síntesis conceptual	conectar objetivos, algoritmos y diagnósticos; resolver preguntas integradoras; reconocer evidencia faltante	Parcial 2, repaso integrador	RA1, RA2, RA4, RA6, RA8
Reproducibilidad y tracking	registrar manifiesto reproducible; reejecutar análisis; distinguir reproducibilidad y validez	manifest del proyecto, reporte reproducible	RA2, RA6, RA7, RA8
Comunicación técnica	construir narrativa técnica; seleccionar figuras funcionales; explicar límites	reporte final, presentación final	RA4, RA6, RA7, RA8
Entrega y defensa	preparar artefactos verificables; defender decisiones; reconocer límites y riesgos	defensa oral, entrega final del proyecto	RA4, RA5, RA6, RA7, RA8

Uso para estudiantes

Antes de leer un capítulo, identifica los verbos de la fila correspondiente: formular, derivar, implementar, diagnosticar, seleccionar, comunicar o defender. Después de estudiar, debes poder producir la evidencia indicada sin mirar una solución oficial completa.

Si solo puedes repetir una definición, todavía no has alcanzado el objetivo. Si puedes resolver un caso pequeño, verificarlo con código y explicar el límite de la conclusión, estás cerca del estándar esperado.

Uso para instructores

Esta matriz permite revisar si una clase, notebook, laboratorio o parcial está evaluando la capacidad correcta. Si una actividad no produce evidencia asociada a un objetivo, debe rediseñarse o declararse como práctica no evaluativa.

Para publicar una nueva edición, el manifiesto debe mantenerse sincronizado con:

- capítulos principales;
- laboratorios y parciales;
- checkpoints de proyecto;
- rúbricas públicas;
- recursos restringidos de instructor.

Criterio de mantenimiento

Cada cambio de capítulo debe revisar tres preguntas:

1. ¿Cambió lo que el estudiante debe poder hacer?
2. ¿Cambió la evidencia que prueba ese aprendizaje?
3. ¿Cambió la alineación con evaluación, laboratorio, parcial o proyecto?

Si la respuesta a alguna pregunta es sí, debe actualizarse `objetivos/manifest_objetivos_aprendizaje.json` y ejecutarse:

```
python3 scripts/auditar_objetivos_aprendizaje_libro.py
```

Mapa de cierres de capítulo

Este mapa muestra cómo los capítulos principales del libro incorporan elementos de cierre de capítulo comparables a un texto profesional: revisión, términos clave, preguntas de práctica, problemas cuantitativos, pensamiento crítico y conexión con proyecto.

La página no sustituye los ejercicios. Sirve para que estudiantes, instructores y revisores puedan verificar rápidamente que cada capítulo termina con práctica clasificada y evidencia observable.

Rasgos de cierre

Rasgo	Función pedagógica	Evidencia esperada
Revisión del capítulo	synetizar ideas centrales antes de practicar	lista de conceptos, criterios o advertencias
Términos clave	fijar vocabulario técnico	definiciones operativas y notación usada
Preguntas conceptuales	comprobar comprensión sin cálculo largo	explicación breve, contraejemplo o distinción
Problemas cuantitativos	verificar cálculo y derivación	fórmula, sustitución, resultado y sentido
Práctica computacional	conectar algoritmo con código	función, notebook, shape, métrica o salida reproducible
Pensamiento crítico	interpretar evidencia y límites	recomendación proporcional con riesgo declarado
Proyecto de cierre	transferir el tema al proyecto	decisión documentada, artefacto o miniinforme

Los cierres de capítulo son públicos. No contienen pruebas ocultas, soluciones oficiales completas ni lógica de autograding.

Mapa por capítulo principal

Parte	Capítulo	Cierre público	Rasgos incluidos
Fundamentos	Incertidumbre y datos observados	revisión, términos, ejercicios y proyecto	variable aleatoria, distribución empírica, ruido, riesgo esperado

Parte	Capítulo	Cierre público	Rasgos incluidos
Regresión	Regresión lineal	revisión, términos, ejercicios y proyecto	conceptual, cuantitativo, computacional, interpretación
Regresión	Descenso del gradiente	revisión, términos, ejercicios y proyecto	derivación, simulación, diagnóstico de convergencia
Regresión	Regularización y validación	revisión, términos, ejercicios y proyecto	validación, leakage, Ridge/Lasso, selección
Regresión	Regresión logística	revisión, términos, ejercicios y proyecto	log-loss, umbral, matriz de confusión, recomendación
Redes	Perceptrón y forward propagation	revisión, términos, ejercicios y proyecto	formas, activaciones, capas, predicción
Redes	Backpropagation	revisión, términos, ejercicios y proyecto	regla de la cadena, gradientes, verificación
Redes	Diagnóstico y regularización en redes	revisión, términos, ejercicios y proyecto	overfitting, dropout, early stopping, curvas
Redes	Optimizadores y PyTorch	revisión, términos, ejercicios y proyecto	loop de entrenamiento, optimizador, reproducibilidad
Estrategia ML	Métricas y particiones	revisión, términos, ejercicios y proyecto	train/dev/test, baseline, costos de error
Estrategia ML	Validación y curvas	revisión, términos, ejercicios y proyecto	learning curves, cross-validation, estabilidad
Estrategia ML	Error analysis y data mismatch	revisión, términos, ejercicios y proyecto	slices, mismatch, diagnóstico, acción
No supervisado	SVD y PCA	revisión, términos, ejercicios y proyecto	centrar, proyectar, reconstruir, explicar varianza
No supervisado	K-Means y clustering	revisión, términos, ejercicios y proyecto	centroides, inercia, escala, estabilidad
No supervisado	Recomendación latente	revisión, términos, ejercicios y proyecto	similitud, ranking, evaluación offline
Cierre	Síntesis conceptual	revisión, términos, ejercicios y proyecto	mapa de decisiones, comparación, límites
Cierre	Reproducibilidad y tracking	revisión, términos, ejercicios y proyecto	manifest, semillas, ambiente, trazabilidad
Cierre	Comunicación técnica	revisión, términos, ejercicios y proyecto	reporte, visualización, audiencia, evidencia
Cierre	Entrega y defensa	revisión, términos, ejercicios y proyecto	defensa oral, autoría, límites, cierre

Bancos integradores

Los capítulos de ejercicios integradores funcionan como cierres de parte. No repiten el patrón de un capítulo conceptual; reúnen práctica acumulativa, problemas integradores, autoevaluación y respuestas seleccionadas.

Parte	Banco público	Uso recomendado
Regresión	Ejercicios de regresión	preparación de Parcial 1 y baseline del proyecto
Redes	Ejercicios de redes	verificación de forward, backpropagation y PyTorch
Estrategia ML	Ejercicios de estrategia	selección de métrica, partición y diagnóstico
No supervisado	Ejercicios de no supervisado	PCA, K-Means, recomendación y evaluación
Cierre	Ejercicios de cierre	reproducibilidad, comunicación y defensa final

Relación con respuestas seleccionadas

Las respuestas seleccionadas viven en los apéndices de cada parte y en el apéndice consolidado de formularios. Su propósito es orientar estudio y revisión, no reemplazar soluciones oficiales.

Recurso	Función
Formularios y respuestas seleccionadas	consulta pública consolidada
Apéndices por parte	notación mínima, fórmulas y respuestas seleccionadas
Guía del estudiante	rutina para usar respuestas sin copiar
Guía de adopción docente	separación entre recurso público y solución restringida

Criterio editorial

Un capítulo principal queda listo para uso de curso cuando:

1. declara objetivos observables;
2. contiene al menos un ejemplo resuelto;
3. incluye revisión del capítulo;
4. define términos clave;
5. ofrece ejercicios conceptuales, cuantitativos, computacionales e interpretativos;
6. cierra con una conexión de proyecto;
7. no expone soluciones oficiales completas ni pruebas ocultas;
8. pasa la auditoría de estructura editorial.

La auditoría automática revisa presencia y densidad mínima. La revisión externa real debe juzgar si esos cierres son matemáticamente correctos y pedagógicamente suficientes.

Apéndices de consulta

Glosario y notación

Este apéndice reúne términos y símbolos que se usan repetidamente en el curso. No reemplaza las definiciones de cada capítulo; sirve como referencia rápida.

Términos clave

Término	Significado operativo en el curso
Observación	Una fila o unidad de análisis del conjunto de datos.
Variable	Una característica medida, construida o codificada para cada observación.
Variable aleatoria	Cantidad incierta antes de observar un caso; modela una columna antes de verla.
Realización	Valor concreto observado de una variable aleatoria.
Distribución empírica	Patrón de valores observado en una muestra disponible.
Valor esperado	Promedio ideal bajo el proceso que genera los datos.
Ruido	Variabilidad no explicada por las variables disponibles o por el modelo usado.
Matriz de diseño	Matriz X que contiene las variables usadas por el modelo.
Parámetro	Cantidad aprendida por el modelo, como un peso o coeficiente.
Predicción	Salida producida por el modelo para una observación o lote de observaciones.
Residual	Diferencia entre valor observado y predicción en regresión.
Pérdida	Función que el algoritmo minimiza durante entrenamiento.
Pérdida empírica	Promedio de pérdidas calculado sobre datos observados.
Riesgo esperado	Error promedio ideal sobre observaciones futuras del proceso relevante.
Métrica	Cantidad usada para evaluar desempeño y tomar decisiones.
Baseline	Modelo o regla simple que sirve como punto mínimo de comparación.

Término	Significado operativo en el curso
Gradiente	Vector de derivadas parciales que indica dirección local de mayor aumento.
Tasa de aprendizaje	Escala del paso en descenso del gradiente.
Regularización	Penalización o restricción que busca mejorar generalización.
Validación	Evaluación en datos no usados para ajustar parámetros.
Leakage	Uso accidental de información que no estaría disponible al predecir en producción.
Regresión logística	Modelo probabilístico para clasificación binaria.
Probabilidad condicional	Probabilidad calculada dado que se conoce cierta información, como $P(Y = 1 \mid X = x)$.
Umbral	Regla que convierte probabilidad o score en decisión discreta.
Matriz de confusión	Tabla de verdaderos/falsos positivos y negativos.
Precision	Proporción de predicciones positivas que fueron correctas.
Recall	Proporción de positivos reales detectados por el modelo.
F1	Media armónica entre precision y recall.
Forward propagation	Cálculo de predicciones capa por capa en una red neuronal.
Backpropagation	Cálculo eficiente de gradientes en una red usando regla de la cadena.
Cross-validation	Evaluación repetida con particiones de entrenamiento y validación.
PCA	Proyección lineal que conserva la mayor varianza posible en pocas dimensiones.
SVD	Factorización matricial que sustenta PCA y otras aproximaciones de bajo rango.
K-Means	Algoritmo de clustering basado en centroides.
Reproducibilidad	Capacidad de reconstruir resultados a partir de datos, código, entorno y decisiones documentadas.

Notación frecuente

Símbolo	Uso
n	Número de observaciones.
p	Número de columnas usadas por el modelo después de codificación.
d	Número de variables originales o dimensión de entrada, según contexto.

Símbolo	Uso
X	Matriz de diseño o lote de entradas.
x_i	Vector de variables de la observación i .
y	Vector de valores observados.
y_i	Valor observado de la observación i .
Y	Variable aleatoria objetivo antes de observar un caso.
$\hat{y}$	Vector de predicciones.
$E[Y]$	Valor esperado de Y .
$P(Y = 1 \mid X = x)$	Probabilidad condicional de clase positiva dado x .
ε	Ruido o componente no explicado en un modelo como $Y = f(X) + \varepsilon$.
θ	Vector de parámetros en modelos lineales.
w, b	Pesos y sesgo de una neurona o capa.
$J(\theta)$	Función de pérdida como función de parámetros.
∇J	Gradiente de la función de pérdida.
λ	Intensidad de regularización.
α	Tasa de aprendizaje o hiperparámetro, según contexto.
Z	Pre-activación o representación proyectada.
A	Activación de una capa.
$U, \Sigma, V^\top$	Factores de la descomposición SVD.

Convención de formas

Cuando se use código Python, el libro favorece esta convención:

Objeto	Forma típica
<code>X</code>	<code>(n, p)</code>
<code>y</code>	<code>(n,)</code> o <code>(n, 1)</code>
<code>theta</code>	<code>(p,)</code> o <code>(p, 1)</code>
<code>X @ theta</code>	<code>(n,)</code> o <code>(n, 1)</code>
<code>W</code> en una capa densa	<code>(d_in, d_out)</code>
<code>b</code> en una capa densa	<code>(d_out,)</code>

Las formas pueden cambiar por biblioteca o por convención local. Lo importante es declarar la convención antes de derivar o implementar.

Índice temático

Este índice ayuda a encontrar conceptos por problema de estudio, no por orden semanal. Úsalo cuando necesites repasar una idea antes de un laboratorio, ubicar una derivación, comparar modelos o preparar una defensa técnica.

Cómo usar este índice

- Si estás empezando un tema, lee primero el capítulo indicado como entrada.
- Si estás resolviendo un ejercicio, revisa también el apéndice o banco integrador asociado.
- Si estás trabajando en proyecto, busca la columna de decisión técnica: allí se indica qué evidencia debes poder defender.

Modelación y representación

Tema	Entrada recomendada	Conexión de proyecto
Observaciones, variables y matriz de datos	Capítulo 0	definir qué representa una fila y qué decisión se quiere informar
Variable aleatoria y realización	Interludio 0.5	distinguir columna observada, objetivo futuro y caso nuevo
Distribución empírica y ruido	Interludio 0.5	describir variabilidad inicial y límites del dataset
Pérdida empírica y riesgo esperado	Interludio 0.5	explicar por qué validar importa para decisiones futuras
Matriz de diseño e intercepto	Capítulo 1	documentar columnas, transformaciones y forma final de X
Parámetros, predicciones y residuales	Capítulo 1	interpretar qué aprende el modelo y qué errores quedan
Features e interacciones	Capítulo 3	justificar transformaciones sin introducir leakage
Baselines	Capítulo 0 y Capítulo 11	comparar contra una regla simple antes de celebrar desempeño
Casos longitudinales	Casos longitudinales	seguir el mismo dataset a través de modelo, validación, diagnóstico y comunicación

Álgebra lineal y optimización

Tema	Entrada recomendada	Conexión de proyecto
Prerrequisitos matemáticos mínimos	Apéndice de prerrequisitos	repasar formas, gradientes, probabilidad mínima y notación antes de resolver ejercicios
Producto matricial y formas	Capítulo 1	detectar errores antes de entrenar
Norma cuadrática y MSE	Capítulo 1	explicar qué error numérico se minimiza
Ecuación normal	Capítulo 1	comparar solución cerrada contra implementación de biblioteca
Gradiente de MSE	Capítulo 2	justificar la actualización iterativa
Escalamiento y condicionamiento	Capítulo 2	decidir cuándo una variable necesita transformación
SVD y bajo rango	Capítulo 15	explicar qué estructura se conserva al reducir dimensión

Regresión, regularización y clasificación

Tema	Entrada recomendada	Conexión de proyecto
Regresión lineal múltiple	Capítulo 1	construir baseline de regresión defendible
Descenso del gradiente	Capítulo 2	diagnosticar convergencia y tasa de aprendizaje
Overfitting y complejidad	Capítulo 3	elegir complejidad usando validación, no test
Ridge y Lasso	Capítulo 3	controlar varianza y seleccionar hiperparámetros
Regresión logística	Capítulo 4	convertir scores en probabilidades interpretables
Probabilidad condicional	Interludio 0.5 y Capítulo 4	interpretar $P(Y = 1 \mid X = x)$
Umbral de decisión	Capítulo 4	antes de escoger umbral
Ejercicios integradores de regresión	Capítulo 5	ajustar decisión al costo relativo de errores
		practicar derivación, código e interpretación conjunta

Validación, métricas y diagnóstico

Tema	Entrada recomendada	Conexión de proyecto
Train, dev, validation y test	Capítulo 3 y Capítulo 11	separar ajuste, selección y estimación final
Leakage	Capítulo 11	impedir que el pipeline vea información de validación o test
Matriz de confusión	Capítulo 4	explicar tipos de error
Precision, recall y F1	Capítulo 4 y Capítulo 11	escoger métrica primaria según costo
Curvas de aprendizaje	Capítulo 12	distinguir sesgo, varianza y límite por datos
Error analysis por slice	Capítulo 13	encontrar fallas que el promedio oculta
Data mismatch	Capítulo 13	decidir si un modelo puede salir del contexto de entrenamiento

Redes neuronales

Tema	Entrada recomendada	Conexión de proyecto
Perceptrón y capas densas	Capítulo 6	decidir si hace falta no linealidad
Forward propagation	Capítulo 6	revisar formas de entradas, pesos, activaciones y logits
Backpropagation	Capítulo 7	verificar gradientes y cache
Diagnóstico de entrenamiento	Capítulo 8	interpretar curvas train/dev
Regularización en redes	Capítulo 8	decidir entre menor capacidad, L2, dropout o más datos
Optimizadores y PyTorch	Capítulo 9	reportar configuración reproducible de entrenamiento

Aprendizaje no supervisado y recomendación

Tema	Entrada recomendada	Conexión de proyecto
PCA	Capítulo 15	reducir dimensión sin afirmar causalidad
Varianza explicada	Capítulo 15	justificar cuántos componentes conservar
K-Means	Capítulo 16	describir clusters antes de convertirlos en decisiones

Tema	Entrada recomendada	Conexión de proyecto
Inercia y silueta	Capítulo 16	evaluar estructura sin absolutizar una métrica
Recomendación item-item	Capítulo 17	explicar similitud, cobertura y sesgo de popularidad
Ejercicios integradores no supervisados	Capítulo 18	practicar PCA, clustering y recomendación con límites

Reproducibilidad y comunicación

Tema	Entrada recomendada	Conexión de proyecto
Síntesis conceptual	Capítulo 19	conectar pérdidas, métricas, validación y decisión
Manifest de experimento	Capítulo 20	registrar datos, código, ambiente, parámetros y artefactos
Trazabilidad de datos	Capítulo 20	permitir auditoría de resultados
Reporte técnico	Capítulo 21	separar evidencia, interpretación y recomendación
Defensa oral	Capítulo 22	responder preguntas difíciles con evidencia concreta
Paquete final	Capítulo 22 y Capítulo 23	entregar reporte, código, datos permitidos y evidencia reproducible

Recursos de consulta rápida

Necesidad	Recurso
Definiciones y símbolos	Glosario y notación
Prerrequisitos de lectura	Prerrequisitos matemáticos mínimos
Algoritmos, datasets y funciones	Índice de algoritmos, datasets y funciones
Respuestas seleccionadas	Formularios y respuestas seleccionadas
Datos descargables	Datos y recursos reproducibles
Accesibilidad y barreras conocidas	Accesibilidad
Errata y correcciones	Errata, revisión y control de calidad

Criterio de mantenimiento

Este índice debe actualizarse cuando:

- se agregue un capítulo;
- cambie el nombre de un capítulo;
- se mueva una derivación central;
- se agregue una métrica, algoritmo o dataset público;
- se modifique la estructura de apéndices;
- una revisión externa pida mejorar navegabilidad o trazabilidad.

Un índice temático profesional no reemplaza la lectura lineal. Sirve para estudiar de forma estratégica y para conectar conceptos que aparecen en partes distintas del curso.

Índice de algoritmos, datasets y funciones

Este apéndice funciona como una tabla de referencia rápida. No enseña los conceptos desde cero; indica dónde aparecen, qué función o clase suele usarse y qué debe revisar el estudiante antes de confiar en un resultado.

Cómo usar este apéndice

Usa este índice cuando encuentres una función, clase o algoritmo en un notebook y necesites ubicar su primer uso conceptual. La columna de revisión necesaria indica qué debe comprobarse antes de convertir un output en evidencia.

El primer uso fuerte no siempre coincide con la primera mención. Se registra el punto donde el libro espera que puedas explicar el objeto matemático, ejecutar o leer una implementación y conectar el resultado con una decisión.

Mapa por primer uso

Capítulo	Objetos técnicos introducidos	Uso posterior
Capítulo 0	<code>train_test_split</code> , <code>DummyClassifier</code> , <code>StandardScaler</code> , <code>LogisticRegression</code> , <code>accuracy_score</code>	flujo mínimo de modelación y baseline
Interludio 0.5	variable aleatoria, distribución empírica, ruido, pérdida empírica, riesgo esperado, <code>np.mean</code> , <code>np.var</code> , <code>np.random.default_rng</code>	lectura probabilística mínima de regresión, clasificación y validación
Capítulo 1	matriz de diseño, MSE, ecuación normal, <code>np.linalg.solve</code>	regresión, comparación con bibliotecas y baselines
Capítulo 2	descenso del gradiente, <code>np.linalg.cond</code> , historial de pérdida	optimización, escala y diagnóstico de convergencia
Capítulo 3	features polinomiales, Ridge, Lasso, validación	selección de modelo, regularización y prevención de leakage

Capítulo	Objetos técnicos introducidos	Uso posterior
Capítulo 4	sigmoide estable, log-loss,	clasificación binaria y métricas de decisión
Capítulo 6	matriz de confusión, umbral	redes neuronales y clasificación multiclase
Capítulo 7	capas densas, activaciones, softmax estable	entrenamiento de redes y verificación numérica
Capítulo 9	regla de la cadena, cache, gradientes vectorizados	loops de entrenamiento reproducibles
Capítulo 11	<code>nn.Module</code> , <code>loss.backward</code> , <code>optimizer.step</code>	evaluación profesional y selección de modelos
Capítulo 12	particiones, <code>Pipeline</code> , métricas y prevención de leakage	diagnóstico de sesgo/varianza e hiperparámetros
Capítulo 15	curvas de aprendizaje, curvas de validación, <code>GridSearchCV</code>	reducción de dimensión e interpretación cautelosa
Capítulo 16	centrado, SVD, PCA, varianza explicada	clustering, segmentación y estabilidad
Capítulo 17	<code>KMeans</code> , inercia, silhouette, escalamiento	recomendación y evaluación offline
Capítulo 20	similitud coseno, ranking y cobertura	trazabilidad y reconstrucción de resultados
Capítulo 20	manifest de experimento, semillas, ambiente	

Referencia de API por capítulo

Primer uso	API o patrón	Qué aprende o calcula	Riesgo principal
Cap. 0	<code>train_test_split</code>	separa observaciones	mezclar selección y test
Cap. 0	<code>DummyClassifier</code>	baseline de clasificación	compararlo con otra partición
Cap. 0	<code>make_pipeline</code>	encadena transformación y modelo	olvidar qué paso aprende parámetros
Int. 0.5	<code>np.mean</code> , <code>np.var</code>	resumen de distribución empírica	confundir resumen muestral con garantía futura
Int. 0.5	<code>np.random.default_rng</code>	simulación reproducible de ruido	interpretar simulación como dato real
Cap. 1	<code>np.column_stack</code> , <code>np.hstack</code>	construye matriz de diseño	duplicar intercepto
Cap. 1	<code>np.linalg.solve</code>	resuelve sistema lineal	usar matriz mal condicionada
Cap. 2	historial de pérdida	registra convergencia	no revisar escala de gradiente

Primer uso	API o patrón	Qué aprende o calcula	Riesgo principal
Cap. 3	<code>Ridge</code> , <code>Lasso</code>	ajusta regularización	escoger hiperparámetro con test
Cap. 4	<code>np.clip</code> , <code>np.log</code>	estabiliza log-loss	ocultar probabilidades extremas sin revisar
Cap. 6	<code>softmax</code> estable	convierte logits en probabilidades	overflow y ejes incorrectos
Cap. 9	<code>loss.backward()</code>	calcula gradientes	acumular gradientes sin limpiar
Cap. 11	<code>Pipeline</code>	evita leakage en transformaciones	ajustar pasos fuera del pipeline
Cap. 12	<code>GridSearchCV</code>	selecciona hiperparámetros	buscar sobre pipeline incompleto
Cap. 15	<code>np.linalg.svd</code>	factoriza matriz centrada	olvidar centrar antes
Cap. 16	<code>KMeans</code>	ajusta centroides	no escalar o no fijar semilla
Cap. 17	<code>cosine_similarity</code>	mide similitud angular	ignorar datos faltantes y popularidad

Algoritmos principales

Algoritmo o método	Primer uso fuerte	Implementación o referencia	Revisión necesaria
Regresión lineal	Capítulo 1	ecuación normal, <code>np.linalg.solve</code>	dimensiones, intercepto, MSE y residual
Descenso del gradiente	Capítulo 2	actualización vectorizada en NumPy	escala de variables, pérdida y convergencia
Regresión polinomial	Capítulo 3	matriz de features y validación	overfitting, grado y partición
Ridge	Capítulo 3	ecuación normal regularizada, <code>Ridge</code>	no penalizar intercepto y validar <code>lambda</code>
Lasso	Capítulo 3	<code>Lasso</code>	<code>sparsity</code> , escalamiento y validación
Regresión logística	Capítulo 4	sigmoide, log-loss, <code>LogisticRegression</code>	umbral, calibración y matriz de confusión
Perceptrón/logística multicapa	Capítulo 6	forward propagation	formas de matrices y activaciones
Backpropagation	Capítulo 7	regla de la cadena vectorizada	gradientes, cache y prueba numérica

Algoritmo o método	Primer uso fuerte	Implementación o referencia	Revisión necesaria
Momentum/Adam	Capítulo 9	loop NumPy o PyTorch	learning rate, estabilidad y validación
PCA	Capítulo 15	centrado y <code>np.linalg.svd</code>	varianza explicada e interpretación
K-Means	Capítulo 16	KMeans	escalamiento, inercia, silhouette y estabilidad
Recomendación item-item	Capítulo 17	similitud coseno	sesgo de popularidad y cobertura

Datasets y generadores

Recurso	Uso en el curso	Advertencia
<code>regresion_lineal_sintetica.csv</code>	caso longitudinal de regresión, gradiente, regularización y comunicación	artificial; no representa una población real
<code>clasificacion_binaria_sintetica.csv</code>	caso longitudinal de clasificación, métricas, validación, error analysis y defensa	el slice reciente ilustra mismatch de forma controlada
<code>pca_sintetico.csv</code>	PCA/SVD, varianza explicada y reconstrucción	estructura latente generada; no afirmar causalidad
<code>clusters_sinteticos.csv</code>	K-Means, inercia, silhouette y estabilidad	clusters compactos por construcción
<code>recomendacion_interacciones.csv</code>	recomendación item-item y cobertura	datos ficticios; no inferir preferencias reales
<code>load_wine</code>	arranque, clasificación multiclase y pipeline básico	dataset limpio; no prueba despliegue real
<code>load_diabetes</code>	regresión, Ridge/Lasso y validación	objetivo clínico simplificado; requiere cautela aplicada
<code>load_breast_cancer</code>	clasificación binaria, métricas y umbrales	no usar como recomendación clínica real
<code>load_digits</code>	redes, PCA y clasificación multiclase	imágenes pequeñas 8x8; no representa visión moderna
<code>make_regression</code>	regresión generada y gradiente	útil para revisar código, no para concluir aplicación
<code>make_blobs</code>	clustering sintético	favorece clusters compactos
<code>make_moons</code>	fronteras no lineales	muestra límites geométricos de modelos lineales

Funciones NumPy frecuentes

Función	Uso típico	Riesgo frecuente
<code>np.asarray</code>	convertir entradas a arreglo numérico	ocultar copias o formas inesperadas
<code>np.ones</code> , <code>np.hstack</code> , <code>np.column_stack</code>	construir matriz de diseño	duplicar intercepto
<code>@</code>	producto matricial	confundir producto elemento a elemento
<code>np.mean</code> , <code>np.sum</code> <code>np.linalg.solve</code>	pérdidas, métricas y gradientes resolver sistemas lineales	olvidar eje o factor $1/n$ usar matrices mal condicionadas
<code>np.linalg.cond</code>	diagnosticar condicionamiento	calcularlo después de leakage o escalamiento incorrecto
<code>np.linalg.svd</code> <code>np.exp</code> , <code>np.log</code> , <code>np.clip</code> <code>np.random.default_rng</code>	PCA y bajo rango sigmoide y log-loss reproducibilidad	olvidar centrar datos overflow o logaritmo de cero no fijar semilla o mezclar generadores

Herramientas scikit-learn frecuentes

Objeto	Uso típico	Revisión necesaria
<code>train_test_split</code>	separar entrenamiento, validación o test	estratificación y semilla
<code>StandardScaler</code>	escalar variables numéricas	ajustar solo con entrenamiento
<code>Pipeline</code> o <code>make_pipeline</code>	evitar leakage en transformaciones	ordenar pasos y revisar qué se aprende
<code>DummyClassifier</code>	baseline de clasificación	comparar antes de celebrar métricas
<code>LinearRegression</code>	referencia profesional para regresión	misma matriz y mismo intercepto
<code>Ridge</code> , <code>Lasso</code>	regularización	escala, alpha y selección por validación
<code>LogisticRegression</code>	clasificación lineal	umbral, regularización y clases
<code>KMeans</code>	clustering por centroides	n_init , semilla y escalamiento
<code>silhouette_score</code>	evaluar separación de clusters	no absolutizar en geometrías no convexas
<code>cosine_similarity</code>	recomendación y similitud	normalización y datos faltantes

Métricas y diagnósticos

Métrica o diagnóstico	Dónde aparece	Pregunta que responde
MSE/MAE	regresión	¿qué tan grande es el error numérico?
matriz de confusión	clasificación	¿qué tipos de error se cometen?
precision	clasificación	¿cuántas predicciones positivas son correctas?
recall	clasificación	¿cuántos positivos reales se detectan?
F1	clasificación	¿cómo balancear precision y recall?
ROC-AUC	clasificación	¿cómo ordena scores el modelo?
learning curve	estrategia ML	¿hay sesgo, varianza o límite por datos?
validation curve	estrategia ML	¿cómo cambia el desempeño con un hiperparámetro?
inertia	K-Means	¿qué tan compactos son los clusters?
silhouette	clustering	¿qué tan separados están los clusters?
varianza explicada	PCA	¿cuánta estructura conserva la proyección?

PyTorch mínimo del curso

Elemento	Uso	Error frecuente
<code>torch.Tensor</code>	datos y parámetros en tensores	mezclar tipos o dispositivos sin control
<code>nn.Module</code>	definir modelo entrenable	esconder formas de entrada/salida
<code>loss_fn</code>	calcular pérdida	usar pérdida incompatible con logits o probabilidades
<code>loss.backward()</code>	calcular gradientes	olvidar limpiar gradientes antes
<code>optimizer.zero_grad()</code>	reiniciar acumulación de gradientes	acumular gradientes sin querer
<code>optimizer.step()</code>	actualizar parámetros	actualizar sin revisar escala de pérdida

Regla de lectura rápida

Cuando encuentres una función o clase en un notebook, pregunta:

1. ¿qué objeto matemático representa?
2. ¿qué aprende con `fit` o durante entrenamiento?
3. ¿qué datos puede ver sin producir leakage?
4. ¿qué métrica verifica que funciona?
5. ¿qué decisión se justifica y cuál no?

Una biblioteca profesional reduce trabajo mecánico, no reduce responsabilidad técnica. El estudiante sigue siendo responsable de la pregunta, la partición, la métrica, el diagnóstico y la interpretación.

Taxonomía de ejercicios y banco de preguntas

Este apéndice organiza los ejercicios del libro por tipo de evidencia. No es un banco privado de parciales ni una colección de soluciones. Su función es ayudar a estudiantes e instructores a reconocer qué habilidad se está practicando y qué cuenta como una respuesta defendible.

Propósito

Los ejercicios del libro siguen una progresión:

1. entender vocabulario y objetos;
2. calcular cantidades pequeñas;
3. implementar procedimientos reproducibles;
4. interpretar resultados con límites;
5. conectar evidencia con una decisión técnica.

Una pregunta profesional no solo pide “obtener un número”. También debe dejar claro qué evidencia permite confiar en ese número y qué conclusión no se puede afirmar.

Tipos de ejercicio

Tipo	Pregunta que responde	Evidencia mínima
Conceptual	¿qué significa este objeto o supuesto?	explicación breve con vocabulario correcto
Cuantitativo	¿puedes calcularlo en un caso pequeño?	cálculo, fórmula, forma o métrica verificable
Computacional	¿puedes implementarlo sin romper la lógica?	función, notebook o salida reproducible
Interpretativo	¿qué decisión se justifica con la evidencia?	recomendación proporcional con limitación
Proyecto	¿cómo se aplica al caso propio?	decisión documentada, artefacto o miniinforme

Niveles cognitivos

El curso usa cinco niveles operativos de dificultad.

	Nivel	Acción observable	Ejemplo de verbo
	1	reconocer o definir	identificar, nombrar, distinguir
	2	calcular o verificar	calcular, derivar, comprobar, estimar
	3	implementar o reproducir	programar, ejecutar, comparar, graficar
	4	diagnosticar o evaluar	justificar, seleccionar, detectar, criticar
	5	diseñar o defender	proponer, integrar, defender, comunicar

Un buen capítulo debe ofrecer ejercicios en varios niveles. Un parcial o laboratorio puede concentrarse en algunos niveles, pero debe declarar qué evidencia espera.

Evidencia esperada

Evidencia	Debe incluir	No basta con
Cálculo matemático	fórmula, sustitución, resultado y unidad o interpretación	solo resultado final
Derivación	hipótesis, pasos algebraicos y forma final	saltar de fórmula inicial a final
Código	función ejecutable, shapes esperados y caso de prueba	celda que funciona solo para un dataset
Métrica	definición, valor, partición usada y baseline	reportar accuracy aislada
Diagnóstico	patrón observado, posible causa y acción siguiente	decir “overfitting” sin evidencia
Recomendación	decisión, métrica, límite y riesgo	afirmar que el modelo “es bueno”

Banco público por módulo

Este banco resume familias de preguntas públicas. Los ejercicios concretos están en cada capítulo; esta tabla ayuda a calibrar estudio y evaluación.

Módulo	Familia de preguntas	Nivel típico	Capítulos
Fundamentos	identificar observación, variable, variable aleatoria, target, ruido, baseline y métrica	1-3	Capítulo 0 e Interludio 0.5
Regresión	construir matriz de diseño, derivar MSE, resolver ecuación normal	2-4	Capítulos 1-2
Regularización	comparar Ridge/Lasso, validar hiperparámetro y detectar leakage	3-4	Capítulo 3
Clasificación	calcular matriz de confusión, precision, recall, F1 y umbral	2-4	Capítulo 4
Redes	verificar formas, forward, backward, gradientes y entrenamiento	2-4	Capítulos 6-9
Estrategia ML	diseñar particiones, curvas, error analysis y diagnóstico por slice	3-5	Capítulos 11-13
No supervisado	centrar datos, aplicar PCA, ejecutar K-Means e interpretar clusters	2-4	Capítulos 15-17
Reproducibilidad	construir manifest, trazabilidad, reporte y defensa técnica	3-5	Capítulos 20-22

Criterios de calibración

Una pregunta es demasiado fácil si puede responderse copiando una definición sin aplicarla. Es demasiado difícil si requiere información no enseñada, datos no disponibles o criterios ocultos.

Criterio	Pregunta de control
Alineación	¿la pregunta mide un objetivo del capítulo?
Verificabilidad	¿hay una respuesta, cálculo o criterio observable?
Independencia	¿puede resolverse sin material privado?
Transferencia	¿obliga a usar la idea en un caso nuevo?

Criterio	Pregunta de control
Interpretación	¿pide límites, riesgo o decisión cuando corresponde?
Automatización	¿qué parte puede calificarse con pruebas o reglas claras?

Uso para estudiantes

Antes de entregar un laboratorio o preparar un parcial, identifica qué tipo de pregunta estás resolviendo:

- si es conceptual, prioriza definiciones y contraejemplos;
- si es cuantitativa, revisa formas, fórmulas y unidades;
- si es computacional, prueba con casos pequeños antes del dataset final;
- si es interpretativa, compara contra baseline y declara límites;
- si es de proyecto, conecta la respuesta con la decisión aplicada.

Uso para instructores

Al diseñar o revisar ejercicios, mantener esta mezcla mínima:

Evaluación	Mezcla recomendada
Quiz corto	conceptual y cuantitativo
Laboratorio	computacional, cuantitativo e interpretativo
Parcial autograded	cuantitativo, computacional y diagnóstico estructurado
Proyecto	interpretativo, reproducible y defensa de decisión

La calificación manual debe reservarse para interpretación, decisión y comunicación. Shapes, métricas, outputs numéricos y funciones pequeñas deben preferirse para autocalificación cuando sea razonable.

Señales de mala pregunta

Una pregunta debe revisarse si:

- premia memorizar una frase sin usarla;
- no indica qué partición de datos usar;
- mezcla entrenamiento y test;
- pide una recomendación sin baseline;
- usa una métrica sin explicar el costo de error;

- requiere una solución oficial o autograder privado;
- tiene ambigüedad matemática que cambia la respuesta;
- no permite distinguir error de código de error conceptual.

Mantenimiento y revisión

Esta taxonomía debe actualizarse cuando:

- cambie la política de evaluación;
- se agreguen laboratorios o parciales;
- se cambie el peso de interpretación manual;
- una revisión externa detecte desalineación entre objetivos y ejercicios;
- un autograder muestre hardcoding o patrones de error no previstos.

Prerrequisitos matemáticos mínimos

Este apéndice resume el lenguaje matemático que el libro usa de forma recurrente. No reemplaza un curso de álgebra lineal, cálculo o probabilidad; es una referencia rápida para leer capítulos, notebooks y ejercicios sin perder la forma de los objetos.

Cómo usar este apéndice

Úsalo cuando aparezca una duda de notación antes de resolver un ejercicio:

1. identifica el objeto: escalar, vector, matriz, función o variable aleatoria;
2. revisa la forma esperada;
3. conecta la fórmula con la operación de NumPy o PyTorch;
4. vuelve al capítulo y verifica si la interpretación tiene sentido.

La pregunta central no es “¿recuerdo la fórmula?”, sino “¿sé qué objeto calcula, con qué forma, sobre qué datos y para qué decisión?”.

Álgebra lineal mínima

Objeto	Notación	Forma típica	Lectura
vector de variables	x	p o $p \times 1$	una observación con p entradas
matriz de diseño	X	$n \times p$	n observaciones y p variables
parámetros	θ o w	p o $p \times 1$	pesos que convierten entradas en predicción
predicción	$\hat{y} = X\theta$	n	una salida por observación
residual	$r = y - \hat{y}$	n	error observado
matriz de pesos	$W^{[\ell]}$	$d_{\ell-1} \times d_\ell$	conecta capas de una red

Producto matricial:

$$(X\theta)_i = \sum_{j=1}^p X_{ij}\theta_j.$$

En NumPy:

```
y_hat = X @ theta
```

Chequeo básico:

Operación	Requisito
<code>X @ theta</code>	columnas de <code>X</code> = largo de <code>theta</code>
<code>X.T @ residual</code>	largo de <code>residual</code> = filas de <code>X</code>
<code>A @ W + b</code>	columnas de <code>A</code> = filas de <code>W</code> ; <code>b</code> broadcast por columnas

Normas, distancias y pérdida cuadrática

La norma euclidiana de un vector es:

$$\|r\|_2 = \sqrt{\sum_i r_i^2}.$$

El MSE usa errores cuadrados:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

En regresión, minimizar MSE equivale a buscar predicciones cercanas a y en promedio cuadrático. En optimización usamos a menudo:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

porque el factor $1/2$ simplifica el gradiente.

Cálculo y gradientes mínimos

Una derivada mide sensibilidad local. Un gradiente reúne derivadas parciales:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

Regla práctica del curso:

Si quieres	Usas
reducir una pérdida	moverte en dirección $-\nabla J$
detectar convergencia	norma del gradiente, cambio de pérdida o cambio de parámetros
entrenar una red	regla de la cadena organizada como backpropagation

Para MSE:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2, \quad \nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Probabilidad mínima

Antes de observar un caso, Y representa una cantidad incierta. Después de observarlo, y_i es una realización.

Idea	Lectura operativa
distribución empírica	lo que se observa en el dataset disponible
valor esperado	promedio ideal bajo un proceso de generación
ruido	variación que el modelo no explica con las variables disponibles
probabilidad condicional	$P(Y = 1 \mid X = x)$: probabilidad dada una entrada
pérdida empírica	error promedio observado en un conjunto finito
riesgo esperado	error promedio ideal en casos futuros

El curso usa esta probabilidad mínima para interpretar validación, logits, probabilidades, umbrales, métricas y generalización.

Notación NumPy y PyTorch

Matemática	NumPy	PyTorch	Cuidado
$X\theta$	<code>X @ theta</code>	<code>X @ theta</code> o <code>model(X)</code>	revisar formas
media	<code>np.mean(x)</code>	<code>torch.mean(x)</code>	eje correcto
exponencial	<code>np.exp(z)</code>	<code>torch.exp(z)</code>	overflow
logaritmo	<code>np.log(p)</code>	<code>torch.log(p)</code>	evitar <code>log(0)</code>
gradiente	cálculo manual	<code>loss.backward()</code>	limpiar gradientes
actualización	<code>theta -= alpha * grad</code>	<code>optimizer.step()</code>	no olvidar <code>zero_grad()</code>

PyTorch acumula gradientes por defecto. Por eso el loop mínimo contiene:

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Checklist de formas

Antes de ejecutar un notebook, verifica:

- ¿cada fila representa una observación?
- ¿la variable objetivo tiene una entrada por observación?
- ¿la matriz de diseño incluye o no intercepto?
- ¿los pesos tienen una dimensión compatible con las entradas?
- ¿la pérdida produce un escalar?
- ¿la métrica se calcula sobre la partición correcta?
- ¿las transformaciones se ajustan solo con entrenamiento?

Ejemplo resuelto: gradiente de MSE en una línea

Sea:

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

El residual es:

$$X\theta - y = \begin{bmatrix} -1 \\ -3 \end{bmatrix}.$$

Como $n = 2$,

$$\nabla J(\theta) = \frac{1}{2} X^\top (X\theta - y) = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -3 \end{bmatrix} = \begin{bmatrix} -2 \\ -1.5 \end{bmatrix}.$$

La actualización con $\alpha = 0.1$ sería:

$$\theta_{\text{nuevo}} = \theta - \alpha \nabla J(\theta) = \begin{bmatrix} 0.2 \\ 0.15 \end{bmatrix}.$$

Respuestas rápidas de autoverificación

1. Si X tiene forma 100×8 y θ tiene largo 8, entonces $X\theta$ tiene largo 100.
2. Si una red produce logits de forma 64×10 , softmax se aplica por fila para obtener una distribución por observación.
3. Si `loss.backward()` se llama dos veces sin `zero_grad()`, los gradientes se acumulan.
4. Si test se usa para escoger hiperparámetros, test deja de ser estimación final.

Formularios y respuestas seleccionadas

Los apéndices reúnen fórmulas, criterios operativos, patrones de código y respuestas seleccionadas de cada bloque. No reemplazan el estudio de los capítulos: sirven como referencia rápida y como verificación después de resolver ejercicios.

Las respuestas seleccionadas son deliberadamente parciales. El objetivo es orientar, no convertir el libro en un solucionario completo. Las soluciones oficiales de laboratorios, parciales y pruebas ocultas pertenecen al paquete restringido de instructor.

Cómo usar esta parte

1. Intenta resolver primero el ejercicio sin mirar la respuesta.
2. Revisa la fórmula o checklist correspondiente.
3. Compara tu resultado con la respuesta seleccionada cuando exista.
4. Si tu respuesta difiere, identifica si el error es de forma, fórmula, cálculo, código o interpretación.
5. Vuelve al capítulo si no puedes explicar por qué la respuesta es correcta.

Mapa de apéndices

Apéndice	Uso principal
Módulo 1	Regresión, gradiente, regularización, logística y Parcial 1.
Módulo 2	Redes densas, backpropagation, diagnóstico y lectura de PyTorch.
Módulo 3	Métricas, particiones, validación, leakage y análisis de errores.
Módulo 4	PCA/SVD, K-Means, recomendación y límites de interpretación.
Módulo 5	Parcial 2, reproducibilidad, reporte técnico y defensa final.

Política de respuestas

Las respuestas públicas cubren una muestra representativa: algunas verifican cálculo, otras verifican razonamiento y otras muestran el nivel esperado de una explicación técnica breve. Los proyectos de cierre no tienen solución única; por eso se evalúan con criterios, evidencia y límites explícitos.

La política completa de separación entre respuestas públicas, manual de estudiante y recursos restringidos está en [Respuestas, solucionarios y recursos restringidos](#).

Apéndice Módulo 1: formulario y respuestas seleccionadas

Resumen de fórmulas, chequeos y verificación

Cómo usar este apéndice

Este apéndice resume las fórmulas y chequeos del primer bloque matemático del curso: regresión, optimización, regularización y clasificación binaria. Úsalo para verificar derivaciones, notebooks y preparación del Parcial 1.

No memorices fórmulas aisladas. Para cada objeto, verifica forma, significado, hipótesis y el lugar que ocupa en el pipeline de modelación.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de diseño
$y \in \mathbb{R}^n$	variable respuesta o etiqueta binaria
$\theta \in \mathbb{R}^p$	vector de parámetros
$\hat{y}$	predicción de regresión
$\hat{p}$	probabilidad predicha en clasificación binaria
λ	intensidad de regularización
τ	umbral de clasificación
TP, TN, FP, FN	conteos de matriz de confusión

Formulario mínimo

Regresión lineal

Predicción:

$$\hat{y} = X\theta.$$

Residual:

$$r = y - X\theta.$$

MSE:

$$\text{MSE}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2.$$

Pérdida escalada:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2.$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Ecuación normal:

$$X^\top X\theta = X^\top y.$$

Solución si $X^\top X$ es invertible:

$$\theta^* = (X^\top X)^{-1} X^\top y.$$

Descenso del gradiente

Actualización:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}).$$

Para MSE:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{1}{n} X^\top (X\theta^{(t)} - y).$$

Estandarización:

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

Ridge

Objetivo:

$$J_{\text{ridge}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \frac{\lambda}{2} \|\theta_{1:p}\|_2^2.$$

Gradiente:

$$\nabla J_{\text{ridge}}(\theta) = \frac{1}{n} X^\top (X\theta - y) + \lambda \tilde{\theta}.$$

Solución sin penalizar intercepto:

$$\theta^* = (X^\top X + n\lambda D)^{-1} X^\top y,$$

donde $D_{00}=0$ y $D_{jj}=1$ para $j>0$.

Lasso

Objetivo:

$$J_{\text{lasso}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \lambda \|\theta_{1:p}\|_1.$$

Regresión logística

Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Probabilidad:

$$\hat{p} = \sigma(X\theta).$$

Log-loss:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (\hat{p} - y).$$

Clasificación por umbral:

$$\hat{y}_i = 1\{\hat{p}_i \geq \tau\}.$$

Métricas

Accuracy:

$$\frac{TP + TN}{TP + TN + FP + FN}.$$

Precision:

$$\frac{TP}{TP + FP}.$$

Recall:

$$\frac{TP}{TP + FN}.$$

F1:

$$2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Patrones de código

```
def add_intercept(X):  
    X = np.asarray(X, dtype=float)  
    return np.hstack([np.ones((X.shape[0], 1)), X])
```

```
def mse_loss(X, y, theta):  
    residual = X @ theta - y  
    return np.mean(residual ** 2)
```

```
def mse_gradient(X, y, theta):  
    return (X.T @ (X @ theta - y)) / X.shape[0]
```

```
def ridge_closed_form(X, y, lam):  
    p = X.shape[1]  
    D = np.eye(p)  
    D[0, 0] = 0  
    return np.linalg.solve(X.T @ X + X.shape[0] * lam * D, X.T @ y)
```

```
def sigmoid(z):
    z = np.asarray(z, dtype=float)
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1 / (1 + np.exp(-z[pos]))
    exp_z = np.exp(z[~pos])
    out[~pos] = exp_z / (1 + exp_z)
    return out
```

Checklist antes de entregar

- El notebook corre desde cero.
- Las funciones no dependen de variables globales ocultas.
- Las formas de matrices y vectores son consistentes.
- El preprocesamiento se ajusta solo con entrenamiento.
- Hay baseline.
- Hay métrica adecuada.
- Hay interpretación breve.
- La conclusión menciona al menos una limitación.

Respuestas seleccionadas

Estas respuestas no reemplazan el desarrollo. Sirven para verificar dirección.

Capítulo 1

Ejercicio 1.1. Si X tiene forma $(120, 4)$, entonces `theta` debe tener forma $(4,)$ o $(4, 1)$ para que `X @ theta` produzca 120 predicciones.

Ejercicio 1.2. La columna de unos convierte el intercepto en un coeficiente más: `theta_0 * 1`.

Ejercicio 1.6. Para

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 2 \\ 2 \\ 5 \end{bmatrix},$$

se obtiene:

$$X^T X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 9 \\ 12 \end{bmatrix}.$$

Capítulo 2

Ejercicio 2.2. Usamos `theta - alpha * grad` porque el gradiente apunta hacia mayor aumento de la pérdida.

Ejercicio 2.4. Escalar ayuda porque reduce diferencias extremas de curvatura entre direcciones. El descenso hace pasos más directos y estables.

Ejercicio 2.8. Una curva que sube y baja violentamente sugiere una tasa de aprendizaje demasiado grande o un problema de escala.

Capítulo 3

Ejercicio 3.2. No se escoge `lambda` con prueba porque entonces prueba deja de ser una evaluación final independiente.

Ejercicio 3.8. Para `theta=(3,4,-1)`, sin intercepto:

$$\|\theta_{1:p}\|_2^2 = 4^2 + (-1)^2 = 17, \quad \|\theta_{1:p}\|_1 = 4 + 1 = 5.$$

Ejercicio 3.16. Preferiría `lambda=0.1` si la meta es generalizar: aumenta el error de entrenamiento, pero reduce mucho el error de validación.

Capítulo 4

Ejercicio 4.6.

$$\sigma(0) = 0.5, \quad \sigma(2) \approx 0.881, \quad \sigma(-2) \approx 0.119.$$

Ejercicio 4.7. Si TP=40, FP=10, TN=80, FN=20:

$$\text{accuracy} = 0.80, \quad \text{precision} = 0.80, \quad \text{recall} = 0.667.$$

F1 es aproximadamente 0.727.

Ejercicio 4.16. Accuracy alta y recall bajo puede indicar clases desbalanceadas: el modelo acierta muchos negativos, pero falla positivos.

Tema	Ejercicios recomendados
------	-------------------------

Guía de estudio para el Parcial 1

Prioriza estos ejercicios si tienes poco tiempo:

Tema	Ejercicios recomendados
Regresión lineal	Cap. 1: 6, 7, 10, 14
Gradiente	Cap. 2: 6, 9, 13, 14
Validación	Cap. 3: 2, 6, 10, 16
Regularización	Cap. 3: 7, 8, 11, 13
Logística	Cap. 4: 6, 7, 11, 12
Umbrales	Cap. 4: 14, 15, 16

Para cerrar, escribe una página de resumen con:

1. una fórmula que todavía te cuesta;
2. un error común que ya sabes evitar;
3. una métrica que usarías en tu proyecto;
4. una pregunta concreta para clase o monitoría.

Apéndice Módulo 2: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice es una referencia para el Módulo 2. Úsalo después de intentar los ejercicios del capítulo o antes de revisar un notebook de redes. La meta no es memorizar símbolos, sino poder auditar cada objeto: forma, significado, operación que lo produjo y riesgo de implementación.

En redes neuronales, una respuesta puede tener código correcto y aun así estar mal argumentada si no explica formas, pérdida, diagnóstico o decisión de entrenamiento.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de entrada: n observaciones, p variables
$Y \in \mathbb{R}^{n \times k}$	etiquetas one-hot para k clases
$W^{[\ell]}, b^{[\ell]}$	pesos y sesgos de la capa ℓ
$Z^{[\ell]}$	preactivación de la capa ℓ
$A^{[\ell]}$	activación de la capa ℓ
$\hat{P}$	probabilidades predichas por softmax
J	pérdida promedio

Fórmulas clave de forward

Forward

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = \text{ReLU}(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$$

$$\hat{P} = \text{softmax}(Z^{[2]})$$

Si $X \in \mathbb{R}^{n \times p}$, una capa oculta con h neuronas y k clases usa las formas:

Objeto	Forma
$W^{[1]}$	$p \times h$
$b^{[1]}$	$h \text{ o } 1 \times h$
$Z^{[1]}, A^{[1]}$	$n \times h$
$W^{[2]}$	$h \times k$
$b^{[2]}$	$k \text{ o } 1 \times k$
$Z^{[2]}, \hat{P}, Y$	$n \times k$

Softmax

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_\ell}}$$

Versión estable:

$$\text{softmax}(z)_j = \frac{e^{z_j - \max(z)}}{\sum_{\ell=1}^k e^{z_\ell - \max(z)}}.$$

Cross-entropy

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k Y_{ij} \log(\hat{P}_{ij})$$

Fórmulas clave de backward

$$dZ^{[2]} = \frac{1}{n}(\hat{P} - Y)$$

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]}, \quad db^{[2]} = \sum_i dZ_i^{[2]}$$

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^\top$$

$$dZ^{[1]} = dA^{[1]} \odot \mathbf{1}\{Z^{[1]} > 0\}$$

$$dW^{[1]} = X^\top dZ^{[1]}, \quad db^{[1]} = \sum_i dZ_i^{[1]}$$

Diagnóstico de entrenamiento

Evidencia	Diagnóstico probable	Acción inicial
Train mejora y validation empeora	overfitting	early stopping, L2, dropout, menor capacidad
Train y validation son malos	underfitting o mala señal	revisar features, arquitectura, learning rate
La pérdida oscila o explota	learning rate alto o escala inestable	bajar learning rate, revisar inicialización
Probabilidades casi uniformes	entrenamiento insuficiente o logits pequeños	revisar inicialización, gradientes y épocas
Gradientes con forma correcta pero norma cero	saturación o camino bloqueado	revisar activación, inicialización y datos

Patrones de código

```
def softmax_stable(Z):
    Z = np.asarray(Z, dtype=float)
    shifted = Z - np.max(Z, axis=1, keepdims=True)
    exp_scores = np.exp(shifted)
    return exp_scores / exp_scores.sum(axis=1, keepdims=True)
```

```
def cross_entropy(P, y):
    eps = 1e-12
    P = np.clip(P, eps, 1 - eps)
    return -np.mean(np.log(P[np.arange(len(y)), y]))
```

```
def relu_backward(dA, Z):
    return dA * (Z > 0)
```

Checklist antes de entregar

- Cada matriz tiene la forma esperada.
- Softmax se aplica por fila.
- La pérdida usa probabilidades recortadas o softmax estable.
- dZ_2 tiene la misma forma que Y .
- Cada gradiente tiene la misma forma que el parámetro que actualiza.
- Los gradientes se dividen por n exactamente una vez.
- El diagnóstico usa curva de entrenamiento y validación.
- La comparación con PyTorch usa misma partición, semilla y métrica.

Respuestas seleccionadas

Estas respuestas verifican dirección. No sustituyen el desarrollo completo ni las pruebas del laboratorio.

Verificaciones de ejemplos resueltos

Softmax estable. Para $z = (2, 1, 0)$, restar el máximo produce $(0, -1, -2)$. La distribución aproximada es $(0.665, 0.245, 0.090)$. La pérdida de una observación se calcula con la probabilidad de la clase correcta, no con la probabilidad más alta en abstracto.

Diferencia finita. Para $J(w) = (w - 3)^2$, $w = 2$ y $\varepsilon = 0.01$, la diferencia central da -2 , igual a la derivada exacta $2(w - 3)$.

Regularización L2. Si $\|W\|_F^2 = 14$ y $\lambda = 0.1$, la penalización $\frac{\lambda}{2}\|W\|_F^2$ vale 0.7. El gradiente suma λW al gradiente de datos.

Momentum. Con $\theta_0 = 5$, $\alpha = 0.1$, $\beta = 0.9$, $v_0 = 0$, $g_1 = 4$ y $g_2 = 2$, se obtiene $v_1 = 0.4$, $\theta_1 = 4.96$, $v_2 = 0.56$ y $\theta_2 = 4.904$.

Capítulo 1

Ejercicio 1.1. Sin activaciones no lineales, una composición de transformaciones lineales sigue siendo una transformación lineal. Por ejemplo:

$$(XW_1)W_2 = X(W_1W_2).$$

La red puede reescribirse como un solo modelo lineal.

Ejercicio 1.3. Si X tiene forma $(250, 20)$ y la capa oculta tiene 12 neuronas, entonces $W1$ debe tener forma $(20, 12)$, $b1$ forma $(12,)$ o $(1, 12)$, y $Z1$ forma $(250, 12)$.

Ejercicio 1.4. Para 5 clases y batch de 32, logits, probabilidades softmax y matriz one-hot tienen forma $(32, 5)$.

Capítulo 2

Ejercicio 2.1. Backpropagation necesita valores guardados del forward porque las derivadas locales dependen de objetos intermedios: por ejemplo, ReLU necesita conocer dónde $Z^{[1]} > 0$, y $dW^{[2]}$ necesita $A^{[1]}$.

Ejercicio 2.2. ReLU vale z cuando $z > 0$ y vale 0 cuando $z \leq 0$. Por tanto, su derivada es 1 en la región positiva y 0 en la región no positiva. En código, $(Z1 > 0)$ actúa como máscara.

Ejercicio 2.4. Si $A^{[1]} \in \mathbb{R}^{80 \times 16}$ y $dZ^{[2]} \in \mathbb{R}^{80 \times 4}$, entonces

$$dW^{[2]} = (A^{[1]})^\top dZ^{[2]} \in \mathbb{R}^{16 \times 4}, \quad db^{[2]} \in \mathbb{R}^4.$$

Capítulo 3

Ejercicio 3.1. Train accuracy 99 % y validation accuracy 70 % sugiere overfitting. El modelo aprendió patrones específicos del entrenamiento que no generalizan. Antes de reportarlo como mejora habría que revisar regularización, capacidad, partición de datos y análisis de errores.

Ejercicio 3.4. Para $w = (2, -1, 0.5)$ y $\lambda = 0.1$,

$$\lambda \sum_j w_j^2 = 0.1(4 + 1 + 0.25) = 0.525.$$

Ejercicio 3.8. Si una métrica global mejora pero un subconjunto crítico empeora, no basta con reportar la mejora. La decisión profesional es aislar el slice, verificar tamaño y costo del error, y proponer una acción antes de usar el modelo.

Capítulo 4

Ejercicio 4.2. PyTorch acumula gradientes porque permite sumar contribuciones de varias pérdidas o batches antes de actualizar. En entrenamiento estándar se llama `zero_grad()` para que el batch actual no se mezcle con gradientes del batch anterior.

Ejercicio 4.3. Con 1024 observaciones y batch size 64, una época tiene $1024/64 = 16$ batches.

Ejercicio 4.4. Si se entrenan 20 épocas con 16 batches por época, `optimizer.step()` se ejecuta $20 \times 16 = 320$ veces.

Ejercicios integradores

Bloque A. Para auditar dimensiones, no empieces por el código. Escribe primero las formas esperadas de X , $W^{[1]}$, $b^{[1]}$, $Z^{[1]}$, $A^{[1]}$, $W^{[2]}$, $b^{[2]}$ y $\hat{P}$. Luego compara cada objeto del notebook contra esa tabla.

Bloque C. Una buena respuesta de diagnóstico tiene tres partes: evidencia observada, causa probable y acción. Ejemplo: “validation loss sube después de la época 15 mientras train loss sigue bajando; esto sugiere overfitting; probaría early stopping y mayor regularización”.

Apéndice Módulo 3: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice resume las reglas que evitan conclusiones engañosas en machine learning aplicado: métrica mal escogida, particiones contaminadas, validación mal interpretada y análisis de error insuficiente. Úsalo como checklist antes de comparar modelos o escribir una recomendación técnica.

Una métrica no es una conclusión. La conclusión aparece cuando conectas métrica, baseline, partición, error principal, costo de error y límite de uso.

Notación mínima

Símbolo	Significado
TP, TN, FP, FN	conteos de la matriz de confusión
s_k	métrica obtenida en el fold k
$\bar{s}$	media de validación cruzada
σ_s	variabilidad entre folds
dev	conjunto de validación para elegir modelos
test	conjunto final para estimar desempeño una vez
slice	subconjunto relevante para análisis de error

Métricas de clasificación binaria

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Validación cruzada

$$\bar{s} = \frac{1}{K} \sum_{k=1}^K s_k, \quad \sigma_s = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (s_k - \bar{s})^2}.$$

Reporta media y variabilidad. Si dos modelos tienen medias similares, la variabilidad puede cambiar la decisión.

Reglas operativas

1. El test se toca una vez.
2. La métrica principal se decide antes de comparar modelos.
3. Las transformaciones se ajustan dentro de cada fold o partición de entrenamiento.
4. Todo modelo se compara contra baseline.
5. Un promedio no reemplaza análisis de slices.

Checklist de evaluación

- La métrica principal está alineada con el costo de error.
- Hay baseline trivial o operativo.
- La partición respeta tiempo, grupos o estratificación cuando aplica.
- El preprocesamiento está dentro de un **Pipeline**.
- La validación no usa test para elegir hiperparámetros.
- El reporte incluye variabilidad o intervalo cuando sea razonable.
- Hay análisis de al menos un slice crítico.
- La recomendación incluye un límite explícito.

Patrones de decisión

Situación	Señal	Decisión prudente
Clase positiva rara	accuracy alta con recall bajo	usar recall, F1, PR-AUC o costo explícito
Métricas train muy superiores a dev	brecha alta	diagnosticar overfitting o leakage
Dev estable y test cae	fuentes/población distinta	investigar data mismatch
Mejor media pero alta variabilidad	folds inconsistentes	preferir modelo más estable si el riesgo importa
Slice crítico empeora	promedio oculta daño	no recomendar despliegue sin mitigación

Patrones de código

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
def metric_by_group(df, group_col, y_col, yhat_col, metric_fn):
    rows = []
    for group, part in df.groupby(group_col):
        rows.append({
            "group": group,
            "n": len(part),
            "score": metric_fn(part[y_col], part[yhat_col])
        })
    return pd.DataFrame(rows)
```

Respuestas seleccionadas

Verificaciones de ejemplos resueltos

Costo de errores. Con umbral A, $FP = 40$, $FN = 15$, costo de FP igual a 1 y costo de FN igual a 10, el costo es $40 + 150 = 190$. Con umbral B, el costo es $10 + 250 = 260$. Si el falso negativo es mucho más costoso, A puede ser preferible aunque tenga menor precisión.

Partición estratificada. Si hay 2000 observaciones y 8% de positivos, hay 160 positivos. En una división 60/20/20 se esperan 96 positivos en train, 32 en dev y 32 en test. Esta cuenta solo verifica proporción de clase; no valida tiempo, grupos ni duplicados.

Learning curve. Si F1 de validación sube de 0.63 a 0.86 al aumentar datos y la brecha train-validación baja de 0.35 a 0.04, la evidencia sugiere que más datos ayuda a reducir varianza. No es una señal para aumentar complejidad como primera acción.

Data mismatch. Si train y dev tienen cerca de 20% de positivos, pero test tiene 8% y recall cae de 0.82 a 0.55, antes de ajustar hiperparámetros hay que auditar fuente, periodo, población y construcción del split.

Brecha por slice. Si el F1 global es 0.81 y en fuente C es 0.57, la brecha es $0.81 - 0.57 = 0.24$. La recomendación debe explicar evidencia, diagnóstico, acción y límite; no basta reportar el promedio global.

Capítulo 1

Ejercicio 1.1. En fraude, preferir recall es razonable cuando perder un fraude es mucho más costoso que revisar falsos positivos. Preferir precision es razonable cuando cada alerta tiene alto costo operativo o afecta injustamente a usuarios legítimos.

Ejercicio 1.4. Con 2000 observaciones y clase positiva de 8 %, hay 160 positivos. En una partición 60/20/20 estratificada se esperan aproximadamente 96 positivos en train, 32 en dev y 32 en test.

Ejercicio 1.7. Imputar la media usando todo el dataset antes de partir es leakage porque la media incorpora información de validación/test. La media debe aprenderse solo con entrenamiento, idealmente dentro de un **Pipeline**.

Capítulo 2

Ejercicio 2.1. Train F1 0.99 y validation F1 0.70 sugiere overfitting. Acciones razonables: aumentar regularización, simplificar el modelo, revisar leakage, recolectar más datos o cambiar features.

Ejercicio 2.2. La desviación estándar de CV importa porque una media alta con mucha variabilidad puede indicar que el modelo depende demasiado de la partición. Si la aplicación exige estabilidad, un modelo con media apenas menor pero menor variabilidad puede ser preferible.

Ejercicio 2.4. Si se evalúan 4 valores de **C**, 3 valores de **penalty** y 5 folds, se entrenan $4 \times 3 \times 5 = 60$ ajustes.

Capítulo 3

Ejercicio 3.2. Un F1 global puede ocultar una falla grave si el modelo funciona bien en grupos numerosos y mal en un grupo pequeño pero importante. Por eso se reportan métricas por slice.

Ejercicio 3.3. Si una tabla por slice muestra F1 global 0.81 y un slice con F1 0.57, la brecha es 0.24. Si ese slice corresponde a una fuente, grupo o periodo relevante para producción, debe tratarse como riesgo técnico aunque el promedio global parezca aceptable.

Ejercicio 3.4. Si dev tiene 30 % de clase positiva y test 12 %, antes de ajustar hiperparámetros sospecharía data mismatch, cambio temporal, sesgo de muestreo o partición no representativa.

Ejercicio 3.7. Data mismatch puede sospecharse si el desempeño en dev es estable pero cae mucho en test, especialmente si test proviene de otra fuente, periodo o población.

Ejercicios integradores

Bloque A. Una respuesta profesional sobre métricas debe nombrar el costo del error. Por ejemplo, no basta decir “F1 es mejor”; hay que explicar si el problema castiga más falsos positivos, falsos negativos o ambos.

Bloque D. La estructura mínima de recomendación técnica es: evidencia observada, diagnóstico probable, acción concreta y límite. Si falta una de esas cuatro partes, la recomendación es difícil de auditar.

Apéndice Módulo 4: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice reúne fórmulas y criterios para aprendizaje no supervisado. En este módulo, el cálculo correcto no basta: también debes decir qué conclusión permite el método y qué conclusión sería una sobreinterpretación.

En aprendizaje no supervisado no hay una etiqueta que confirme automáticamente que la estructura encontrada es útil. La evidencia debe combinar cálculo, visualización, estabilidad, interpretación y límite aplicado.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de datos
X_c	matriz centrada por columnas
$U\Sigma V^\top$	descomposición SVD
V_k	primeros k vectores derechos
$Z = X_c V_k$	scores o coordenadas PCA
μ_j	centroide del cluster j
c_i	asignación de cluster de la observación i
R	matriz usuario-ítem

SVD

$$X = U\Sigma V^\top$$

Si $X \in \mathbb{R}^{n \times p}$ y $r = \text{rank}(X)$, entonces:

Objeto	Forma
U reducida	$n \times r$
Σ reducida	$r \times r$
V reducida	$p \times r$
V_k	$p \times k$
$Z = X_c V_k$	$n \times k$

PCA

$$X_c = X - \bar{X}$$

$$Z = X_c V_k$$

$$\hat{X} = Z V_k^\top + \bar{X}$$

Varianza explicada por el componente j :

$$\frac{\sigma_j^2}{\sum_\ell \sigma_\ell^2}.$$

Varianza explicada acumulada con k componentes:

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_\ell \sigma_\ell^2}.$$

K-Means

$$\min_{\mu, c} \sum_{i=1}^n \|x_i - \mu_{c_i}\|_2^2$$

Silhouette

$$s = \frac{b - a}{\max(a, b)}$$

Cosine similarity

$$\cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}$$

Checklist de interpretación

- Los datos se centraron o escalaron según el método.
- La forma de la proyección PCA es consistente.
- La elección de k tiene evidencia: varianza, reconstrucción, curva o uso.
- K-Means se ejecutó con variables comparables en escala.
- Inertia no se interpreta como métrica absoluta de calidad.
- Silhouette se usa como señal, no como verdad final.
- Los clusters se describen con variables originales.
- El recomendador declara cold start, popularidad o sesgo cuando aplica.

Patrones de código

```
def pca_scores_from_svd(X, k):
    X = np.asarray(X, dtype=float)
    mean = X.mean(axis=0, keepdims=True)
    Xc = X - mean
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    Z = Xc @ Vt[:k].T
    return Z, Vt[:k], mean, S
```

```
def cosine_similarity_matrix(A):
    A = np.asarray(A, dtype=float)
    norms = np.linalg.norm(A, axis=1, keepdims=True)
    norms = np.maximum(norms, 1e-12)
    A_norm = A / norms
    return A_norm @ A_norm.T
```

Respuestas seleccionadas

Capítulo 1

Ejercicio 1.1. PCA centra porque busca direcciones de variación alrededor de la media. Sin centrar, el primer componente puede capturar desplazamiento de la nube respecto al origen, no estructura de variabilidad.

Ejercicio 1.3. Si X tiene forma (500, 64) y usamos $k=2$, la proyección PCA tiene forma (500, 2).

Ejercicio 1.4. Que los primeros 10 componentes expliquen 85 % de la varianza significa que, en el subespacio de 10 dimensiones, se conserva 85 % de la variabilidad cuadrática medida por PCA. No significa que esos componentes expliquen causalmente el fenómeno.

Verificación de proyección. Para

$$X = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \quad \bar{X} = (1, 1),$$

se obtiene:

$$X_c = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Con $v_1 = (1, -1)^\top / \sqrt{2}$, los scores son $(\sqrt{2}, -\sqrt{2})^\top$. La reconstrucción con ese único componente recupera exactamente X_c , porque la matriz centrada tiene rango 1.

Capítulo 2

Ejercicio 2.1. Inertia baja al aumentar k porque cada punto puede quedar más cerca de algún centroide. En el extremo $k = n$, cada punto puede ser su propio centroide y la inertia llega a cero.

Ejercicio 2.2. K-Means debe usar escalamiento cuando las variables tienen unidades distintas porque la distancia euclídea queda dominada por variables con mayor escala numérica.

Ejercicio 2.4. Si un cluster contiene 90 % de las observaciones, revisaría escalamiento, elección de k , inicializaciones, variables dominantes y si la estructura real quizá no es aproximadamente esférica.

Verificación de K-Means. Para puntos 1, 2, 8, 10 y centroides iniciales $\mu_1 = 1$, $\mu_2 = 10$, las asignaciones son $\{1, 2\}$ y $\{8, 10\}$. La inertia inicial es

$$0^2 + 1^2 + 2^2 + 0^2 = 5.$$

Los centroides actualizados son 1.5 y 9. Con ellos, la inertia baja a

$$0.5^2 + 0.5^2 + 1^2 + 1^2 = 2.5.$$

Esto verifica el objetivo geométrico, no la calidad aplicada de la segmentación.

Capítulo 3

Ejercicio 3.1. La similitud coseno puede ser útil cuando algunos ítems son más populares porque compara dirección de interacción, no solo magnitud. Aun así, no elimina por completo el sesgo de popularidad.

Ejercicio 3.2. El problema de usuario nuevo se conoce como cold start: sin historial, no hay vector de preferencias suficiente para comparar o inferir gustos.

Ejercicio 3.8. Para detectar si un recomendador amplifica popularidad, compararía la popularidad media de los ítems recomendados contra la popularidad del catálogo y contra un baseline simple de popularidad.

Verificación item-item. Si

$$A = (1, 1, 0, 0), \quad B = (1, 1, 0, 1), \quad C = (0, 1, 1, 0), \quad D = (0, 0, 1, 1),$$

entonces:

$$\cos(A, B) = \frac{2}{\sqrt{2}\sqrt{3}} \approx 0.816, \quad \cos(A, C) = 0.5, \quad \cos(A, D) = 0.$$

Por similitud con A , el primer candidato es B . Si B ya fue visto por el usuario, el siguiente candidato entre estos ítems sería C .

Ejercicios integradores

Bloque A. Si dos componentes explican poca varianza pero separan grupos en un gráfico, puedes afirmar que la proyección muestra una separación visual en esos datos. No puedes afirmar causalidad ni que la separación generalice sin más evidencia.

Bloque B. Una buena descripción de cluster debe volver a las variables originales. Nombrar clusters solo por la posición en el gráfico suele producir etiquetas frágiles.

Bloque C. Una recomendación item-item mínima debe excluir ítems ya vistos y declarar qué ocurre con usuarios o ítems sin historial.

Apéndice Módulo 5: formulario y respuestas seleccionadas

Cómo usar este apéndice

Este apéndice sirve para cerrar el curso: preparar el Parcial 2, revisar reproducibilidad, escribir el reporte final y defender una decisión técnica. La pregunta central ya no es solo “¿cuál fórmula uso?”, sino “¿qué evidencia puedo defender y qué límite debo declarar?”.

Una defensa profesional no exagera. Presenta evidencia suficiente, diagnóstico proporcional, acción concreta y límites claros.

Notación mínima

Símbolo	Significado
$\hat{p}_{i,y_i}$	probabilidad asignada a la clase correcta
$\bar{s}, \sigma_s$	media y variabilidad de validación
Z_k	representación PCA con k componentes
$\hat{X}$	reconstrucción aproximada
a_i, b_i	distancia intra-cluster y vecino más cercano en silhouette
manifest	archivo que registra datos, parámetros, métricas y artefactos

Fórmulas clave

Softmax

$$\text{softmax}(z)_j = \frac{e^{z_j - \max(z)}}{\sum_{k=1}^K e^{z_k - \max(z)}}.$$

Entropía cruzada multiclase

$$L = -\frac{1}{n} \sum_{i=1}^n \log(\hat{p}_{i,y_i}).$$

Score CV

$$\bar{s} = \frac{1}{K} \sum_{k=1}^K s_k, \quad \sigma_s = \sqrt{\frac{1}{K} \sum_{k=1}^K (s_k - \bar{s})^2}.$$

PCA desde SVD

$$X_c = U \Sigma V^\top, \quad Z_k = X_c V_k, \quad \hat{X} = Z_k V_k^\top + \mu.$$

Silhouette

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}.$$

Hash SHA-256

Un hash no prueba que un resultado sea correcto. Prueba que el archivo no cambió respecto a la versión registrada.

Checklist de paquete reproducible

- Hay una instrucción clara para ejecutar el flujo principal.
- Datos, scripts y resultados tienen rutas documentadas.
- La semilla y las particiones están registradas.
- El entorno está especificado con **requirements**, **environment** o equivalente.
- El preprocesamiento final está dentro del pipeline o descrito paso a paso.
- Las métricas finales se pueden recalcular.
- Los artefactos importantes tienen versión, nombre estable o hash.
- La conclusión distingue evidencia, interpretación y recomendación.

Estructura mínima de manifest

```
{
  "run_id": "2026-09-30-baseline-logreg",
  "seed": 2105,
  "data": {
    "train": "data/processed/train.csv",
    "test": "data/processed/test.csv"
  },
}
```



```

"model": {
  "name": "logistic_regression",
  "params": {"C": 1.0}
},
"metrics": {
  "f1_test": 0.72,
  "recall_test": 0.68
},
"artifacts": {
  "report": "reports/final.html",
  "model": "models/model.joblib"
},
"decision": "no desplegar sin revisar slice crítico"
}

```

Lenguaje de defensa

Frase débil	Versión defendible
“El modelo es bueno.”	“El modelo supera el baseline en F1, pero falla en un slice crítico.”
“PCA demuestra grupos.”	“La proyección PCA sugiere separación visual; falta validación adicional.”
“El experimento es reproducible.”	“El experimento tiene semilla, entorno, manifest y script de evaluación.”
“Recomendamos usarlo.”	“Recomendamos una prueba limitada bajo estas restricciones.”

Respuestas seleccionadas

Verificaciones de ejemplos resueltos

Softmax estable. Para logits (1000, 1001, 1002), restar el máximo produce $(-2, -1, 0)$. La distribución queda aproximadamente (0.090, 0.245, 0.665). La resta evita overflow y no cambia softmax.

Manifest. Si el manifest registra `sha256:abc123` y el archivo actual tiene `sha256:def987`, el artefacto no coincide con la corrida registrada. La métrica asociada debe considerarse no verificada hasta rerunear o recuperar el archivo correcto.

Mejora relativa. Si F1 pasa de 0.60 a 0.72, la mejora absoluta es 0.12 y la relativa es $0.12/0.60 = 20\%$.

Rúbrica ponderada. Con puntajes 12, 16, 20, 13, 12, 8 sobre pesos que suman 100, la nota final es 81%.

Síntesis para Parcial 2

Capítulo 1, ejercicio 1. Una mejora global no basta si un slice crítico empeora porque la métrica agregada puede ocultar daño concentrado. Una respuesta profesional pide revisar tamaño del slice, costo del error, estabilidad de la medición y acción de mitigación.

Capítulo 1, ejercicio 4. Para logits (1000, 1001, 1002), resta el máximo: $(-2, -1, 0)$. Softmax queda proporcional a $(e^{-2}, e^{-1}, 1)$, es decir aproximadamente (0.090, 0.245, 0.665).

Reproducibilidad

Capítulo 2, ejercicio 1. Una semilla fija no garantiza reproducibilidad completa porque pueden cambiar librerías, hardware, orden de datos, implementaciones paralelas o archivos de entrada.

Capítulo 2, ejercicio 4. Un hash SHA-256 detecta si un archivo cambió respecto a la versión registrada. No dice si el contenido es correcto, ético o suficiente para la decisión.

Capítulo 2, ejercicio 8. Si no se puede reproducir una métrica final, revisa primero partición, versión de datos, pipeline de preprocesamiento, semilla, parámetros del modelo y script de evaluación.

Comunicación técnica

Capítulo 3, ejercicio 1. “El modelo es bueno” no es suficiente porque no dice contra qué baseline, con qué métrica, en qué partición, bajo qué costo de error ni con qué limitaciones.

Capítulo 3, ejercicio 4. Si el baseline tiene F1 0.60 y el modelo final F1 0.72, la mejora absoluta es 0.12 puntos de F1 y la mejora relativa es $(0.72 - 0.60)/0.60 = 20\%$.

Capítulo 3, ejercicio 7. Una conclusión exagerada debe reescribirse con cuatro partes: evidencia, límite, acción y condición de uso. Por ejemplo: “El modelo mejora F1 frente al baseline, pero su recall cae en clientes nuevos; recomendamos analizar ese slice antes de una prueba controlada”.

Entrega y defensa

Capítulo 4, ejercicio 3. Con puntajes 12/15, 16/20, 20/25, 13/15, 12/15 y 8/10, el total es 81/100.

Capítulo 4, ejercicio 4. Si una defensa usa cinco criterios 0/1/2, el puntaje máximo es 10. Una conversión transparente a porcentaje es $\text{porcentaje} = 10 \times \text{puntaje}$.

Capítulo 4, ejercicio 7. Una respuesta defendible a “¿qué cambiaría con dos semanas más?” debe nombrar una acción priorizada y una razón: “ampliaría análisis del slice con peor recall porque ahí está el riesgo operativo principal”.

Ejercicios integradores

Ejercicio 1. Restar el máximo evita overflow porque reemplaza exponenciales enormes por valores en una escala estable. La operación no cambia softmax porque multiplica numerador y denominador por la misma constante.

Ejercicio 2. Una respuesta sólida: el modelo mejora el baseline, pero tiene variabilidad alta, cae en test y falla en un slice crítico. La acción prudente es diagnosticar ese slice y ajustar datos, features o umbral antes de recomendar uso operativo.

Ejercicio 3. Con más componentes, el subespacio de reconstrucción contiene al anterior. Por eso la mejor aproximación con $k + 1$ componentes no puede tener mayor error que con k .

Ejercicio 4. Un manifest válido incluye parámetros, métricas, rutas de artefactos, hashes y una decisión. Su valor principal es auditar qué corrida produjo una conclusión.

Ejercicio 5. Una defensa final fuerte debe declarar al menos un límite: datos insuficientes, slice inestable, métrica parcial, supuestos de despliegue o riesgo de cambio de distribución.

Información editorial y publicación

Información editorial

Audiencia

Este libro está escrito para estudiantes de pregrado que cursan **MATE-2105 Matemáticas y Algoritmos en Ciencia de Datos Aplicada**. También puede servir como texto de apoyo para cursos que conecten álgebra lineal, cálculo, optimización, programación científica y aprendizaje automático aplicado.

El texto asume disposición para leer matemáticas y programar en Python, pero no asume experiencia previa avanzada en machine learning.

Prerrequisitos recomendados

Para aprovechar el libro conviene tener familiaridad básica con:

- vectores, matrices y productos matriciales;
- funciones, derivadas y gradientes;
- estadística descriptiva introductoria y nociones básicas de incertidumbre;
- programación básica en Python;
- lectura de tablas y gráficos.

Cuando un capítulo necesita una herramienta específica, la introduce en contexto y la conecta con implementación. El libro no exige un curso formal previo de probabilidad: el Interludio 0.5 fija el vocabulario mínimo de variable aleatoria, muestra, ruido, probabilidad condicional y generalización que se usa en los capítulos posteriores.

Alcance

El libro se concentra en el razonamiento matemático-algorítmico detrás de modelos usados en ciencia de datos aplicada:

- regresión y optimización;
- regularización y validación;
- clasificación y métricas;
- redes neuronales;
- estrategia de machine learning;
- reducción de dimensión y clustering;
- reproducibilidad y comunicación técnica.

No busca ser una enciclopedia de ciencia de datos. Su propósito es formar criterio: qué se calcula, por qué se calcula, cómo se valida y con qué límites se recomienda una decisión.

Formato recomendado

La versión web es el formato principal porque mantiene navegación, búsqueda, enlaces y lectura móvil. El portal del curso publica la versión vigente en:

<https://cesarneyit-code.github.io/mate2105/libro/>

La versión PDF se genera desde Quarto junto con la edición web. Antes de publicarla como material final de impresión, deben revisarse figuras, tablas, fórmulas largas y saltos de página.

Cada publicación web incluye además un `manifest-publicacion.json` con inventario de archivos, tamaños y hashes SHA-256 de los artefactos públicos del libro. Ese archivo permite auditar que una copia publicada corresponde a una construcción concreta.

Cómo citar

Referencia sugerida:

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: Libro del curso MATE-2105*. Universidad de los Andes.

<https://cesarneyit-code.github.io/mate2105/libro/>

Accesibilidad

La edición web debe revisarse con estos criterios mínimos:

- navegación por teclado en páginas principales;
- contraste suficiente en texto, enlaces y cajas pedagógicas;
- tablas legibles en pantallas pequeñas;
- fórmulas renderizadas correctamente;
- texto alternativo o descripción para figuras no decorativas;
- enlaces descriptivos y funcionales.

Si una figura o tabla no puede leerse en móvil, debe rediseñarse antes de considerarse lista para publicación.

La declaración pública completa está en la página **Accesibilidad**, que separa objetivo, evidencia automática, revisión manual requerida, barreras conocidas y plantillas de remediación.

Errata y mejoras

Este libro es una edición viva del curso. Los errores matemáticos, problemas de código, enlaces rotos o sugerencias de mejora deben registrarse como tareas de revisión en el repositorio docente antes de publicar una nueva versión.

Cada corrección importante debe indicar:

- capítulo o sección afectada;
- problema encontrado;
- corrección propuesta;
- impacto sobre notebooks, laboratorios o evaluaciones;
- fecha de revisión.

Separación de materiales

El libro público no contiene:

- soluciones oficiales completas;
- pruebas ocultas;
- autograders;
- paquetes Gradescope;
- guías docentes privadas;
- criterios internos de calificación.

Esa separación protege la integridad académica y permite que el libro funcione como texto de estudio.

Accesibilidad

Esta página declara el estado de accesibilidad del libro y el proceso de mejora continua. La versión web es el formato principal de lectura porque permite navegación, búsqueda, enlaces, ajuste de tamaño de texto y auditorías HTML. El PDF se conserva como formato offline y archivo de versión, pero no reemplaza la versión web como opción accesible principal.

Objetivo de accesibilidad

El objetivo editorial es que la versión web del libro sea usable con:

- navegación por teclado;
- lectores de pantalla modernos;
- zoom del navegador;
- lectura en pantallas pequeñas;
- texto alternativo o descripción equivalente para figuras no decorativas;
- tablas con encabezados;
- enlaces descriptivos;
- bloques de código legibles y desplazables;
- contraste suficiente en texto, enlaces y recursos pedagógicos.

La meta técnica de esta edición es avanzar hacia criterios compatibles con WCAG 2.2 AA para la versión web. Las pruebas automáticas ayudan a detectar problemas, pero no prueban por sí solas conformidad completa.

Alcance evaluado

La revisión actual cubre:

Elemento	Estado
páginas HTML del libro	verificadas con auditoría estática
páginas representativas	verificadas con axe
figuras nuevas	tienen texto alternativo o descripción equivalente
tablas	se auditan encabezados <th> en HTML
enlaces	se auditan textos accesibles básicos
bloques de código	reciben foco de teclado mediante ajuste HTML
PDF	texto extraíble, tamaño carta y metadatos verificados

Elemento	Estado
PDF etiquetado	no disponible; existe decisión formal de usar HTML como formato accesible principal

Evidencia automática

Antes de publicar, deben pasar estas verificaciones:

```
python3 scripts/auditar_accesibilidad_libro.py sitio_curso/_site/libro
bash scripts/auditar_axe_libro.sh http://127.0.0.1:8765/libro
python3 scripts/auditar_pdf_libro.py sitio_curso/_site/libro/Matemáticas-y-Algoritmos-en-Ciencia
```

La auditoría estática revisa mínimos estructurales: idioma, título, encabezado principal, texto alternativo, enlaces con texto, tablas con encabezados e ids duplicados. Axe revisa una muestra representativa de páginas renderizadas.

Revisión manual requerida

Una edición publicable fuera del curso requiere además revisión manual. Como mínimo, debe probarse:

1. navegación por teclado desde portada hasta capítulos y apéndices;
2. orden de foco en menús, búsqueda, enlaces y bloques de código;
3. lectura con lector de pantalla en portada, página editorial, capítulo con matemática, capítulo con figuras, tabla de datos y apéndice;
4. comprensión de figuras mediante alt text o texto cercano;
5. lectura de fórmulas en el navegador usado por estudiantes;
6. zoom al 200 % sin pérdida de contenido esencial;
7. lectura móvil de tablas y bloques de código;
8. contraste de cajas pedagógicas y enlaces;
9. descarga de CSV y PDF mediante enlaces claros.

El paquete preparado para ejecutar esta revisión está registrado en `libro_curso/registros/accesibilidad/paquete-07-07.md`. Ese documento define páginas mínimas, pruebas, evidencia esperada y plantillas de trabajo; no sustituye la ejecución manual.

Matemática, código y figuras

La accesibilidad de un libro matemático exige revisar tres zonas de riesgo.

Zona	Riesgo	Criterio
Fórmulas	lectura ambigua con tecnologías asistivas	acompañar fórmulas centrales con explicación textual
Figuras	pérdida de información si la imagen no se ve	incluir alt text o lectura de figura cercana
Código	líneas largas o bloques no navegables	permitir desplazamiento, copia y foco de teclado

Las figuras complejas deben tener una **Lectura de la figura** en el capítulo o un texto alternativo suficientemente descriptivo.

Barreras conocidas

Barrera	Estado	Acción
PDF no etiquetado	conocido y decidido para esta edición	mantener HTML como formato accesible principal; no presentar PDF como equivalente accesible
revisión manual con lector de pantalla	pendiente	completar checklist antes de edición publicable final
revisión manual de teclado completa	pendiente	completar checklist antes de edición publicable final
fórmulas complejas en PDF	pendiente de revisión visual/manual	revisar capítulos matemáticos antes de impresión

Reporte de barreras

Si un estudiante, instructor o revisor encuentra una barrera de accesibilidad, el reporte debe incluir:

- página o archivo afectado;
- descripción de lo que ocurrió;
- navegador y sistema operativo;
- tecnología asistiva usada, si aplica;
- captura o ejemplo mínimo, si es posible;
- impacto sobre lectura, navegación, descarga o comprensión.

Las barreras se registran como errata de accesibilidad y se priorizan por impacto pedagógico.

Plantillas de revisión

Las plantillas operativas están publicadas con el libro:

- [checklist_teclado.md](#)
- [checklist_lector_pantalla.md](#)
- [reporte_barrera_accesibilidad.md](#)
- [registro_remediacion_accesibilidad.md](#)

La decisión editorial sobre PDF no etiquetado queda registrada en `libro_curso/registros/pdf/decision_accesibilidad_07-07.md`. Ese registro declara que el HTML es el formato accesible principal y que el PDF se publica como respaldo offline y archivo de versión.

Criterio de cierre

Una barrera de accesibilidad queda cerrada cuando:

1. la causa está identificada;
2. la corrección está aplicada en la fuente;
3. el libro se renderiza de nuevo;
4. la auditoría automática relevante pasa;
5. la persona revisora confirma la mejora cuando el problema era manual;
6. la corrección queda registrada en el historial editorial o en errata.

Equipo, revisión y reconocimientos

Esta página documenta quién produce, revisa y mantiene el libro. En una versión publicable, esta información no es decorativa: permite al lector identificar responsabilidades, estado de revisión y límites de la edición.

Autoría y responsabilidad editorial

Rol	Responsable	Alcance
Autor del curso y del libro	César Galindo	diseño curricular, selección de contenidos, redacción matemática, notebooks y evaluación
Edición técnica	César Galindo	consistencia de notación, código, ejemplos, figuras y referencias internas
Edición pedagógica	César Galindo	secuencia de aprendizaje, ejercicios, respuestas seleccionadas y conexión con laboratorios
Mantenimiento de publicación	César Galindo	render Quarto, portal, PDF, auditorías y control de versiones

Esta es una edición de trabajo. Cuando el libro tenga revisión externa o apoyo institucional formal, esta página debe registrar nombres, afiliaciones, fecha de revisión y alcance de cada contribución.

Estado de revisión

El libro tiene tres niveles de revisión:

1. **Revisión estructural.** Verifica que cada capítulo tenga objetivos, ejemplos, revisión, términos clave, ejercicios y proyecto de cierre.
2. **Revisión técnica.** Verifica fórmulas, dimensiones, código, resultados numéricos y coherencia con notebooks o laboratorios.
3. **Revisión editorial.** Verifica legibilidad, figuras, accesibilidad, enlaces, PDF y separación entre materiales públicos y privados.

Nivel	Estado actual	Evidencia
estructura de capítulos	automatizada	<code>scripts/auditar_estructura_editorial_1</code>
publicación pública	automatizada	<code>scripts/auditar_publicacion.py</code> y verificación del portal
accesibilidad estática	automatizada	<code>scripts/auditar_accesibilidad_libro.py</code>
accesibilidad con navegador	muestra representativa	<code>scripts/auditar_axe_libro.sh</code>
preparación OpenStax-like	automatizada para curso	<code>scripts/auditar_preparacion_openstax_1</code>
revisión matemática externa	pendiente	requiere lector técnico externo
revisión pedagógica externa	pendiente	requiere instructor o par académico externo

La página **Matriz de preparación editorial** detalla el estado por parte y por capítulo para distinguir uso en curso, publicación web y publicación abierta. La página **Protocolo de revisión externa** define el procedimiento y las rúbricas que deben usarse antes de declarar una edición publicable final. Ese protocolo incluye plantillas de carta, checklist, dictamen, rúbrica y registro de correcciones para que la revisión sea trazable. La página **Registro público de errata** mantiene el estado visible de reportes, correcciones cerradas y cambios editoriales de la edición vigente.

Cómo reportar una corrección

Una corrección útil debe incluir:

- capítulo o sección;
- descripción precisa del problema;
- tipo de problema: matemático, código, redacción, enlace, figura, evaluación o accesibilidad;
- propuesta de corrección, si existe;
- evidencia mínima: cálculo, captura, error de ejecución o enlace roto.

Las correcciones que afecten laboratorios, parciales o autograders deben revisarse también en el repositorio docente privado antes de publicar una nueva versión.

Reconocimientos

Este libro se construye como material vivo del curso MATE-2105. Su forma combina texto matemático, notebooks, laboratorios autocalificados, proyecto aplicado y portal de curso. Las mejoras futuras deben reconocer explícitamente a revisores, asistentes docentes, estudiantes o colegas que aporten correcciones sustantivas.

Criterio para versión publicable

Antes de declarar una edición como publicable fuera del curso, deben cumplirse estos requisitos:

- licencia formal definida en el repositorio;

- revisión matemática externa de capítulos centrales;
- revisión pedagógica externa de secuencia, ejercicios y evaluación;
- dictamen externo trazable y correcciones obligatorias cerradas;
- registro de errata abierto o canal institucional equivalente;
- portada, cita sugerida, accesibilidad y recursos complementarios visibles;
- separación verificada entre libro público y materiales privados de evaluación.

Protocolo de revisión externa

Este protocolo define cómo debe revisarse el libro antes de declararlo una edición publicable fuera del curso. Su objetivo es convertir la revisión externa en evidencia verificable, no en una aprobación informal.

Propósito de la revisión

La revisión externa debe responder cuatro preguntas:

1. ¿Las derivaciones matemáticas son correctas y usan notación consistente?
2. ¿Los ejemplos, ejercicios y respuestas seleccionadas apoyan el aprendizaje?
3. ¿La secuencia del libro prepara las evaluaciones y el proyecto del curso?
4. ¿El texto puede publicarse sin exponer material privado ni inducir decisiones técnicas exageradas?

Una revisión externa no reemplaza las auditorías automáticas. Las complementa: las auditorías detectan estructura, enlaces, accesibilidad básica y material sensible; el revisor externo juzga corrección matemática, claridad pedagógica y criterio disciplinar.

Roles de revisión

Rol	Perfil sugerido	Alcance
Revisor matemático	profesor o investigador con experiencia en álgebra lineal, optimización, estadística o aprendizaje automático	derivaciones, notación, resultados, ejemplos y ejercicios cuantitativos
Revisor pedagógico	docente de curso STEM o ciencia de datos aplicada	secuencia, objetivos, carga, evaluación, retroalimentación y autonomía
Revisor computacional	persona con experiencia en Python científico y reproducibilidad	código, notebooks, dependencias, datasets y comparación con librerías
Revisor de accesibilidad	persona con experiencia en accesibilidad web o materiales educativos	navegación, texto alternativo, tablas, contraste, PDF y lector de pantalla

Una misma persona puede cubrir más de un rol si declara su alcance explícitamente.

Evidencia que recibe el revisor

Antes de revisar, el revisor debe tener acceso a:

- libro web vigente;
- PDF vigente;
- fuente Quarto de capítulos públicos;
- matriz de preparación editorial;
- resultados de aprendizaje y alineación;
- página de referencias y atribuciones;
- reporte de auditoría editorial;
- reporte de publicación sin material sensible;
- evidencia de render y link check;
- indicación clara de qué materiales son privados y no se revisan públicamente.

Plantillas de revisión

El paquete operativo para revisión externa está disponible en:

Plantilla	Uso
Carta de solicitud Checklist del paquete	invitar a un revisor y declarar alcance esperado confirmar que el revisor recibe los materiales correctos
Formulario de dictamen Rúbrica de revisión	registrar aprobación, correcciones y trazabilidad evaluar criterios matemáticos, pedagógicos, computacionales y de accesibilidad
Registro de correcciones	cerrar correcciones obligatorias y recomendadas

Estas plantillas deben completarse solo con revisiones reales. No son evidencia de revisión por sí mismas; son el formato para producir esa evidencia.

El paquete preparado para enviar a revisores externos está registrado en `libro_curso/registros/revision_externa/07-07.md`. Ese documento organiza rutas, alcance y evidencia disponible, pero no cuenta como revisión completada.

Rúbrica matemática

Criterio	Aceptable	Requiere corrección
Notación	símbolos, dimensiones y convenciones son consistentes	un símbolo cambia de significado sin aviso
Derivaciones	los pasos principales son correctos y suficientes para el nivel	falta un paso que cambia el resultado o confunde gradientes

Criterio	Aceptable	Requiere corrección
Ejemplos	los cálculos pueden reproducirse y coinciden con el texto	resultados numéricos no coinciden o no se justifican
Ejercicios	practican conceptos centrales con dificultad razonable	preguntan algo no preparado o demasiado ambiguo
Interpretación	las conclusiones declaran límites	se sugieren decisiones no justificadas por la evidencia

Rúbrica pedagógica

Criterio	Aceptable	Requiere corrección
Objetivos	son observables y se conectan con evidencias	son vagos o no se evalúan
Secuencia	los capítulos avanzan de apoyo guiado a autonomía	aparecen saltos conceptuales sin transición
Actividades	ejercicios y proyectos preparan laboratorios y parciales	la práctica no se alinea con evaluación
Carga	la cantidad de lectura y práctica es viable por semana	una semana concentra demasiada demanda nueva
Retroalimentación	respuestas seleccionadas orientan sin ser solucionario	faltan verificaciones públicas donde se necesitan

Rúbrica computacional

Criterio	Aceptable	Requiere corrección
Reproducibilidad	ejemplos declaran datos, semilla y dependencia relevante	un resultado depende de estado oculto
Código	funciones y shapes son claros	código no ejecuta o no coincide con la fórmula
Comparación profesional	se compara contra referencia sin cambiar el experimento	se comparan modelos con datos, métricas o particiones distintas
Datasets	fuentes y límites están documentados	se usa un dataset sin advertencia de interpretación
Leakage	transformaciones se aprenden solo en entrenamiento	se filtra información de validación o test

Dictamen de revisión

Cada revisión externa debe cerrar con un dictamen breve:

Revisor:
Afiliación:
Rol de revisión:
Fecha:
Versión revisada:
Alcance:
Dictamen: aprobar / aprobar con correcciones menores / revisar de nuevo
Correcciones obligatorias:
Correcciones recomendadas:
Evidencia revisada:

Registro de revisiones externas

Fecha	Revisor	Rol	Alcance	Dictamen	Estado
pendiente	pendiente	pendiente	pendiente	pendiente	pendiente

Este registro debe actualizarse solo con revisiones reales. No se deben inventar revisores, afiliaciones ni aprobaciones.

Criterio de cierre

La revisión externa se considera cerrada cuando:

1. existe dictamen firmado o trazable;
2. las correcciones obligatorias están registradas;
3. las correcciones aplicadas tienen evidencia de render y validación;
4. las correcciones rechazadas tienen justificación editorial;
5. la matriz de preparación editorial refleja el nuevo estado.

Hasta que esto ocurra, el libro puede estar listo para el curso, pero no debe declararse como edición publicable final.

Matriz de preparación editorial

Esta matriz resume el estado de preparación del libro como texto de curso y como posible publicación abierta. Su función es separar tres cosas que a menudo se confunden: material listo para usar en clase, material verificado por auditorías automáticas y material revisado externamente.

Niveles de preparación

Nivel	Estado actual	Evidencia	Pendiente
Estructura editorial	verificada	capítulos con objetivos, ejemplos, revisión, términos clave, ejercicios y proyecto	mantener auditoría al editar
Técnica matemática	revisión interna	derivaciones, ejemplos, código y respuestas seleccionadas	revisión externa de capítulos centrales
Diseño pedagógico	revisión interna	secuencia por módulos, laboratorios, notebooks y proyecto	revisión externa de alineación
Publicación web	verificada	render Quarto, link check, auditoría de material sensible y LICENSE formal de uso docente restringido	evaluación-aprendizaje decidir si una edición futura será abierta
PDF e impresión	verificado técnicamente, pendiente de prueba de impresión	auditoría PDF, guía de impresión y plantillas de revisión	revisión visual completa antes de impresión pública
Accesibilidad	verificación automática y declaración pública	auditoría estática, axe en páginas representativas y página de accesibilidad	revisión manual con teclado y lector de pantalla
Preparación OpenStax-like	verificada para curso, pendiente para edición final	<code>scripts/auditar_preparacion_openstax_libro.py</code>	cuando existan revisión externa, registros manuales y decisión de PDF accesible

En esta edición, **listo para curso** no significa todavía **publicación abierta final**. El libro puede usarse como texto del curso mientras conserva pendientes editoriales explícitos para una versión publicable.

Matriz por partes

Parte	Alcance	Estado para el curso	Revisión pendiente
I. Fundamentos	lenguaje común, incertidumbre mínima, flujo aplicado y notación inicial	usable como arranque	revisión de transición hacia regresión
II. Regresión, optimización y clasificación	mínimos cuadrados, gradiente, regularización, logística y métricas	núcleo fuerte y verificable	revisión externa de derivaciones y ejercicios
III. Redes neuronales	perceptrón, backpropagation, diagnóstico y PyTorch	reforzado con derivaciones, ejemplos, verificaciones y caso <code>load_digits</code>	revisión externa de backpropagation y criterios de entrenamiento
IV. Estrategia de ML	métricas, validación, curvas y análisis de errores	reforzado con figuras, casos por slice y verificaciones seleccionadas	revisión pedagógica de decisiones aplicadas
V. No supervisado	PCA/SVD, K-Means y recomendación latente	reforzado con derivaciones, ejemplos calculados y datasets longitudinales	revisar si una edición futura requiere más ejercicios avanzados
VI. Reproducibilidad y defensa	síntesis, tracking, comunicación y defensa final	reforzado con plantillas, casos longitudinales, comunicación y defensa	revisión de rúbricas y defensa oral

Matriz por capítulo principal

	Capítulo	Tema	Estado editorial	Siguiente revisión
	0	lenguaje común de ciencia de datos aplicada	estructura base	coherencia con primera clase
	0.5	incertidumbre, datos observados y generalización	puente probabilístico mínimo	revisión pedagógica con estudiantes sin probabilidad previa

Capítulo	Tema	Estado editorial	Siguiente revisión
1	regresión lineal múltiple	fuerte	revisión externa de notación y ejercicios
2	descenso del gradiente	fuerte	revisión externa de gradiente y convergencia
3	validación y regularización	fuerte	revisión de ejemplos y casos de leakage
4	regresión logística	fuerte	revisión de umbrales y métricas
5	ejercicios integradores de regresión	banco estructurado	calibración de dificultad
6	perceptrón y propagación hacia adelante	reforzado	revisión de formas y ejemplo <code>load_digits</code>
7	backpropagation	reforzado	revisión técnica externa de derivación softmax-log-loss
8	diagnóstico y regularización en redes	reforzado	revisión de criterios de diagnóstico y parada
9	optimizadores y PyTorch	reforzado	revisar dependencia entre teoría, Adam y API
10	ejercicios integradores de redes	banco estructurado	calibración de dificultad
11	métricas y particiones	reforzado	revisar ejemplos por costo de error y partición
12	validación y curvas	reforzado	revisar interpretación de curvas y estabilidad
13	análisis de errores y data mismatch	reforzado	revisar slices, sesgos y acciones correctivas
14	ejercicios integradores de estrategia	banco estructurado	calibración de decisiones
15	SVD y PCA	reforzado	revisión externa de álgebra lineal
16	K-Means y clustering	reforzado	revisión de limitaciones y diagnóstico
17	recomendación latente	reforzado	revisión de supuestos y evaluación

Capítulo	Tema	Estado editorial	Siguiente revisión
18	ejercicios integradores no supervisado	banco estructurado	calibración de dificultad
19	síntesis conceptual	reforzado	revisión de preparación para parcial
20	reproducibilidad y tracking	reforzado	revisión de manifest y trazabilidad
21	comunicación técnica	reforzado	revisión de rúbricas comunicativas
22	entrega y defensa	reforzado	revisión de defensa oral
23	ejercicios integradores de cierre	banco estructurado	calibración final

Evidencia mínima por nueva edición

Antes de publicar una nueva edición, el registro editorial debe incluir:

- fecha de construcción;
- resumen de cambios;
- auditoría de estructura;
- auditoría de material sensible;
- verificación de enlaces;
- revisión de accesibilidad estática;
- pasada axe sobre páginas representativas;
- declaración de accesibilidad actualizada;
- checklist manual de teclado y lector de pantalla cuando aplique;
- revisión de navegabilidad transversal: glosario, índice temático e índice de algoritmos;
- revisión de la taxonomía de ejercicios frente a capítulos, laboratorios y bancos integradores;
- auditoría de preparación OpenStax-like en modo curso;
- registro público de errata actualizado;
- revisión visual del PDF;
- dictamen de revisión externa;
- checklist y formulario de revisión externa completados;
- auditoría de preparación OpenStax-like en modo `--final`;
- estado de correcciones obligatorias;
- decisión de licencia;
- registro de decisión de licencia completado;
- estado de revisión externa;
- checklist de impresión o registro de prueba cuando el PDF se distribuya como versión imprimible.

Señales de riesgo editorial

Un capítulo debe volver a revisión si ocurre cualquiera de estos casos:

- cambia una derivación central;
- cambia un dataset, métrica o semilla de ejemplo;
- se modifica un laboratorio o parcial asociado;
- se agrega una figura sin texto alternativo;
- se cambia una respuesta seleccionada;
- se cambia la taxonomía, dificultad o propósito de un banco de ejercicios;
- se detecta desborde visual en PDF;
- se publica por error material privado de evaluación.

Características pedagógicas

Este libro está diseñado como texto de curso, no como una colección de apuntes. La estructura editorial busca que cada capítulo pueda leerse de forma independiente, pero también como parte de una secuencia de formación matemática y computacional.

Estructura recurrente

Cada capítulo debe tender hacia la siguiente organización:

1. **Pregunta aplicada.** Sitúa el problema antes del formalismo.
2. **Objetivos de aprendizaje.** Declara lo que el estudiante debería poder hacer al terminar.
3. **Modelo matemático.** Define objetos, dimensiones, supuestos y notación.
4. **Algoritmo.** Explica qué se calcula y por qué.
5. **Implementación.** Traduce el algoritmo a código reproducible.
6. **Validación.** Conecta pérdida, métrica, partición y evidencia.
7. **Ejemplo resuelto.** Muestra un caso pequeño que pueda verificarse a mano o con pocas líneas de código.
8. **Para explorar más.** Propone una extensión matemática, una extensión computacional y una conexión con proyecto.
9. **Ejercicios.** Separa práctica conceptual, cuantitativa, computacional e interpretativa.
10. **Respuestas seleccionadas.** Da retroalimentación sin reemplazar el trabajo del estudiante.

En la Parte I, esta estructura se adapta a capítulos puente. Allí el objetivo no es demostrar teoremas largos, sino fijar lenguaje: qué es una observación, qué es una variable aleatoria, qué significa ruido y por qué una pérdida calculada en datos observados no equivale automáticamente a desempeño futuro.

Desde la Parte II en adelante, varios capítulos vuelven sobre casos longitudinales. La intención es que el estudiante no vea regresión, regularización, validación, análisis de errores, comunicación y defensa como episodios aislados. Un mismo dataset puede reaparecer con una pregunta nueva: primero para ajustar un modelo, después para seleccionar una métrica, luego para diagnosticar slices y finalmente para comunicar una recomendación proporcional.

Criterio editorial. Un capítulo profesional debe permitir tres lecturas: una lectura rápida para ubicar el tema, una lectura lenta para seguir derivaciones y una lectura de práctica para resolver ejercicios.

Figuras guía

El libro debe usar figuras cuando aclaran una decisión o proceso. Las figuras no reemplazan la derivación matemática; sirven como mapa para que el estudiante ubique qué papel cumple cada objeto.

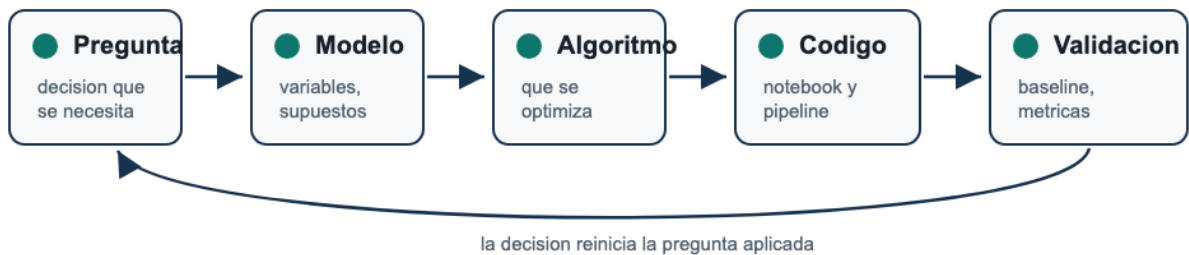


Figura 40: Ciclo de modelación aplicada del curso: pregunta aplicada, modelo matemático, algoritmo, código, validación y retorno a la pregunta.

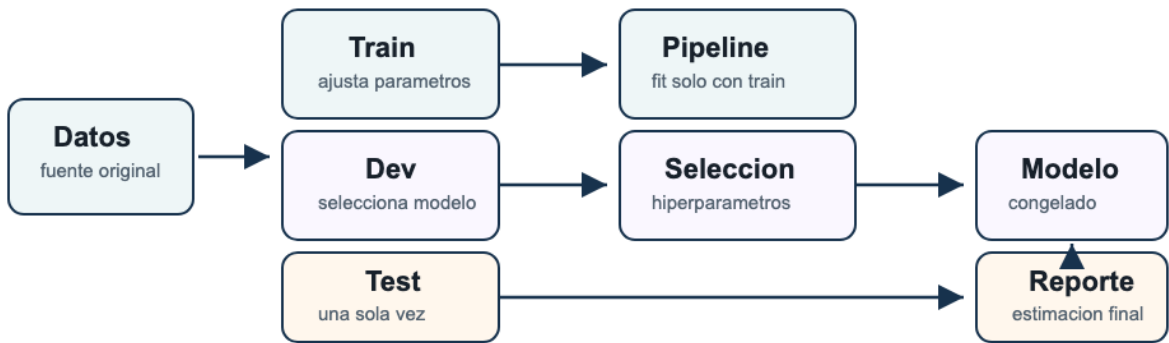


Figura 41: Flujo de validación: los datos se separan en train, dev y test; train ajusta parámetros, dev selecciona el modelo y test se usa una sola vez para reporte final.

Tipos de recursos

El libro usa una familia de recursos recurrentes.

Recurso	Uso esperado
Definición	Fijar vocabulario técnico y notación.
Proposición	Registrar un resultado matemático que se usa después.
Ejemplo resuelto	Mostrar una aplicación completa, con cálculo o código.

Recurso	Uso esperado
Criterio	Dar una regla de decisión o diagnóstico.
Nota	Advertir sobre errores frecuentes o sutilezas.
Para explorar más	Extender el capítulo con una tarea matemática, una computacional y una de proyecto.
Ejercicio	Proponer práctica verificable.
Proyecto	Conectar el capítulo con una decisión técnica más amplia.
Caso longitudinal	Mantener continuidad entre capítulos mediante un dataset o decisión que reaparece.
Índice temático	Ayudar a estudiar por concepto, modelo, métrica o decisión, no solo por semana.
Taxonomía de ejercicios	Calibrar tipo de pregunta, nivel cognitivo y evidencia esperada.

Estándar de ejercicios

Los ejercicios del libro deben cubrir cuatro niveles.

Nivel	Evidencia de aprendizaje
Conceptual	El estudiante explica el significado de un objeto, supuesto o métrica.
Cuantitativo	El estudiante calcula una cantidad, gradiente, métrica o dimensión.
Computacional	El estudiante implementa o verifica un procedimiento.
Interpretativo	El estudiante defiende una decisión técnica con evidencia y límites.

La página [Taxonomía de ejercicios y banco de preguntas](#) convierte estos niveles en criterios de diseño, estudio y revisión. Esa página es pública porque orienta la práctica; no contiene pruebas ocultas ni soluciones oficiales completas.

La página [Mapa de cierres de capítulo](#) muestra cómo cada capítulo principal incorpora revisión, términos clave, preguntas conceptuales, problemas cuantitativos, práctica computacional, pensamiento crítico y proyecto de cierre.

Relación con evaluación

El libro prepara para laboratorios y parciales, pero no contiene pruebas ocultas ni soluciones oficiales completas. La separación es intencional:

- el libro desarrolla conocimiento;

- el portal organiza entregas;
- los notebooks entrenan implementación;
- los laboratorios y parciales verifican desempeño.

Estándar de revisión

Antes de publicar una versión del libro, cada capítulo debe revisarse con esta lista mínima:

- objetivos claros y evaluables;
- notación consistente;
- al menos un ejemplo resuelto;
- derivación guiada cuando el capítulo introduce una función objetivo, gradiente, factorización o regla de decisión;
- respuestas o verificaciones seleccionadas visibles al final del capítulo;
- sección de exploración adicional conectada con matemática, código y proyecto;
- ejercicios suficientes y clasificados;
- una conexión explícita con validación o decisión técnica;
- conexión con un caso longitudinal cuando el tema se beneficie de continuidad;
- ausencia de dependencias privadas;
- enlaces funcionales;
- lectura legible en móvil y escritorio.

Recursos para estudiantes e instructores

Este libro se publica como texto de estudio. El curso completo incluye además portal, notebooks, laboratorios, rúbricas y guías docentes. Esta página separa qué recursos son públicos, cuáles son de uso del estudiante y cuáles deben permanecer restringidos para proteger la integridad académica.

Recursos públicos para estudiantes

Libro web y PDF. Texto principal del curso: definiciones, derivaciones, ejemplos, figuras, ejercicios, respuestas seleccionadas y apéndices.

Datasets públicos. CSV de práctica descargables y manifest de datos con hashes para practicar regresión, clasificación, clustering, PCA y recomendación sin depender de internet.

Casos longitudinales. Hilos de datos que reaparecen en varios capítulos para conectar modelo, validación, diagnóstico, comunicación y defensa.

Portal del curso. Calendario, ruta semanal, enlaces a materiales, proyecto y configuración técnica. El portal organiza el trabajo; el libro organiza las ideas.

Notebooks de clase. Documentos ejecutables para practicar implementación, validar fórmulas y conectar teoría con código. Deben poder ejecutarse sin exponer soluciones oficiales de evaluaciones.

Laboratorios publicados. Enunciados, notebooks de estudiante, datos permitidos, criterios de entrega y rúbricas públicas. Las pruebas ocultas no se publican.

Respuestas seleccionadas. Retroalimentación parcial para ejercicios del libro. Su función es calibrar el estudio, no reemplazar soluciones completas.

Política de integridad académica y uso de IA. Reglas públicas sobre ayuda permitida, declaración de uso sustancial, defensa oral y límites de colaboración.

Guías públicas de inicio

El libro incluye dos guías de uso que deben permanecer públicas:

Guía	Audiencia	Propósito
Guía de estudio del estudiante	estudiantes	convertir lectura, notebook, laboratorio y proyecto en una rutina semanal verificable

Guía	Audiencia	Propósito
Guía pública de adopción docente	instructores y revisores	explicar adopción, separación público/privado, evaluación automatizada y controles de publicación
Mapa de objetivos de aprendizaje	estudiantes, instructores y revisores	ubicar objetivos observables, evidencia primaria y alineación con resultados globales
Mapa de cierres de capítulo	estudiantes, instructores y revisores	localizar términos clave, repaso, problemas, pensamiento crítico y proyecto de cierre
Casos longitudinales	estudiantes, instructores y revisores	seguir datasets públicos a través de varias decisiones técnicas
Respuestas y solucionarios	estudiantes, instructores y revisores	separar respuestas públicas, manual de estudiante y recursos restringidos
Prerrequisitos matemáticos mínimos	estudiantes	repasar formas, gradientes, probabilidad mínima y notación de código

Estas guías no contienen soluciones oficiales, pruebas ocultas ni instrucciones internas de calificación. Su función es transparentar el método de trabajo.

Recursos restringidos para instructores

Los recursos restringidos no deben publicarse en el libro ni en el portal público.

Recurso	Motivo de restricción
soluciones oficiales completas	preservan el valor formativo de ejercicios y evaluaciones
pruebas ocultas de autograding paquetes Gradescope/Otter guías docentes de clase	evitan hardcoding y filtración de criterios contienen lógica de calificación automática incluyen tiempos, decisiones de conducción y notas internas
rúbricas internas detalladas	apoyan consistencia docente sin revelar casos límite
bancos de parcial no liberados	mantienen integridad de evaluación

Una versión pública del curso nunca debe contener autograders, soluciones oficiales completas, zips de Gradescope ni notebooks con respuestas incrustadas. Antes de publicar, debe correrse la auditoría de material sensible del repositorio.

Cómo estudiar con los recursos

La secuencia recomendada para cada tema es:

1. leer la pregunta aplicada y los objetivos del capítulo;
2. revisar definiciones, proposiciones y figuras;
3. reproducir al menos un ejemplo resuelto;
4. ejecutar el notebook de clase;
5. resolver ejercicios conceptuales y cuantitativos;
6. implementar las funciones pequeñas del laboratorio;
7. escribir una interpretación corta con evidencia y límites;
8. comparar solo después con respuestas seleccionadas o feedback automático.

Criterio de dominio. Un estudiante domina un tema cuando puede explicar el objeto matemático, calcularlo en un caso pequeño, implementarlo en Python, validarlo con una métrica apropiada y decir qué decisión sí o no se justifica.

Cómo usar los recursos como instructor

El instructor puede usar el libro como base estable y el portal como capa operativa. Una preparación semanal mínima debería revisar:

Recurso	Pregunta docente
capítulo del libro	¿qué concepto central debe quedar claro?
notebook de clase	¿qué implementación se hará en vivo?
laboratorio	¿qué evidencia autocalifica el sistema?
rúbrica pública	¿qué debe saber el estudiante antes de entregar?
guía docente privada	¿qué errores frecuentes conviene anticipar?
reporte de autograding	¿qué casos requieren revisión humana?

Flujo de publicación

Antes de publicar una nueva versión:

1. renderizar libro y portal;
2. ejecutar verificación de enlaces del sitio público;
3. ejecutar auditoría de material sensible;
4. ejecutar auditoría estática de accesibilidad;
5. ejecutar axe sobre páginas representativas;
6. revisar visualmente portada, índice, figuras, tablas y páginas nuevas;
7. registrar cambios importantes en la auditoría editorial.

Política de respuestas

El libro puede incluir respuestas seleccionadas si cumplen estas reglas:

- explican el razonamiento central;
- no sustituyen una solución completa de laboratorio o parcial;
- no revelan pruebas ocultas;
- no publican código final de evaluaciones autograded;
- ayudan al estudiante a detectar errores de concepto, dimensión o interpretación.

Las soluciones completas pueden existir en materiales docentes privados, pero no deben sincronizarse con el portal público ni con el libro publicado.

La página [Respuestas, solucionarios y recursos restringidos](#) formaliza esta separación y enlaza plantillas públicas para controlar la liberación de respuestas.

Guía pública de adopción docente

Esta guía ayuda a usar el libro en un curso real sin exponer materiales restringidos. Está pensada para instructores, monitores y revisores que necesitan entender qué se puede publicar, qué debe permanecer privado y cómo sostener una evaluación con baja carga manual.

Para dictar MATE-2105 semana a semana, use también el panel docente privado del repositorio del curso. Ese panel no se publica en el portal de estudiantes y conecta esta guía con la ruta semanal, los capítulos del libro, las diapositivas, notebooks, entregas y fechas del curso.

Sistema de sincronización del curso

La adopción del curso debe mantener tres capas sincronizadas:

Capa	Función	Riesgo si se desactualiza
Portal	operación del curso, enlaces, programa, calendario y entregas	el estudiante no sabe qué hacer ni qué entregar
Libro	explicación conceptual, matemática y ejercicios	el estudiante estudia sin estructura o con notación inconsistente
Panel docente privado	preparación, conducción y cierre de clase	el profesor dicta sin verificar recursos, hitos o evaluación

Antes de dictar una semana, el profesor debe confirmar que las tres capas dicen lo mismo sobre tema, fecha, lectura, notebook, entrega e hito de proyecto.

Rutina mínima del profesor

Momento	Acción
7 días antes	revisar calendario, ruta semanal y programa
48 horas antes	abrir capítulo, slides, notebook y laboratorio público
antes de entrar a clase	definir pregunta aplicada, derivación central y cierre técnico
durante clase	derivar, implementar, validar e interpretar

Momento	Acción
después de clase	publicar recordatorio y registrar bloqueos frecuentes
antes de calificar	correr autograding y revisar solo casos que requieren juicio

Usos posibles del libro

Uso	Qué adoptar	Qué ajustar
Curso completo MATE-2105	todas las partes, portal, notebooks, labs y proyecto	fechas, plataforma de entrega y datasets de proyecto
Módulo de regresión aplicada	Parte II y apéndice de regresión	profundidad de derivaciones y número de laboratorios
Módulo de redes	Parte III y notebooks asociados	librería, hardware y alcance de entrenamiento
Módulo de estrategia ML	Parte IV y proyecto aplicado	dominio, métricas y criterios de decisión
Taller de reproducibilidad	Parte VI, manifiestos y guías de publicación	herramientas institucionales y política de datos

El libro permite una lectura modular, pero la secuencia completa fue diseñada para pasar de apoyo guiado a autonomía: derivar, implementar, validar, comunicar y defender.

Diseño semanal recomendado

Bloque	Tiempo sugerido	Producto
Activación	10-15 min	pregunta aplicada y error común
Desarrollo matemático	35-45 min	objeto, derivación o criterio de decisión
Implementación guiada	40-60 min	notebook con funciones y verificaciones
Discusión de validación	20-30 min	métrica, partición, curva o diagnóstico
Cierre	10-15 min	checklist de laboratorio o proyecto

Para una clase más corta, conserve pregunta aplicada, derivación mínima, notebook y cierre. Para una clase larga, agregue comparación con librería, diagnóstico de errores y discusión de límites.

Separación entre recursos públicos y privados

Público	Restringido
libro web y PDF	soluciones oficiales completas
datasets públicos y manifest	pruebas ocultas de autograding
notebooks de clase sin soluciones finales	zips Gradescope/Otter
enunciados y rúbricas públicas	bancos de parcial no liberados
respuestas seleccionadas	guías docentes con decisiones internas
guías de estudiante y adopción	reportes individuales de estudiantes

Esta separación es parte del diseño pedagógico. El estudiante debe tener criterios de éxito claros, pero no la lógica completa de evaluación.

Evaluación con baja carga manual

La calificación manual debe concentrarse en interpretación, decisiones técnicas y casos dudosos marcados por el autograder.

Evidencia	Autocalificable	Revisión humana mínima
funciones numéricas	shapes, valores, tolerancias y pruebas ocultas	patrones sospechosos o hardcoding
notebooks	ejecución limpia, outputs y métricas	interpretación corta
parciales notebook	implementación y resultados	justificación de fórmula o decisión
proyecto	checklist, manifest, métricas y reproducibilidad	defensa, límites y autoría

Principio operativo. Automatice código, resultados y consistencia. Revise manualmente solo aquello que requiere juicio: interpretación, decisión, ética, comunicación y autoría.

Alineación mínima antes de dictar

Antes de abrir una semana, confirme:

- el capítulo asociado está publicado;
- el objetivo observable de la semana aparece en el mapa de objetivos;
- el notebook de clase corre desde cero;
- el laboratorio no expone soluciones ni pruebas ocultas;
- la rúbrica pública coincide con lo que se autocalifica;
- los datasets no requieren internet para evaluación;
- las dependencias están en la guía de configuración;
- hay una pregunta de cierre conectada con proyecto o parcial.

Adaptación a otro contexto

Si el libro se usa fuera de MATE-2105, documente estos cambios:

Decisión	Debe quedar explícito
alcance de capítulos	partes asignadas y partes omitidas
calendario	semanas, fechas y entregas
evaluación	pesos, plataformas y reglas de colaboración
datasets	fuentes, licencia, fecha y restricciones
software	ambiente, versiones y fallback local
accesibilidad	formato principal y canal de reporte
licencia	permisos de uso, redistribución y adaptación

No declare una adaptación como edición abierta si la licencia vigente no lo permite. En esta edición, el repositorio tiene una licencia formal de uso docente restringido.

Revisión de calidad antes de publicar

Use esta secuencia como control mínimo:

1. renderizar libro y portal;
2. ejecutar auditoría de estructura editorial;
3. ejecutar auditoría de objetivos de aprendizaje;
4. ejecutar auditoría de datasets;
5. ejecutar verificación del sitio público;
6. ejecutar auditoría de material sensible;
7. ejecutar auditoría de accesibilidad HTML y axe;
8. revisar visualmente portada, descargas, capítulos modificados y apéndices;
9. generar o verificar manifiesto de publicación;
10. registrar bloqueadores editoriales;
11. no cerrar revisión externa ni accesibilidad manual sin evidencia humana real.

Señales de una adopción saludable

Una adopción del libro funciona cuando:

- los estudiantes saben qué producir antes de cada evaluación;
- los notebooks no son solo demostraciones, sino práctica verificable;
- las rúbricas públicas explican criterios sin revelar tests ocultos;
- el proyecto avanza por checkpoints con evidencia;
- las decisiones técnicas se discuten con límites;
- el instructor revisa pocos casos a mano y con mejor evidencia;
- la versión publicada puede reconstruirse y auditarse.

Referencias y atribuciones

Esta página centraliza las fuentes técnicas, referencias editoriales y atribuciones del libro. No reemplaza las referencias locales de cada capítulo; funciona como registro de trazabilidad para estudiantes, instructores y revisores.

Referencia editorial

El benchmark editorial usado para esta edición es:

- OpenStax. *Principles of Data Science*. Página del libro: <https://openstax.org/details/books/principles-data-science>.
- OpenStax. Prefacio de *Principles of Data Science*: <https://openstax.org/books/principles-data-science/pages/preface>.
- OpenStax. Elementos de cierre de capítulo: términos clave, revisión de capítulo y problemas cuantitativos.
- OpenStax. Separación entre Answer Key, Student Solution Manual e Instructor Solution Manual descrita en el prefacio.

Esta referencia se usa como estándar de formato: front matter, recursos, accesibilidad, errata, atribución, separación estudiante-instructor y cierres de capítulo. No se usa como fuente de contenido matemático para copiar.

Referencias matemáticas y algorítmicas

Estas referencias son útiles para revisión técnica y profundización:

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Trefethen, L. N., & Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.

Documentación técnica

El curso usa documentación oficial como referencia para implementación:

- NumPy documentation: <https://numpy.org/doc/stable/>.
- pandas documentation: <https://pandas.pydata.org/docs/>.
- scikit-learn user guide and API: <https://scikit-learn.org/stable/>.
- scikit-learn toy datasets: https://scikit-learn.org/stable/datasets/toy_dataset.html.
- Matplotlib documentation: <https://matplotlib.org/stable/index.html>.
- seaborn documentation: <https://seaborn.pydata.org/>.
- PyTorch documentation: <https://docs.pytorch.org/docs/stable/index.html>.
- Project Jupyter documentation: <https://docs.jupyter.org/>.
- Quarto books documentation: <https://quarto.org/docs/books/>.

Evaluación automatizada

Las referencias técnicas para autograding y publicación de evaluaciones son:

- Otter-Grader documentation: <https://otter-grader.readthedocs.io/en/latest/>.
- Gradescope autograder documentation: <https://gradescope-autograders.readthedocs.io/en/latest/>.

Los autograders, pruebas ocultas y paquetes de evaluación son recursos restringidos. Esta página referencia las plataformas, no publica material privado de evaluación.

Datasets y generadores

El libro incluye un paquete público de CSV de práctica generados por `scripts/generar_datasets_libro.py` y documentados en `datos/manifest_datos.json`. Estos archivos son originales del curso y se usan para práctica reproducible.

Además, algunos ejemplos y laboratorios pueden usar datasets de `scikit-learn` o generadores de datos controlados con semilla fija:

- `load_wine`: clasificación multiclase en el módulo de arranque.
- `load_diabetes`: regresión y regularización.
- `load_breast_cancer`: clasificación binaria, métricas y umbrales.
- `load_digits`: redes neuronales, PCA y clasificación multiclase.
- `make_regression`: regresión generada y descenso del gradiente.
- `make_blobs`: clustering y geometría de grupos.
- `make_moons`: fronteras no lineales y límites de modelos lineales.

La página **Datos y recursos reproducibles** describe el uso pedagógico y las advertencias de interpretación de estos recursos.

Figuras

Las figuras del atlas visual y de las aperturas de parte son originales del curso MATE-2105. Fueron creadas para explicar objetos matemáticos y decisiones técnicas del libro. El inventario auditable está publicado en [figuras/manifest_figuras.json](#) e incluye, para cada figura, archivo SVG, archivo PNG, caption, texto alternativo, autoría, estado de derechos, primer uso significativo y propósito pedagógico.

Cada figura debe conservar:

- título o caption;
- texto alternativo o descripción equivalente;
- archivo fuente o versión editable cuando exista;
- indicación de autoría del curso si se reutiliza fuera del contexto original.

Si una edición futura incorpora imágenes, tablas, datasets o textos de terceros, debe registrar fuente, autor, licencia, enlace original y modificaciones realizadas.

Cómo citar este libro

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: Libro del curso MATE-2105*. Universidad de los Andes.

<https://cesarneyit-code.github.io/mate2105/libro/>

El repositorio incluye `CITATION.cff` para herramientas compatibles con GitHub y gestores bibliográficos.

Integridad académica y uso de IA

El libro está diseñado para estudiar, practicar y preparar evaluaciones. Esa apertura requiere una regla clara: el estudiante puede recibir ayuda, pero debe poder explicar, modificar y defender lo que entrega.

Principio central

Puedes usar apoyo técnico, bibliográfico o computacional, incluido apoyo de IA generativa, siempre que mantengas responsabilidad sobre cada línea relevante de código, cada cálculo, cada métrica y cada conclusión que presentes.

En este curso, aprender no significa producir una respuesta que pase una prueba. Significa construir criterio técnico verificable.

Uso permitido

Está permitido usar recursos externos o IA para:

- aclarar mensajes de error;
- repasar definiciones, fórmulas o derivaciones;
- proponer pruebas pequeñas para una función;
- mejorar redacción, visualización o estructura de un reporte;
- comparar una implementación con documentación oficial;
- generar preguntas de estudio;
- preparar una defensa oral mediante preguntas de práctica.

El uso permitido siempre exige verificación: ejecutar código, revisar dimensiones, comprobar métricas y contrastar conclusiones con evidencia.

Uso que requiere declaración

Debe declararse de forma breve cualquier uso sustancial de IA o apoyo externo cuando influya en una entrega evaluada. Una declaración suficiente puede tener este formato:

Usé IA para: depurar la función de pérdida y mejorar la redacción de la conclusión.

Verifiqué el resultado mediante: pruebas del laboratorio, comparación con `sklearn` y revisión de métricas.

No es necesario declarar correcciones menores de ortografía, autocompletado trivial o consulta puntual de documentación.

Uso no permitido

No está permitido:

- entregar código que no puedes explicar;
- copiar una solución completa de laboratorio, parcial o proyecto;
- pedir a una herramienta que fabrique interpretaciones sin verificarlas;
- inventar fuentes, métricas, resultados o limitaciones;
- publicar, solicitar o compartir pruebas ocultas, autograders o soluciones oficiales;
- ocultar colaboración sustancial en una entrega individual;
- usar IA para evadir una defensa oral individual.

Evidencia de trabajo propio

Una entrega defendible debe dejar evidencia de:

Evidencia	Pregunta que debe responder
código ejecutable	¿puede reproducirse el resultado?
pruebas o checks	¿cómo se sabe que la implementación calcula lo esperado?
comparación razonable	¿hay baseline, referencia o cálculo manual pequeño?
interpretación	¿qué decisión se justifica y con qué límite?
trazabilidad	¿de dónde vienen datos, fórmulas, fuentes o ayudas externas?

Defensa oral

La defensa oral existe para verificar comprensión individual. Un estudiante debe poder explicar:

- qué representan los datos;
- qué modelo o algoritmo se usó;
- qué función objetivo, pérdida o métrica se optimizó;
- qué errores aparecen y cómo se diagnosticaron;
- qué límites tiene la recomendación;
- qué parte del trabajo fue propia y qué apoyo externo recibió.

Si una respuesta, notebook o conclusión no puede defenderse sin consultar la herramienta que la produjo, todavía no cumple el estándar académico del curso.

Regla práctica antes de entregar

Antes de entregar, responde:

Si el profesor me pide explicar esta línea, esta fórmula, esta métrica o esta conclusión en una defensa oral, ¿puedo hacerlo con mis propias palabras?

Si la respuesta es no, la entrega debe revisarse antes de enviarse.

Respuestas, solucionarios y recursos restringidos

Esta página declara qué tipo de respuestas puede publicar el libro y qué debe permanecer fuera de la publicación. El objetivo es dar retroalimentación útil al estudiante sin debilitar laboratorios, parciales, proyectos ni defensas orales.

Principio editorial

El libro puede publicar respuestas seleccionadas, fórmulas, verificaciones y criterios de interpretación. No debe publicar soluciones oficiales completas de evaluaciones, pruebas ocultas ni lógica de calificación.

Regla de separación. Lo público ayuda a estudiar y verificar dirección. Lo restringido permite calificar, comparar entregas y preservar integridad académica.

Tipos de recurso

Recurso	Audiencia	Estado	Contenido permitido
Respuestas seleccionadas del libro	estudiantes	público	resultado breve, pista, criterio o verificación parcial
Formularios y notación	estudiantes	público	fórmulas, formas, métricas y convenciones
Manual de estudio del estudiante	estudiantes	público	guía de uso de respuestas, rutina y preparación
Rúbricas públicas	estudiantes	público	criterios de evaluación sin pruebas ocultas
Guía de adopción docente	instructores y revisores	público	diseño de curso, separación de recursos y validaciones
Manual de instructor	equipo docente verificado	restringido	desarrollos completos, casos límite y criterios internos

Recurso	Audiencia	Estado	Contenido permitido
Paquetes de calificación	equipo docente verificado	restringido	pruebas, tolerancias, reportes y lógica de evaluación
Bancos no liberados	equipo docente verificado	restringido	variantes de parcial, quiz o defensa

Qué puede aparecer en respuestas públicas

Una respuesta pública es aceptable si cumple al menos una función formativa:

- verificar el signo, dimensión, fórmula o métrica esperada;
- mostrar el nivel de detalle de una interpretación breve;
- señalar un error frecuente;
- conectar un resultado con una limitación;
- dar una pista suficiente para corregir sin revelar todo el desarrollo.

Qué no debe publicarse

El libro y el portal público no deben incluir:

- desarrollo completo de laboratorios evaluados;
- código final de evaluaciones;
- pruebas ocultas, tolerancias o casos límite privados;
- respuestas completas de parciales no liberados;
- soluciones únicas para proyectos abiertos;
- reportes individuales de estudiantes;
- instrucciones internas para decidir casos dudosos de calificación.

Política por tipo de pregunta

Tipo de pregunta	Respuesta pública adecuada	Recurso restringido adecuado
Conceptual	definición breve o distinción clave	ejemplos de respuestas completas y distractores
Cuantitativa	valor final, fórmula o paso crítico	desarrollo algebraico completo y variantes
Computacional	shape, output esperado o test visible mínimo	pruebas ocultas y casos límite
Interpretativa	estructura evidencia-límite-decisión	rúbrica detallada y ejemplos calibrados
Proyecto	checklist, criterio y ejemplo abstracto	retroalimentación individual y notas de defensa

Respuestas de proyectos

Los proyectos no tienen solución oficial única. Una buena respuesta depende de datos, contexto, métrica, costo de error, limitaciones y defensa técnica. Por eso el libro puede publicar:

- estructura de reporte;
- rúbricas;
- ejemplos abstractos;
- checklists de reproducibilidad;
- criterios de defensa.

No debe publicar una solución modelo que los equipos puedan convertir en plantilla mecánica para evitar la toma de decisiones.

Proceso para liberar una respuesta

Antes de pasar una respuesta de restringida a pública, revisar:

1. ¿ayuda a aprender sin reemplazar el ejercicio?
2. ¿evita revelar pruebas, tolerancias o casos límite privados?
3. ¿no corresponde a una evaluación activa?
4. ¿declara límites cuando hay interpretación?
5. ¿puede mantenerse si cambia el autograder o la rúbrica?
6. ¿respeto privacidad y autoría?
7. ¿está enlazada desde el capítulo, apéndice o guía adecuada?

Plantillas de control

El libro publica plantillas para mantener la separación:

Plantilla	Uso
plantillas/solucionarios/manual_estudiante	respuestas en curso público con pistas y verificaciones parciales
plantillas/solucionarios/manual_instructores	restringido en curso restringido sin mezclarlo con la publicación
plantillas/solucionarios/registro_liberacion	decisiones de si una respuesta puede pasar a pública

Estas plantillas son públicas porque describen proceso editorial, no contenido de evaluación.

Criterio de auditoría

Una edición publicable debe poder demostrar que:

- el apéndice de respuestas seleccionadas existe y está enlazado;
- la política de respuestas está publicada;
- los recursos restringidos no aparecen en el sitio público;
- el auditor de publicación no detecta material sensible;
- la guía de estudiante explica cómo usar respuestas sin copiar;
- la guía de adopción docente explica la separación público/restringido.

Errata, revisión y control de calidad

Un libro de curso debe poder corregirse sin perder trazabilidad. Esta página define cómo registrar errores, cómo priorizarlos y qué evidencia se requiere para cerrar una corrección.

Qué cuenta como errata

Debe registrarse como errata cualquier problema que pueda afectar aprendizaje, evaluación, reproducibilidad o confianza editorial:

- fórmula incorrecta o ambigua;
- derivación incompleta;
- ejemplo con resultado numérico inconsistente;
- código que no ejecuta o no coincide con el texto;
- figura sin explicación suficiente;
- tabla rota en HTML o PDF;
- enlace roto;
- instrucción de evaluación confusa;
- material privado publicado por error.

Severidad

Severidad	Criterio	Acción
Alta	afecta una fórmula, resultado, evaluación o integridad académica	corregir antes de publicar
Media	confunde una explicación, ejemplo o ejercicio	corregir en la siguiente revisión
Baja	estilo, ortografía o formato menor	agrupar en una pasada editorial

Flujo de corrección

1. registrar capítulo, sección y descripción del problema;
2. clasificar severidad;
3. proponer corrección mínima;
4. revisar impacto en notebooks, laboratorios, rúbricas o autograders;

5. aplicar cambio en fuente, no solo en HTML generado;
6. renderizar y validar;
7. registrar evidencia de cierre.

Plantillas de errata

El paquete operativo de errata está disponible en:

Plantilla	Uso
Reporte de errata	reportar problema con ubicación, evidencia y corrección sugerida
Registro público de errata	mantener historial público de cambios aceptados o cerrados
Cierre de corrección	documentar evidencia después de corregir
Clasificación de severidad	decidir prioridad, evidencia mínima y manejo privado

Estas plantillas se publican para transparencia editorial. Los reportes que incluyan soluciones oficiales, pruebas ocultas o lógica de autograding deben moverse al espacio docente privado.

Evidencia de cierre

Una errata se considera cerrada solo si existe evidencia verificable:

- archivo fuente corregido;
- render actualizado del libro;
- prueba o auditoría relevante aprobada;
- revisión visual cuando el problema era de formato;
- nota breve del impacto sobre materiales asociados.

Registro público

La página [Registro público de errata](#) muestra el estado vigente de reportes abiertos, errata cerrada y cambios editoriales de la edición. Aunque no haya errata confirmada, el registro debe existir para que el lector sepa dónde consultar correcciones futuras.

Separación de errata pública y docente

La errata pública puede mencionar problemas de texto, código público o enlaces. La errata docente privada debe manejar por separado cualquier asunto que revele:

- soluciones oficiales;
- pruebas ocultas;
- bancos de parcial;
- criterios internos de calificación;
- casos límite del autograder.

Si una corrección involucra material privado, primero se retira del sitio público y luego se revisa el historial de publicación. La integridad académica tiene prioridad sobre la estética editorial.

Registro público de errata

Esta página mantiene el estado público de errata y correcciones del libro. Su propósito es que estudiantes, instructores y revisores puedan distinguir entre problemas confirmados, correcciones cerradas y cambios editoriales de una nueva edición.

Estado actual

Campo	Estado
Edición	Edición de trabajo 2026-20
Fecha de actualización	2026-07-07
Errata confirmada abierta	0
Errata confirmada cerrada	0
Correcciones editoriales registradas	1
Material privado afectado	no

Que no haya errata confirmada no significa que el libro esté libre de errores. Significa que, al momento de esta edición, no hay reportes públicos aceptados y pendientes de corrección.

Estados de un reporte

Estado	Significado
recibido	el reporte llegó, pero todavía no fue triageado
en revisión	el problema se está verificando
aceptado	el problema fue confirmado y requiere corrección
corregido	la corrección fue aplicada en la fuente
cerrado	la corrección fue renderizada, validada y documentada
no reproducible	no se pudo confirmar con la evidencia disponible
privado	el reporte toca soluciones, autograders o evaluación y se mueve al repositorio docente

Errata abierta

ID	Fecha	Ubicación	Severidad	Estado	Nota
—	—	—	—	sin errata abierta	—

Errata cerrada

ID	Fecha	Ubicación	Tipo	Evidencia de cierre
—	—	—	—	sin errata cerrada registrada

Cambios editoriales de edición

Estos cambios no son necesariamente errores. Se registran porque afectan la forma pública del libro.

Fecha	Alcance	Tipo	Evidencia
2026-07-07	Libro completo	consolidación editorial de edición de trabajo: front matter, accesibilidad, errata, datasets, índices, figuras, auditorías y publicación	render del libro, auditoría editorial, auditoría PDF, manifest y verificación del portal

Criterio de publicación de errata

Un reporte se publica en esta página cuando:

1. afecta contenido público del libro;
2. no revela soluciones oficiales, pruebas ocultas ni lógica de autograding;
3. tiene ubicación verificable;
4. fue clasificado por severidad;
5. existe una decisión editorial: aceptar, cerrar, rechazar o mover a privado.

Reportes privados

Los reportes que mencionen evaluaciones, soluciones completas, pruebas ocultas, rúbricas internas, bancos de parcial o paquetes de autograding no se publican aquí. Deben manejarse en el repositorio docente privado y, si hubo exposición accidental, también debe revisarse el historial de publicación.

Cómo reportar

Para reportar una errata pública, usar la plantilla:

- [reporte_errata.md](#)

Para cerrar una corrección, usar:

- [cierre_correccion.md](#)

Para decidir severidad, usar:

- [clasificacion_severidad.md](#)

Criterio de cierre

Una errata pública queda cerrada cuando:

- el archivo fuente fue corregido;
- el libro fue renderizado nuevamente;
- la auditoría relevante pasó;
- el manifest publicado fue regenerado;
- la corrección no expuso material privado;
- el registro público fue actualizado.

Edición, versión y publicación

Esta página funciona como ficha editorial de la edición vigente. Su propósito es que el lector, el estudiante y el instructor sepan qué versión están usando, qué estado tiene el material y qué revisiones deben cumplirse antes de publicarlo como texto estable.

Ficha de edición

Campo	Valor
Título	<i>Matemáticas y Algoritmos en Ciencia de Datos Aplicada</i>
Curso	MATE-2105
Autor	César Galindo
Edición	Edición de trabajo 2026-20
Fecha de construcción	2026-07-07
Formatos	libro web, PDF, datasets públicos, guía de impresión, manifiesto de publicación y capítulos fuente Quarto
Sitio público previsto	https://cesarneyit-code.github.io/mate2105/libro/
Estado	versión docente en revisión editorial

Criterio de versión estable

Una versión puede considerarse estable cuando cumple estas condiciones:

1. todos los capítulos principales tienen objetivos, ejemplos resueltos, revisión, términos clave, ejercicios y proyecto de cierre;
2. los apéndices tienen notación mínima, fórmulas y respuestas seleccionadas;
3. las figuras tienen texto alternativo o explicación equivalente;
4. el PDF no tiene tablas, fórmulas o títulos visualmente rotos;
5. el portal no contiene soluciones oficiales completas ni autograders privados;
6. la auditoría PDF verifica metadatos, tamaño carta, texto extraíble y ausencia de JavaScript o encriptación;
7. las auditorías de estructura, enlaces, publicación y accesibilidad pasan;
8. el manifiesto de publicación existe y registra hashes de artefactos primarios;
9. el paquete de datasets públicos tiene manifest y auditoría de hashes;

10. la declaración de accesibilidad está publicada y distingue evidencia automática, revisión manual pendiente y barreras conocidas;
11. el registro público de errata está publicado y actualizado para la edición;
12. la licencia y el estado de uso están documentados explícitamente;
13. el estado de impresión del PDF está documentado cuando se distribuya como versión archivable o imprimible.
14. cada página HTML incluye un pie editorial con cita sugerida, estado de uso y enlaces a licencia, accesibilidad, errata y atribuciones.

Canales de publicación

El libro debe publicarse en dos niveles:

Canal	Función	Ritmo de cambio
libro web	lectura principal, búsqueda y navegación	puede actualizarse durante el semestre
PDF	lectura offline y archivo de versión	debe actualizarse solo después de revisión visual
datasets públicos	práctica reproducible y descargas del libro	se regeneran con semilla fija antes de publicación
accesibilidad	declaración pública, pruebas y remediación	se actualiza con cada barrera o auditoría
errata pública	reportes abiertos, cerrados y cambios editoriales	se actualiza con cada corrección aceptada
guía de impresión	revisión de PDF archivable e imprimible	se completa antes de impresión pública
formatos y descargas	entrada pública a HTML, PDF, datasets y manifiestos	debe actualizarse con cada cambio de artefactos
metadatos bibliográficos	citation tags, Dublin Core, Open Graph y JSON-LD	deben coincidir con título, autor, fecha, URL y estado de uso
capturas visuales	evidencia de portada, páginas editoriales, descargas, capítulos y apéndices	se regeneran con cada candidato de publicación
manifiesto	inventario, tamaños y hashes de publicación	se regenera con cada render público
portal del curso	calendario, materiales y entregas	cambia por cohorte
repositorio fuente	historial, edición y auditoría	cambia con cada mejora aprobada

El portal organiza el curso. El libro organiza el conocimiento. Una publicación profesional mantiene esa separación incluso cuando ambos viven en el mismo sitio.

Checklist de publicación

Antes de publicar una nueva edición:

- renderizar libro y portal desde cero;
- verificar enlaces del sitio público;
- auditar material sensible;
- auditar estructura editorial;
- auditar PDF generado;
- auditar accesibilidad estática;
- correr **axe** sobre páginas representativas;
- revisar y actualizar la declaración de accesibilidad;
- revisar y actualizar el registro público de errata;
- revisar visualmente portada, índice, capítulos modificados, tablas y figuras;
- completar checklist de impresión si el PDF se usará como copia archivable o imprimible;
- confirmar que PDF, HTML y descargas corresponden a la misma versión;
- revisar que la página de formatos y descargas apunte a los artefactos vigentes;
- verificar metadatos bibliográficos en HTML;
- generar capturas visuales representativas en escritorio y móvil;
- generar y revisar **manifest-publicacion.json**;
- registrar cambios editoriales relevantes;
- confirmar que licencia, cita sugerida y política de errata están visibles.
- verificar que el pie editorial persistente aparezca en páginas representativas del libro.

Registro de edición

Fecha	Cambio editorial	Evidencia esperada
2026-07-07	Consolidación de estructura tipo libro, front matter, auditorías, figuras, ejemplos resueltos, apéndices, guía de impresión y registro público de errata	render del libro, auditoría editorial y validación del portal

Registros de candidato

Cada construcción que se quiera tratar como candidato de publicación debe tener un registro interno en **libro_curso/registros/publicacion/**. Ese registro no reemplaza el manifiesto de publicación: el manifiesto prueba qué archivos fueron publicados; el registro editorial explica qué evidencia se revisó, qué estado se declara y qué bloqueadores quedan abiertos.

Para esta edición, el registro vigente es:

libro_curso/registros/publicacion/candidato_2026-07-07.md

Paquete de publicación

Esta página define qué se entrega cuando el libro se publica como versión web del curso. El objetivo es que cada edición pueda auditarse con artefactos identificables, checksums y comandos de validación, no solo con una carpeta HTML.

Artefactos públicos

Cada publicación debe incluir, como mínimo:

Artefacto	Ruta esperada	Propósito
Libro web	<code>index.html</code> y capítulos HTML	lectura principal, búsqueda y navegación
PDF	<code>Matemáticas-y-Algoritmos-en-Ciencia-de-Datos-Aplicada.pdf</code>	lectura offline y archivo de versión
Manifiesto	<code>manifest-publicacion.json</code>	inventario de archivos, tamaños y hashes
Sitemap	<code>sitemap.xml</code>	descubrimiento por buscadores
Recursos visuales	<code>figuras/</code> , <code>figuras/manifest_figuras.json</code> , <code>portada.svg</code> , <code>portada.png</code>	figuras, portada, atribuciones y assets del libro
Datasets públicos	<code>datos/*.csv</code> y <code>datos/manifest_datos.json</code>	práctica reproducible y descargas del libro
Estilos	<code>styles.css</code>	formato pedagógico y ajustes de accesibilidad
Apéndices de consulta	<code>apendices/</code>	glosario, índice temático, algoritmos y respuestas seleccionadas
Taxonomía de ejercicios	<code>apendices/taxonomia-ejercicios.html</code>	tipos de pregunta, niveles cognitivos y banco público por módulo
Guías de inicio	<code>guia_estudiante.html</code> y <code>guia_adopcion_docente.html</code>	rutina de estudio, adopción docente y separación público/privado
Mapa de objetivos de aprendizaje	<code>mapa_objetivos_aprendizaje.html</code> y <code>objetivos/manifest_objetivos_aprendizaje.json</code>	objetivos observables, evidencia y alineación por capítulo

Artefacto	Ruta esperada	Propósito
Mapa de cierres de capítulo	<code>mapa_cierres_capitulo.html</code>	revisión, términos clave, práctica y proyecto por capítulo
Política de respuestas y solucionarios	<code>solucionarios_respuestas.html</code> y <code>plantillas/solucionarios/</code>	separación entre respuestas públicas y recursos restringidos
Guía de impresión	<code>guia_impresion.html</code>	criterios para PDF archivable e impresión controlada
Plantillas de impresión	<code>plantillas/impresion/</code>	checklist visual y registro de prueba de impresión
Declaración de accesibilidad	<code>accesibilidad.html</code>	estado de accesibilidad, barreras conocidas y proceso de remediación
Plantillas de accesibilidad	<code>plantillas/accesibilidad/</code>	revisión de teclado, lector de pantalla y reporte de barreras
Formatos y descargas	<code>formatos_descargas.html</code>	acceso directo a HTML, PDF, datasets, manifiestos y estado de uso
Metadatos bibliográficos	<code><head></code> de páginas HTML	citation tags, Dublin Core, Open Graph y JSON-LD Schema.org
Registro público de errata	<code>registro_errata_publica.html</code>	estado de reportes, correcciones y cambios editoriales
Pie editorial persistente	todas las páginas HTML del libro	cita sugerida, estado de uso, licencia, accesibilidad, errata y atribuciones

El manifiesto no reemplaza la revisión editorial. Sirve para probar que una carpeta publicada corresponde a una construcción concreta y para detectar cambios accidentales en archivos públicos.

Manifiesto de publicación

El archivo público [manifest-publicacion.json](#) se genera después de renderizar el sitio. Contiene:

- fecha de generación;
- título y URL pública prevista;
- lista de artefactos primarios;
- inventario de archivos públicos del libro;
- tamaño en bytes;
- hash SHA-256;
- comandos recomendados de validación.

El manifiesto excluye librerías vendorizadas de Quarto en `site_libs/`, porque su contenido depende de la herramienta de render y no del manuscrito del libro. Sí incluye páginas HTML, figuras, datasets públicos, PDF, sitemap, estilos y archivos de búsqueda.

El archivo [figuras/manifest_figuras.json](#) registra las figuras originales con SVG, PNG, texto alternativo, atribución, estado de derechos, primer uso significativo y propósito pedagógico.

El archivo [objetivos/manifest_objetivos_aprendizaje.json](#) registra los objetivos observables de los capítulos principales, la evidencia esperada y la alineación con resultados globales.

Comandos de construcción

Desde la raíz del repositorio:

```
bash sitio_curso/scripts/sync_libro.sh
```

Para preparar una carpeta pública separada:

```
bash scripts/preparar_repo_publico.sh
```

Ambos flujos generan el manifiesto del libro publicado.

Comandos de validación

Antes de publicar, deben pasar:

```
python3 scripts/generar_datasets_libro.py
python3 scripts/auditar_datasets_libro.py
python3 scripts/auditar_figuras_libro.py --published-root sitio_curso/_site/libro
python3 scripts/auditar_objetivos_aprendizaje_libro.py --published-root sitio_curso/_site/libro
python3 scripts/auditar_estructura_editorial_libro.py
python3 scripts/auditar_pdf_libro.py sitio_curso/_site/libro/Matemáticas-y-Algoritmos-en-Cienc
python3 scripts/auditar_manifest_publicacion_libro.py sitio_curso/_site/libro
python3 scripts/auditar_publicacion.py sitio_curso/_site
python3 scripts/verificar_sitio_publico.py sitio_curso/_site
python3 scripts/auditar_accesibilidad_libro.py sitio_curso/_site/libro
python3 scripts/generar_capturas_visuales_libro.py
python3 scripts/auditar_preparacion_openstax_libro.py
bash scripts/auditar_axe_libro.sh http://127.0.0.1:8765/libro
```

Además, el PDF debe revisarse visualmente en portada, índice, capítulos modificados, tablas, figuras, fórmulas largas y páginas de apéndices. Si la edición se va a imprimir, también debe completarse la guía de impresión y PDF del libro.

Para una edición publicable fuera del curso, también deben completarse las plantillas manuales de accesibilidad para teclado y lector de pantalla. Los paquetes de trabajo preparados para esas revisiones están en `libro_curso/registros/revision_externa/paquete_revision_externa_2026-07-07.md` y `libro_curso/registros/accesibilidad/paquete_revision_manual_2026-07-07.md`. El cierre final se verifica con:

```
python3 scripts/auditar_preparacion_openstax_libro.py --final
```

Criterio de aceptación

Una publicación del libro se considera aceptable para el curso cuando:

1. el sitio renderizado contiene HTML, PDF y manifiesto;
2. el manifiesto registra hashes de los artefactos primarios;
3. el paquete de datasets públicos pasa auditoría de filas, columnas y hashes;
4. no hay enlaces internos rotos;
5. no hay material sensible en el sitio público;
6. la auditoría editorial pasa;
7. la auditoría PDF pasa y declara explícitamente el estado de etiquetado;
8. la auditoría HTML de accesibilidad pasa;
9. **axe** no reporta violaciones en el muestreo vigente;
10. la versión PDF se revisó visualmente.
11. la declaración de accesibilidad está publicada y no promete más de lo que la evidencia puede probar.
12. el registro público de errata está actualizado para la edición.
13. si el PDF se presenta como versión imprimible, existe checklist de impresión o registro de prueba asociado.
14. la taxonomía pública de ejercicios está sincronizada con los capítulos, laboratorios y bancos integradores vigentes.
15. el pie editorial persistente aparece en páginas representativas del libro.
16. la página de formatos y descargas enlaza los artefactos públicos vigentes.
17. existe un registro de capturas visuales representativas para escritorio y móvil.
18. la portada HTML incluye metadatos bibliográficos machine-readable.
19. las figuras publicadas tienen manifiesto auditable con atribución, texto alternativo, formatos SVG/PNG y primer uso significativo.
20. los capítulos principales tienen objetivos observables auditables, evidencia primaria y alineación con resultados globales.

Para una edición publicable fuera del curso, además se requiere revisión externa registrada y revisión manual de accesibilidad. La decisión explícita sobre accesibilidad del PDF ya está registrada: HTML es el formato accesible principal y el PDF es respaldo offline/archivo de versión. Para una edición abierta tipo OER, también debe documentarse una licencia abierta que reemplace o complemente el LICENSE restringido vigente.

Guía de impresión y PDF

Esta página define el estándar mínimo para tratar el PDF como una edición archivable e imprimible del libro. La versión web sigue siendo el formato principal de lectura y accesibilidad; el PDF funciona como respaldo estable, lectura offline y fuente para impresión controlada.

Relación entre web, PDF e impresión

Formato	Uso principal	Criterio editorial
Web	lectura principal, búsqueda, enlaces, actualización y accesibilidad	debe ser la versión más actual y navegable
PDF	lectura offline, archivo de versión y distribución cerrada	debe corresponder al mismo contenido renderizado del libro web
Impresión	lectura prolongada, archivo físico o copia institucional	requiere revisión visual adicional antes de ordenar copias

El PDF no reemplaza la revisión de accesibilidad de la versión web. Mientras el PDF no sea etiquetado, la publicación debe declarar que la versión HTML es la opción accesible principal.

Especificación de PDF

La edición vigente usa estos parámetros de construcción:

- tamaño de página: carta;
- márgenes: 1 pulgada;
- tabla de contenido hasta nivel de subsección;
- enlaces en color;
- código con ajuste de línea;
- figuras con captions;
- metadatos de título y autor;
- texto extraíble;
- sin JavaScript;
- sin encriptación.

La auditoría técnica se ejecuta con:

```
python3 scripts/auditar_pdf_libro.py sitio_curso/_site/libro/Matemáticas-y-Algoritmos-en-Cienc.
```

Para generar evidencia visual HTML representativa:

```
python3 scripts/generar_capturas_visuales_libro.py
```

Revisión visual obligatoria

Antes de usar el PDF como versión de referencia, revisar una muestra que cubra:

1. portada;
2. índice;
3. información editorial;
4. páginas con tablas anchas;
5. páginas con fórmulas largas;
6. páginas con bloques de código;
7. páginas con figuras;
8. ejercicios integradores;
9. apéndices;
10. páginas finales.

La revisión debe confirmar:

- que no hay títulos aislados al final de página;
- que las tablas no se salen del área imprimible;
- que las fórmulas largas no quedan cortadas;
- que los bloques de código mantienen legibilidad;
- que las figuras no pierden su caption;
- que las referencias internas no aparecen como enlaces rotos;
- que el contraste es suficiente en impresión a color y en escala de grises;
- que el número total de páginas coincide con el registro de edición.

Prueba de impresión

Para una edición pública fuera del curso, la prueba de impresión debe hacerse antes de anunciar el PDF como versión final. La prueba mínima consiste en:

- imprimir portada, índice, una parte introductoria, un capítulo matemático, un capítulo con código, un capítulo con figuras y un apéndice;
- revisar lectura en escala de grises;
- confirmar que los enlaces visibles no son indispensables para entender el contenido impreso;
- verificar que tablas y fórmulas conservan alineación;
- registrar cualquier corrección en el flujo de errata.

Plantillas de revisión

Las plantillas operativas están publicadas con el libro:

- [checklist_revision_impresion.md](#)
- [registro_prueba_impresion.md](#)

Estado de impresión de esta edición

Elemento	Estado	Evidencia
PDF generado	verificado automáticamente	auditoría PDF
tamaño carta	verificado automáticamente	pdfinfo y auditoría PDF
texto extraíble	verificado automáticamente	extracción de texto
JavaScript/encryptación	verificado automáticamente	auditoría PDF
PDF etiquetado	no disponible en la edición vigente	decisión formal: HTML es el formato accesible principal
capturas HTML representativas	verificado automáticamente	scripts/generar_capturas_visuales_libro
revisión visual completa	pendiente para impresión pública	checklist de impresión
prueba física o simulada	pendiente para impresión pública	registro de prueba

Criterio de aceptación para impresión

Una edición puede enviarse a impresión controlada cuando:

1. el libro web, el PDF y el manifiesto provienen del mismo render;
2. la auditoría PDF pasa sin errores;
3. la revisión visual obligatoria está registrada;
4. las páginas impresas de muestra son legibles en color y escala de grises;
5. las correcciones obligatorias de maquetación están cerradas;
6. la edición, licencia y cita sugerida están visibles;
7. el estado de accesibilidad del PDF se declara explícitamente.

La decisión vigente está registrada en **libro_curso/registros/pdf/decision_accesibilidad_pdf_2026-07-07.md**: el PDF se usa como respaldo offline y archivo de versión; el HTML es el formato accesible principal.

Licencia y uso del material

Estado de licencia

Este libro es material docente del curso **MATE-2105 Matemáticas y Algoritmos en Ciencia de Datos Aplicada**.

El repositorio incluye un archivo **LICENSE** formal de uso docente restringido. Esa licencia permite lectura, estudio, revisión y uso dentro del curso, pero no es una licencia abierta ni una licencia Creative Commons. La redistribución, adaptación pública o uso comercial requiere autorización explícita del autor.

Esta página separa el estado legal del material de su uso pedagógico. La intención futura puede ser publicar una versión abierta, pero esa decisión debe quedar documentada de manera explícita en el repositorio y reemplazar o complementar el **LICENSE** vigente.

La página **Decisión de licencia abierta** compara opciones de licencia y define el procedimiento para migrar de uso docente restringido a licencia abierta sin confundir intención editorial con permiso legal vigente.

Uso permitido en el curso

Los estudiantes pueden:

- leer el libro en línea;
- descargar materiales públicos del portal;
- citar secciones del libro en sus reportes de curso;
- usar fragmentos de código educativo dentro de sus notebooks de clase y laboratorios, con atribución cuando corresponda.

Uso no permitido sin autorización

No está permitido, salvo autorización explícita:

- redistribuir el libro completo como producto propio;
- vender copias o adaptaciones;
- remover autoría o contexto institucional;
- publicar soluciones oficiales, pruebas ocultas o autograders;
- mezclar este texto con materiales externos de licencia incompatible.

Cita sugerida

Galindo, C. (2026). *Matemáticas y Algoritmos en Ciencia de Datos Aplicada: Libro del curso MATE-2105*. Universidad de los Andes.

<https://cesarneyit-code.github.io/mate2105/libro/>

Materiales de terceros

Si una versión futura incorpora imágenes, datasets o textos de terceros, cada recurso debe incluir:

- fuente;
- autor o entidad responsable;
- licencia;
- enlace original;
- modificaciones realizadas, si aplica;
- texto alternativo cuando el recurso sea visual.

Integridad académica

El libro público no contiene soluciones oficiales completas ni pruebas privadas. Esa separación permite estudiar con transparencia sin debilitar laboratorios, parciales o autograders.

La política operativa de autoría, colaboración y uso de IA se documenta en la página **Integridad académica y uso de IA** de este mismo libro.

Decisión de licencia abierta

Esta página documenta cómo debe tomarse una eventual decisión de licencia abierta para el libro. No reemplaza el archivo LICENSE del repositorio. En la edición vigente, ese archivo declara una licencia formal de **uso docente restringido**; por tanto, el libro no es todavía un recurso educativo abierto en sentido estricto.

Por qué esta decisión importa

Un libro profesional debe decir con claridad qué puede hacer el lector con el material:

- leerlo;
- compartirlo;
- adaptarlo;
- traducirlo;
- imprimirlo;
- usar fragmentos en clase;
- reutilizar código;
- reutilizar figuras;
- usarlo con fines comerciales o no comerciales.

OpenStax declara explícitamente que *Principles of Data Science* usa una licencia Creative Commons BY-NC-SA 4.0 para el contenido del libro. Ese es un referente editorial útil, pero no obliga a este curso a elegir la misma licencia.

Opciones razonables

Opción	Qué permite	Ventaja	Cuidado
Uso restringido del curso	lectura y uso dentro de MATE-2105	protege evaluaciones y futuras versiones	no es una licencia abierta
CC BY 4.0	reutilización amplia con atribución	máxima circulación académica	permite uso comercial y adaptaciones permisivas
CC BY-SA 4.0	reutilización con atribución y misma licencia	mantiene adaptaciones abiertas	puede limitar compatibilidad con algunos materiales
CC BY-NC 4.0	reutilización no comercial con atribución	reduce uso comercial no autorizado	“no comercial” puede ser ambiguo

Opción	Qué permite	Ventaja	Cuidado
CC BY-NC-SA 4.0	reutilización no comercial, atribución y misma licencia	cercano al modelo OpenStax revisado	limita uso comercial y exige compartir adaptaciones bajo la misma licencia

La decisión de licencia debe cubrir el libro, las figuras, el código educativo y los materiales descargables. No debe cubrir soluciones oficiales, pruebas ocultas, autograders ni guías docentes privadas.

Separación por tipo de material

La licencia no tiene que ser idéntica para todos los artefactos.

Artefacto	Decisión recomendada antes de publicar
texto del libro	uso docente restringido vigente; escoger licencia abierta solo si se quiere OER
figuras originales	alinear con la licencia del texto o declarar atribución propia
fragmentos de código	declarar si siguen la licencia del libro o una licencia de software
notebooks públicos	declarar si son material docente abierto o solo del curso
datasets externos	conservar licencia y términos de la fuente original
autograders y pruebas ocultas	mantener privados y fuera de la licencia pública
soluciones oficiales	mantener privadas o publicar solo respuestas seleccionadas

Criterios de decisión

Antes de reemplazar o complementar el LICENSE restringido vigente con una licencia abierta, resolver estas preguntas:

1. ¿Se quiere permitir adaptación por otros profesores?
2. ¿Se quiere permitir uso comercial?
3. ¿Las adaptaciones deben conservar la misma licencia?
4. ¿Las figuras y el texto tendrán la misma licencia?
5. ¿Los notebooks públicos seguirán la licencia del libro?
6. ¿Hay material institucional que requiera aprobación previa?
7. ¿Hay recursos de terceros incompatibles con la licencia deseada?
8. ¿Cómo se separan libro público y materiales privados de evaluación?

Registro de decisión pendiente

Elemento	Estado	Evidencia requerida
licencia del texto	uso docente restringido	archivo LICENSE
licencia abierta del texto	pendiente	decisión explícita de Creative Commons u otra licencia abierta
licencia de figuras originales	alineada al uso docente restringido salvo indicación contraria	nota en referencias/atribuciones
licencia de código educativo	uso docente restringido salvo indicación contraria	archivo LICENSE y README
tratamiento de notebooks públicos	pendiente de normalización por módulo	política en portal o README
exclusión de autograders	definida	política de recursos restringidos
revisión de terceros	pendiente	tabla de atribuciones completa

Procedimiento de cierre

Para cerrar la decisión de licencia:

1. elegir licencia abierta o mantener uso restringido de forma explícita;
2. reemplazar o complementar **LICENSE** si se adopta una licencia abierta;
3. actualizar `licencia_uso.qmd`, `README.md`, `CITATION.cff` y el portal;
4. revisar atribuciones de figuras, datasets y fragmentos de código;
5. confirmar que materiales privados no quedan incluidos;
6. renderizar libro y portal;
7. ejecutar auditorías de publicación;
8. registrar la decisión en la matriz editorial.

La plantilla fuente para documentar esta decisión está en [registro_decision_licencia.md](#).